# Volume 1: The Foundational Mathematics Primer for Advanced AI, ML, and LLMs – An Exhaustive PM Masterclass

## Table of Contents

# Reader Orientation

This volume is written for Prabhjot S. Ahluwalia, a senior fintech Product Manager rebuilding advanced AI and ML mathematical fluency with production judgment. The manuscript treats mathematics as a product operating system: each definition is connected to Paytm-scale merchant economics, fraud detection, retention automation, reconciliation, and LLM systems. Every topic follows the same six-part rhythm: intuition, formal definition, two numeric fintech examples, common pitfalls, PM relevance, and runnable NumPy.

| PM Relevance |
| --- |
| - Why a PM must master this: Mathematical fluency lets a product leader challenge model behavior, pricing experiments, fraud thresholds, and AI roadmap claims without outsourcing judgment. |
| - Metrics to own: Revenue, precision, recall, false-positive cost, churn prevention, model latency, calibration, experiment power, and risk-adjusted performance. |
| - Strategic tradeoffs: Accuracy vs latency, revenue vs retention, automation vs human review, interpretability vs model capacity, and short-term conversion vs long-term trust. |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: A Georgia Tech-trained fintech PM should be able to translate linear algebra, probability, calculus, optimization, information theory, graph logic, and time-series math into product decisions at payment-network scale. |

# Chapter 1 — Linear Algebra: The Geometry Engine of Machine Learning

Linear algebra is the coordinate system of modern AI. Every embedding, every attention head, every recommender, every fraud-risk score lives inside a vector space. If you cannot reason about vectors, matrices, and their decompositions, you cannot reason about an LLM's behavior — you can only reason about its output. As a Paytm PM owning merchant intelligence, you need both.

## 1.1 Vectors

### 1.1.1 Plain-English Intuition

A vector is an ordered list of numbers that together describe something. One number alone is a scalar — "the temperature is 31". Two numbers together — "31°C, 78% humidity" — start to describe a state. A vector is just that idea, generalized: a tuple of measurements that, taken jointly, locate an object inside a space of possibilities.

Everyday analogy — GPS. Your phone's location is a vector $(\text{lat}, \text{lon}, \text{altitude})$. No single coordinate tells you where you are; only the three together do. Move any one of them and you are somewhere else. (Plain English: GPS shows three numbers — north-south, east-west, and height — and only all three together pin you to a real spot on Earth.)

Fintech analogy — a Paytm merchant profile. A kirana store can be represented as the vector

$$\mathbf{m} = (\text{avg ticket size}, \text{txns/day}, \text{refund rate}, \text{QR scans}, \text{nights-open ratio})$$

Plain English: each merchant is a list of five numbers describing how they actually behave. Two merchants with similar vectors are similar businesses, even if their names are different. This is the seed of every clustering, recommendation, and risk model you will ever ship.

### 1.1.2 Formal Mathematical Definition

A vector in $\mathbb{R}^n$ is an element of the $n$-dimensional real coordinate space:

$$\mathbf{v} = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix} \in \mathbb{R}^n.$$

Plain English: v is a column of n real numbers.

Vector addition is componentwise: $(\mathbf{u} + \mathbf{v})_i = u_i + v_i$. Plain English: add coordinate-by-coordinate.

Scalar multiplication: $(\alpha \mathbf{v})_i = \alpha v_i$. Plain English: stretch every coordinate by the same factor.

Inner (dot) product:

$$\mathbf{u} \cdot \mathbf{v} = \sum_{i=1}^n u_i v_i = \|\mathbf{u}\| \, \|\mathbf{v}\| \cos\theta.$$

Plain English: multiply matching coordinates, add them up; equivalently, it equals the product of the two lengths times the cosine of the angle between them.

Euclidean norm (length):

$$\|\mathbf{v}\| = \sqrt{\sum_{i=1}^n v_i^2} = \sqrt{\mathbf{v} \cdot \mathbf{v}}.$$

Plain English: the straight-line length of the arrow from the origin to the point v.

Derivation that $\mathbf{u}\cdot\mathbf{v} = \|\mathbf{u}\|\|\mathbf{v}\|\cos\theta$. Consider the triangle with sides $\mathbf{u}, \mathbf{v}, \mathbf{u}-\mathbf{v}$. The Law of Cosines gives

$$\|\mathbf{u}-\mathbf{v}\|^2 = \|\mathbf{u}\|^2 + \|\mathbf{v}\|^2 - 2\|\mathbf{u}\|\|\mathbf{v}\|\cos\theta.$$

But also $\|\mathbf{u}-\mathbf{v}\|^2 = (\mathbf{u}-\mathbf{v})\cdot(\mathbf{u}-\mathbf{v}) = \|\mathbf{u}\|^2 - 2\mathbf{u}\cdot\mathbf{v} + \|\mathbf{v}\|^2$. Equating the two and cancelling yields $\mathbf{u}\cdot\mathbf{v} = \|\mathbf{u}\|\|\mathbf{v}\|\cos\theta$. QED.

Symbol Table.

| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| $\mathbf{v}$ | A vector | $n \times 1$ | A merchant / user / product represented as features |
| $v_i$ | i-th component | scalar | One feature value (e.g., avg ticket size) |
| $n$ | Dimensionality | scalar | Number of features in the representation |
| $\|\mathbf{v}\|$ | Norm / length | scalar | "Size" or magnitude of the entity in feature space |
| $\mathbf{u}\cdot\mathbf{v}$ | Inner product | scalar | Alignment / similarity between two entities |
| $\theta$ | Angle between vectors | scalar (radians) | Inverse-similarity: 0 = identical direction, π/2 = unrelated |

### 1.1.3 Two Fintech Worked Examples

Example A — Two Paytm merchants, similarity by dot product.

Let $\mathbf{m}_1 = (250, 80, 0.02, 95, 0.6)$ — a busy tea stall: avg ticket ₹250, 80 txns/day, 2% refund rate, 95 QR scans/day, open 60% of nights. Let $\mathbf{m}_2 = (240, 75, 0.03, 90, 0.55)$ — a similar paan shop next door.

Dot product step by step:

- $250 \times 240 = 60{,}000$
- $80 \times 75 = 6{,}000$
- $0.02 \times 0.03 = 0.0006$
- $95 \times 90 = 8{,}550$
- $0.6 \times 0.55 = 0.33$
- Sum: $60{,}000 + 6{,}000 + 0.0006 + 8{,}550 + 0.33 = 74{,}550.3306$.

The number itself is hard to interpret because the features have wildly different scales (avg ticket dominates). That motivates normalization, which we will do in §1.13 (cosine similarity).

Example B — Stretching a merchant vector by promotion uplift.

Suppose a Diwali promotion lifts every behavioral coordinate by 1.4×. Then

$$1.4 \cdot \mathbf{m}_1 = (350, 112, 0.028, 133, 0.84).$$

The new norm:

$$\|1.4\,\mathbf{m}_1\| = 1.4 \cdot \|\mathbf{m}_1\| = 1.4 \cdot \sqrt{250^2 + 80^2 + 0.02^2 + 95^2 + 0.6^2}.$$

Compute under the square root: $62{,}500 + 6{,}400 + 0.0004 + 9{,}025 + 0.36 = 77{,}925.36$. So $\|\mathbf{m}_1\| \approx \sqrt{77{,}925.36} \approx 279.15$. After uplift, $\|1.4\,\mathbf{m}_1\| \approx 390.81$. Plain English: the merchant's "size" in feature space grew by exactly 40%, as expected.

### 1.1.4 Common Pitfalls

- Treating raw vectors as comparable across features of different units. ₹ and counts and ratios cannot be summed meaningfully without scaling.
- Confusing the dot product with similarity. A large dot product can mean "both vectors are huge" rather than "they point the same way".
- Forgetting that vectors are direction-and-magnitude, not just lists. Geometry matters; ordering of features must be consistent across all rows.
- Using Euclidean distance in very high dimensions (e.g., 768-dim embeddings) — distances concentrate; prefer cosine.

| PM Relevance |
| --- |
| - Why a PM must master this: Every merchant, customer, transaction, or query you ever score at Paytm is implicitly a vector. Decisions about which features go into that vector are product decisions, not just modeling decisions. |
| - Metrics to own: Feature coverage (% of merchants with non-null vectors), feature drift (KL between this month's distribution and last), embedding norm distribution (proxy for representation health). |
| - Strategic tradeoffs: More features  richer representation but sparser data and slower indexing; fewer features  faster but lossy. |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "At Paytm I treat each of our 30M merchants as a vector in a behavioral feature space. The dimensionality, scaling, and choice of features are product calls that determine downstream model ceiling. My Georgia Tech MSCS taught me that representation is the model — pick the wrong space and no amount of compute saves you." |

### 1.1.5 NumPy Implementation

```
import numpy as np

m1 = np.array([250.0, 80.0, 0.02, 95.0, 0.6])
m2 = np.array([240.0, 75.0, 0.03, 90.0, 0.55])

dot = np.dot(m1, m2)
norm_m1 = np.linalg.norm(m1)
norm_m2 = np.linalg.norm(m2)
uplift = 1.4 * m1
norm_uplift = np.linalg.norm(uplift)

print(f"Dot product           : {dot:.4f}")
print(f"||m1||                : {norm_m1:.4f}")
print(f"||m2||                : {norm_m2:.4f}")
print(f"||1.4 * m1||          : {norm_uplift:.4f}")
print(f"Sanity 1.4 * ||m1||   : {1.4 * norm_m1:.4f}")
```

## 1.2 Matrices

### 1.2.1 Plain-English Intuition

A matrix is a table of vectors, or equivalently a machine that transforms vectors. Two minds, same object.

- As a table: rows could be merchants, columns could be features. The whole Paytm merchant base is one big matrix.
- As a machine: multiply a matrix by an input vector and you get an output vector — rotation, projection, scaling, mixing. Every neural network layer is a matrix acting on its input.

Everyday analogy — photo compression. A grayscale photo is literally a matrix of pixel intensities. JPEG works by recognizing that most photos lie close to a low-rank approximation — a few "basis pictures" added together — and throwing away the rest. (Plain English: a photo is a grid of brightness numbers, and most photos can be described well using just a handful of templates.)

Fintech analogy — Paytm's daily settlement. Stack all merchants as rows and all settlement banks as columns; the entry $A_{ij}$ is how much money flowed from merchant $i$ to bank $j$ today. That matrix is the settlement state of the platform.

### 1.2.2 Formal Mathematical Definition

A real matrix $A \in \mathbb{R}^{m \times n}$ has $m$ rows and $n$ columns:

$$ A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}. $$

Plain English: A is a rectangle of numbers, m tall and n wide.

Transpose: $ (A^\top)_{ij} = A_{ji} $. Plain English: flip rows and columns.

Matrix-vector product for $A\in\mathbb{R}^{m\times n},\ \mathbf{x}\in\mathbb{R}^n$:

$$ (A\mathbf{x})_i = \sum_{j=1}^n a_{ij} x_j. $$

Plain English: each output entry is the dot product of that row of A with x.

Geometric reading. Writing $A = [\mathbf{a}_1\ \mathbf{a}_2\ \cdots\ \mathbf{a}_n]$ by columns,

$$ A\mathbf{x} = x_1 \mathbf{a}_1 + x_2 \mathbf{a}_2 + \cdots + x_n \mathbf{a}_n. $$

Plain English: A x is a recipe — take $x_1$ scoops of column 1, $x_2$ scoops of column 2, continuing through every column, and mix.

Symbol Table.

| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| $A$ | Matrix | $m \times n$ | Dataset (rows=entities, cols=features), or a transformation |
| $a_{ij}$ | Entry | scalar | One cell in the data table |
| $A^\top$ | Transpose | $n \times m$ | Same data, axes swapped (e.g., features-as-rows) |
| $A\mathbf{x}$ | Matrix-vector product | $m\times 1$ | Apply the transformation A to input x |
| $I_n$ | Identity matrix | $n\times n$ | "Do nothing" transformation |

### 1.2.3 Two Fintech Worked Examples

Example A — A 3-merchant × 2-feature matrix applied to a feature-weight vector.

Let

$$ A = \begin{bmatrix} 250 & 80 \\ 600 & 30 \\ 150 & 200 \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} 0.001 \\ 0.5 \end{bmatrix}. $$

Rows are merchants; columns are (avg ticket ₹, txns/day). The weight vector turns each merchant into a single "engagement score" = 0.001 · ticket + 0.5 · txns.

- Row 1: $0.001\times 250 + 0.5\times 80 = 0.25 + 40 = 40.25.$
- Row 2: $0.001\times 600 + 0.5\times 30 = 0.6 + 15 = 15.6.$
- Row 3: $0.001\times 150 + 0.5\times 200 = 0.15 + 100 = 100.15.$

So $A\mathbf{w} = (40.25, 15.6, 100.15)^\top$. Merchant 3 wins by activity volume even though its ticket size is lowest.

Example B — Settlement matrix transpose.

Let $S\in\mathbb{R}^{4\times 3}$ be 4 merchants × 3 banks, with row totals = merchant settlement, column totals = bank inflow.

$$ S = \begin{bmatrix} 1000 & 0 & 200 \\ 500 & 500 & 0 \\ 0 & 800 & 300 \\ 700 & 100 & 0 \end{bmatrix}\,(\text{₹ thousands}). $$

- Total per merchant (row sums): 1200, 1000, 1100, 800.
- Total per bank (column sums = row sums of $S^\top$): $1000+500+0+700 = 2200$; $0+500+800+100 = 1400$; $200+0+300+0 = 500$. Grand total ₹4,100k. Transposing converts a merchant-centric view into a bank-centric one without losing a paisa.

### 1.2.4 Common Pitfalls

- Index order. $A_{ij}$ is row $i$, column $j$. Reversing in code = silent bugs.
- Shape mismatches in multiplication. $A\in\mathbb{R}^{m\times n}$ times $B\in\mathbb{R}^{p\times q}$ requires $n=p$.
- Forgetting that matrix multiplication is non-commutative. $AB \ne BA$ generally.
- Confusing the data matrix with the transformation matrix. Same notation, different semantics — be explicit in design docs.

| PM Relevance |
| --- |
| - Why a PM must master this: Every "table" in your PRD is secretly a matrix, and every "transformation" in your pipeline is a matrix multiply. Roadmapping at the matrix level lets you reason about latency, memory, and parallelism without leaving English. |
| - Metrics to own: Row coverage, column sparsity, transformation latency (ms per Ax), memory footprint per merchant matrix in production. |
| - Strategic tradeoffs: Dense vs sparse matrices; precision (fp32 vs fp16); store-and-multiply vs streaming. |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "Paytm's settlement engine, fraud scoring, and merchant clustering all share one mathematical object: a matrix. My Georgia Tech MSCS trained me to ask, what is the shape, what does each axis mean, and what is the cost of multiplying it — those three questions surface 80% of system risks before a single line of code." |

### 1.2.5 NumPy Implementation

```
import numpy as np

A = np.array([[250.0, 80.0],
              [600.0, 30.0],
              [150.0, 200.0]])
w = np.array([0.001, 0.5])
scores = A @ w

S = np.array([[1000, 0, 200],
              [500, 500, 0],
              [0, 800, 300],
              [700, 100, 0]], dtype=float)
merchant_totals = S.sum(axis=1)
bank_totals = S.sum(axis=0)

print("Engagement scores per merchant:", scores)
print("Row sums (merchants):", merchant_totals)
print("Col sums (banks)    :", bank_totals)
print("Grand total via S.sum():", S.sum())
```

## 1.3 Matrix Multiplication Mechanics

### 1.3.1 Plain-English Intuition

If matrices are machines, matrix multiplication is chaining two machines into one. $C = AB$ means "first apply B, then apply A". The composition is itself a matrix — that's the magical closure property that makes deep learning work: any stack of linear layers collapses (mathematically) into a single matrix, which is why we need non-linearities between them.

Everyday analogy — laundry pipeline. Washer is matrix B (turns dirty clothes into wet clothes); dryer is matrix A (turns wet clothes into dry clothes). $AB$ is the combined "dirty  dry" machine. Order matters: $BA$ (dryer then washer) is a disaster.

Fintech analogy — KYC pipeline. Matrix B maps a raw merchant document image to extracted fields; matrix A maps fields to a risk score. $AB$ is your end-to-end OCR-to-risk model. You cannot put the risk scorer before OCR.

### 1.3.2 Formal Mathematical Definition

For $A\in\mathbb{R}^{m\times n}$ and $B\in\mathbb{R}^{n\times p}$, the product $C = AB \in \mathbb{R}^{m\times p}$ is defined by

$$ C_{ij} = \sum_{k=1}^n A_{ik} B_{kj}. $$

Plain English: entry (i,j) of the product is the dot product of row i of A with column j of B.

Equivalent views.

1. Row view: row $i$ of $C$ = row $i$ of $A$ times $B$.
2. Column view: column $j$ of $C$ = $A$ times column $j$ of $B$.
3. Outer-product view: $AB = \sum_{k=1}^n A_{:,k}\, B_{k,:}$ — a sum of rank-1 matrices. This is the view that powers SVD.

Properties (with one-line proofs).

- Associativity: $ (AB)C = A(BC) $. Proof: both equal $\sum_{k,\ell} A_{ik}B_{k\ell}C_{\ell j}$ by reindexing.
- Distributivity: $ A(B+C) = AB + AC $. Proof: linearity of the sum.
- Non-commutativity: in general $AB \ne BA$; even shapes may differ.
- Transpose: $ (AB)^\top = B^\top A^\top $. Proof: $ ((AB)^\top)_{ij} = (AB)_{ji} = \sum_k A_{jk}B_{ki} = \sum_k (B^\top)_{ik}(A^\top)_{kj} $.

Computational cost. A naive $m\times n$ by $n\times p$ multiplication takes $O(mnp)$ flops. For square $n\times n$ matrices, that is $O(n^3)$. Strassen's algorithm achieves $O(n^{2.807})$; state-of-the-art theoretical bound is $\approx O(n^{2.371})$.

Symbol Table.

| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| $A$ | Left operand | $m\times n$ | Second-stage transformation |
| $B$ | Right operand | $n\times p$ | First-stage transformation |
| $C = AB$ | Product | $m\times p$ | Composed pipeline |
| $n$ | Shared "contracting" dimension | scalar | Bottleneck of the pipeline |
| flops | $2mnp$ approx. | scalar | Compute cost — directly drives GPU bill |

### 1.3.3 Two Fintech Worked Examples

Example A — Compose feature-engineering with scoring.

Stage 1 (B): take raw (ticket, txns) and produce engineered features (log-ticket, txns, ticket·txns / 1000). Approximate "log" linearly around the centroid for this toy:

$$ B = \begin{bmatrix} 0.004 & 0 \\ 0 & 1 \\ 0.05 & 0.05 \end{bmatrix}. $$

(B is 3×2: 3 output features from 2 inputs.)

Stage 2 (A): combine into a single risk score with weights:

$$ A = \begin{bmatrix} 0.3 & 0.5 & 0.2 \end{bmatrix}. $$

(A is 1×3.)

Composed pipeline: $C = AB \in \mathbb{R}^{1\times 2}$. Compute column by column.

- Column 1: $A \cdot B_{:,1} = 0.3(0.004) + 0.5(0) + 0.2(0.05) = 0.0012 + 0 + 0.01 = 0.0112.$
- Column 2: $A \cdot B_{:,2} = 0.3(0) + 0.5(1) + 0.2(0.05) = 0 + 0.5 + 0.01 = 0.51.$

So $C = [0.0112,\ 0.51]$. The composed weight on (ticket, txns) is now explicit. Apply to merchant 1 (ticket=250, txns=80):

$$ \text{score} = 0.0112\times 250 + 0.51\times 80 = 2.8 + 40.8 = 43.6. $$

Example B — Verify non-commutativity with a tiny 2×2.

Let

$$ P = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix},\ Q = \begin{bmatrix} 1 & 0 \\ 3 & 1 \end{bmatrix}. $$

- $PQ$: row1·col1 = 1·1+2·3 = 7; row1·col2 = 1·0+2·1 = 2; row2·col1 = 0·1+1·3 = 3; row2·col2 = 0·0+1·1 = 1. So $PQ = \begin{bmatrix} 7 & 2 \\ 3 & 1 \end{bmatrix}$.
- $QP$: row1·col1 = 1·1+0·0 = 1; row1·col2 = 1·2+0·1 = 2; row2·col1 = 3·1+1·0 = 3; row2·col2 = 3·2+1·1 = 7. So $QP = \begin{bmatrix} 1 & 2 \\ 3 & 7 \end{bmatrix}$.

Different. Plain English: order of pipeline stages changes the result — this is why "let's just swap the layers" is never free.

### 1.3.4 Common Pitfalls

- Shape errors hidden by broadcasting. NumPy's broadcasting can silently produce a result that compiles but is mathematically wrong.
- Assuming AB = BA. Disastrous in attention layers and graph convolutions.
- Ignoring cost. Doubling the contracting dimension doubles compute and memory bandwidth pressure.
- Batched matmul confusion. In deep learning, the leading axes are batch; only the last two participate.

| PM Relevance |
| --- |
| - Why a PM must master this: Inference latency and GPU spend are determined by matrix-multiply shapes. Negotiating these shapes with engineering = negotiating P&L. |
| - Metrics to own: flops per request, tokens/sec, GPU utilization, batch-size sweet spot. |
| - Strategic tradeoffs: Bigger matrices = better quality but worse latency/cost; quantization trades precision for throughput. |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "When my team debates an LLM upgrade at Paytm, I translate the spec into matrix shapes first. A 7B  13B move roughly doubles per-token flops; my Georgia Tech MSCS gave me the vocabulary to challenge those numbers in the same room as the ML leads." |

### 1.3.5 NumPy Implementation

```
import numpy as np

B = np.array([[0.004, 0.0],
              [0.0,   1.0],
              [0.05,  0.05]])
A = np.array([[0.3, 0.5, 0.2]])

C = A @ B  # shape (1, 2)
merchant = np.array([250.0, 80.0])
score = (C @ merchant).item()
print("Composed weight C =", C)
print("Risk score for merchant 1 =", score)

# Non-commutativity
P = np.array([[1, 2], [0, 1]])
Q = np.array([[1, 0], [3, 1]])
print("PQ =\n", P @ Q)
print("QP =\n", Q @ P)
print("Equal?", np.array_equal(P @ Q, Q @ P))
```

## 1.4 Linear Independence and Bases

### 1.4.1 Plain-English Intuition

Vectors are linearly independent if none of them is "already explainable" by the others — no one is redundant. A basis is a minimal set of independent vectors that can build everything in the space by combination.

Everyday analogy — colors on a screen. Red, Green, Blue are a basis for screen color: every pixel is a unique mix $rR + gG + bB$. Add "Yellow" and you have redundancy (Yellow = Red + Green); your basis is no longer minimal. (Plain English: three independent colors are enough to make every screen color; a fourth is wasteful.)

Fintech analogy — merchant feature redundancy. If "monthly GMV" and "monthly txn count × avg ticket" both appear as features, you have linear dependence. Your "basis" has hidden redundancy and your model will be ill-conditioned. PMs cause this all the time by stuffing dashboards into feature stores.

### 1.4.2 Formal Mathematical Definition

Vectors $\mathbf{v}_1, \ldots, \mathbf{v}_k \in \mathbb{R}^n$ are linearly independent iff

$$ \sum_{i=1}^k c_i \mathbf{v}_i = \mathbf{0} \implies c_1 = c_2 = \cdots = c_k = 0. $$

Plain English: the only way to add them up to zero is to use all-zero weights.

Otherwise they are linearly dependent: there exist coefficients not all zero giving $\mathbf{0}$. Equivalently, at least one vector is a linear combination of the others.

The span of $\{\mathbf{v}_i\}$ is the set of all linear combinations: $ \text{span}\{\mathbf{v}_i\} = \{\sum c_i \mathbf{v}_i : c_i \in \mathbb{R}\}$.

A basis of a subspace $V$ is a set that is (i) linearly independent and (ii) spans $V$. The dimension of $V$ is the size of any basis (well-defined: all bases of a given subspace have the same cardinality — a standard theorem).

Proof sketch (basis size is invariant). Suppose $B_1$ has $k$ vectors and $B_2$ has $k+1$. Express each $B_2$ vector in coordinates of $B_1$. That gives a $k\times(k+1)$ matrix; by rank considerations, its columns are dependent — contradicting $B_2$'s independence.

Test for independence. Stack vectors as columns of a matrix $M$. They are independent iff $\text{rank}(M) = k$ iff (for square $M$) $\det(M) \ne 0$.

Symbol Table.

```
| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| \(\{\mathbf{v}_i\}\) | A set of vectors | each \(n\times 1\) | A candidate feature set |
| span | All linear combinations | subspace of \(\mathbb{R}^n\) | The space of states reachable with these
features |
| basis | Independent + spanning | size = dim | A minimal, non-redundant feature set |
| dim | Cardinality of any basis | scalar | Effective number of features after de-duplication |
| rank | Dimension of column span | scalar | Effective information content of a data matrix |
```

### 1.4.3 Two Fintech Worked Examples

Example A — Detect redundant features.

Three merchant features in three sample merchants (rows):

\[ X = \begin{bmatrix} 100 & 50 & 5000 \\ 200 & 80 & 16000 \\ 150 & 60 & 9000 \end{bmatrix}, \]

where column 1 = avg ticket, column 2 = txns/day, column 3 = "GMV proxy" = col1·col2.

Check: \(100\times 50 = 5000\), \(200\times 80 = 16000\), \(150\times 60 = 9000\). Column 3 is exactly the elementwise product, but that's not linear dependence. Test for linear dependence via determinant.

- Expand along row 1: \(\det = 100(80\cdot 9000 - 16000\cdot 60) - 50(200\cdot 9000 - 16000\cdot 150) + 5000(200\cdot 60 - 80\cdot 150).\)
- Term 1: \(100(720000 - 960000) = 100(-240000) = -24{,}000{,}000.\)
- Term 2: \(-50(1{,}800{,}000 - 2{,}400{,}000) = -50(-600000) = 30{,}000{,}000.\)
- Term 3: \(5000(12000 - 12000) = 0.\)
- Sum: \(-24{,}000{,}000 + 30{,}000{,}000 + 0 = 6{,}000{,}000 \ne 0.\)

So these three columns are linearly independent despite the product relationship. Plain English: products aren't linear, so they don't create linear dependence — though they will hurt linear models via multicollinearity in scaled/centered space.

Example B — A genuinely dependent set.

\[ Y = \begin{bmatrix} 100 & 50 & 200 \\ 200 & 80 & 360 \\ 150 & 60 & 270 \end{bmatrix}, \]

where column 3 = \(2 \cdot \text{col1} + 0 \cdot \text{col2}\)? Check: row 1: \(2\times 100 + 0 \times 50 = 200\) ; row 2: \(2\times 200 + ? = 360 \Rightarrow\) need also some col 2 contribution: try \(c_1 \cdot 200 + c_2 \cdot 80 = 360\) with row 1: \(c_1 \cdot 100 + c_2 \cdot 50 = 200 \Rightarrow 2c_1 + c_2 = 4\). Two unknowns. From row 2: \(200c_1 + 80c_2 = 360 \Rightarrow 5c_1 + 2c_2 = 9\). Solve: from first, \(c_2 = 4 - 2c_1\). Substitute: \(5c_1 + 2(4-2c_1) = 9 \Rightarrow 5c_1 + 8 - 4c_1 = 9 \Rightarrow c_1 = 1, c_2 = 2\). Verify row 3: \(1\times 150 + 2 \times 60 = 150 + 120 = 270\) .

So col 3 = 1·col1 + 2·col2 exactly. The determinant must be zero:

- Term 1: \(100(80\cdot 270 - 360\cdot 60) = 100(21600 - 21600) = 0.\)
- Term 2: \(-50(200\cdot 270 - 360\cdot 150) = -50(54000 - 54000) = 0.\)
- Term 3: \(200(200\cdot 60 - 80\cdot 150) = 200(12000 - 12000) = 0.\)
- Sum: 0. Confirmed: rank(Y) = 2, not 3 — one column is redundant.

### 1.4.4 Common Pitfalls

- Confusing functional with linear redundancy. Two features can be perfectly predictive of each other without being linearly dependent (e.g., $y = x^2$).
- Numerical near-dependence. In float arithmetic, "rank-deficient" becomes "small singular value"; pick a tolerance.
- Categorical dummy variables. k one-hot columns are linearly dependent — must drop one or use the right regularizer.
- Assuming more features = more information. Adding linearly dependent features adds zero information and increases collinearity pain.

| PM Relevance |
| --- |
| - Why a PM must master this: Feature requests pile up; without an independence lens, your team drowns in redundant signals that hurt models and slow training. |
| - Metrics to own: Feature-set rank, max pairwise correlation, variance inflation factor (VIF) per feature. |
| - Strategic tradeoffs: Removing dependent features shrinks model size but risks discarding interpretability some stakeholders rely on. |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "When a Paytm team proposes a new feature, I ask: is it linearly independent of what's already in the store? My Georgia Tech MSCS taught me that a feature catalog with hidden dependencies is a model bug waiting to happen — it inflates variance and confuses ablation studies." |

### 1.4.5 NumPy Implementation

```
import numpy as np

X = np.array([[100, 50, 5000],
              [200, 80, 16000],
              [150, 60, 9000]], dtype=float)

Y = np.array([[100, 50, 200],
              [200, 80, 360],
              [150, 60, 270]], dtype=float)

print("det(X) =", np.linalg.det(X))   # ~6,000,000 -> independent
print("det(Y) =", np.linalg.det(Y))   # ~0          -> dependent
print("rank(X) =", np.linalg.matrix_rank(X))
print("rank(Y) =", np.linalg.matrix_rank(Y))

# Show the dependence in Y explicitly: col3 = 1*col1 + 2*col2
coeffs, *_ = np.linalg.lstsq(Y[:, :2], Y[:, 2], rcond=None)
print("Coefficients reconstructing col3 from col1,col2:", coeffs)
```

## 1.5 Determinants

### 1.5.1 Plain-English Intuition

The determinant of a square matrix is the signed volume scaling factor of the transformation it represents. $\det(A) = 3$ means "this transformation triples volumes"; $\det(A) = -1$ means "preserves volume, flips orientation"; $\det(A) = 0$ means "squashes space into a lower dimension — irreversibly".

Everyday analogy — a photocopier with a zoom dial. $\det$ is the area ratio between the copy and the original. A copier that flips the page upside-down has negative determinant.

Fintech analogy — currency exchange consistency. A 3-currency exchange matrix $\begin{bmatrix} \text{INRINR} & \text{USDINR} & \text{EURINR} \\ \text{INRUSD} & \cdots & \cdots \\ \cdots \end{bmatrix}$ has determinant exactly 1 only if it is internally arbitrage-free across all triples. A non-unit determinant in a closed loop = money created or destroyed = arbitrage opportunity (or bug).

### 1.5.2 Formal Mathematical Definition

For a square matrix $A \in \mathbb{R}^{n\times n}$, the determinant is defined by the Leibniz formula:

\[ \det(A) = \sum_{\sigma \in S_n} \text{sgn}(\sigma) \prod_{i=1}^n A_{i, \sigma(i)}, \]

where \(S_n\) is the set of permutations of \(\{1,\ldots,n\}\). Plain English: sum over all ways to pick exactly one entry from each row and column, multiplying picks together, with a sign that depends on the permutation's parity.

2×2 case: \(\det \begin{bmatrix} a & b \\ c & d \end{bmatrix} = ad - bc.\)

3×3 case (cofactor / Sarrus): \(\det = a(ei - fh) - b(di - fg) + c(dh - eg)\) for \( \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}\).

Key properties (each with a one-line justification).

1. \(\det(I) = 1\) (identity scales nothing).
2. \(\det(AB) = \det(A)\det(B)\) (composition of scalings).
3. \(\det(A^\top) = \det(A)\).
4. Swapping two rows flips the sign.
5. Multiplying a row by \(c\) multiplies the determinant by \(c\).
6. \(\det(A) = 0 \iff A\) is singular (not invertible) \(\iff\) rows/columns are linearly dependent.
7. \(\det(A^{-1}) = 1/\det(A)\).

Geometric interpretation (proof sketch in 2D). The unit square has vertices at \((0,0), (1,0), (0,1), (1,1)\). Applying \(A = \begin{bmatrix} a & b \\ c & d\end{bmatrix}\) maps these to \((0,0), (a,c), (b,d), (a+b, c+d)\). The signed area of this parallelogram is the cross-product \(ad - bc\).

Symbol Table.

| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| \(\det(A)\) | Determinant | scalar | Volume scaling / arbitrage indicator |
| \(\text{sgn}(\sigma)\) | Permutation sign | ±1 | Orientation preserved/reversed |
| \(A^{-1}\) | Inverse (if det ≠ 0) | \(n\times n\) | "Undo" the transformation |
| singular | det = 0 | — | Information was destroyed, cannot be reversed |

### 1.5.3 Two Fintech Worked Examples

Example A — Two-feature transformation determinant.

A 2×2 normalization matrix scales features before clustering:

\[ N = \begin{bmatrix} 1/300 & 0 \\ 0 & 1/100 \end{bmatrix}. \]

\(\det(N) = (1/300)(1/100) - 0 = 1/30000 \approx 3.33 \times 10^{-5}\). Plain English: this transformation shrinks 2-D area by 30,000×; sensible because ticket size and txns/day live on very different scales, and post-normalization they fit into a unit-ish box.

Example B — Arbitrage-free 3-currency exchange.

Spot rates: 1 INR = 0.012 USD, 1 USD = 0.92 EUR, 1 EUR = 90 INR. Build the matrix \(R\) where \(R_{ij}\) = "amount of currency \(i\) per unit of currency \(j\)". Rows/cols ordered (INR, USD, EUR):

\[ R = \begin{bmatrix} 1 & 1/0.012 & 90 \\ 0.012 & 1 & 1/0.92 \\ 1/90 & 0.92 & 1 \end{bmatrix} \approx \begin{bmatrix} 1 & 83.333 & 90 \\ 0.012 & 1 & 1.0870 \\ 0.01111 & 0.92 & 1 \end{bmatrix}. \]

For arbitrage-free triangulation we need INRUSDEURINR to multiply to 1: $0.012 \times 0.92 \times 90 = 0.99360$.
Plain English: 0.6% short of 1 — so a round-trip loses 0.6%, no arbitrage in this direction. Determinant of $R$ won't be exactly 1, but the cycle product is the right invariant. Compute it: $0.99360$. If you started with ₹1,000,000, you'd end with ₹993,600 after the loop — the 0.6% is bid-ask plus FX margin.

### 1.5.4 Common Pitfalls

- Computing det of large matrices directly. Numerically unstable; prefer log-det via LU decomposition.
- Reading sign as "bad". Negative determinant is fine — just an orientation flip.
- Tiny determinant $\ne$ zero. Float noise; check condition number, not just det.
- Assuming det = product of eigenvalues for non-diagonalizable matrices. True with multiplicity, but careful with defective matrices.

| PM Relevance |
| --- |
| - Why a PM must master this: Determinant = "did my transformation preserve information?". In risk and FX pipelines, a near-zero determinant is the math word for "you just destroyed signal". |
| - Metrics to own: Log-determinant of feature covariance, condition number, FX cycle deviation from 1. |
| - Strategic tradeoffs: Dimension-reduction pipelines deliberately drive det  0 (information loss for compression); arbitrage controllers must keep \|cycle product  1\| < threshold. |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "On the Paytm FX desk, every triangular cycle has a theoretical product of 1. The deviation is your spread. My Georgia Tech MSCS framing helps me explain why this is just the determinant of a 3×3 — same math, different domain." |

### 1.5.5 NumPy Implementation

```
import numpy as np

N = np.array([[1/300, 0.0],
              [0.0,   1/100]])
print("det(N) =", np.linalg.det(N), "expected 1/30000 =", 1/30000)

R = np.array([
    [1.0,        1/0.012,  90.0     ],
    [0.012,      1.0,      1/0.92   ],
    [1/90.0,     0.92,     1.0      ],
])
print("det(R) =", np.linalg.det(R))
cycle = 0.012 * 0.92 * 90.0
print("INR->USD->EUR->INR cycle product =", cycle, "loss% =", (1 - cycle)*100)
```

## 1.6 Trace

### 1.6.1 Plain-English Intuition

The trace is the sum of the diagonal entries of a square matrix. Tiny definition, huge utility: trace tracks total variance, expected energy, and eigenvalue sum — quantities that matter every day in PCA, attention, and covariance audits.

Everyday analogy — a daily diary's total. If a matrix's diagonal entries are "Monday's gain, Tuesday's gain, continuing through the week", the trace is the weekly total ignoring cross-effects.

Fintech analogy — diagonal of a transaction covariance matrix. That diagonal lists per-channel variance (UPI, wallet, cards). Their sum (the trace) is total platform-level transactional variance — a top-line risk number.

### 1.6.2 Formal Mathematical Definition

For $A \in \mathbb{R}^{n\times n}$:

$$ \text{tr}(A) = \sum_{i=1}^n A_{ii}. $$

Plain English: add up the diagonal.

Key properties.

1. Linearity: $\text{tr}(\alpha A + \beta B) = \alpha\,\text{tr}(A) + \beta\,\text{tr}(B).$
2. Cyclic property: $\text{tr}(ABC) = \text{tr}(BCA) = \text{tr}(CAB).$ (But not arbitrary permutations.)
3. Eigenvalue sum: $\text{tr}(A) = \sum_i \lambda_i$, counting algebraic multiplicities. Proof: characteristic polynomial $\det(\lambda I - A)$ expanded has coefficient of $\lambda^{n-1}$ equal to $-\text{tr}(A)$, which by Vieta's equals $-\sum \lambda_i$.
4. Frobenius norm: $\|A\|_F^2 = \text{tr}(A^\top A) = \sum_{i,j} A_{ij}^2.$ Hugely used as a regularizer.

Symbol Table.

| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| $\text{tr}(A)$ | Trace | scalar | Diagonal sum — "total variance" in covariance settings |
| $A_{ii}$ | Diagonal entries | scalar | Per-axis variance / self-loop weight |
| $\|A\|_F$ | Frobenius norm | scalar | Total "energy" of the matrix; common regularizer |
| cyclic | tr(ABC)=tr(BCA) | — | Lets you reorder products in proofs and code |

### 1.6.3 Two Fintech Worked Examples

Example A — Total UPI/wallet/card variance.

Daily-rupee variance covariance matrix (₹² × $10^9$):

$$ \Sigma = \begin{bmatrix} 4.0 & 1.2 & 0.7 \\ 1.2 & 2.5 & 0.4 \\ 0.7 & 0.4 & 1.0 \end{bmatrix}. $$

Trace = 4.0 + 2.5 + 1.0 = 7.5 (×$10^9$ ₹²). Plain English: total daily-rupee variance across the three channels is 7.5 billion ₹². The off-diagonals (cross-channel covariances) cancel out in the trace — useful, because total variance is a basis-independent quantity, exactly what an exec dashboard wants.

Example B — Frobenius norm of a small embedding matrix.

$E = \begin{bmatrix} 0.3 & -0.4 \\ 0.5 & 0.1 \\ -0.2 & 0.6 \end{bmatrix}$ (3 merchants × 2-D embedding).

$\|E\|_F^2 = 0.09 + 0.16 + 0.25 + 0.01 + 0.04 + 0.36 = 0.91$. So $\|E\|_F = \sqrt{0.91} \approx 0.9539$. Verify via trace: $E^\top E = \begin{bmatrix} 0.3^2+0.5^2+(-0.2)^2 & 0.3(-0.4)+0.5(0.1)+(-0.2)(0.6) \\ \cdot & (-0.4)^2+0.1^2+0.6^2\end{bmatrix}$. Diagonal: $0.09+0.25+0.04 = 0.38$; $0.16+0.01+0.36 = 0.53$. Trace = 0.91 .

### 1.6.4 Common Pitfalls

- Trace  determinant. Trace adds eigenvalues; det multiplies them. Confusing them in a design doc is embarrassing.
- Trace is invariant under similarity ($\text{tr}(P^{-1}AP) = \text{tr}(A)$) — but not arbitrary transformations.
- Frobenius regularization is not the same as nuclear norm; the former penalizes squared sums, the latter encourages low rank.

| PM Relevance |
|---|
| - Why a PM must master this: Trace is the simplest "single-number summary" of a matrix that respects geometry — perfect for dashboards over covariance or attention matrices. |

| |
|---|
| - Metrics to own: Total variance (trace of feature covariance), Frobenius regularization weight, attention-matrix trace as a "self-attention saturation" signal. |
| - Strategic tradeoffs: Tracking trace hides off-diagonal structure; pair with off-diagonal max for full picture. |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "When my Paytm dashboards need one number to summarize variability across UPI, cards, and wallet, I report the trace of the covariance — a quantity my Georgia Tech MSCS taught me is basis-invariant, which is exactly the property an executive needs." |

### 1.6.5 NumPy Implementation

```
import numpy as np

Sigma = np.array([[4.0, 1.2, 0.7],
                  [1.2, 2.5, 0.4],
                  [0.7, 0.4, 1.0]])
print("trace(Sigma) =", np.trace(Sigma))
print("sum of eigenvalues =", np.linalg.eigvalsh(Sigma).sum())

E = np.array([[0.3, -0.4],
              [0.5,  0.1],
              [-0.2, 0.6]])
print("||E||_F^2  =", (E**2).sum(), " == trace(E^T E) =", np.trace(E.T @ E))
print("||E||_F    =", np.linalg.norm(E, 'fro'))
```

## 1.7 Inverse Matrices and Systems of Linear Equations

### 1.7.1 Plain-English Intuition

The inverse $A^{-1}$ "undoes" $A$: $A A^{-1} = A^{-1} A = I$. If $A$ is your transformation, $A^{-1}$ takes outputs back to inputs. It exists iff $\det(A) \ne 0$.

A system of linear equations $A\mathbf{x} = \mathbf{b}$ asks: "what input $\mathbf{x}$ produced output $\mathbf{b}$?" If $A$ is invertible, the answer is unique: $\mathbf{x} = A^{-1}\mathbf{b}$. If $A$ is singular, there are either no solutions or infinitely many.

Everyday analogy — decoding a Caesar cipher. Encoding is "shift letters by 3"; decoding is "shift letters by 3". The decode operation is the inverse, and it exists only because shifting is reversible.

Fintech analogy — netting between Paytm merchants and banks. Given the net flows per bank, can we recover the per-merchant settlements? Yes, if the merchant-to-bank matrix is invertible. If two merchants always use the same bank in the same ratio, the matrix is singular and you can't disentangle them — a real operational problem.

### 1.7.2 Formal Mathematical Definition

$A \in \mathbb{R}^{n\times n}$ is invertible iff there exists $A^{-1} \in \mathbb{R}^{n\times n}$ with $AA^{-1} = A^{-1}A = I_n$. Equivalent conditions:

- $\det(A) \ne 0$.
- $\text{rank}(A) = n$.
- The columns of $A$ are linearly independent.
- $A\mathbf{x} = \mathbf{0}$ implies $\mathbf{x} = \mathbf{0}$.

Formula for 2×2: $ A = \begin{bmatrix} a & b \\ c & d \end{bmatrix},\quad A^{-1} = \frac{1}{ad-bc}\begin{bmatrix} d & -b \\ -c & a \end{bmatrix}.$

General inverse via cofactors: $A^{-1} = \frac{1}{\det(A)} \text{adj}(A)$, where $\text{adj}(A)$ is the adjugate (transpose of the cofactor matrix). In practice we never use this for $n > 3$; we use LU or QR.

Solving \(A\mathbf{x} = \mathbf{b}\).

- Invertible square A: unique solution \(\mathbf{x} = A^{-1}\mathbf{b}\) (but `np.linalg.solve(A, b)` is faster and more stable than computing the inverse).
- Overdetermined (\(m > n\)): no exact solution in general; least-squares \(\mathbf{x}^ = (A^\top A)^{-1} A^\top \mathbf{b}\) (the normal equations*).
- Underdetermined (\(m < n\)): infinite solutions; minimum-norm solution \(\mathbf{x}^* = A^\top (A A^\top)^{-1} \mathbf{b}\).

Derivation of normal equations. Minimize \(\|A\mathbf{x} - \mathbf{b}\|^2\). Differentiating with respect to \(\mathbf{x}\): \(\nabla = 2 A^\top (A\mathbf{x} - \mathbf{b}) = 0\), giving \(A^\top A \mathbf{x} = A^\top \mathbf{b}\).

Symbol Table.

```
| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| \(A^{-1}\) | Inverse | \(n\times n\) | The "undo" transformation |
| \(\text{adj}(A)\) | Adjugate | \(n\times n\) | Building block for inverse formula |
| \(\mathbf{x} = A^{-1}\mathbf{b}\) | Linear system solution | \(n\times 1\) | Inputs that produced observed
outputs |
| \((A^\top A)^{-1}A^\top\) | Pseudo-inverse | \(n\times m\) | Best-fit solution in least-squares sense |
| condition number \(\kappa(A)\) | \(\|A\|\|A^{-1}\|\) | scalar | Numerical sensitivity of the system |
```

### 1.7.3 Two Fintech Worked Examples

Example A — 2×2 inverse, recovering merchant inputs.

A reconciliation engine reports: total UPI revenue = ₹3,000; total wallet revenue = ₹1,200. Merchant 1 earns ₹40 per UPI and ₹10 per wallet; Merchant 2 earns ₹20 per UPI and ₹30 per wallet. How many transactions of each type did each contribute? Wait, that's the wrong shape; let's pose it properly.

Let \(x_1\) = #merchant-1 settlements, \(x_2\) = #merchant-2 settlements. Suppose the aggregated payouts per settlement form:

\[ A = \begin{bmatrix} 40 & 20 \\ 10 & 30 \end{bmatrix},\quad \mathbf{b} = \begin{bmatrix} 3000 \\ 1200 \end{bmatrix}. \]

\(\det(A) = 40\cdot 30 - 20\cdot 10 = 1200 - 200 = 1000.\)

\(A^{-1} = \frac{1}{1000}\begin{bmatrix} 30 & -20 \\ -10 & 40 \end{bmatrix} = \begin{bmatrix} 0.030 & -0.020 \\ -0.010 & 0.040 \end{bmatrix}.\)

\(\mathbf{x} = A^{-1}\mathbf{b} = \begin{bmatrix} 0.030 \cdot 3000 - 0.020 \cdot 1200 \\ -0.010 \cdot 3000 + 0.040 \cdot 1200 \end{bmatrix} = \begin{bmatrix} 90 - 24 \\ -30 + 48 \end{bmatrix} = \begin{bmatrix} 66 \\ 18 \end{bmatrix}.\)

So 66 settlements from merchant 1 and 18 from merchant 2. Verify: \(40\cdot 66 + 20\cdot 18 = 2640 + 360 = 3000\) ; \(10\cdot 66 + 30\cdot 18 = 660 + 540 = 1200\) .

Example B — Overdetermined system: fit a line via normal equations.

Three observations: (txns=80, GMV=20k), (txns=120, GMV=31k), (txns=200, GMV=49k). Fit \(\text{GMV} = a \cdot \text{txns} + b\) via least squares.

\[ A = \begin{bmatrix} 80 & 1 \\ 120 & 1 \\ 200 & 1 \end{bmatrix},\quad \mathbf{b} = \begin{bmatrix} 20000 \\ 31000 \\ 49000 \end{bmatrix}. \]

\(A^\top A = \begin{bmatrix} 80^2+120^2+200^2 & 80+120+200 \\ 80+120+200 & 3 \end{bmatrix} = \begin{bmatrix} 6400+14400+40000 & 400 \\ 400 & 3 \end{bmatrix} = \begin{bmatrix} 60800 & 400 \\ 400 & 3 \end{bmatrix}.\)

$A^\top \mathbf{b} = \begin{bmatrix} 80\cdot 20000 + 120\cdot 31000 + 200\cdot 49000 \\ 20000 + 31000 + 49000 \end{bmatrix} = \begin{bmatrix} 1{,}600{,}000 + 3{,}720{,}000 + 9{,}800{,}000 \\ 100{,}000 \end{bmatrix} = \begin{bmatrix} 15{,}120{,}000 \\ 100{,}000 \end{bmatrix}.$

$\det(A^\top A) = 60800 \cdot 3 - 400^2 = 182400 - 160000 = 22400.$

$(A^\top A)^{-1} = \frac{1}{22400}\begin{bmatrix} 3 & -400 \\ -400 & 60800 \end{bmatrix}.$

$\mathbf{x}^* = \frac{1}{22400}\begin{bmatrix} 3 \cdot 15{,}120{,}000 - 400 \cdot 100{,}000 \\ -400 \cdot 15{,}120{,}000 + 60800 \cdot 100{,}000 \end{bmatrix} = \frac{1}{22400}\begin{bmatrix} 45{,}360{,}000 - 40{,}000{,}000 \\ -6{,}048{,}000{,}000 + 6{,}080{,}000{,}000 \end{bmatrix} = \frac{1}{22400}\begin{bmatrix} 5{,}360{,}000 \\ 32{,}000{,}000 \end{bmatrix} = \begin{bmatrix} 239.286 \\ 1428.571 \end{bmatrix}.$

So GMV  239.29·txns + 1428.57. Sanity check on point 2: $239.286 \cdot 120 + 1428.571 = 28714.3 + 1428.6 = 30142.9$ vs actual 31000 — residual ₹857, very reasonable.

### 1.7.4 Common Pitfalls

- Computing the inverse explicitly when you just need to solve. Use `solve`; it's 3× faster and numerically better.
- Near-singular matrices. A condition number above $10^{10}$ means your solution is effectively noise.
- Normal equations explode the condition number (squared). Prefer QR or SVD for least squares in production.
- Forgetting the intercept column in regression — yields a forced-through-origin fit.

| PM Relevance |
| --- |
| - Why a PM must master this: Reconciliation, settlement netting, and regression are all linear systems. Knowing when one has a unique solution, none, or infinitely many is the difference between a clean dashboard and a dispute. |
| - Metrics to own: % of reconciliations solved uniquely, condition number of feature matrix, mean residual ₹ in fitted models. |
| - Strategic tradeoffs: Exact inversion vs least squares vs regularized solutions (ridge); each has different bias-variance behavior. |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "At Paytm, reconciliation is essentially solving a system Ax=b at 30M-merchant scale. My Georgia Tech MSCS taught me that the condition number of A is a product metric — if it spikes, my reconciliation team will see disputes before the model team sees error metrics." |

### 1.7.5 NumPy Implementation

```
import numpy as np

A = np.array([[40.0, 20.0],
              [10.0, 30.0]])
b = np.array([3000.0, 1200.0])
x = np.linalg.solve(A, b)
print("Reconciled counts:", x)              # [66, 18]
print("Condition number :", np.linalg.cond(A))

# Least-squares regression
X = np.array([[80.0, 1.0],
              [120.0, 1.0],
              [200.0, 1.0]])
y = np.array([20000.0, 31000.0, 49000.0])
coef, residuals, rank, sv = np.linalg.lstsq(X, y, rcond=None)
print("Slope, intercept :", coef)            # ~239.29, ~1428.57
print("Predictions       :", X @ coef)
```

## 1.8 Eigenvalues and Eigenvectors

### 1.8.1 Plain-English Intuition

An eigenvector of $A$ is a direction that the transformation $A$ does not rotate — it only stretches or flips. The amount of stretch is the eigenvalue.

$$ A\mathbf{v} = \lambda \mathbf{v}. $$

Plain English: applying A to v gives you back v itself, just scaled by λ.

Everyday analogy — a stretchy sheet of rubber. Pull the sheet diagonally. Some directions on the sheet (the principal stretch axes) don't rotate; they only get longer or shorter. Those are eigenvectors; the stretch factors are eigenvalues. Every other direction does rotate.

Fintech analogy — Paytm merchant clusters. The covariance matrix of merchant features has eigenvectors pointing along the natural axes of variation in your population — for example, "GMV scale" and "engagement frequency" might emerge as the top two eigen-directions. Eigenvalues tell you how much of total merchant diversity lies along each direction.

### 1.8.2 Formal Mathematical Definition and Characteristic Equation

$\lambda \in \mathbb{C}$ is an eigenvalue of $A \in \mathbb{R}^{n\times n}$ iff there exists $\mathbf{v} \ne \mathbf{0}$ with $A\mathbf{v} = \lambda \mathbf{v}$. Such $\mathbf{v}$ is the corresponding eigenvector.

Derivation of the characteristic equation. Rewrite:

$$ A\mathbf{v} = \lambda \mathbf{v} \iff (A - \lambda I)\mathbf{v} = \mathbf{0}. $$

A nonzero $\mathbf{v}$ in the nullspace of $(A - \lambda I)$ exists iff $(A - \lambda I)$ is singular, i.e.

$$ \det(A - \lambda I) = 0. $$

This is the characteristic polynomial, of degree $n$ in $\lambda$. Its roots are the eigenvalues.

For a 2×2 $A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$:

$$ \det\begin{bmatrix} a-\lambda & b \\ c & d-\lambda \end{bmatrix} = (a-\lambda)(d-\lambda) - bc = \lambda^2 - (a+d)\lambda + (ad-bc) = 0. $$

So $\lambda^2 - \text{tr}(A)\lambda + \det(A) = 0$, giving

$$ \lambda = \frac{\text{tr}(A) \pm \sqrt{\text{tr}(A)^2 - 4\det(A)}}{2}. $$

Plain English: for 2×2 matrices, eigenvalues are the two roots of a quadratic whose coefficients are minus the trace and the determinant.

Spectral theorem (symmetric A). If $A = A^\top$, all eigenvalues are real and there exists an orthonormal eigenbasis: $A = Q\Lambda Q^\top$ with $Q$ orthogonal and $\Lambda$ diagonal. This is the foundation of PCA.

Symbol Table.

| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| $\lambda$ | Eigenvalue | scalar | Importance / stretch factor along an axis |
| $\mathbf{v}$ | Eigenvector | $n\times 1$ | An invariant direction in feature space |
| char. poly. | $\det(A-\lambda I)$ | polynomial | Equation whose roots are eigenvalues |
| $\Lambda$ | Diagonal eigenvalue matrix | $n\times n$ | Importance ranking diagonalized |
| $Q$ | Orthogonal eigenvector matrix | $n\times n$ | Rotation to the natural axes |

### 1.8.3 Two Fintech Worked Examples

Example A — 2×2 eigen-decomposition of a tiny covariance.

$$ C = \begin{bmatrix} 5 & 2 \\ 2 & 2 \end{bmatrix}. $$

Trace = 7, det = 5·2  2·2 = 10  4 = 6. Char eqn: $\lambda^2 - 7\lambda + 6 = 0$. Factor: $(\lambda-6)(\lambda-1) = 0$. So $\lambda_1 = 6, \lambda_2 = 1$.

Eigenvector for $\lambda_1 = 6$: solve $(C - 6I)\mathbf{v} = 0$:

$$ \begin{bmatrix} -1 & 2 \\ 2 & -4 \end{bmatrix}\mathbf{v} = 0. $$

Row 1: $-v_1 + 2v_2 = 0 \Rightarrow v_1 = 2 v_2$. Pick $v_2 = 1$: $\mathbf{v}_1 = (2, 1)^\top$. Normalize: $\|\mathbf{v}_1\| = \sqrt{5}$, so $\hat{\mathbf{v}}_1 = (2/\sqrt{5}, 1/\sqrt{5}) \approx (0.8944, 0.4472)$.

Eigenvector for $\lambda_2 = 1$: $(C - I)\mathbf{v} = 0$:

$$ \begin{bmatrix} 4 & 2 \\ 2 & 1 \end{bmatrix}\mathbf{v} = 0. $$

Row 1: $4v_1 + 2v_2 = 0 \Rightarrow v_2 = -2 v_1$. Pick $v_1 = 1$: $\mathbf{v}_2 = (1, -2)$. Normalize: $\hat{\mathbf{v}}_2 = (1/\sqrt{5}, -2/\sqrt{5}) \approx (0.4472, -0.8944)$. Orthogonal to $\hat{\mathbf{v}}_1$ .

Plain English: 6/(6+1)  85.7% of variance lies along (0.894, 0.447); the remaining 14.3% along the perpendicular direction. If these were (avg ticket scaled, txns scaled), the dominant axis is "everything scaling together" — a generic "size" factor for merchants.

Example B — Power iteration on a merchant-transition matrix.

Two merchant states (active, dormant) with monthly transition matrix:

$$ T = \begin{bmatrix} 0.9 & 0.4 \\ 0.1 & 0.6 \end{bmatrix} $$

(columns sum to 1; $T_{ij}$ = prob of moving from state $j$ to state $i$). Find the steady-state distribution = eigenvector with eigenvalue 1.

Trace = 1.5, det = 0.9·0.6  0.4·0.1 = 0.54  0.04 = 0.5. Char eqn: $\lambda^2 - 1.5\lambda + 0.5 = 0$. Discriminant: $2.25 - 2.0 = 0.25$. $\lambda = (1.5 \pm 0.5)/2 = \{1.0, 0.5\}$.

For $\lambda = 1$: $(T - I)\mathbf{v} = 0$): $\begin{bmatrix} -0.1 & 0.4 \\ 0.1 & -0.4 \end{bmatrix}\mathbf{v} = 0$. Row 1: $-0.1 v_1 + 0.4 v_2 = 0 \Rightarrow v_1 = 4 v_2$. Pick $v_2 = 1, v_1 = 4$. Normalize to probabilities (sum = 1): $(4/5, 1/5) = (0.8, 0.2)$. Plain English: in the long run, 80% of merchants are active and 20% dormant — that is your steady-state churn equilibrium.

### 1.8.4 Common Pitfalls

- Eigenvectors are not unique — any nonzero scalar multiple works. Always normalize.
- Complex eigenvalues appear for non-symmetric matrices. They encode rotations (oscillations).
- Defective matrices lack a full eigenbasis; use Jordan form or SVD.
- Confusing eigenvectors of $A$ with singular vectors of a non-square matrix.

PM Relevance

| |
|---|
| - Why a PM must master this: Every dimensionality-reduction, recommender, or PageRank-style ranker is at heart "find the dominant eigenvector". This is the single most reused result in ML systems. |
| - Metrics to own: Explained variance ratio per component, condition number (= λ_max / λ_min for symmetric A), spectral gap. |
| - Strategic tradeoffs: Top-k eigenvectors = compression vs fidelity; iterative methods (power iteration, Lanczos) trade exactness for scale. |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "At 30M merchants, full eigen-decomposition is infeasible. My Georgia Tech MSCS taught me power iteration and randomized methods — I can frame which components matter without ever materializing the full matrix, which is what Paytm's infra constraints demand." |

### 1.8.5 NumPy Implementation

```
import numpy as np

C = np.array([[5.0, 2.0],
              [2.0, 2.0]])
vals, vecs = np.linalg.eigh(C)   # eigh: symmetric, returns sorted ascending
print("Eigenvalues  :", vals)     # [1, 6]
print("Eigenvectors :\n", vecs)   # columns are eigenvectors
# Reconstruct: C = Q Lambda Q^T
recon = vecs @ np.diag(vals) @ vecs.T
print("Reconstruction error:", np.linalg.norm(C - recon))

T = np.array([[0.9, 0.4],
              [0.1, 0.6]])
vals_T, vecs_T = np.linalg.eig(T)
print("T eigenvalues:", vals_T)
idx = np.argmin(np.abs(vals_T - 1.0))
ss = vecs_T[:, idx].real
ss = ss / ss.sum()
print("Steady state (active, dormant):", ss)
```

## 1.9 Principal Component Analysis (PCA) — End-to-End Derivation

### 1.9.1 Plain-English Intuition

PCA finds the axes along which your data varies most, then lets you represent each point in fewer numbers by keeping only the dominant axes. It is the canonical compression-while-preserving-structure tool.

Everyday analogy — photographing a needle. A 3-D needle has length but almost no width. Photograph it from the side and you keep nearly all its information in 2-D; photograph it head-on and you lose almost everything. PCA picks the side view automatically: the direction with the most variance.

Fintech analogy — Paytm merchant segmentation. You have hundreds of features per merchant but, empirically, 3–5 directions explain ~80% of behavioral variance ("scale", "frequency", "seasonality", "risk", "category mix"). PCA finds those directions and lets you cluster, visualize, and flag outliers in a 3–5-D space rather than a 100+-D one.

### 1.9.2 Formal Derivation (Step by Step)

Let $X \in \mathbb{R}^{m\times n}$ be a data matrix with $m$ samples (rows) and $n$ features (columns). Assume columns are mean-centered: $\sum_i X_{ij} = 0$ for each $j$. (If not, subtract column means first.)

Step 1 — Define the objective. Find a unit vector $\mathbf{w}\in\mathbb{R}^n$ such that the projection $\mathbf{z} = X\mathbf{w} \in \mathbb{R}^m$ has maximum variance:

$$ \max_{\|\mathbf{w}\|=1} \; \text{Var}(\mathbf{z}) = \max_{\|\mathbf{w}\|=1} \; \frac{1}{m-1}\, \mathbf{w}^\top X^\top X \, \mathbf{w}. $$

The covariance matrix is $C = \frac{1}{m-1} X^\top X \in \mathbb{R}^{n\times n}$.

Step 2 — Lagrangian. Add the constraint via multiplier $\lambda$:

$$ \mathcal{L}(\mathbf{w}, \lambda) = \mathbf{w}^\top C \mathbf{w} - \lambda(\mathbf{w}^\top \mathbf{w} - 1). $$

Set gradient w.r.t. $\mathbf{w}$ to zero:

$$ \nabla_{\mathbf{w}}\mathcal{L} = 2 C \mathbf{w} - 2\lambda \mathbf{w} = 0 \implies C\mathbf{w} = \lambda \mathbf{w}. $$

Result: $\mathbf{w}$ must be an eigenvector of $C$, and $\lambda$ the corresponding eigenvalue. The objective value at that solution: $\mathbf{w}^\top C \mathbf{w} = \mathbf{w}^\top (\lambda \mathbf{w}) = \lambda$. So the maximum variance equals the largest eigenvalue, attained at its eigenvector.

Step 3 — Subsequent components. The second principal component maximizes variance subject to being orthogonal to the first; this gives the second-largest eigenvalue/eigenvector, and so on. So the eigen-decomposition of $C$ yields the entire ordered set of principal directions.

Step 4 — Explained variance ratio.

$$ \text{EVR}_k = \frac{\lambda_k}{\sum_{j=1}^n \lambda_j} = \frac{\lambda_k}{\text{tr}(C)}. $$

Plain English: each eigenvalue's share of the trace tells you how much of the total variance that component carries.

Step 5 — Projection (dimensionality reduction). Stack the top $k$ eigenvectors as columns of $W_k \in \mathbb{R}^{n\times k}$. The compressed representation is $Z = X W_k \in \mathbb{R}^{m\times k}$. Reconstruction: $\hat X = Z W_k^\top$. Reconstruction error in Frobenius norm: $\|X - \hat X\|_F^2 = \sum_{j > k} \lambda_j (m-1)$.

Symbol Table.

| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| $X$ | Centered data matrix | $m\times n$ | All merchants × all features, mean removed |
| $C$ | Covariance matrix | $n\times n$ | Pairwise feature variability |
| $\mathbf{w}_k$ | k-th principal direction | $n\times 1$ | k-th "natural axis of variation" |
| $\lambda_k$ | k-th eigenvalue | scalar | Variance explained along axis k |
| $W_k$ | Top-k loadings | $n\times k$ | Projection matrix into low-D space |
| $Z$ | Scores | $m\times k$ | Each merchant's coordinates in PC space |
| EVR | Explained variance ratio | scalar in [0,1] | % of merchant diversity captured |

### 1.9.3 Two Fintech Worked Examples

Example A — PCA on a 3-merchant, 2-feature toy.

Raw data: merchants × (avg ticket, txns).

$$ X_{\text{raw}} = \begin{bmatrix} 200 & 80 \\ 400 & 40 \\ 300 & 60 \end{bmatrix}. $$

Column means: ticket mean = 300, txns mean = 60. Centered:

$$ X = \begin{bmatrix} -100 & 20 \\ 100 & -20 \\ 0 & 0 \end{bmatrix}. $$

Covariance $C = \frac{1}{2} X^\top X$:

$$ X^\top X = \begin{bmatrix} (-100)^2 + 100^2 + 0 & (-100)(20)+(100)(-20)+0 \\ \cdot & 20^2 + 20^2 + 0 \end{bmatrix} = \begin{bmatrix} 20000 & -4000 \\ -4000 & 800 \end{bmatrix}. $$

$C = \begin{bmatrix} 10000 & -2000 \\ -2000 & 400 \end{bmatrix}$. Trace = 10400. Det = $10000\cdot 400 - 2000^2 = 4{,}000{,}000 - 4{,}000{,}000 = 0$. So eigenvalues are $\lambda_1 = 10400, \lambda_2 = 0$ — the centered data lies on a single line.

Eigenvector for $\lambda_1 = 10400$: $(C - 10400 I)\mathbf{v} = 0$: $\begin{bmatrix} -400 & -2000 \\ -2000 & -10000\end{bmatrix}\mathbf{v} = 0$. Row 1: $-400 v_1 - 2000 v_2 = 0 \Rightarrow v_1 = -5 v_2$. Pick $v_2 = -1\Rightarrow v_1 = 5$, so $(5, -1)^\top$. Normalize: $\|(5,-1)\| = \sqrt{26}$, giving $\hat{\mathbf{w}}_1 \approx (0.9806, -0.1961)$.

Projection of each merchant onto PC1: $z_i = X_i \cdot \hat{\mathbf{w}}_1$.

- Merchant 1: $-100 \cdot 0.9806 + 20 \cdot (-0.1961) = -98.06 - 3.92 = -101.98.$
- Merchant 2: $100 \cdot 0.9806 + (-20) \cdot (-0.1961) = 98.06 + 3.92 = 101.98.$
- Merchant 3: 0.

$EVR_1 = 10400 / 10400 = 100\%$. Plain English: in this toy, all variance is captured by PC1 — perfect compression from 2-D to 1-D. PC1 mostly reflects ticket size (loading 0.98 on ticket) with a small negative tilt on txns, telling you "high-ticket merchants tend to have slightly fewer transactions".

Example B — PCA explained-variance interpretation at scale.

Suppose Paytm's covariance over 200 merchant features has eigenvalues whose first ten are $(52.3, 18.1, 9.4, 6.2, 4.5, 3.1, 2.4, 1.8, 1.5, 1.1)$ and trace = 130. Then:

- $EVR_1 = 52.3/130 = 40.2\%$
- Cumulative through 5: $(52.3+18.1+9.4+6.2+4.5)/130 = 90.5/130 = 69.6\%$.
- Cumulative through 10: $(52.3+18.1+9.4+6.2+4.5+3.1+2.4+1.8+1.5+1.1)/130 = 100.4/130 = 77.2\%$.

Plain English: 5 components capture ~70% of merchant variance, 10 capture ~77% — a strong product signal that segmentation should default to 5 axes for clustering and use 10 only for richer downstream models like risk.

### 1.9.4 Common Pitfalls

- Forgetting to center the data — your first PC will then point at the mean.
- Forgetting to scale features to comparable units — the highest-variance feature dominates artificially. Standardize (zero mean, unit variance) when units differ.
- Confusing loadings with scores. Loadings (W) are the feature weights; scores (Z) are the projected merchant coordinates.
- Treating PCA as feature selection. PC directions are linear combinations of features — they are not raw features.
- Reading sign of a PC as meaningful. Eigenvectors are defined up to sign flip.

| PM Relevance |
| --- |
| - Why a PM must master this: Segmentation, visualization, anomaly detection, and feature compression at 30M-merchant scale all start with PCA. Knowing what EVR you need to hit is a product call. |
| - Metrics to own: Cumulative EVR vs k, downstream model lift after PCA, storage saved per merchant. |
| - Strategic tradeoffs: Aggressive k = fast, cheap, blurrier; conservative k = expensive but faithful. Some teams use PCA only for visualization; others bake it into production features. |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "When Paytm leadership asks 'how many real dimensions does merchant behavior have?', the answer is a PCA scree plot. My Georgia Tech MSCS gave me the derivation — variance maximization under unit-norm constraint via Lagrange multipliers gives you the covariance eigenproblem — and the language to defend a chosen k as a product decision, not just a model knob." |

### 1.9.5 NumPy Implementation

```
import numpy as np

X_raw = np.array([[200.0, 80.0],
                  [400.0, 40.0],
                  [300.0, 60.0]])

# 1. Center
mu = X_raw.mean(axis=0)
X = X_raw - mu

# 2. Covariance
C = (X.T @ X) / (X.shape[0] - 1)
print("Cov =\n", C)

# 3. Eigendecomposition (symmetric)
vals, vecs = np.linalg.eigh(C)
order = np.argsort(vals)[::-1]
vals, vecs = vals[order], vecs[:, order]
print("Eigenvalues (sorted):", vals)
print("Loadings W (columns are PCs):\n", vecs)
evr = vals / vals.sum()
print("Explained variance ratio:", evr)

# 4. Scores
Z = X @ vecs
print("Scores (each merchant in PC space):\n", Z)

# 5. Reconstruct using top-1 component
W1 = vecs[:, :1]
X_hat = X @ W1 @ W1.T
print("Recon error (Frobenius):", np.linalg.norm(X - X_hat))
```

## 1.10 Singular Value Decomposition (SVD): $A = U\Sigma V^\top$

### 1.10.1 Plain-English Intuition

SVD is the most useful matrix factorization in all of applied math. It says: any matrix $A$ — square or rectangular, full rank or rank-deficient — can be written as

$$ A = U\Sigma V^\top, $$

where $U$ and $V$ are rotations (orthogonal matrices) and $\Sigma$ is a diagonal matrix of non-negative scaling factors called singular values. Reading the equation right-to-left, $A$ does: rotate the input, scale each axis, rotate the output. Three clean steps. That is what every matrix does, full stop.

Everyday analogy — Netflix recommendations. Netflix's user-by-movie rating matrix is gigantic and sparse. SVD finds latent "taste axes" (drama-vs-action, indie-vs-blockbuster) such that every user is a small vector of taste weights and every movie is a small vector of axis loadings; their dot product approximates the rating. (Plain English: SVD invents a few invisible "taste channels" that explain millions of ratings.)

Fintech analogy — Paytm merchant × service-category matrix. Rows = merchants, columns = service categories (food, grocery, electronics, and other operating categories). Entry = % of GMV in that category. SVD finds latent "merchant types" (e.g., kirana mix, mobile-recharge specialist, restaurant) and lets you recommend new services to each merchant by completing the low-rank approximation.

### 1.10.2 Formal Definition and Derivation

For any $A \in \mathbb{R}^{m\times n}$ of rank $r$, there exist orthogonal matrices $U \in \mathbb{R}^{m\times m}$ and $V \in \mathbb{R}^{n\times n}$ and a diagonal matrix $\Sigma \in \mathbb{R}^{m\times n}$ with non-negative entries $\sigma_1 \ge \sigma_2 \ge \cdots \ge \sigma_r > 0$, $\sigma_{r+1} = \cdots = 0$, such that

$$ A = U\Sigma V^\top. $$

The $\sigma_i$ are singular values; columns of $U$ are left singular vectors; columns of $V$ are right singular vectors.

Derivation via the symmetric matrix $A^\top A$.

Consider $A^\top A \in \mathbb{R}^{n\times n}$. It is symmetric and positive semi-definite, hence diagonalizable with non-negative eigenvalues:

$$ A^\top A = V \Lambda V^\top, \quad \Lambda = \text{diag}(\lambda_1, \ldots, \lambda_n), \quad \lambda_i \ge 0. $$

Define $\sigma_i = \sqrt{\lambda_i}$. For each $i$ with $\sigma_i > 0$, define

$$ \mathbf{u}_i = \frac{1}{\sigma_i} A \mathbf{v}_i \in \mathbb{R}^m. $$

Check that the $\mathbf{u}_i$ are orthonormal:

$$ \mathbf{u}_i^\top \mathbf{u}_j = \frac{1}{\sigma_i \sigma_j} \mathbf{v}_i^\top A^\top A \mathbf{v}_j = \frac{\lambda_j}{\sigma_i \sigma_j} \mathbf{v}_i^\top \mathbf{v}_j = \frac{\sigma_j^2}{\sigma_i \sigma_j}\delta_{ij} = \delta_{ij}. $$

Extend $\{\mathbf{u}_i\}_{i\le r}$ to an orthonormal basis of $\mathbb{R}^m$. Then

$$ A \mathbf{v}_i = \sigma_i \mathbf{u}_i, $$

which is the columnwise statement $AV = U\Sigma$, giving $A = U\Sigma V^\top$.

Equivalent forms.

- Compact (thin) SVD: $A = U_r \Sigma_r V_r^\top$ using only the first $r$ columns of $U, V$ and the $r\times r$ block of $\Sigma$.
- Outer-product: $A = \sum_{i=1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^\top$ — a sum of $r$ rank-1 matrices.

Eckart–Young theorem. The best rank-$k$ approximation of $A$ in Frobenius (and spectral) norm is the truncated SVD:

$$ A_k = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^\top, \quad \|A - A_k\|_F^2 = \sum_{i=k+1}^r \sigma_i^2. $$

This is the theoretical basis for compression (JPEG, embeddings, latent-factor recommenders).

Connection to PCA. If $X$ is mean-centered, the right singular vectors $V$ of $X$ are exactly the principal components, and singular values relate to eigenvalues of the covariance via $\lambda_i = \sigma_i^2 / (m-1)$.

Symbol Table.

| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| $A$ | Any data/transform matrix | $m\times n$ | Merchant × category, user × movie, query × doc |
| $U$ | Left singular vectors | $m\times m$ | Row-space "themes" (merchant types) |
| $\Sigma$ | Singular values (diag) | $m\times n$ | Strength of each latent theme |
| $V$ | Right singular vectors | $n\times n$ | Column-space "themes" (category mixes) |
| $\sigma_i$ | i-th singular value | scalar | Importance of latent theme i |
| $A_k$ | Rank-k approximation | $m\times n$ | Compressed / denoised version of A |

### 1.10.3 Two Fintech Worked Examples

Example A — Hand SVD of a 2×2 merchant matrix.

$$ A = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix}. $$

A is symmetric, so its SVD coincides with its eigen-decomposition (singular values = |eigenvalues|).

Char eqn: $\lambda^2 - 6\lambda + 8 = 0 \Rightarrow \lambda = (6 \pm \sqrt{36-32})/2 = (6 \pm 2)/2 = \{4, 2\}$. So $\sigma_1 = 4, \sigma_2 = 2$.

Eigenvector for 4: $(A - 4I)\mathbf{v} = 0 \Rightarrow \begin{bmatrix} -1 & 1 \\ 1 & -1\end{bmatrix}\mathbf{v} = 0 \Rightarrow v_1 = v_2$. Normalize: $\mathbf{v}_1 = (1/\sqrt 2, 1/\sqrt 2)$. Eigenvector for 2: $(A - 2I)\mathbf{v} = 0 \Rightarrow \begin{bmatrix} 1 & 1 \\ 1 & 1\end{bmatrix}\mathbf{v} = 0 \Rightarrow v_1 = -v_2$. Normalize: $\mathbf{v}_2 = (1/\sqrt 2, -1/\sqrt 2)$.

So $V = U = \frac{1}{\sqrt 2}\begin{bmatrix} 1 & 1 \\ 1 & -1\end{bmatrix},\ \Sigma = \begin{bmatrix} 4 & 0 \\ 0 & 2\end{bmatrix}.$

Verify: $U\Sigma V^\top = \frac{1}{2}\begin{bmatrix} 1 & 1 \\ 1 & -1\end{bmatrix}\begin{bmatrix} 4 & 0 \\ 0 & 2\end{bmatrix}\begin{bmatrix} 1 & 1 \\ 1 & -1\end{bmatrix} = \frac{1}{2}\begin{bmatrix} 4 & 2 \\ 4 & -2\end{bmatrix}\begin{bmatrix} 1 & 1 \\ 1 & -1\end{bmatrix} = \frac{1}{2}\begin{bmatrix} 4+2 & 4-2 \\ 4-2 & 4+2\end{bmatrix} = \begin{bmatrix} 3 & 1 \\ 1 & 3\end{bmatrix}$ .

Rank-1 approximation: $A_1 = \sigma_1 \mathbf{u}_1 \mathbf{v}_1^\top = 4 \cdot \frac{1}{2}\begin{bmatrix} 1 \\ 1\end{bmatrix}\begin{bmatrix} 1 & 1\end{bmatrix} = 2\begin{bmatrix} 1 & 1 \\ 1 & 1\end{bmatrix} = \begin{bmatrix} 2 & 2 \\ 2 & 2\end{bmatrix}.$

Reconstruction error: $\|A - A_1\|_F^2 = (3-2)^2 + (1-2)^2 + (1-2)^2 + (3-2)^2 = 4$, which equals $\sigma_2^2 = 4$ .

Example B — Latent merchant-category factors (compressed recommender).

A 4-merchant × 3-category GMV-share matrix (rows sum to 1):

$$ M = \begin{bmatrix} 0.7 & 0.2 & 0.1 \\ 0.6 & 0.3 & 0.1 \\ 0.1 & 0.2 & 0.7 \\ 0.2 & 0.3 & 0.5 \end{bmatrix} $$

(Rows: Merchants 1–4; columns: Food, Recharge, Electronics.) Compute SVD (we will show the result of numerical computation in §1.10.5; here we explain the use). Suppose singular values come out as $ \sigma = (1.345, 0.502, 0.073) $. Then EVR via SVD: $ 1.345^2 / (1.345^2 + 0.502^2 + 0.073^2) = 1.809/(1.809+0.252+0.0053) = 1.809/2.066 = 87.6\% $ for the first latent factor.

Plain English: a single hidden axis already explains 87.6% of the variability across merchants and categories — that axis is something like "food-heavy  electronics-heavy". Truncating to rank 2 captures $ (1.809 + 0.252)/2.066 = 99.7\% $, so a 2-factor approximation lets us recommend missing category exposures with near-perfect reconstruction at a fraction of storage.

### 1.10.4 Common Pitfalls

- Treating SVD as expensive to compute — randomized SVD on tall-skinny matrices is fast and parallel.
- Confusing left and right singular vectors. $U$ lives in row-space dimension, $V$ in column-space dimension.
- Forgetting sign ambiguity in singular vectors (any matched sign flip in a $(\mathbf{u}_i, \mathbf{v}_i)$ pair is valid).
- Using SVD on raw, uncentered data when you wanted PCA — center first.
- Storing the full SVD when truncated is what you need; full SVD eats the savings.

| PM Relevance |
| --- |
| - Why a PM must master this: SVD underlies modern recommenders (matrix factorization), embedding compression, denoising, and the analysis of attention matrices. The same equation $A = U\Sigma V^\top$ recurs across systems. |
| - Metrics to own: Singular-value spectrum, rank-k reconstruction error, latent-factor coverage, recommender hit-rate uplift after SVD-based candidate generation. |

- Strategic tradeoffs: Higher k = better recommendations, larger index, slower retrieval; aggressive truncation = cheap and noisy.

- 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "When Paytm's merchant-recommendation team picks rank k for matrix factorization, that's a product decision wearing a math hat. My Georgia Tech MSCS made me fluent in the Eckart–Young guarantee — truncated SVD is provably the best low-rank approximation in Frobenius norm — so I can defend k = 32 vs k = 128 against vendor pitches that ignore the spectrum entirely."

### 1.10.5 NumPy Implementation

```
import numpy as np

A = np.array([[3.0, 1.0],
              [1.0, 3.0]])
U, s, Vt = np.linalg.svd(A)
print("Singular values:", s)              # [4, 2]
print("U =\n", U)
print("V^T =\n", Vt)
recon = U @ np.diag(s) @ Vt
print("Reconstruction error:", np.linalg.norm(A - recon))

# Rank-1 approximation
A1 = s[0] * np.outer(U[:, 0], Vt[0, :])
print("A_1 =\n", A1)
print("||A - A_1||_F^2 =", np.linalg.norm(A - A1)**2, "expected sigma2^2 =", s[1]**2)

# Merchant x category latent factors
M = np.array([[0.7, 0.2, 0.1],
              [0.6, 0.3, 0.1],
              [0.1, 0.2, 0.7],
              [0.2, 0.3, 0.5]])
Um, sm, Vmt = np.linalg.svd(M, full_matrices=False)
print("Singular values:", sm)
energy = sm**2
print("Cumulative EVR:", np.cumsum(energy)/energy.sum())
# Rank-2 reconstruction
M2 = Um[:, :2] @ np.diag(sm[:2]) @ Vmt[:2, :]
print("Rank-2 recon error:", np.linalg.norm(M - M2))
```

## 1.11 High-Dimensional Vector Spaces

### 1.11.1 Plain-English Intuition

In 2-D and 3-D, intuition works. In 768-D (a typical LLM embedding) or 1536-D (OpenAI's `text-embedding-3-small`), intuition breaks. Two surprises:

1. Concentration of measure. Random points on a high-dim sphere are nearly equidistant from each other — Euclidean distance loses discriminating power.
2. Almost everything is orthogonal. Two random unit vectors in $\mathbb{R}^{768}$ have an inner product whose absolute value is, with overwhelming probability, less than $0.1$. The world is "wider than it is curved".

Everyday analogy — a city block in 3-D vs 100-D. In 3-D, you can describe a city as "downtown, midtown, suburbs". In 100-D, every point looks like an outlier from any other point's neighborhood — there is no clear "downtown". This is the curse (and gift) of dimensionality.

Fintech analogy — merchant fraud embeddings. Fraud detection embeddings live in 256–1024 dimensions. A rule like "find merchants within Euclidean distance < 5 of this fraudulent one" almost certainly returns nothing useful — distances all look similar. You must switch to angle-based similarity (cosine), and you must reason probabilistically about what neighborhoods even mean.

### 1.11.2 Formal Mathematical Concepts

Volume of a unit ball in $\mathbb{R}^n$:

$$ V_n = \frac{\pi^{n/2}}{\Gamma(n/2 + 1)}. $$

$V_1 = 2,\ V_2 = \pi \approx 3.14,\ V_3 = 4\pi/3 \approx 4.19,$ but $V_{10} \approx 2.55,\ V_{100} \approx 2.4\times 10^{-40}.$ Plain English: in high dimensions, the unit ball is essentially empty. Volume concentrates in a thin shell near the surface.

Concentration of distance. For i.i.d. random points in $[0,1]^n$, the ratio of maximum to minimum pairwise distance $\to 1$ as $n \to \infty$. Proof uses CLT applied to coordinate sums.

Random orthogonality. Let $\mathbf{u}, \mathbf{v}$ be independent uniform unit vectors in $\mathbb{R}^n$. Then $\mathbf{u}\cdot\mathbf{v}$ has mean 0 and variance $1/n$; by CLT, $\sqrt{n}\,\mathbf{u}\cdot\mathbf{v}$ is approximately standard normal. So $|\mathbf{u}\cdot\mathbf{v}|$ is typically $O(1/\sqrt{n})$ — for $n = 768$, about $0.036$.

Johnson–Lindenstrauss lemma. $N$ points in any dimension can be projected to $O(\log N / \epsilon^2)$ dimensions preserving all pairwise distances within factor $1\pm\epsilon$. This justifies aggressive dimensionality reduction in retrieval.

Symbol Table.

| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| $n$ | Embedding dimensionality | scalar | Choice of embedding model (e.g., 768, 1536) |
| $V_n$ | Unit-ball volume | scalar | "Density" of the embedding space |
| $\mathbf{u}\cdot\mathbf{v}$ | Dot product | scalar | Raw similarity (dominated by magnitudes) |
| JL distortion $\epsilon$ | Distance preservation tolerance | scalar | How aggressively we can compress without breaking ANN |
| ANN | Approximate Nearest Neighbor | algo | Production retrieval (FAISS, HNSW) |

### 1.11.3 Two Fintech Worked Examples

Example A — How orthogonal are 768-D random embeddings?

Generate two random unit vectors in 768-D. Theoretically $\text{Var}(\mathbf{u}\cdot\mathbf{v}) = 1/768$, so std $1/\sqrt{768} \approx 0.0361$. The 99% confidence interval for $\mathbf{u}\cdot\mathbf{v}$ is roughly $\pm 2.58 \cdot 0.0361 = \pm 0.0932$. Plain English: random embedded pairs almost never exceed cosine similarity of ~0.1. So when your similarity-search hits 0.85, that is enormously significant — a thousand standard deviations from random.

Example B — JL projection budget.

Suppose Paytm wants approximate retrieval over 30M merchant embeddings with 5% distortion ($\epsilon = 0.05$). JL gives target dimension

$$ k \approx \frac{8 \ln N}{\epsilon^2} = \frac{8 \ln(3\times 10^7)}{0.0025} = \frac{8 \cdot 17.22}{0.0025} \approx \frac{137.8}{0.0025} \approx 55{,}127. $$

That's higher than typical embedding dimensions — JL is a worst-case bound. In practice, FAISS/Chroma store 256–1024-D embeddings and rely on the empirical structure (clustering) of merchant data rather than worst-case JL.

### 1.11.4 Common Pitfalls

- Trusting Euclidean intuition in high dim. "Nearest" stops meaning what you think.
- Underestimating storage. 30M merchants × 1024 dims × float32 = 30M × 4KB = 120 GB just for embeddings.
- Ignoring quantization losses in production ANN (e.g., FAISS PQ codes can drop 4-bit per dim — fine for retrieval recall, terrible for downstream arithmetic).
- Mixing models. Two different embedding models do not produce comparable vectors. Never mix.

| PM Relevance |
| --- |
| - Why a PM must master this: Choosing embedding dim, similarity metric, and ANN configuration are the defining product decisions for any RAG / semantic-search / recommendation feature at Paytm. |
| - Metrics to own: Recall@k, embedding storage cost, p99 retrieval latency, drift in mean cosine similarity vs anchor set. |
| - Strategic tradeoffs: Larger dim  richer semantics, more cost; quantized index  cheap, blurrier; multiple embedding models  richer but no cross-comparability. |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "When Paytm Mall debates 768 vs 1536 dim for product retrieval, the answer hinges on concentration of measure and JL bounds. My Georgia Tech MSCS taught me the math — but more importantly, the heuristics for when the math is and isn't the bottleneck." |

### 1.11.5 NumPy Implementation

```
import numpy as np

rng = np.random.default_rng(42)
n = 768
N = 2000
# Sample N random unit vectors
X = rng.standard_normal((N, n))
X /= np.linalg.norm(X, axis=1, keepdims=True)

# Pairwise dot products of a subset
dots = X[:200] @ X[200:400].T  # 200x200 random pairs
print("Mean |dot|:", np.mean(np.abs(dots)), "expected ~", 1/np.sqrt(n))
print("Std  dot   :", np.std(dots), "expected ~", 1/np.sqrt(n))
print("Max |dot| :", np.max(np.abs(dots)))

# Unit ball volume in n dimensions
from math import gamma, pi
def unit_ball_volume(n):
    return pi**(n/2) / gamma(n/2 + 1)
for d in [1, 2, 3, 10, 100, 500]:
    print(f"V_{d} =", unit_ball_volume(d))
```

## 1.12 Cosine Similarity

### 1.12.1 Plain-English Intuition

Cosine similarity strips out magnitude and measures direction alignment alone:

$$ \cos\theta = \frac{\mathbf{u}\cdot\mathbf{v}}{\|\mathbf{u}\|\|\mathbf{v}\|}. $$

It returns 1 if vectors point identically, 0 if perpendicular, 1 if antiparallel.

Everyday analogy — comparing musical taste. Two listeners might play very different volumes of music (one a casual user, one a power user), but if their genre proportions match, cosine similarity = 1. We don't care that one plays 10× more — we care that they like the same music.

Fintech analogy — Paytm merchant clustering. A small kirana doing ₹50k/month and a large kirana doing ₹5M/month can be "the same kind of merchant" in terms of category mix and behavior pattern. Euclidean distance would flag them as far apart; cosine sees them as twins. That is exactly what you want for clustering.

### 1.12.2 Formal Definition and Properties

For nonzero $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$:

$$ \text{cos\_sim}(\mathbf{u}, \mathbf{v}) = \frac{\sum_{i=1}^n u_i v_i}{\sqrt{\sum_{i=1}^n u_i^2}\sqrt{\sum_{i=1}^n v_i^2}}. $$

Range: $[-1, 1]$, by Cauchy–Schwarz. Proof: $(\mathbf{u}\cdot\mathbf{v})^2 \le \|\mathbf{u}\|^2 \|\mathbf{v}\|^2$, so $|\cos\theta| \le 1$.

Cosine distance: $d_{\cos} = 1 - \cos\theta \in [0, 2]$. Not a true metric (fails triangle inequality on raw vectors), but angular distance $\theta = \arccos(\cos\theta)$ is.

Relationship to Euclidean distance for unit vectors. If $\|\mathbf{u}\| = \|\mathbf{v}\| = 1$:

$$ \|\mathbf{u} - \mathbf{v}\|^2 = 2 - 2\cos\theta. $$

So on the unit sphere, Euclidean and cosine rank identically — which is why production ANN indices typically L2-normalize first and then use inner product.

Symbol Table.

| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| $\cos\theta$ | Cosine similarity | scalar in [−1,1] | Direction-only similarity |
| $d_{\cos}$ | Cosine distance | scalar in [0,2] | Quasi-metric for ranking |
| $\theta$ | Angle between vectors | radians | True metric of separation |
| L2-normalized | $\hat{\mathbf{u}} = \mathbf{u}/\|\mathbf{u}\|$ | unit vector | Standard preprocessing for cosine retrieval |

### 1.12.3 Two Fintech Worked Examples

Example A — Two Paytm merchants at very different scales.

$\mathbf{m}_A = (50000, 300, 0.02, 200)$ (small but active kirana). $\mathbf{m}_B = (5000000, 30000, 0.02, 20000)$ (chain). Note $\mathbf{m}_B = 100\,\mathbf{m}_A$ exactly.

$$ \cos\theta = \frac{\mathbf{m}_A \cdot \mathbf{m}_B}{\|\mathbf{m}_A\|\|\mathbf{m}_B\|} = \frac{100\|\mathbf{m}_A\|^2}{\|\mathbf{m}_A\| \cdot 100\|\mathbf{m}_A\|} = 1. $$

Plain English: scale doesn't matter — cosine sees them as identical types. Euclidean distance would be enormous.

Example B — Compute cosine on real-looking numbers.

$\mathbf{u} = (3, 4, 0)$, $\mathbf{v} = (4, 3, 0)$.

- Dot: $3\cdot 4 + 4\cdot 3 + 0 = 12 + 12 = 24.$
- $\|\mathbf{u}\| = \sqrt{9+16} = 5$. $\|\mathbf{v}\| = \sqrt{16+9} = 5.$
- $\cos\theta = 24/25 = 0.96.$
- Angle $\theta = \arccos(0.96) \approx 0.2838 \text{ rad} \approx 16.26°.$

Very similar direction. Now try $\mathbf{w} = (-3, 4, 0)$: dot = $-9+16 = 7$; $\|\mathbf{w}\| = 5$; $\cos\theta = 7/25 = 0.28$ — much weaker similarity, even though magnitudes are unchanged.

### 1.12.4 Common Pitfalls

- Using cosine on un-normalized count features where magnitude is meaningful (e.g., raw transaction counts) can erase the signal you wanted.
- Negative cosines in non-negative spaces. If your features are TF-IDF or counts, cosine is in [0,1]; if embeddings, it's in [-1,1].
- Treating cosine as a metric. It's a similarity, not a distance; in some libraries 1  cosine is also not a metric.
- Comparing cosines across different embedding models. Distributions differ wildly.

| PM Relevance |
| --- |
| - Why a PM must master this: Cosine similarity is the default similarity metric in every vector DB you will ever spec. Retrieval quality lives or dies by it. |
| - Metrics to own: Mean cosine of relevant pairs, separation between positive and negative pair distributions (a calibration curve), recall@k under cosine. |
| - Strategic tradeoffs: Cosine ignores magnitude — gain when magnitude is noise, loss when magnitude is signal (e.g., merchant size genuinely matters). |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "At Paytm, our merchant-similarity index uses cosine because a ₹50k/month kirana and a ₹5M/month kirana are still the same kind of business. Magnitude lives in a separate feature. My Georgia Tech MSCS taught me to be explicit about which axis I want to compare — and cosine vs Euclidean is a product decision I make, not the data scientist." |

### 1.12.5 NumPy Implementation

```
import numpy as np

def cos_sim(u, v):
    return float(np.dot(u, v) / (np.linalg.norm(u) * np.linalg.norm(v)))

mA = np.array([50000.0, 300.0, 0.02, 200.0])
mB = 100.0 * mA
print("Cos(mA, mB) =", cos_sim(mA, mB))           # 1.0
print("Euclid(mA, mB) =", np.linalg.norm(mA - mB)) # huge

u = np.array([3.0, 4.0, 0.0])
v = np.array([4.0, 3.0, 0.0])
w = np.array([-3.0, 4.0, 0.0])
print("Cos(u, v) =", cos_sim(u, v))               # 0.96
print("Angle(u,v) deg =", np.degrees(np.arccos(cos_sim(u, v))))
print("Cos(u, w) =", cos_sim(u, w))               # 0.28

# Batch cosine similarity (FAISS / Chroma style)
def batch_cos_sim(X, Y):
    Xn = X / np.linalg.norm(X, axis=1, keepdims=True)
    Yn = Y / np.linalg.norm(Y, axis=1, keepdims=True)
    return Xn @ Yn.T

X = np.array([mA, u*1.0, v*1.0])
Y = np.array([mB, w*1.0])
print("Batch cos sim:\n", batch_cos_sim(X, Y))
```

## 1.13 Embeddings in Production: Chroma, FAISS, and Paytm Merchant Clustering

### 1.13.1 Plain-English Intuition

An embedding is a learned vector representation: a model maps every merchant, query, document, or transaction into the same vector space such that semantic similarity becomes geometric proximity. A vector database (Chroma, FAISS, Pinecone, pgvector) stores millions to billions of these and answers "find the nearest K vectors to this query" in milliseconds via Approximate Nearest Neighbor (ANN) indices.

Everyday analogy — Netflix again. Netflix doesn't store "what movies a user likes"; it stores a vector for each user and each movie in a shared latent space. Recommendation = nearest-movie search in that space. (Plain English: instead of remembering preferences as words, Netflix encodes them as coordinates.)

Fintech analogy — Paytm merchant clustering at 30M scale. Train (or fine-tune) a model that maps each merchant's behavior into a 256-D vector. Store in FAISS with an HNSW or IVF index. Now: (1) "find merchants like this fraudulent one" is a single ANN query; (2) clustering = k-means on the vectors; (3) cold-start onboarding = embed the new merchant's first-week behavior and seed offers from its nearest neighbors. All of fraud, growth, and product personalization sit on top of this one substrate.

### 1.13.2 Formal Framing

A vector index stores $\{(\mathbf{x}_i, \text{metadata}_i)\}_{i=1}^N$ and supports a query: given $\mathbf{q}$, return $\text{argtop}_k^{i} \cos(\mathbf{q}, \mathbf{x}_i)$. Exact search is $O(Nd)$; ANN reduces to $O(\log N \cdot d)$ (HNSW) or $O(\sqrt{N}\cdot d)$ (IVF-Flat) at the cost of small recall loss.

HNSW (Hierarchical Navigable Small World): layered graph where higher layers are sparse; search descends layers greedily. Build time $O(N \log N)$; search $O(\log N)$ with strong empirical recall.

IVF (Inverted File Index): cluster vectors into $\sqrt N$ buckets (Voronoi cells via k-means on a sample); at query time, probe `nprobe` nearest buckets. Trades recall for speed.

Product Quantization (PQ): split each $d$-dim vector into $m$ sub-vectors of $d/m$ dims; quantize each sub-vector to one of 256 centroids. Storage drops from $4d$ bytes to $m$ bytes per vector — 1024-D float32 (4 KB)  32 bytes with $m=32$. At 30M merchants: 4 KB · 30M = 120 GB  32B · 30M = 960 MB. A 125× storage reduction.

Recall@k vs latency are the two key product metrics; they are coupled via index parameters (`efSearch` in HNSW, `nprobe` in IVF). The PM owns the curve.

Symbol Table.

| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| $\mathbf{x}_i$ | i-th embedding | $d\times 1$ | One merchant in vector space |
| $N$ | Number of indexed items | scalar | Population size |
| $d$ | Embedding dim | scalar | Model choice |
| Recall@k | % of true top-k retrieved | scalar in [0,1] | Quality of ANN |
| `efSearch`/`nprobe` | Search width parameter | scalar | Latency-recall dial |
| PQ bytes per vec | After compression | scalar | Index storage cost |

### 1.13.3 Two Fintech Worked Examples

Example A — Capacity planning for 30M merchant embeddings.

Choose $d = 256$, float32. Raw size: $30 \times 10^6 \times 256 \times 4 \text{ bytes} = 3.07 \times 10^{10}$ bytes = 30.7 GB. Plus HNSW overhead (~30%)  ~40 GB. Achievable on a single 64-GB-RAM machine. If we instead chose $d = 1024$, it would be ~160 GB — multi-node, more expensive, slower. With PQ ($m = 32$) at $d = 1024$: vectors shrink to 30M × 32 B = 960 MB. PM-level decision: 1024-D + PQ is the right tradeoff when semantic richness matters; 256-D float is right when latency and simplicity matter more.

Example B — Tiny end-to-end retrieval.

Three merchant embeddings (2-D, toy):

- M1 = (0.9, 0.1) — food-heavy
- M2 = (0.85, 0.15) — also food-heavy
- M3 = (0.1, 0.95) — electronics-heavy

Query: a new merchant Q = (0.88, 0.12). Compute cosine similarity to each (L2-normalize first):

- $\|M_1\| = \sqrt{0.81+0.01} = \sqrt{0.82} \approx 0.9055$.
- $\|M_2\| = \sqrt{0.7225+0.0225} = \sqrt{0.7450} \approx 0.8631$.
- $\|M_3\| = \sqrt{0.01+0.9025} = \sqrt{0.9125} \approx 0.9552$.
- $\|Q\| = \sqrt{0.7744+0.0144} = \sqrt{0.7888} \approx 0.8881$.

Cosine similarities:

- Q·M1 = 0.88·0.9 + 0.12·0.1 = 0.792 + 0.012 = 0.804. Cos = 0.804/(0.8881·0.9055)  0.804/0.8042  0.9998.
- Q·M2 = 0.88·0.85 + 0.12·0.15 = 0.748 + 0.018 = 0.766. Cos  0.766/(0.8881·0.8631)  0.766/0.7665  0.9993.
- Q·M3 = 0.88·0.1 + 0.12·0.95 = 0.088 + 0.114 = 0.202. Cos  0.202/(0.8881·0.9552)  0.202/0.8483  0.2381.

Ranked nearest: M1 (0.9998), M2 (0.9993), M3 (0.238). Plain English: Q is unambiguously a "food-heavy" merchant; recommend the offers M1 and M2 already use.

### 1.13.4 Common Pitfalls

- Mixing embedding models across cohorts — vectors from different models are not in the same space.
- Not L2-normalizing before cosine search — silent miscalibration.
- Over-quantizing. PQ at 8 bytes/vec is often catastrophic; 32–64 B is a typical sweet spot.
- No metadata filters in ANN. Without filtering, you'll return "similar" merchants from wrong cities/countries.
- Stale embeddings. Merchant behavior drifts; embeddings need refresh cadence.

| PM Relevance |
| --- |
| - Why a PM must master this: Embedding choice + index parameters = the foundation of every modern AI product surface. Get these wrong and no model layer on top can fix it. |
| - Metrics to own: Recall@k, p50/p99 latency, storage cost per vector, refresh cadence and freshness lag, drift KL of embedding distribution. |
| - Strategic tradeoffs: Dim × precision × index type × refresh cadence — pick any three. Document the chosen point on the curve explicitly. |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "At Paytm scale, the embedding store is the central nervous system of merchant intelligence. My Georgia Tech MSCS taught me the math (cosine, JL, PQ); my PM work taught me that the operating point on the recall-latency-cost surface is a quarterly business decision, not a one-time spec." |

### 1.13.5 NumPy Implementation (Mini-FAISS)

```
import numpy as np

# A toy in-memory "vector DB"
class TinyVectorIndex:
    def __init__(self):
        self.vecs = None
        self.ids = []

    def add(self, ids, X):
        Xn = X / np.linalg.norm(X, axis=1, keepdims=True)
        self.vecs = Xn if self.vecs is None else np.vstack([self.vecs, Xn])
        self.ids.extend(ids)

    def search(self, q, k=3):
        qn = q / np.linalg.norm(q)
        sims = self.vecs @ qn
        order = np.argsort(-sims)[:k]
        return [(self.ids[i], float(sims[i])) for i in order]

idx = TinyVectorIndex()
M = np.array([[0.9, 0.1],
              [0.85, 0.15],
              [0.10, 0.95]])
idx.add(["M1", "M2", "M3"], M)

q = np.array([0.88, 0.12])
print("Top-3 neighbors:", idx.search(q, k=3))

# Storage estimate for 30M, d=256, float32
N, d = 30_000_000, 256
raw_gb = N * d * 4 / (1024**3)
print(f"Raw storage @ d={d}: {raw_gb:.2f} GB")
# With PQ m=32
pq_gb = N * 32 / (1024**3)
print(f"PQ storage   m=32 : {pq_gb:.4f} GB")
```

# Chapter 2 — Probability and Statistics: The Decision Engine of Product

If linear algebra is geometry, probability is belief. Every model you ship is a belief: "this user will repay", "this merchant is fraudulent", "this experiment moved the metric". Probability is the discipline of making those beliefs precise, updating them when data arrives, and admitting uncertainty when honest. As a Paytm PM, you make probability statements every single day — usually without realizing it. This chapter makes them rigorous.

## 2.1 Probability Spaces

### 2.1.1 Plain-English Intuition

A probability space is the mathematical "stage" on which random events happen. It has three pieces:

1. Sample space $\Omega$: all possible outcomes (e.g., all possible transactions).
2. Event space $\mathcal{F}$: the collection of subsets of $\Omega$ we are willing to talk about ("transaction was fraudulent", "transaction was above ₹500").
3. Probability measure $P$: assigns a number in $[0,1]$ to each event, with rules.

Everyday analogy — Bengaluru weather. Sample space = {clear, cloudy, drizzle, downpour, fog}. Event "it rains" = {drizzle, downpour}. $P(\text{rains})$ is the long-run fraction of days that are drizzle or downpour. (Plain English: the stage lists all possible weather, the event groups outcomes you care about, and the measure assigns chances.)

Fintech analogy — Paytm UPI transaction. Sample space = every possible (amount, time, merchant, customer, success/failure, fraud-label) tuple. Event "failed UPI in last 1 min above ₹10k" is a subset; $P$ of that event is the rate at which the platform sees those — a SRE metric and a product KPI at the same time.

### 2.1.2 Formal Definition

A probability space is the triple $(\Omega, \mathcal{F}, P)$ where:

- $\Omega$ is a non-empty set (sample space).
- $\mathcal{F}$ is a σ-algebra over $\Omega$: non-empty, closed under complement, closed under countable unions.
- $P : \mathcal{F} \to [0,1]$ is a measure with $P(\Omega) = 1$ and countable additivity: for disjoint $A_1, A_2, \ldots$,

$$ P\left(\bigcup_{i=1}^\infty A_i\right) = \sum_{i=1}^\infty P(A_i). $$

Plain English: probability of a union of mutually exclusive events equals the sum of their probabilities.

Basic consequences (derivations).

- Complement: $P(A^c) = 1 - P(A)$. Proof: $A \cup A^c = \Omega$, disjoint; sum to 1.
- Monotonicity: $A \subseteq B \Rightarrow P(A) \le P(B)$. Proof: $B = A \cup (B\setminus A)$, disjoint; $P(B) = P(A) + P(B\setminus A) \ge P(A)$.
- Inclusion-Exclusion: $P(A\cup B) = P(A) + P(B) - P(A\cap B)$. Proof: decompose $A\cup B$ into disjoint $A\setminus B,\ B\setminus A,\ A\cap B$; add and rearrange.
- Union bound: $P(\bigcup_i A_i) \le \sum_i P(A_i)$. Proof: by induction from inclusion-exclusion, dropping non-negative $\cap$ terms.

Symbol Table.

| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| $\Omega$ | Sample space | set | All possible outcomes (transactions, sessions) |
| $\mathcal{F}$ | Event σ-algebra | collection of subsets | Things we can "measure" — i.e., log, monitor |
| $P$ | Probability measure | function | Belief / long-run rate assignment |
| $A$ | Event | subset of $\Omega$ | Something we care about (fraud, churn, conversion) |
| $A^c$ | Complement | subset | "Did not happen" |

### 2.1.3 Two Fintech Worked Examples

Example A — UPI success/failure. Let $\Omega = \{S, F\}$ for one UPI attempt, with $P(S) = 0.992$, $P(F) = 0.008$. Then the probability of at least one failure in 5 independent attempts:

$$ P(\ge 1 \text{ fail in 5}) = 1 - P(\text{all succeed}) = 1 - 0.992^5. $$

Compute $0.992^5$:

- $0.992^2 = 0.984064$
- $0.992^3 = 0.984064 \times 0.992 = 0.976191$
- $0.992^4 = 0.976191 \times 0.992 = 0.968382$
- $0.992^5 = 0.968382 \times 0.992 = 0.960635$

So $P(\ge 1,F) = 1 - 0.960635 = 0.039365$, or 3.94%. Plain English: even with a healthy 99.2% per-attempt success rate, ~4% of 5-attempt sessions see at least one failure — a customer-experience metric the rails team must care about.

Example B — Inclusion-exclusion on two failure modes. Define events on a single transaction:

- $A$ = "PSP timeout", $P(A) = 0.005$.
- $B$ = "Bank rejection", $P(B) = 0.004$.
- $P(A \cap B) = 0.0005$ (sometimes both).

Then $P(A \cup B) = 0.005 + 0.004 - 0.0005 = 0.0085$, or 0.85%. Union-bound (without intersection info) would have overestimated: 0.005 + 0.004 = 0.9%. Plain English: the union bound is a safe upper estimate if you don't know the overlap.

### 2.1.4 Common Pitfalls

- Ignoring dependence and multiplying probabilities anyway. Independence must be argued, not assumed.
- Confusing "event" with "outcome". An event is a set of outcomes.
- Forgetting σ-additivity for countable infinities (matters in queueing models).
- Using "probability 0" as "impossible". In continuous spaces, individual points have probability 0 yet can occur.

| PM Relevance |
|---|
| - Why a PM must master this: Every operational metric (failure rate, fraud rate, conversion) is a probability. Reasoning about combinations (union, complement, intersection) is the language of incident postmortems. |
| - Metrics to own: Per-event base rates, joint failure rates (intersection probabilities), p99 failure-mode incidence. |
| - Strategic tradeoffs: Reporting marginals is simple; reporting joints reveals true risk but is expensive. |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "When Paytm's UPI SLA degrades, my first decomposition is into disjoint events — PSP timeout vs bank reject vs client error — because inclusion-exclusion tells me which intersections to investigate. My Georgia Tech MSCS gave me the formalism; my PM scars taught me to insist on the joint table, not just marginals." |

### 2.1.5 NumPy Implementation

```
import numpy as np

p_success = 0.992
n_attempts = 5
p_all_success = p_success ** n_attempts
p_at_least_one_fail = 1 - p_all_success
print(f"P(all 5 succeed) = {p_all_success:.6f}")
print(f"P(>=1 fail in 5) = {p_at_least_one_fail:.6f}")

# Simulation cross-check
rng = np.random.default_rng(0)
trials = 1_000_000
sessions = rng.random((trials, n_attempts)) > p_success  # True = fail
emp_fail = sessions.any(axis=1).mean()
print(f"Simulated P(>=1 fail) ~ {emp_fail:.6f}")

# Inclusion-exclusion
pA, pB, pAB = 0.005, 0.004, 0.0005
p_union = pA + pB - pAB
p_union_bound = pA + pB
print(f"P(A union B) exact  = {p_union}")
print(f"Union-bound upper    = {p_union_bound}")
```

## 2.2 Joint, Marginal, and Conditional Probability

### 2.2.1 Plain-English Intuition

- Joint $P(A, B)$: probability that both events happen.
- Marginal $P(A)$: probability of $A$, ignoring $B$ (sum/integrate out $B$).
- Conditional $P(A \mid B)$: probability of $A$ given that $B$ already happened.

The relationships: $P(A \mid B) = \frac{P(A, B)}{P(B)}$, and $P(A) = \sum_b P(A, B=b)$.

Everyday analogy — cricket umpire calls. Joint: P(LBW appeal AND given out). Marginal: P(given out) across all dismissal types. Conditional: P(given out | LBW appeal). The conditional is what a batsman fears in real time; the marginal is what the scorebook records.

Fintech analogy — Paytm fraud labels. Joint: P(merchant is high-risk AND transaction is chargeback). Marginal: overall chargeback rate. Conditional: P(chargeback | merchant is high-risk). The third is the one that drives risk pricing.

### 2.2.2 Formal Definitions and Derivations

For events $A, B$ with $P(B) > 0$:

$$ P(A \mid B) = \frac{P(A \cap B)}{P(B)}. $$

Plain English: divide joint by marginal.

Chain rule (proof). Rearranging:

$$ P(A, B) = P(A \mid B) P(B) = P(B \mid A) P(A). $$

Generalizes to $n$ events: $P(A_1, \ldots, A_n) = \prod_{i=1}^n P(A_i \mid A_1, \ldots, A_{i-1})$.

Law of total probability. If $\{B_i\}$ is a partition of $\Omega$,

$$ P(A) = \sum_i P(A \mid B_i) P(B_i). $$

Plain English: average the conditional probabilities, weighted by how often each condition occurs.

Independence. $A, B$ are independent iff $P(A, B) = P(A)\,P(B)$, equivalently $P(A \mid B) = P(A)$.

Symbol Table.

```
| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| $P(A,B)$ | Joint probability | scalar in [0,1] | Both events happen |
| $P(A)$ | Marginal | scalar | One event, ignoring the other |
| $P(A\mid B)$ | Conditional | scalar | Updated belief after observing B |
| partition $\{B_i\}$ | Disjoint cover of $\Omega$ | sets | Mutually exclusive scenarios |
| independence | $P(A,B)=P(A)P(B)$ | — | "Knowing B tells you nothing about A" |
```

### 2.2.3 Fintech Worked Examples

Example A — Chargeback rate among high-risk merchants. Out of 1,000,000 transactions in a month, 50,000 are from merchants flagged "high-risk." Total chargebacks: 4,000. Of those, 1,800 came from high-risk merchants.

- $P(\text{HR}) = 50{,}000 / 1{,}000{,}000 = 0.05$.
- $P(\text{CB}) = 4{,}000 / 1{,}000{,}000 = 0.004$ (marginal).
- $P(\text{CB} \cap \text{HR}) = 1{,}800 / 1{,}000{,}000 = 0.0018$ (joint).
- $P(\text{CB} \mid \text{HR}) = 0.0018 / 0.05 = 0.036$ — high-risk merchants chargeback at 3.6%, about 9× the baseline of 0.4%.
- Independence check: $P(\text{CB})P(\text{HR}) = 0.004 \times 0.05 = 0.0002 \neq 0.0018$. Not independent — the high-risk label is genuinely informative.

Example B — Total UPI failure rate via law of total probability. Three rails carry traffic: Rail A 60%, Rail B 30%, Rail C 10%. Failure rates: A 0.5%, B 1.2%, C 3.0%.

$$ P(\text{fail}) = 0.6(0.005) + 0.3(0.012) + 0.1(0.030) = 0.003 + 0.0036 + 0.003 = 0.0096. $$

Plain English: overall failure 0.96%. If you move 5 percentage points of traffic from C to A, the new rate is $0.65(0.005) + 0.3(0.012) + 0.05(0.030) = 0.00325 + 0.0036 + 0.0015 = 0.00835$, i.e., 0.84% — a 12.5% relative reduction in failures from a routing change alone.

### 2.2.4 Common Pitfalls

- Confusing $P(A\mid B)$ with $P(B\mid A)$ (prosecutor's fallacy — covered fully in Bayes).
- Assuming independence because two metrics "feel unrelated." Always check $P(A,B)$ vs $P(A)P(B)$.
- Conditioning on the future. $P(\text{churn} \mid \text{future refund})$ leaks the label — a classic data-leakage bug.
- Simpson's paradox. Conditional rates can reverse when you aggregate across subgroups; always inspect strata.

| PM Relevance |
|---|
| - Why master this: Every funnel metric, cohort comparison, and risk score is a conditional probability. Mis-stating the condition is the most common PM error in dashboards. |
| - Metrics to own: segment-conditional conversion, conditional chargeback rate, P(activation \| first-session length), P(retention \| acquisition channel). |
| - Strategic tradeoffs: narrower conditioning  higher signal but smaller sample  wider confidence intervals. Trade specificity for statistical power. |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "At Paytm I never quote a raw chargeback rate; I quote it conditional on merchant tier and rail, because aggregation hides the Simpson's-paradox traps that Georgia Tech's CS-7641 drilled into me." |

### 2.2.5 NumPy Snippet

```
import numpy as np

# Example A: joint table
N = 1_000_000
HR = 50_000
CB = 4_000
CB_and_HR = 1_800

P_HR = HR / N
P_CB = CB / N
P_joint = CB_and_HR / N
P_CB_given_HR = P_joint / P_HR
print(P_HR, P_CB, P_joint, P_CB_given_HR)  # 0.05 0.004 0.0018 0.036

# Independence check
print("indep gap:", P_joint - P_HR * P_CB)  # 0.0016

# Example B: law of total probability
mix = np.array([0.60, 0.30, 0.10])
fail = np.array([0.005, 0.012, 0.030])
print("overall fail:", mix @ fail)         # 0.0096

new_mix = np.array([0.65, 0.30, 0.05])
print("after reroute:", new_mix @ fail)    # 0.00835
```

## 2.3 Bayes' Theorem — Derivation and Fintech Use

### 2.3.1 Plain-English Intuition

Bayes' rule is the mathematics of updating beliefs when new evidence arrives. You start with a prior (what you believed before), observe evidence (data), and compute a posterior (revised belief).

Everyday analogy — weather forecasting. Prior: based on the season, 20% chance of rain today. You then look outside and see dark clouds (evidence). The likelihood P(dark clouds | rain) is high, P(dark clouds | no rain) is lower. Bayes mathematically tells you how to revise the 20% upward.

Fintech analogy — fraud alerts at Paytm. Prior: 0.1% of transactions are fraudulent. A model flags this transaction. Likelihood: P(flag | fraud) = 95%, P(flag | legit) = 2%. Most flags are still on legitimate transactions, because legit transactions vastly outnumber fraud. Bayes computes the actual P(fraud | flag) — usually a humbling number.

### 2.3.2 Formal Derivation

Start from the symmetry of the joint:

$$ P(A, B) = P(A \mid B)P(B) = P(B \mid A)P(A). $$

Divide both sides by $P(B)$:

$$ \boxed{\; P(A \mid B) = \frac{P(B \mid A) \, P(A)}{P(B)}. \;} $$

Plain English: posterior = (likelihood × prior) / evidence.

Expanding the denominator via law of total probability with partition $\{A, A^c\}$:

$$ P(B) = P(B\mid A)P(A) + P(B\mid A^c)P(A^c). $$

Odds form. Dividing the Bayes formula for $A$ by the one for $A^c$:

$$ \underbrace{\frac{P(A\mid B)}{P(A^c\mid B)}}_{\text{posterior odds}} = \underbrace{\frac{P(B\mid A)}{P(B\mid A^c)}}_{\text{likelihood ratio}} \cdot \underbrace{\frac{P(A)}{P(A^c)}}_{\text{prior odds}}. $$

Plain English: posterior odds = likelihood ratio × prior odds. The likelihood ratio (LR) is the "strength" of the evidence — independent of the base rate.

Symbol Table.

```
| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| \(P(A)\) | Prior | scalar | Belief before seeing evidence |
| \(P(B\mid A)\) | Likelihood | scalar | How well A explains B |
| \(P(A\mid B)\) | Posterior | scalar | Updated belief |
| \(P(B)\) | Evidence / marginal likelihood | scalar | Normalizing constant |
| LR | \(P(B\mid A)/P(B\mid A^c)\) | scalar | Strength of one piece of evidence |
| posterior odds | \(P(A\mid B)/P(A^c\mid B)\) | scalar | Decision-ready ratio |
```

### 2.3.3 Fintech Worked Examples

Example A — Fraud-flag posterior (the classic base-rate trap). Prior P(fraud) = 0.001. Detector: P(flag | fraud) = 0.95 (sensitivity), P(flag | legit) = 0.02 (false-positive rate).

\[ P(\text{flag}) = 0.95(0.001) + 0.02(0.999) = 0.00095 + 0.01998 = 0.02093. \] \[ P(\text{fraud} \mid \text{flag}) = \frac{0.95 \cdot 0.001}{0.02093} = \frac{0.00095}{0.02093} \approx 0.0454. \]

Plain English: even with a 95%-sensitive detector, only 4.5% of flagged transactions are actually fraud. If Paytm auto-blocks every flag, 95.5% of blocks hit honest merchants — a CX disaster. This is why thresholding and tiered review queues exist.

Example B — Sequential evidence on a merchant. Prior odds that a merchant is laundering: \(0.002 / 0.998 \approx 0.002\) (so ~1 in 500). Two evidences arrive:

- E1: night-time burst of transactions; LR1 = 8.
- E2: round-number amounts dominating; LR2 = 5.

Assuming conditional independence given the laundering label:

\[ \text{posterior odds} = 0.002 \times 8 \times 5 = 0.080. \] \[ P(\text{launder}\mid E_1, E_2) = \frac{0.080}{1+0.080} \approx 0.0741. \]

From 0.2% prior to 7.4% posterior — not yet "block," but absolutely "send to enhanced due diligence." Each subsequent LR compounds multiplicatively in log-odds space (the basis of logistic regression).

### 2.3.4 Common Pitfalls

- Prosecutor's fallacy: equating P(evidence | innocent) with P(innocent | evidence). The 4.5% example above is the antidote.
- Ignoring base rates. A 99% accurate detector on a 0.01% base rate is mostly false alarms.
- Double-counting correlated evidence. Multiplying LRs assumes conditional independence; correlated features inflate confidence.
- Hard priors. Setting P(A) = 0 means no evidence can ever update you — be careful with "impossible" categories.

| PM Relevance |
|---|
| - Why master this: every risk score, fraud alert, recommender confidence, and A/B "early-stop" decision is implicit Bayesian updating. The PM who understands posterior odds writes better thresholds than the PM who only sees AUC. |
| - Metrics to own: precision at threshold (= posterior), recall (= likelihood), false-alarm cost per flag, lift over base rate. |
| - Strategic tradeoffs: stricter threshold  higher precision, lower recall, more missed fraud; looser threshold  opposite. Bayes makes the tradeoff explicit in monetary terms. |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "When a model team brags about 95% accuracy, my first question is what's the base rate, because Bayes tells me posterior precision can still be in single |

digits — that intuition is straight from Georgia Tech's CS-7641 and ISYE-6420 Bayesian Methods."

### 2.3.5 NumPy Snippet

```
import numpy as np

# Example A: fraud flag
prior = 0.001
P_flag_given_fraud = 0.95
P_flag_given_legit = 0.02

P_flag = P_flag_given_fraud*prior + P_flag_given_legit*(1-prior)
posterior = (P_flag_given_fraud * prior) / P_flag
print("P(flag):", P_flag)               # 0.02093
print("P(fraud|flag):", posterior)       # ~0.0454

# Example B: sequential evidence in odds form
prior_odds = 0.002 / 0.998
LR1, LR2 = 8.0, 5.0
post_odds = prior_odds * LR1 * LR2
post_prob = post_odds / (1 + post_odds)
print("posterior odds:", post_odds)      # approximately 0.0801
print("posterior P:", post_prob)         # approximately 0.0742

# Log-odds form (numerically stable, basis of logistic regression)
log_post = np.log(prior_odds) + np.log(LR1) + np.log(LR2)
print("log posterior odds:", log_post)
```

## 2.4 Random Variables

### 2.4.1 Plain-English Intuition

A random variable (RV) is a function that assigns a number to each outcome in your sample space. Instead of saying "the coin landed heads," you say $X = 1$; instead of "merchant performed 47 transactions today," you say $X = 47$. RVs let us do arithmetic on randomness.

- Discrete RV: countable values (transaction counts, fraud flags 0/1).
- Continuous RV: uncountable values (transaction amount in ₹, latency in ms).

Everyday analogy — weather. Temperature is a continuous RV; "did it rain?" is a discrete (Bernoulli) RV; rainfall in mm is continuous-but-mixed (a point mass at 0 plus a continuous tail).

Fintech analogy — Paytm daily GMV. GMV per merchant per day is continuous (₹ amount); number of transactions is discrete (count); whether the merchant logged in is Bernoulli (0/1).

### 2.4.2 Formal Definitions

A random variable on $(\Omega, \mathcal{F}, P)$ is a measurable function $X:\Omega \to \mathbb{R}$.

- PMF (discrete): $p_X(x) = P(X = x)$, with $\sum_x p_X(x) = 1$.
- PDF (continuous): density $f_X(x) \geq 0$ with $\int f_X(x)\,dx = 1$; $P(a \leq X \leq b) = \int_a^b f_X(x)\,dx$.
- CDF (both): $F_X(x) = P(X \leq x)$, non-decreasing from 0 to 1.

Plain English: PMF gives probability of a single value (discrete); PDF gives a density (continuous, you integrate over intervals); CDF gives "probability of being at most x."

Transformations. If $Y = g(X)$ and $g$ is monotone differentiable, the change-of-variables formula gives $f_Y(y) = f_X(g^{-1}(y)) \left|\frac{d}{dy} g^{-1}(y)\right|$. Plain English: transformations stretch or compress the density.

Symbol Table.

```
| Symbol | Definition | Dimensions | PM Interpretation |
```

```
|---|---|---|---|
| \(X\) | Random variable | scalar function | A numeric quantity that varies stochastically |
| \(p_X(x)\) | PMF | [0,1] | Probability of exact value |
| \(f_X(x)\) | PDF | density (1/units) | Probability per unit width; integrate to get probability |
| \(F_X(x)\) | CDF | [0,1] | "At most x" probability |
| support | \(\{x: p_X(x)>0\}\) | set | Possible values the metric can take |
```

### 2.4.3 Fintech Worked Examples

Example A — Transaction count PMF (truncated geometric-like). A merchant's daily transactions \(X \in \{0,1,2,3,4,5\}\) with PMF \(p(0)=0.10, p(1)=0.20, p(2)=0.30, p(3)=0.25, p(4)=0.10, p(5)=0.05\). Sums to 1.00. Compute CDF:

```
| x | p(x) | F(x) |
|---|---|---|
| 0 | 0.10 | 0.10 |
| 1 | 0.20 | 0.30 |
| 2 | 0.30 | 0.60 |
| 3 | 0.25 | 0.85 |
| 4 | 0.10 | 0.95 |
| 5 | 0.05 | 1.00 |
```

\(P(X \geq 3) = 1 - F(2) = 1 - 0.60 = 0.40\). Forty percent of merchants do 3+ transactions on this day.

Example B — Continuous ticket size, exponential-style. Suppose Paytm ticket size \(X\) (in ₹) has PDF \(f(x) = \lambda e^{-\lambda x}\) for \(x \geq 0\), with \(\lambda = 1/500\) (mean ₹500). Then

\[ P(X > 1000) = e^{-1000/500} = e^{-2} \approx 0.1353. \] \[ P(200 \leq X \leq 800) = e^{-200/500} - e^{-800/500} = e^{-0.4} - e^{-1.6} \approx 0.6703 - 0.2019 = 0.4684. \]

About 13.5% of tickets exceed ₹1000; 46.8% fall in the ₹200–₹800 band. Used for tier-pricing decisions.

### 2.4.4 Common Pitfalls

- Treating a PDF value as a probability. \(f(x) = 0.003\) does not mean 0.3% — it's a density and can exceed 1 for narrow distributions.
- Forgetting the PMF must sum to exactly 1. Off-by-rounding errors creep in when reading from dashboards.
- Mixing units. A latency PDF in ms vs seconds differ by 1000× in density — units matter.
- Ignoring atoms in mixed distributions. Refund-amount has a giant spike at ₹0 (no refund) plus a continuous tail.

| PM Relevance |
|---|
| - Why master this: Every metric a PM owns is a random variable. Means hide tails; PMs who think in distributions, not averages, ship better products. |
| - Metrics to own: P50/P90/P99 of latency, ticket-size distribution, daily-active-transaction distribution per merchant. |
| - Strategic tradeoffs: optimize for the mean (mass market) vs. the tail (high-value customers); a product can win at one and lose at the other. |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "I never report just the mean GMV — at Paytm I report the full quantile vector, because the median merchant and the P99 merchant have different needs, and Georgia Tech's stochastic-processes lens made that automatic for me." |

### 2.4.5 NumPy Snippet

```
import numpy as np

# Example A
pmf = np.array([0.10, 0.20, 0.30, 0.25, 0.10, 0.05])
xs = np.arange(6)
assert np.isclose(pmf.sum(), 1.0)
cdf = np.cumsum(pmf)
print("CDF:", cdf)
print("P(X>=3):", 1 - cdf[2])              # 0.40

# Sample 100k merchants to validate
```

```
rng = np.random.default_rng(0)
samples = rng.choice(xs, size=100_000, p=pmf)
print("empirical P(X>=3):", (samples >= 3).mean())

# Example B: exponential PDF
lam = 1/500
print("P(X>1000):", np.exp(-lam*1000))     # 0.1353
print("P(200..800):", np.exp(-lam*200) - np.exp(-lam*800))  # 0.4684

# Simulate
xs_cont = rng.exponential(scale=1/lam, size=200_000)
print("emp P(X>1000):", (xs_cont > 1000).mean())
```

## 2.5 Expectation, Variance, Covariance, and Correlation

### 2.5.1 Plain-English Intuition

- Expectation $E[X]$: the long-run average; the "center of mass" of the distribution.
- Variance $\mathrm{Var}(X) = E[(X-E[X])^2]$: how spread-out values are.
- Covariance $\mathrm{Cov}(X,Y)$: do X and Y move together (positive) or in opposition (negative)?
- Correlation $\rho$: covariance rescaled to [-1, 1], so it's comparable across units.

Everyday analogy — cricket batting. Expectation = batting average. Variance = consistency (low variance = reliable; high variance = boom or bust). Covariance with opener's score = team chemistry. Correlation between batsman score and team total = his individual contribution magnitude.

Fintech analogy — merchant cohort metrics. Mean monthly GMV = expectation. Spread across the cohort = variance. Co-movement of GMV with customer-acquisition spend = covariance. Standardized = correlation. PMs use correlation to detect levers and avoid colinear vanity-metric pairs.

### 2.5.2 Formal Definitions and Properties

Discrete: $E[X] = \sum_x x\, p(x)$. Continuous: $E[X] = \int x f(x)\,dx$.

$$ \mathrm{Var}(X) = E[(X-\mu)^2] = E[X^2] - (E[X])^2. $$

Proof of variance shortcut: $E[(X-\mu)^2] = E[X^2 - 2\mu X + \mu^2] = E[X^2] - 2\mu E[X] + \mu^2 = E[X^2] - \mu^2$.

Linearity of expectation: $E[aX + bY + c] = aE[X] + bE[Y] + c$, always (no independence needed).

Variance of a sum: $\mathrm{Var}(X+Y) = \mathrm{Var}(X) + \mathrm{Var}(Y) + 2\mathrm{Cov}(X,Y)$. If independent, the cross term vanishes.

Covariance: $\mathrm{Cov}(X,Y) = E[(X-\mu_X)(Y-\mu_Y)] = E[XY] - E[X]E[Y].$

Correlation: $\rho_{XY} = \frac{\mathrm{Cov}(X,Y)}{\sigma_X \sigma_Y} \in [-1, 1].$ Bounded by Cauchy–Schwarz: $\mathrm{Cov}(X,Y)^2 \leq \mathrm{Var}(X)\mathrm{Var}(Y)$.

Symbol Table.

| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| $\mu = E[X]$ | Mean | units of X | Long-run average outcome |
| $\sigma^2 = \mathrm{Var}(X)$ | Variance | units² | Risk / spread |
| $\sigma$ | Std deviation | units | Typical deviation, plottable |
| $\mathrm{Cov}(X,Y)$ | Covariance | units of X × units of Y | Raw co-movement |
| $\rho$ | Correlation | unitless [-1,1] | Strength + direction of linear relation |

### 2.5.3 Fintech Worked Examples

Example A — Expected GMV and variance for a merchant tier. Five merchants, daily GMV (₹): 8000, 12000, 10000, 15000, 5000.

- \(\mu = (8000+12000+10000+15000+5000)/5 = 50000/5 = 10000.\)
- Deviations: -2000, 2000, 0, 5000, -5000.
- Squared deviations: 4M, 4M, 0, 25M, 25M; sum = 58M.
- Variance (population): \(58{,}000{,}000 / 5 = 11{,}600{,}000.\) Std dev \(\approx 3{,}406.\)
- Sample variance (divide by n1=4): \(58M/4 = 14{,}500{,}000\); std  3,808.

Example B — Correlation between GMV and active-customer count. Five days:

```
| Day | GMV (₹L) X | Active customers Y |
|---|---|---|
| Mon | 10 | 800 |
| Tue | 12 | 950 |
| Wed | 15 | 1100 |
| Thu | 11 | 850 |
| Fri | 17 | 1300 |
```

Means: \(\bar X = 13, \bar Y = 1000\). Deviations: X: 3,1,2,2,4. Y: 200,50,100,150,300.

Cross products: (3)(200)=600, (1)(50)=50, (2)(100)=200, (2)(150)=300, (4)(300)=1200. Sum = 2350.

Sample covariance: \(2350 / 4 = 587.5\) (units ₹L·customer).

\(\sum (X-\bar X)^2 = 9+1+4+4+16 = 34\); sample var X = 34/4 = 8.5; \(\sigma_X \approx 2.915\).

\(\sum (Y-\bar Y)^2 = 40000+2500+10000+22500+90000 = 165000\); sample var Y = 41250; \(\sigma_Y \approx 203.10\).

\(\rho = 587.5 / (2.915 \times 203.10) = 587.5 / 592.04 \approx 0.992.\) Near-perfect linear relation — GMV and active customers move in lockstep, so they're not independent levers; lifting one doesn't decouple from the other.

### 2.5.4 Common Pitfalls

- Correlation  causation. Cricket-on-TV correlates with cola sales; neither causes the other.
- Confusing zero correlation with independence. \(\rho = 0\) only rules out linear relations; \(Y = X^2\) with symmetric X has \(\rho=0\) but is fully determined.
- Outliers wreck Pearson \(\rho\). Use Spearman (rank-based) for monotone-but-not-linear relations.
- Variance scales with units squared. Always report std dev (same units) in dashboards.

| PM Relevance |
| --- |
| - Why master this: Expectations drive forecasts; variances drive risk; correlations drive feature selection. The PM who confuses them ships fragile experiments. |
| - Metrics to own: mean and std of GMV per cohort, weekly volatility, correlation matrix of north-star with input metrics. |
| - Strategic tradeoffs: chase mean lift (growth) vs reduce variance (reliability). A 5% GMV lift with 3× variance is worse than 3% lift with stable variance — finance team will tell you. |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "I never present a single number without its dispersion — at Paytm I quote GMV ± std and the top correlation with input metrics, because Georgia Tech taught me that decisions on point estimates without covariance are decisions in the dark." |

### 2.5.5 NumPy Snippet

```
import numpy as np
```

```
# Example A
gmv = np.array([8000, 12000, 10000, 15000, 5000], dtype=float)
print("mean:", gmv.mean())                      # 10000
print("pop var:", gmv.var())                    # 11_600_000
print("sample var:", gmv.var(ddof=1))           # 14_500_000
print("sample std:", gmv.std(ddof=1))           # ~3807.9

# Example B
X = np.array([10, 12, 15, 11, 17], dtype=float)
Y = np.array([800, 950, 1100, 850, 1300], dtype=float)
cov_mat = np.cov(X, Y, ddof=1)   # 2x2 sample covariance matrix
corr_mat = np.corrcoef(X, Y)
print("cov(X,Y):", cov_mat[0,1])                # 587.5
print("rho:", corr_mat[0,1])                    # ~0.9923
```

## 2.6 Key Distributions: Normal, Binomial, Poisson, Power Law, Multinomial

### 2.6.1 Plain-English Intuition

Each distribution is a "shape" of randomness suited to a class of phenomena:

- Normal (Gaussian): sums and averages of many small influences  bell curve. Daily aggregate GMV across millions of merchants.
- Binomial: number of "successes" in n independent yes/no trials. Number of users who completed KYC out of 1000 invitees.
- Poisson: count of rare events in a fixed interval. Number of UPI failures per minute on a healthy rail.
- Power Law: few giants, long tail of small players. Merchant GMV — top 1% generates the majority.
- Multinomial: generalizes Binomial to k categories. Distribution of payments across UPI / Wallet / Card / NetBanking.

Cricket umpire analogy. Sum of many small line-call errors per match  Normal. Number of out calls in 10 LBW appeals  Binomial. Number of no-balls per over  Poisson. Player Test runs across history  Power law (Tendulkar is the giant tail). Distribution of dismissal types  Multinomial.

Weather analogy. Daily temperature  Normal. Rain/no-rain over 30 days  Binomial. Lightning strikes per square km per hour  Poisson. Hurricane intensity  Power law. Cloud-cover category (clear/partly/cloudy/overcast)  Multinomial.

### 2.6.2 Formal Definitions

Normal $\mathcal{N}(\mu, \sigma^2)$:

$$ f(x) = \frac{1}{\sqrt{2\pi}\,\sigma} \exp\!\left(-\frac{(x-\mu)^2}{2\sigma^2}\right), \quad x \in \mathbb{R}. $$

Mean $\mu$, variance $\sigma^2$. Standardize via $Z = (X-\mu)/\sigma \sim \mathcal{N}(0,1)$. 68/95/99.7 rule: $\mu \pm 1/2/3\sigma$ contains 68%, 95%, 99.7%.

Binomial $\text{Bin}(n, p)$:

$$ P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}, \quad k = 0,1,\ldots,n. $$

Mean $np$, variance $np(1-p)$.

Poisson $\text{Pois}(\lambda)$:

$$ P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}, \quad k = 0,1,2,\ldots $$

Mean = variance = $\lambda$. Limit of Binomial with $n \to \infty, p \to 0, np = \lambda$.

Power Law (Pareto):

\[ f(x) = \alpha x_{\min}^\alpha x^{-(\alpha+1)}, \quad x \geq x_{\min}. \]

Tail \(P(X > x) = (x_{\min}/x)^\alpha\). For \(\alpha \leq 2\), variance is infinite; for \(\alpha \leq 1\), even the mean diverges.

Multinomial \(\text{Mult}(n, \mathbf{p})\) with \(\mathbf{p} = (p_1, \ldots, p_k)\), \(\sum p_i = 1\):

\[ P(X_1=n_1,\ldots,X_k=n_k) = \frac{n!}{n_1!\cdots n_k!} \prod_i p_i^{n_i}, \quad \sum n_i = n. \]

Marginal of each \(X_i\) is \(\text{Bin}(n, p_i)\).

Symbol Table.

| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| \(\mu, \sigma\) | Normal mean / std | units of X | Center and spread |
| \(n, p\) | Binomial trials, success prob | count, [0,1] | Sample size & conversion rate |
| \(\lambda\) | Poisson rate | events per unit | Expected count per interval |
| \(\alpha\) | Power-law exponent | unitless | Heaviness of tail (lower = heavier) |
| \(p_i\) | Multinomial category prob | [0,1] | Share of traffic per channel |

### 2.6.3 Fintech Worked Examples

Example A — Binomial KYC drop-off. 1000 new sign-ups, true KYC completion rate p = 0.62. Expected completions: \(np = 620\). Std: \(\sqrt{1000 \cdot 0.62 \cdot 0.38} = \sqrt{235.6} \approx 15.35\). With Normal approximation, P(at least 650) \(P\!\left(Z \geq \frac{649.5 - 620}{15.35}\right) = P(Z \geq 1.922) \approx 0.0273\). So seeing 650+ completions in a 1000-cohort is a ~2.7% tail event — worth investigating if marketing claims a "650 baseline."

Example B — Poisson UPI failures. Average failure rate \(\lambda = 4\) failures per minute on Rail A. P(0 failures in a minute) = \(e^{-4} \approx 0.0183\). P(10 in a minute) = \(1 - \sum_{k=0}^{9} \frac{4^k e^{-4}}{k!}\). Computing partial sums of the PMF gives F(9)  0.99187, so P(10)  0.00813 — an alert threshold for SRE. If you observe 12 in one minute, that's a ~0.0009 tail event, almost certainly a real incident.

(Multinomial check) Traffic mix \(\mathbf{p} = (0.55, 0.20, 0.15, 0.10)\) for (UPI, Wallet, Card, NetBanking). Out of n=10,000 transactions, expected counts are (5500, 2000, 1500, 1000); per-channel std \(\sqrt{n p_i (1-p_i)}\): UPI \(\sqrt{10000\cdot0.55\cdot0.45} = \sqrt{2475} \approx 49.75\), so 5500 ± 50 is the noise band.

### 2.6.4 Common Pitfalls

- Assuming Normal when the data is heavy-tailed. GMV per merchant is power law; reporting "mean ± std" is misleading and the std may be infinite.
- Using mean and std for a power law. Always report quantiles and tail exponent.
- Poisson with overdispersion. Real-world counts often have variance > mean (use Negative Binomial).
- Multinomial independence illusion. The \(X_i\) are negatively correlated — if one channel grows, others must shrink at fixed n.

| PM Relevance |
|---|
| - Why master this: the right distribution is the single biggest determinant of whether your A/B test, forecast, or anomaly detector works. |
| - Metrics to own: distributional fit of each KPI (normal-ish vs power-law), Poisson rates for incidents per service, channel share via multinomial. |
| - Strategic tradeoffs: Normal-based forecasts are tractable but understate tails; power-law forecasts capture tails but resist closed-form ROI math. |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "On Paytm's 30M-merchant base, GMV follows a power law — so I never quote 'average merchant' GMV. I quote the head, the body, and the tail separately. |

Georgia Tech's stochastic-processes coursework wired that distinction into me."

### 2.6.5 NumPy Snippet

```
import numpy as np
from math import erf, sqrt, exp, factorial

rng = np.random.default_rng(42)

# Binomial example
n, p = 1000, 0.62
mu, sd = n*p, np.sqrt(n*p*(1-p))
# Normal approx, continuity correction
z = (649.5 - mu) / sd
P_tail = 0.5 * (1 - erf(z / sqrt(2)))
print("P(>=650) normal-approx:", P_tail)         # ~0.0273

emp = rng.binomial(n, p, size=200_000)
print("emp P(>=650):", (emp >= 650).mean())

# Poisson example
lam = 4
def pois_pmf(k, lam): return (lam**k) * np.exp(-lam) / factorial(k)
print("P(0):", pois_pmf(0, lam))                 # 0.0183
F9 = sum(pois_pmf(k, lam) for k in range(10))
print("P(>=10):", 1 - F9)                         # ~0.00813

# Multinomial
probs = np.array([0.55, 0.20, 0.15, 0.10])
draw = rng.multinomial(10_000, probs)
print("draw:", draw, "expected:", 10_000*probs)
print("UPI std:", np.sqrt(10_000*0.55*0.45))      # ~49.75

# Power-law sanity: tail probability for alpha=1.5, xmin=1000
alpha, xmin = 1.5, 1000
def tail(x): return (xmin/x)**alpha
print("P(GMV>10000):", tail(10000))               # ~0.0316
```

## 2.7 Central Limit Theorem (CLT)

### 2.7.1 Plain-English Intuition

Take any distribution with finite mean and variance. Sample n values, average them. Repeat. The distribution of those averages becomes approximately Normal as n grows — regardless of the original shape. This is the single most important theorem in applied statistics: it justifies almost every confidence interval and A/B test.

Cricket umpire analogy. Individual line-call errors per ball can be skewed; but the average error per match (over hundreds of calls) is essentially Normal — which is why match-level umpire performance scores work.

Fintech analogy — Paytm daily metrics. Individual transaction amounts are wildly skewed (power-law), but daily average ticket size per merchant (averaged over ~50 transactions) is Normal enough for control charts. The CLT is why dashboards work at all.

### 2.7.2 Formal Statement and Sketch of Proof

Let $X_1, \ldots, X_n$ be i.i.d. with $E[X_i] = \mu$ and $\mathrm{Var}(X_i) = \sigma^2 < \infty$. Then

$$ \frac{\bar X_n - \mu}{\sigma / \sqrt{n}} \xrightarrow{d} \mathcal{N}(0, 1), \quad \text{as } n \to \infty. $$

Equivalently $\bar X_n \approx \mathcal{N}(\mu, \sigma^2/n)$. Plain English: sample-mean standard error shrinks as $1/\sqrt{n}$.

Sketch of proof (characteristic functions). Let $\varphi_X(t) = E[e^{itX}]$. For i.i.d., $\varphi_{\bar X_n}(t) = \varphi_X(t/n)^n$. Taylor-expand: $\varphi_X(t/n) \approx 1 + i\mu t/n - (\sigma^2 + \mu^2)t^2/(2n^2)$. Centering and rescaling, the standardized sum has characteristic function $\to e^{-t^2/2}$, the c.f. of standard Normal. By Lévy's continuity theorem, convergence of c.f. implies convergence in distribution.

Rule of thumb: n  30 is usually enough for moderately-skewed data; n  200+ for heavy-tailed data like GMV.

Symbol Table.

```
| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| \(\bar X_n\) | Sample mean | units of X | Average over n observations |
| \(\sigma/\sqrt{n}\) | Standard error | units | Uncertainty of the sample mean |
| n | Sample size | count | Larger n → tighter estimate |
| \(\mu\) | True population mean | units | The target you're estimating |
```

### 2.7.3 Fintech Worked Examples

Example A — Daily conversion rate sample-mean. True daily app-to-purchase conversion p = 0.08 with variance \(p(1-p) = 0.0736\). Sample 5000 users per day. \(\bar p_n\) has SE = \(\sqrt{0.0736/5000} = \sqrt{0.00001472} \approx 0.003836\). 95% Normal CI: \(0.08 \pm 1.96 \times 0.003836 = [0.0725, 0.0875]\). Any sample mean outside that band, two days running, warrants investigation.

Example B — Average ticket size on a heavy-tailed metric. True \(\mu = 500\), \(\sigma = 2000\) (power-law tail truncated). With n = 100, SE = 200 — too wide. With n = 10,000, SE = 20 — much tighter. To halve the SE you need 4× the data; quartering the SE needs 16×. Plan your experiment durations accordingly: for a ₹10 detectable lift on a ₹500 base, you need \(n \approx (1.96 \cdot 2000/10)^2 \approx 153,664\) per arm per day (two-arm A/B).

### 2.7.4 Common Pitfalls

- Using CLT on the median or max. CLT is for sums/means; extremes follow Extreme Value Theory.
- Heavy tails kill convergence. If \(\sigma^2 = \infty\) (power law with \(\alpha \leq 2\)), CLT fails. Use Lévy stable distributions or log-transform.
- Dependence violates i.i.d. Time-series data needs block bootstraps or HAC standard errors.
- Tiny n with skew. A 10-merchant pilot is not "approximately Normal" — it's whatever the underlying skew gives you.

| PM Relevance |
|---|
| - Why master this: every A/B test confidence interval, every dashboard error bar, every sample-size calculation rests on CLT. |
| - Metrics to own: SE of north-star, n required for a given MDE, p99 ticket size where CLT may not apply. |
| - Strategic tradeoffs: running an experiment longer reduces SE but costs opportunity; understanding CLT prices that tradeoff. |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "When stakeholders ask 'is this lift real?', I anchor the answer in standard error, not p-values. Georgia Tech reinforced that CLT is what makes mean-based decisions defensible — but only when the data is light-tailed enough; for GMV I switch to log-transforms." |

### 2.7.5 NumPy Snippet

```
import numpy as np
rng = np.random.default_rng(7)

# Example A: Bernoulli conversion
p = 0.08
n = 5000
draws = rng.binomial(1, p, size=(20_000, n)).mean(axis=1)
print("mean of means:", draws.mean())                      # ~0.08
print("SE empirical:", draws.std())                        # ~0.00384
print("SE theory:", np.sqrt(p*(1-p)/n))                    # 0.003836

# Example B: heavy-tailed ticket size (log-normal toy)
mu_log, sigma_log = np.log(500) - 0.5*1.0**2, 1.0  # so mean ~500
samples = rng.lognormal(mean=mu_log, sigma=sigma_log, size=(20_000, 100))
sample_means = samples.mean(axis=1)
print("mean of sample means:", sample_means.mean())        # ~500
print("SE n=100:", sample_means.std())
# Required n for MDE of 10, sd assumed 2000:
print("required n:", (1.96 * 2000 / 10) ** 2)              # ~153664
```

## 2.8 Maximum Likelihood Estimation (MLE)

### 2.8.1 Plain-English Intuition

MLE answers: "what parameter values make the data I observed most probable?" Pick the parameter $\theta$ that maximizes the likelihood of the actual data. It's the default fitter behind logistic regression, neural networks (cross-entropy loss), and most "training" in ML.

Cricket analogy. You watch a bowler take 7 wickets in 100 deliveries. What is his "true" wicket probability per ball? MLE says $\hat p = 7/100 = 0.07$ — the value that maximizes the binomial likelihood of exactly 7 successes.

Fintech analogy — Paytm KYC funnel. Out of 1000 sign-ups, 620 completed KYC. The MLE for the completion rate is 0.62. Behind every conversion rate on every dashboard sits a silent MLE.

### 2.8.2 Formal Definition and Derivations

Likelihood for i.i.d. data $\mathcal{D} = \{x_1, \ldots, x_n\}$ under model $p(x \mid \theta)$:

$$ L(\theta) = \prod_{i=1}^n p(x_i \mid \theta), \qquad \ell(\theta) = \log L(\theta) = \sum_{i=1}^n \log p(x_i \mid \theta). $$

MLE: $\hat\theta_{MLE} = \arg\max_\theta \ell(\theta).$

Worked derivation — Bernoulli. Data $x_i \in \{0,1\}$, $p(x_i \mid p) = p^{x_i}(1-p)^{1-x_i}$.

$$ \ell(p) = \sum_i x_i \log p + (1 - x_i)\log(1-p) = k \log p + (n-k)\log(1-p), $$

where $k = \sum x_i$. Set derivative to zero:

$$ \frac{d\ell}{dp} = \frac{k}{p} - \frac{n-k}{1-p} = 0 \implies \hat p = k/n. $$

Plain English: the MLE for a Bernoulli rate is just the sample proportion.

Worked derivation — Normal $\mu$ and $\sigma^2$. Log-likelihood:

$$ \ell(\mu, \sigma^2) = -\frac{n}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_i (x_i - \mu)^2. $$

$\partial_\mu \ell = 0 \Rightarrow \hat\mu = \bar x$. Then $\partial_{\sigma^2} \ell = 0 \Rightarrow \hat\sigma^2 = \frac{1}{n}\sum (x_i - \bar x)^2$ (note divisor n, not n1: MLE is biased; the n1 version is unbiased).

Properties. Under regularity, MLE is consistent ($\hat\theta \to \theta^*$), asymptotically Normal ($\sqrt{n}(\hat\theta - \theta^*) \to \mathcal{N}(0, I^{-1})$ where I is Fisher information), and invariant ($\widehat{g(\theta)} = g(\hat\theta)$).

Symbol Table.

| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| $\theta$ | Parameter(s) | model-dependent | Knobs to tune |
| $L(\theta)$ | Likelihood | scalar product | How probable the data is at this $\theta$ |
| $\ell(\theta)$ | Log-likelihood | scalar sum | Numerically tractable form |
| $\hat\theta_{MLE}$ | Maximizer | same as $\theta$ | Best single guess from data |
| I($\theta$) | Fisher information | matrix | Curvature; inverse ≈ asymptotic variance |

### 2.8.3 Fintech Worked Examples

Example A — MLE for KYC completion rate. k = 620 completed of n = 1000. $\hat p = 0.62$. Asymptotic SE = $\sqrt{\hat p(1-\hat p)/n} = \sqrt{0.62 \cdot 0.38/1000} = \sqrt{0.0002356} \approx 0.01535$. 95% CI $[0.62 - 1.96 \cdot 0.01535, 0.62

+ 1.96 \cdot 0.01535] = [0.590, 0.650].\)

Example B — MLE for Poisson failure rate. Observed failure counts over 5 minutes: 4, 6, 3, 5, 7. MLE: \(\hat\lambda = (4+6+3+5+7)/5 = 25/5 = 5\) failures/min. Asymptotic SE = \(\sqrt{\hat\lambda/n} = \sqrt{5/5} = 1\). 95% CI \([3.04, 6.96]\). If your SLO says λ should not exceed 4, you cannot reject the SLO with this data — escalate when CI's lower bound exceeds SLO, not on the point estimate alone.

### 2.8.4 Common Pitfalls

- Overfitting. MLE on a flexible model with sparse data memorizes noise. Use regularization (= MAP, next section).
- Local maxima. Non-convex likelihoods (neural nets) need multiple restarts or convex relaxations.
- Identifiability. Two different \(\theta\) values can give identical likelihoods (e.g., mixture-component labels). MLE picks one arbitrarily.
- Confidence vs. likelihood. A high likelihood at \(\hat\theta\) doesn't mean tight uncertainty — check Fisher information.

| PM Relevance |
| --- |
| - Why master this: every fitted KPI rate, every conversion model, every recommender score is MLE under the hood. Understanding when MLE breaks (sparse data, multimodal likelihoods) is the difference between trusting a model and being misled by one. |
| - Metrics to own: point estimates with asymptotic CIs, model fit (log-likelihood, AIC). |
| - Strategic tradeoffs: MLE is unbiased asymptotically but can overfit on small samples — switch to MAP/regularization when n is small per cell. |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "Every conversion rate on my dashboard is an MLE — sample mean for Bernoulli. I always pair it with its asymptotic CI, because Georgia Tech taught me that a point estimate without uncertainty is product-management malpractice." |

### 2.8.5 NumPy Snippet

```
import numpy as np

# Example A: Bernoulli MLE
k, n = 620, 1000
p_hat = k / n
se = np.sqrt(p_hat * (1 - p_hat) / n)
ci = (p_hat - 1.96*se, p_hat + 1.96*se)
print("p_hat:", p_hat, "SE:", se, "95% CI:", ci)

# Example B: Poisson MLE
counts = np.array([4, 6, 3, 5, 7])
lam_hat = counts.mean()
se_lam = np.sqrt(lam_hat / len(counts))
print("lam_hat:", lam_hat, "SE:", se_lam, "CI:", (lam_hat-1.96*se_lam, lam_hat+1.96*se_lam))

# Verify by grid search on log-likelihood
ks = counts
lams = np.linspace(0.1, 15, 5000)
# log p(k|lam) = k log lam - lam - log(k!)
from math import lgamma
loglik = (ks[:, None] * np.log(lams) - lams - np.array([lgamma(k+1) for k in ks])[:, None]).sum(axis=0)
print("argmax lam:", lams[loglik.argmax()])  # ~5
```

## 2.9 Maximum A Posteriori (MAP) Estimation

### 2.9.1 Plain-English Intuition

MAP = MLE + a prior. Instead of asking "what \(\theta\) makes the data most probable?", MAP asks "what \(\theta\) is most probable given both my prior beliefs and the data?" Mathematically, MAP maximizes the posterior \(p(\theta \mid \mathcal{D})\). When the prior is uniform, MAP = MLE; when the prior is informative, MAP shrinks estimates toward prior beliefs — useful for small samples.

Cricket analogy. A new bowler debuts and takes 1 wicket in 6 balls. MLE says p = 1/6  0.167 (absurdly high). MAP with a Beta prior centered on the league average ~0.04 will pull this estimate down to something like 0.05 — far more realistic.

Fintech analogy — Paytm cold-start merchants. A merchant onboarded yesterday has 2 transactions and 1 chargeback. MLE: 50% chargeback rate (alarm!). MAP with prior centered on the segment average pulls the estimate to ~0.5%. Without MAP, every new merchant gets falsely flagged.

### 2.9.2 Formal Definition

By Bayes:

$$ p(\theta \mid \mathcal{D}) = \frac{p(\mathcal{D} \mid \theta) p(\theta)}{p(\mathcal{D})} \propto L(\theta) p(\theta). $$

MAP: $\hat\theta_{MAP} = \arg\max_\theta \log L(\theta) + \log p(\theta).$ The added term $\log p(\theta)$ is a regularizer.

Connection to regularization. With a Gaussian prior $\theta \sim \mathcal{N}(0, \tau^2 I)$, $\log p(\theta) = -\|\theta\|_2^2 / (2\tau^2) + \text{const}$ — this is L2 / ridge regularization. With a Laplace prior, you get L1 / lasso. Every regularized loss in ML is a MAP problem in disguise.

Beta–Binomial conjugate example. Prior $p \sim \text{Beta}(\alpha, \beta)$ with PDF $\propto p^{\alpha-1}(1-p)^{\beta-1}$. Data: k successes in n trials.

$$ \log p(p \mid \mathcal{D}) = (k + \alpha - 1)\log p + (n - k + \beta - 1)\log(1-p) + \text{const}. $$

$\partial/\partial p = 0 \Rightarrow \hat p_{MAP} = \frac{k + \alpha - 1}{n + \alpha + \beta - 2}.$

Plain English: MAP for Beta–Binomial is "pseudo-counts $\alpha-1$ successes and $\beta-1$ failures added to the data." Posterior mean (slightly different): $(k+\alpha)/(n+\alpha+\beta)$.

Symbol Table.

| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| $p(\theta)$ | Prior | density | Belief before seeing data |
| $L(\theta)$ | Likelihood | scalar | Same as MLE |
| $\hat\theta_{MAP}$ | MAP estimate | model-dim | Regularized estimate |
| $\alpha, \beta$ | Beta prior shape | scalars | Pseudo-counts of prior successes/failures |
| $\tau^2$ | Gaussian prior variance | units² | Strength of L2 shrinkage |

### 2.9.3 Fintech Worked Examples

Example A — New merchant chargeback rate. Observed k=1 chargeback in n=2 transactions. Prior Beta($\alpha=2, \beta=200$) (pseudo-counts: 2 chargebacks, 200 OK — reflects ~1% segment baseline).

- MLE: $\hat p = 1/2 = 0.50.$
- MAP: $\hat p_{MAP} = (1 + 2 - 1)/(2 + 2 + 200 - 2) = 2/202 \approx 0.0099.$ About 1% — sane.
- Posterior mean: $(1+2)/(2+2+200) = 3/204 \approx 0.0147.$

As more data comes in (n=200, k=5), MAP and MLE converge: MAP = (5+1)/(200+200) = 6/400 = 0.015 vs MLE = 5/200 = 0.025. Prior weight fades as data accumulates.

Example B — Ridge regression for ETA prediction. Predict delivery time $y$ from features $\mathbf{x}$ with model $y = \mathbf{x}^\top \boldsymbol\beta + \epsilon$), $\epsilon \sim \mathcal{N}(0, \sigma^2)$, prior $\boldsymbol\beta \sim \mathcal{N}(0, \tau^2 I)$. MAP closed form:

$$ \hat{\boldsymbol\beta}_{MAP} = (X^\top X + \lambda I)^{-1} X^\top y, \quad \lambda = \sigma^2/\tau^2. $$

With X being a 5×3 toy design and λ=1, the MAP shrinks coefficients toward zero compared to OLS (\(\lambda = 0\)). For sparse-feature ETA modeling on new geos, MAP stabilizes predictions.

### 2.9.4 Common Pitfalls

- Choosing priors arbitrarily. A too-strong prior wins the fight against data; a too-weak prior is useless. Calibrate on holdout.
- Improper priors. \(p(\theta) \propto 1\) on the real line may not integrate; the MAP may still exist, but Bayesian intervals get strange.
- Confusing MAP with posterior mean. Same when symmetric; different for skewed posteriors (e.g., Beta).
- Treating MAP as the "answer." MAP is a point; the full posterior carries the uncertainty — for risk decisions, use the distribution.

| PM Relevance |
| --- |
| - Why master this: every regularized model and every empirical-Bayes shrinkage estimate (e.g., Wilson interval, James–Stein) is a MAP. MAP is how you avoid embarrassing alerts on cold-start merchants. |
| - Metrics to own: shrinkage rate per segment, pseudo-count calibration, "stability" of metric as data accumulates. |
| - Strategic tradeoffs: strong prior = stable but slow to react; weak prior = responsive but noisy. Tune per use case. |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "On 30M merchants, raw rates are catastrophically noisy at the long tail. I use empirical-Bayes shrinkage — MAP under a Beta prior — to produce stable risk scores. Georgia Tech's Bayesian methods course made that a default reflex." |

### 2.9.5 NumPy Snippet

```
import numpy as np

# Example A: Beta-Binomial MAP
alpha, beta = 2, 200
k, n = 1, 2
mle = k / n
map_est = (k + alpha - 1) / (n + alpha + beta - 2)
post_mean = (k + alpha) / (n + alpha + beta)
print("MLE:", mle, "MAP:", map_est, "post mean:", post_mean)
# 0.5, ~0.0099, ~0.0147

# Larger sample shows convergence
k2, n2 = 5, 200
print("MLE:", k2/n2, "MAP:", (k2+alpha-1)/(n2+alpha+beta-2))

# Example B: Ridge regression closed form
rng = np.random.default_rng(0)
X = rng.normal(size=(50, 3))
true_b = np.array([1.5, -2.0, 0.0])
y = X @ true_b + rng.normal(scale=1.0, size=50)
lam = 1.0
beta_ols = np.linalg.solve(X.T @ X, X.T @ y)
beta_map = np.linalg.solve(X.T @ X + lam * np.eye(3), X.T @ y)
print("OLS:", beta_ols)
print("MAP/ridge:", beta_map)   # shrinks toward 0
```

## 2.10 Hypothesis Testing: p-values, Type I/II, α, Effect Size, Power, Confidence Intervals

### 2.10.1 Plain-English Intuition

Hypothesis testing is a formal courtroom for data: assume the null \(H_0\) is true (no effect), and ask whether the observed data is implausible enough to reject \(H_0\) in favor of an alternative \(H_1\) (an effect exists).

- p-value: P(data this extreme or more | \(H_0\) true). Small p  data is hard to explain under \(H_0\).
- α (significance level): the maximum false-positive rate you tolerate (typically 0.05). Reject \(H_0\) iff p < α.

- Type I error: rejecting $H_0$ when it's true (false positive). Rate = α.
- Type II error: failing to reject $H_0$ when $H_1$ is true (false negative). Rate = β.
- Power: $1 - \beta$, the probability of detecting a real effect of a given size.
- Effect size: the magnitude of the true difference (independent of n).
- Confidence interval: range that, over repeated experiments, would contain the true value 100(1α)% of the time.

Cricket umpire analogy. $H_0$: "not out." Evidence: the appeal. Type I error: giving an innocent batsman out (false positive). Type II error: missing a real dismissal (false negative). α is how strict the umpire is; power is how good they are at spotting real outs. DRS (Decision Review System) is essentially raising power by adding more evidence channels — the same idea as increasing sample size.

Fintech analogy — Paytm A/B test on checkout button. $H_0$: new button has same conversion as old. Reject if observed lift is bigger than what random noise would produce 5% of the time. Type I: shipping a placebo. Type II: rejecting a real winner. Power = your chance of catching a true 2% lift with the sample you have.

### 2.10.2 Formal Definitions and Derivations

Test statistic $T = T(\mathcal{D})$. Null distribution $T \mid H_0$. p-value (two-sided): $p = P(|T| \geq |t_\text{obs}| \mid H_0).$

Decision rule: reject $H_0$ if $p < \alpha$.

Two-proportion z-test (the A/B workhorse). Control conversion $\hat p_C = x_C/n_C$, treatment $\hat p_T = x_T/n_T$. Pooled rate $\hat p = (x_C + x_T)/(n_C + n_T)$.

$$ Z = \frac{\hat p_T - \hat p_C}{\sqrt{\hat p(1-\hat p)\left(\frac{1}{n_C} + \frac{1}{n_T}\right)}} \;\sim\; \mathcal{N}(0,1) \text{ under } H_0. $$

Two-sided p-value: $p = 2(1 - \Phi(|Z|))$.

Power for a two-sided z-test at level α with true lift $\delta$:

$$ \text{Power} = \Phi\!\left(\frac{|\delta|}{SE} - z_{1-\alpha/2}\right) + \Phi\!\left(-\frac{|\delta|}{SE} - z_{1-\alpha/2}\right) \approx \Phi\!\left(\frac{|\delta|}{SE} - z_{1-\alpha/2}\right) \text{ for small } \delta. $$

Sample size for two-proportion test (per arm):

$$ n \approx \frac{(z_{1-\alpha/2} + z_{1-\beta})^2 \,[p_C(1-p_C) + p_T(1-p_T)]}{(p_T - p_C)^2}. $$

Confidence interval (proportion difference).

$$ \hat\delta \pm z_{1-\alpha/2}\sqrt{\frac{\hat p_C(1-\hat p_C)}{n_C} + \frac{\hat p_T(1-\hat p_T)}{n_T}}. $$

A CI that excludes 0 corresponds to rejecting $H_0$ at level α (duality).

Effect size measures. Cohen's h for proportions: $h = 2\arcsin\sqrt{p_T} - 2\arcsin\sqrt{p_C}$. Cohen's d for means: $d = (\bar x_T - \bar x_C)/s_\text{pooled}$. Rules of thumb: 0.2 small, 0.5 medium, 0.8 large.

Symbol Table.

```
| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| \(H_0, H_1\) | Null & alternative | hypotheses | "No effect" vs "effect" |
| α | Significance level | [0,1] | Tolerable false-positive rate |
| β | Type II rate | [0,1] | Missed real-effect rate |
```

```
| 1−β | Power | [0,1] | Detection probability |
| p | p-value | [0,1] | Surprise under \(H_0\) |
| δ | True effect size | units | What you'd ship for |
| CI | Confidence interval | range | Plausible parameter values |
| MDE | Minimum detectable effect | units | Smallest δ you can reliably catch |
```

### 2.10.3 Fintech Worked Examples

Example A — Checkout button A/B test. Control: 12,000 sessions, 1,440 conversions (\(\hat p_C = 0.120\)). Treatment: 12,000 sessions, 1,560 conversions (\(\hat p_T = 0.130\)).

- Lift: \(\hat\delta = 0.010\) (1 percentage point, 8.3% relative).
- Pooled: \(\hat p = (1440+1560)/24000 = 0.125\).
- SE under \(H_0\): \(\sqrt{0.125 \cdot 0.875 \cdot (1/12000 + 1/12000)} = \sqrt{0.109375 \cdot 0.0001667} = \sqrt{0.00001823} \approx 0.004270\).
- Z = 0.010 / 0.004270  2.342.
- Two-sided p = 2(1  Φ(2.342))  2(1  0.99041)  0.0192. Significant at α=0.05.
- 95% CI for δ (unpooled SE): \(\sqrt{0.12 \cdot 0.88/12000 + 0.13 \cdot 0.87/12000} = \sqrt{0.0000088 + 0.00000943} = \sqrt{0.00001823} \approx 0.004270\). CI = \(0.010 \pm 1.96 \cdot 0.004270 = [0.00163, 0.01837]\). Excludes 0  consistent with rejection.
- Cohen's h = \(2\arcsin\sqrt{0.13} - 2\arcsin\sqrt{0.12} = 2(0.3690) - 2(0.3537) = 0.0307\). Tiny — but PM-meaningful because the base is huge.

Example B — Sample-size planning for a 0.5pp MDE. Plan an experiment to detect a true lift from \(p_C = 0.120\) to \(p_T = 0.125\) (0.5pp absolute, ~4.2% relative) at α = 0.05, power = 0.80.

- \(z_{0.975} = 1.96\), \(z_{0.80} = 0.8416\). Sum = 2.8016, squared = 7.849.
- Numerator: \(7.849 \cdot (0.12 \cdot 0.88 + 0.125 \cdot 0.875) = 7.849 \cdot (0.1056 + 0.109375) = 7.849 \cdot 0.214975 \approx 1.6873.\)
- Denominator: \((0.005)^2 = 0.000025.\)
- n per arm  67,492. With 30M-merchant daily traffic this is fine; with a niche segment of 5,000/day it would take ~14 days per arm.

### 2.10.4 Common Pitfalls

- p-hacking and peeking. Repeatedly looking at p-values inflates Type I error. Use sequential tests (mSPRT, group sequential, Bayesian decision rules) or pre-commit to n.
- Multiple testing. With 20 tests at α=0.05, expect 1 false positive. Apply Bonferroni (α/m), Holm–Bonferroni, or BH for FDR control.
- Significance vs practical importance. With n=10M you can detect 0.001pp lifts that are meaningless commercially. Always report effect size and CI, not just p.
- Asymmetric harm. "Insignificant" doesn't mean "safe" — a 50% miss probability of a harmful regression is not acceptable.
- Variance reduction skipped. CUPED, stratification, and pre-period covariates can halve required n.

| PM Relevance |
| --- |
| - Why master this: experimentation discipline is what separates real PMs from dashboard tourists. Every ship/no-ship call rests on these primitives. |
| - Metrics to own: MDE, power, sample size, expected experiment duration, false-discovery rate across the experiment portfolio. |

- Strategic tradeoffs: stricter α reduces false ships but raises false kills; longer experiments raise power but cost opportunity.

- 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "I price every experiment in MDE × duration × cost-of-mistake. At Paytm scale, 'statistically significant' is cheap; I demand pre-registered effect-size thresholds. Georgia Tech's experimental-design coursework drilled the dual α/β discipline into me."

### 2.10.5 NumPy Snippet

```
import numpy as np
from math import erf, sqrt

def Phi(z):  # standard normal CDF
    return 0.5 * (1 + erf(z / sqrt(2)))

# Example A
nC, xC = 12000, 1440
nT, xT = 12000, 1560
pC, pT = xC/nC, xT/nT
p_pool = (xC + xT) / (nC + nT)
SE_null = np.sqrt(p_pool*(1-p_pool)*(1/nC + 1/nT))
Z = (pT - pC) / SE_null
p_val = 2 * (1 - Phi(abs(Z)))
print("Z:", Z, "p:", p_val)                                  # 2.342, 0.0192

# 95% CI for delta (unpooled SE)
SE_unpool = np.sqrt(pC*(1-pC)/nC + pT*(1-pT)/nT)
ci = (pT - pC - 1.96*SE_unpool, pT - pC + 1.96*SE_unpool)
print("delta CI:", ci)                                       # ~(0.00163, 0.01837)

# Cohen's h
h = 2*np.arcsin(np.sqrt(pT)) - 2*np.arcsin(np.sqrt(pC))
print("Cohen's h:", h)                                       # ~0.0307

# Example B: required sample size per arm
z_a, z_b = 1.96, 0.8416
pC2, pT2 = 0.120, 0.125
num = (z_a + z_b)**2 * (pC2*(1-pC2) + pT2*(1-pT2))
den = (pT2 - pC2)**2
n_req = num / den
print("n per arm:", n_req)                                   # ~67492

# Empirical power check via simulation
rng = np.random.default_rng(11)
sims = 5000
sig = 0
n = int(np.ceil(n_req))
for _ in range(sims):
    c = rng.binomial(n, pC2)
    t = rng.binomial(n, pT2)
    pp = (c + t) / (2*n)
    SE = np.sqrt(pp*(1-pp)*(2/n))
    z = (t/n - c/n) / SE
    if abs(z) > 1.96:
        sig += 1
print("empirical power:", sig/sims)                          # ~0.80
```

## 2.11 Rigorous Multivariable A/B Testing at 30M-Merchant Scale

### 2.11.1 Plain-English Intuition

A simple A/B test compares one treatment to one control on one metric. Real Paytm decisions are not that clean:

- Multiple variants (A/B/C/D/E — "multi-armed bandits" or factorial designs).
- Multiple metrics (north-star + guardrails like crashes, latency, refunds).
- Multiple segments (tier-1 merchants, kirana stores, e-com, new vs. mature).
- Network effects (merchants influence each other; treatment leaks via WhatsApp).
- Heavy tails (GMV is power-law; classical t-tests are unreliable).
- Sequential decisions (you want to stop early when an arm wins or hurts).

Rigorous multivariable A/B testing is the discipline of running these tests without lying to yourself.

Cricket umpire analogy — DRS multi-evidence. Modern DRS combines audio (snicko), thermal (hotspot), and ball-tracking. Each is a separate test; the umpire must combine evidence without double-counting. The PM equivalent is multiple metrics: each is informative, but you must adjust for correlated false positives.

Weather forecasting analogy. Ensemble models combine many noisy forecasts to a single calibrated probability. A/B at scale is an ensemble: pre-period covariates, segmented effects, and Bayesian shrinkage combine into one ship/no-ship call.

### 2.11.2 Formal Methods

1. Stratified / blocked randomization. Pre-bucket merchants by tier and city; randomize within strata. Reduces between-strata variance, equivalent to ANOVA with strata fixed effects.

2. Variance reduction (CUPED). Adjust the metric $Y$ using a pre-experiment covariate $X$:

$$ Y_{\text{adj}} = Y - \theta(X - \bar X), \quad \theta = \mathrm{Cov}(Y, X) / \mathrm{Var}(X). $$

Plain English: subtract the predictable pre-period component. Required n drops by factor $1/(1-\rho^2)$, where $\rho$ is correlation between $Y$ and $X$. With $\rho = 0.7$, n drops ~2×.

3. Multiple-testing correction. With m hypotheses, the family-wise error rate (FWER) is bounded by $\sum \alpha_i$. Methods:

- Bonferroni: test each at $\alpha/m$. Conservative.
- Holm–Bonferroni: sequential, uniformly more powerful.
- Benjamini–Hochberg (BH): controls false discovery rate $\mathrm{FDR} = E[\text{FP}/\max(\text{FP+TP},1)]$. Sort p-values; reject the largest k where $p_{(k)} \leq k\alpha/m$. Standard at scale.

4. Heavy-tail handling. For GMV, use:

- Log-transform: $\log(1 + Y)$ (mass at zero handled by +1).
- Winsorize at p99.
- Trimmed means.
- Bootstrap CI (resample-with-replacement, take 2.5/97.5 percentiles of bootstrap means — robust to non-Normal).
- Quantile regression for median lift.

5. Sequential testing (mSPRT, group sequential). The mixture sequential probability ratio test allows continuous monitoring with controlled Type I rate. Equivalently, "always valid" Bayesian decision rules using posterior probability of being best (Thompson sampling).

6. Network / spillover. Cluster-randomize at city or merchant-cluster level when treatments diffuse; estimate via difference-in-differences with cluster-robust SEs.

7. Heterogeneous treatment effects (HTE). Estimate per-segment effects via interaction terms or causal forests (X-learner, R-learner). Report a CATE map, not just the ATE.

8. Guardrails. Pre-register acceptable degradations on guardrail metrics (crash rate, latency P95, refunds). Even a "winning" north-star ships only if guardrails hold.

Symbol Table.

| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| ATE | Average treatment effect | units | Mean lift on north-star |
| CATE | Conditional ATE | function | Lift per segment |

| FWER | Family-wise error rate | [0,1] | P(any false rejection) |
| FDR | False discovery rate | [0,1] | Expected fraction of false rejections |
| MDE | Min detectable effect | units | Floor of test sensitivity |
| CUPED θ | Variance-reduction slope | unitless | Pre-period regression coefficient |
| guardrail | Pre-registered metric | various | Non-degradation constraint |

### 2.11.3 Fintech Worked Examples

Example A — Five-arm checkout test with BH correction. Test 4 treatments vs control on Paytm checkout. Observed two-sided p-values for treatment vs control on the primary conversion metric:

| Arm | p-value |
|---|---|
| T1 | 0.004 |
| T2 | 0.021 |
| T3 | 0.045 |
| T4 | 0.300 |

m = 4 tests. BH procedure at FDR = 0.05: sort ascending (0.004, 0.021, 0.045, 0.300). Compare to thresholds \(k \cdot 0.05/4 = 0.0125, 0.025, 0.0375, 0.05\):

- k=1: 0.004  0.0125
- k=2: 0.021  0.025
- k=3: 0.045  0.0375
- k=4: 0.300  0.05

Largest k with success = 2. Reject T1 and T2 (also any earlier in sorted order). T3 is "Bonferroni-significant in isolation" but does not survive FDR control.

Contrast with Bonferroni at FWER=0.05: threshold = 0.0125. Only T1 (0.004) survives. BH is uniformly more powerful for screening many arms.

Example B — CUPED variance reduction at 30M scale. A guardrail metric: weekly GMV per merchant. Pre-period mean \(\bar X = ₹4{,}800\); experiment-period mean by arm:

- Control: \(\bar Y_C = ₹5{,}040\), \(\hat\sigma_Y^2 = 4{,}000{,}000\).
- Treatment: \(\bar Y_T = ₹5{,}160\), \(\hat\sigma_Y^2 = 4{,}050{,}000\).
- Correlation \(\rho(Y, X) = 0.75\). \(\theta \approx \rho \sigma_Y/\sigma_X\); for simplicity assume \(\sigma_X \approx \sigma_Y \approx 2000\), so \(\theta \approx 0.75\).

CUPED variance: \(\sigma_{Y\text{-adj}}^2 = \sigma_Y^2 (1 - \rho^2) = 4{,}000{,}000 \cdot (1 - 0.5625) = 1{,}750{,}000\). Halves variance  halves required n (or doubles power at fixed n).

Required n per arm to detect ₹100 lift at 80% power, α=0.05, with original variance: \(n = (1.96 + 0.84)^2 \cdot 2 \cdot 4{,}000{,}000 / 100^2 = 7.849 \cdot 8{,}000{,}000 / 10{,}000 = 6{,}279\). With CUPED variance 1.75M: \(n = 7.849 \cdot 3{,}500{,}000/10{,}000 = 2{,}747\). At 30M merchants, the experiment that took 4 days now takes ~2 days.

Guardrail check. Latency P95 must not regress by more than 5ms. Observed: Control 180ms, Treatment 184ms. Δ = +4ms, within the pre-registered band. Plus: bootstrap 95% CI of P95-difference: [+1, +7]ms — upper bound crosses 5ms guardrail. Hold ship for an SRE investigation, even though the conversion lift is statistically significant.

### 2.11.4 Common Pitfalls

- Looking at the dashboard daily and "calling it" — peeking inflates Type I from 5% to 25%+ over a week.
- Naïvely averaging GMV. A single ₹50L outlier merchant can flip the test sign. Always winsorize or log.

- Forgetting the SUTVA assumption. If treatment merchants chat with control merchants on WhatsApp and share offers, randomization is contaminated  cluster-randomize.
- Reporting only ATE. A flat ATE often hides +10% lift for tier-1 and 5% for kirana; segment the effect.
- Ignoring guardrails. "Conversion is up, but app crash rate is up too" — guardrails are non-negotiable.
- Treating new-merchant cohorts as steady-state. Novelty effects fade; run long enough to capture decay.
- Forgetting CUPED. Free variance reduction; refusal to use it is leaving sample size on the table.

| PM Relevance |
| --- |
| - Why master this: at 30M-merchant scale a single bad ship can cost crores in GMV and trust. Multivariable test rigor is a profitability lever, not a research nicety. |
| - Metrics to own: north-star (e.g., 7-day active checkout), per-segment CATE, guardrail compliance vector, FDR across the experiment portfolio. |
| - Strategic tradeoffs: more variants accelerate learning but multiply error correction; sequential tests ship faster but require strict pre-registration; CUPED is free if pre-period data exists. |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "At Paytm I treat experimentation as a portfolio: I budget FDR across all concurrent tests with Benjamini–Hochberg, use CUPED to halve sample sizes, and pre-register guardrails on latency and refund rate. Georgia Tech's research-methods rigor turned that into instinct — and turned me into the PM who ships fewer, surer wins." |

### 2.11.5 NumPy Snippet

```
import numpy as np
from math import erf, sqrt

def Phi(z): return 0.5 * (1 + erf(z / sqrt(2)))

# Example A: Benjamini-Hochberg
pvals = np.array([0.004, 0.021, 0.045, 0.300])
labels = np.array(['T1', 'T2', 'T3', 'T4'])
m = len(pvals)
alpha = 0.05

order = np.argsort(pvals)
sorted_p = pvals[order]
sorted_labels = labels[order]
thresholds = (np.arange(1, m+1) / m) * alpha
passes = sorted_p <= thresholds
# Largest k that passes; reject all up to and including that k
if passes.any():
    k_max = np.max(np.where(passes)[0])
    rejected = sorted_labels[:k_max+1]
else:
    rejected = []
print("BH rejected:", list(rejected))               # ['T1', 'T2']

# Bonferroni for comparison
bonf_threshold = alpha / m
bonf_rej = labels[pvals < bonf_threshold]
print("Bonferroni rejected:", list(bonf_rej))       # ['T1']

# Example B: CUPED required-n calculation
sigma2_Y = 4_000_000
rho = 0.75
sigma2_adj = sigma2_Y * (1 - rho**2)

z_a, z_b = 1.96, 0.8416
delta = 100  # ₹ lift to detect
n_orig = (z_a + z_b)**2 * 2 * sigma2_Y / delta**2
n_cuped = (z_a + z_b)**2 * 2 * sigma2_adj / delta**2
print("n original:", int(np.ceil(n_orig)))           # 6279
print("n CUPED:", int(np.ceil(n_cuped)))             # 2747
print("savings ratio:", 1 - n_cuped/n_orig)          # 0.4375

# Simulate CUPED estimate on synthetic data
rng = np.random.default_rng(0)
N = 20_000
X_pre = rng.normal(4800, 2000, size=(2, N))
true_lift = 100
Y_post = X_pre + rng.normal(0, np.sqrt(sigma2_Y*(1-rho**2)), size=(2, N))
Y_post[1] += true_lift  # treatment

# Pool to estimate theta
all_Y = np.concatenate([Y_post[0], Y_post[1]])
all_X = np.concatenate([X_pre[0], X_pre[1]])
theta = np.cov(all_Y, all_X, ddof=1)[0,1] / np.var(all_X, ddof=1)
```

```
Y_adj = Y_post - theta * (X_pre - X_pre.mean())

naive_lift = Y_post[1].mean() - Y_post[0].mean()
cuped_lift = Y_adj[1].mean() - Y_adj[0].mean()
print("naive var:", Y_post[0].var(ddof=1))
print("cuped var:", Y_adj[0].var(ddof=1))
print("naive lift:", naive_lift, "cuped lift:", cuped_lift)

# Guardrail bootstrap example
control_lat = rng.normal(180, 30, 5000)
treat_lat = rng.normal(184, 30, 5000)
def p95(x): return np.quantile(x, 0.95)
B = 2000
diffs = np.empty(B)
for b in range(B):
    c = rng.choice(control_lat, size=len(control_lat), replace=True)
    t = rng.choice(treat_lat, size=len(treat_lat), replace=True)
    diffs[b] = p95(t) - p95(c)
print("P95 diff 95% CI:", (np.quantile(diffs, 0.025), np.quantile(diffs, 0.975)))
```

## Closing Notes — Volume 1 Chapters 1–2

You have now rebuilt, from first principles, the two pillars on which every modern AI/ML system rests: linear algebra (the geometry of representations) and probability/statistics (the calculus of decisions). Every embedding in Chroma or FAISS, every Bayesian fraud score at Paytm, every multivariable test on a 30M-merchant base, draws directly on the equations in these pages.

A PM who masters this material does three things that most PMs cannot:

1. Read papers without flinching at \(A = U\Sigma V^\top\), eigendecompositions, or posterior odds.
2. Push back on ML teams when an AUC of 0.95 hides a posterior precision of 5%, or when a "significant" p hides a meaningless effect size.
3. Design experiments that ship the right thing — fewer, surer wins, with guardrails and FDR control.

Chapters 3 onward (calculus and optimization, information theory, the geometry of deep learning) build on these foundations directly. Reread any section whose 30-second interview talking point you cannot deliver from memory. Treat the NumPy snippets as your private lab — every fintech worked example here was chosen to map to a real Paytm dashboard you can recreate.

— End of Chapters 1 and 2.

# Chapter 3 — Calculus & Matrix Calculus

> "Calculus is the language in which nature writes its rate of change. For a PM, it is the language in which a product writes its sensitivity to a single knob."

Prabhjot — you have built and shipped pricing experiments at Paytm, you have argued MDR with merchants, you have watched your discount engine bleed and recover. Every one of those experiences is a story about derivatives in disguise. When you ask "what happens to take-rate if I drop MDR by 5 basis points?" — that is a partial derivative. When you ask "which combination of MDR, cashback, and onboarding fee maximizes contribution margin?" — that is a gradient pointing uphill. When you ask "is this revenue surface smooth or am I about to fall off a cliff?" — that is continuity and the Hessian.

This chapter rebuilds your calculus muscles from the ground up, in matrix form, with fintech worked examples. You do not need to memorize identities. You need to see the geometry. By the end you will be able to walk into any ML architecture review at Paytm and reason about gradient flow the way you currently reason about funnel drop-off.

## 3.1 Limits and Continuity

### 1. Plain-English Intuition

A limit answers a question that sounds almost philosophical: "As I get arbitrarily close to a point, what value is my function trying to be?" The function might never actually equal that value at the point — maybe it is undefined there, maybe there is a hole — but the trend is unmistakable.

Everyday analogy. You are driving toward a tunnel entrance. You cannot be inside the tunnel and outside it at the same time, but as you approach, your speed, your engine RPM, and the temperature inside the car all settle toward predictable values. The limit is what those values converge to, regardless of whether you ever stop exactly at the entrance.

Fintech analogy. Consider Paytm's UPI transaction success rate as a function of payload size. At payload size zero, success rate is undefined (no transaction). But as payload approaches zero from above, success rate approaches some asymptotic ceiling — say 99.7%. That ceiling is the limit. A function is continuous at a point if (a) it is defined there, (b) the limit exists, and (c) the two agree. Continuity is the mathematical statement of "no surprises, no cliffs". ML loss functions had better be continuous in their parameters — otherwise gradient descent is walking blindfolded near a chasm.

### 2. Formal Mathematical Definition

The $\varepsilon$–$\delta$ definition of a limit:

$$\lim_{x \to a} f(x) = L \quad \iff \quad \forall \varepsilon > 0,\; \exists \delta > 0 \;\text{ such that }\; 0 < |x - a| < \delta \implies |f(x) - L| < \varepsilon.$$

Plain English: "For every tolerance epsilon you give me on the output, I can hand you back a tolerance delta on the input such that if x stays within delta of a (but not equal to a), then f(x) stays within epsilon of L."

A function $f:\mathbb{R}^n \to \mathbb{R}^m$ is continuous at $\mathbf{a}$ iff:

$$\lim_{\mathbf{x} \to \mathbf{a}} f(\mathbf{x}) = f(\mathbf{a}).$$

Plain English: "The value the function approaches equals the value it actually takes."

Sketch proof that polynomials are continuous everywhere. Let $f(x) = c_0 + c_1 x + \cdots + c_n x^n$. By the algebra of limits (sum of limits is limit of sum, product of limits is limit of product), and because $\lim_{x\to a} x = a$, it follows that $\lim_{x\to a} f(x) = f(a)$. QED. The takeaway: any loss function built from polynomial primitives is automatically

continuous — a fact you will quietly rely on every time you train a regression head.

```
| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| \(f\) | Function under study | \(\mathbb{R}^n \to \mathbb{R}^m\) | Mapping from product inputs (price,
latency) to outcomes (conversion, revenue) |
| \(a\) | Point of interest in domain | \(\mathbb{R}^n\) | The current operating point (today's MDR, today's
traffic) |
| \(L\) | Limit value | \(\mathbb{R}^m\) | The KPI the system is heading toward as inputs shift |
| \(\varepsilon\) | Output tolerance | scalar \(>0\) | Acceptable error band on the KPI (e.g., ±5 bps on take-
rate) |
| \(\delta\) | Input tolerance | scalar \(>0\) | How tightly you must control the lever to stay in band |
```

## 3. Fintech Worked Numeric Examples

Example A — Paytm Wallet top-up success rate near zero amount. Empirically, the success rate model is $S(x) = \dfrac{99.5 x + 0.02}{x + 0.0002}$ for top-up amount $x$ in INR. We want $\lim_{x \to 0^+} S(x)$.

Step 1. Substitute small values. At $x = 0.01$: numerator $= 99.5(0.01) + 0.02 = 0.995 + 0.02 = 1.015$; denominator $= 0.01 + 0.0002 = 0.0102$; ratio $= 1.015 / 0.0102 = 99.5098$. Step 2. At $x = 0.001$: numerator $= 0.0995 + 0.02 = 0.1195$; denominator $= 0.0012$; ratio $= 99.5833$. Step 3. At $x = 0.0001$: numerator $= 0.00995 + 0.02 = 0.02995$; denominator $= 0.0003$; ratio $= 99.8333$. Step 4. Apply L'Hôpital or just divide leading constants: $\lim_{x\to 0^+} S(x) = 0.02 / 0.0002 = 100$. Interpretation: as you drive top-ups toward zero, the model breaks — success rate asymptotically goes to 100%, which is non-physical. This is a PM red flag: the model is not continuous at $x = 0$ and your dashboard near the floor is lying.

Example B — MDR vs merchant conversion continuity check. Suppose merchant onboarding conversion is $C(m)$ where $m$ is the MDR in basis points. Data points: $C(50) = 0.82$, $C(60) = 0.78$, $C(70) = 0.73$, $C(80) = 0.67$, $C(90) = 0.59$, $C(100) = 0.49$. Differences: $-0.04, -0.05, -0.06, -0.08, -0.10$. The function is decreasing and the differences are themselves changing smoothly — no jumps. So $C$ appears continuous on $[50,100]$ bps. Now suppose at $m = 95$, regulation forces a flat compliance fee, and $C(95) = 0.40$. Then $\lim_{m \to 95^-} C(m) \approx 0.54$ but $C(95) = 0.40$ — a jump discontinuity. PM consequence: A/B tests straddling 95 bps will produce uninterpretable variance; you must segment.

## 4. Common Pitfalls

- Confusing "limit exists" with "function is defined." A function can have a limit at a hole.
- Forgetting that a multivariate limit must be the same along every path of approach. $f(x,y) = xy/(x^2 + y^2)$ has different limits along $y = x$ versus $y = 0$ — so the limit at the origin does not exist.
- Assuming continuity implies differentiability. $|x|$ is continuous everywhere, differentiable nowhere at zero. Many regularizers (L1) live on this knife edge.
- Treating discrete KPIs (count of merchants) as continuous without acknowledging the approximation.

## 5. PM Relevance

| PM Relevance |
|---|
| - Why a PM must master this: Every dashboard, every funnel, every ML loss is a function. If you cannot reason about whether it is continuous near your operating point, you cannot reason about whether small changes will produce small or catastrophic effects on revenue. |
| - Metrics to own: KPI sensitivity bands, discontinuity audits at policy thresholds (KYC tiers, regulatory MDR caps), model-vs-reality residuals near boundaries. |
| - Strategic tradeoffs: Smoother (more continuous) systems are easier to optimize but may be slower to respond to legitimate regime changes; sharper systems can chase signal but punish A/B testers with high variance. |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "At Paytm I treated MDR-vs-conversion as a continuous curve in the interior but flagged the regulatory cap at 95 bps as a jump discontinuity. My Georgia Tech MSCS work on optimization made me suspicious of any pipeline that assumed global smoothness — that |

suspicion saved an A/B test from a six-figure misread."

### 6. NumPy-Only Python Snippet

```
import numpy as np

# Example A: limit of S(x) = (99.5x + 0.02)/(x + 0.0002) as x -> 0+
xs = np.array([1e-2, 1e-3, 1e-4, 1e-5, 1e-6])
S = (99.5 * xs + 0.02) / (xs + 0.0002)
print("x values:", xs)
print("S(x):    ", S)
print("Apparent limit:", 0.02 / 0.0002)  # leading-term ratio = 100.0

# Example B: continuity check on MDR-vs-conversion
m = np.array([50, 60, 70, 80, 90, 100], dtype=float)
C = np.array([0.82, 0.78, 0.73, 0.67, 0.59, 0.49])
diffs = np.diff(C) / np.diff(m)            # discrete derivative
second_diffs = np.diff(diffs)              # discrete curvature
print("First differences (slope):", diffs)
print("Second differences (curvature):", second_diffs)
# Inject the regulatory jump and detect it
m2 = np.append(m, 95.0)
C2 = np.append(C, 0.40)
order = np.argsort(m2)
m2, C2 = m2[order], C2[order]
local_jump = np.abs(np.diff(C2)) / np.abs(np.diff(m2) + 1e-9)
print("Local jump magnitudes:", local_jump)  # spike near m=95 flags the discontinuity
```

## 3.2 Derivatives — The Single-Variable Foundation

### 1. Plain-English Intuition

A derivative is the instantaneous rate of change. If your function tells you where you are, the derivative tells you which way and how fast you are moving.

Everyday analogy — the hiker. You are a hiker in fog on a Himalayan trail, and you want to reach the valley floor. You cannot see the valley. But you can feel the slope under your boots. The slope at your current spot is the derivative. If it tilts down to your east, you step east. The steeper the tilt, the bigger your step. This is the whole story of gradient descent in one sentence.

Fintech analogy — the MDR knob. Imagine the merchant discount rate is a single dial on your dashboard, and revenue $R(m)$ is a number that updates when you turn the dial. $R'(m)$, the derivative, tells you "if I nudge MDR up by one tiny bp, revenue changes by $R'(m)$ rupees." If $R'(m) > 0$, raise MDR. If $R'(m) < 0$, drop it. The optimal MDR is exactly where $R'(m) = 0$ — the dial where nudging in either direction loses you money. That is the first-order optimality condition that drives every pricing model Paytm runs.

### 2. Formal Mathematical Definition

The derivative of $f:\mathbb{R} \to \mathbb{R}$ at $x$:

$$ f'(x) \;=\; \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}. $$

Plain English: "The slope of the secant line through $(x, f(x))$ and $(x+h, f(x+h))$, in the limit as the second point slides into the first."

Derivation of the power rule. Let $f(x) = x^n$, $n$ a positive integer. Then by the binomial theorem,

$$ f(x+h) = (x+h)^n = \sum_{k=0}^{n} \binom{n}{k} x^{n-k} h^{k} = x^n + n x^{n-1} h + \binom{n}{2} x^{n-2} h^2 + \cdots + h^n. $$

So

\[ \frac{f(x+h) - f(x)}{h} = n x^{n-1} + \binom{n}{2} x^{n-2} h + \cdots + h^{n-1}. \]

Taking the limit as \(h \to 0\), every term with a positive power of \(h\) vanishes, leaving \(f'(x) = n x^{n-1}\). QED. Plain English: "Multiply by the exponent, drop the exponent by one."

Product rule, derived. If \(u,v\) are differentiable, \[ (uv)'(x) = \lim_{h\to 0} \frac{u(x+h)v(x+h) - u(x)v(x)}{h}. \] Add and subtract \(u(x+h)v(x)\) in the numerator: \[ = \lim_{h\to 0} \frac{u(x+h)[v(x+h) - v(x)]}{h} + \lim_{h\to 0} \frac{v(x)[u(x+h) - u(x)]}{h} = u(x) v'(x) + v(x) u'(x). \] Plain English: "Differentiate one, hold the other; swap, add."

```
| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| \(f(x)\) | Scalar function of scalar | \(\mathbb{R} \to \mathbb{R}\) | Revenue as a function of a single
lever |
| \(f'(x)\), \(\frac{df}{dx}\) | Derivative at \(x\) | scalar | Marginal revenue per unit lever change |
| \(h\) | Increment | scalar | A/B test step size on the lever |
| \(\Delta f / \Delta x\) | Discrete slope | scalar | Empirical sensitivity measured from data |
```

## 3. Fintech Worked Numeric Examples

Example A — Marginal revenue from MDR. Model: \(R(m) = 1000 \cdot m \cdot (1 - 0.008 m)\), where \(m\) is MDR in bps, \(R\) is revenue in INR per merchant per month. Expanding: \(R(m) = 1000 m - 8 m^2\). Step 1. Differentiate term-by-term: \(R'(m) = 1000 - 16 m\). Step 2. Evaluate at the current operating point \(m = 50\): \(R'(50) = 1000 - 800 = 200\) INR per bp. Step 3. Interpretation: raising MDR by 1 bp increases monthly per-merchant revenue by about 200 INR — at the current point. Worth doing if elasticity from churn is less than 200 INR/bp. Step 4. Find the revenue-maximizing MDR: set \(R'(m) = 0 \Rightarrow m^\star = 62.5\) bps. Maximum revenue \(R(62.5) = 62{,}500 - 8(62.5)^2 = 62{,}500 - 31{,}250 = 31{,}250\) INR per merchant per month. Step 5. Beyond \(m = 62.5\), the derivative is negative — every additional bp now costs you because conversion erodes faster than pricing gains.

Example B — Cashback budget burn rate. Daily cashback spend \(B(t) = 5000 + 1200 t - 30 t^2\) in thousands of INR, where \(t\) is days since campaign launch. Step 1. \(B'(t) = 1200 - 60 t\) (thousands of INR per day). Step 2. At \(t = 5\): \(B'(5) = 1200 - 300 = 900\). The campaign is burning 9 lakh INR/day more each day; we are still accelerating. Step 3. At \(t = 20\): \(B'(20) = 1200 - 1200 = 0\). Burn rate has peaked — daily spend is stationary. Step 4. At \(t = 30\): \(B'(30) = 1200 - 1800 = -600\). Spend is decelerating; campaign exhaustion has set in. PM action: this is the right window to either inject fresh creative or end the campaign cleanly.

## 4. Common Pitfalls

- Confusing the average rate of change \((\Delta f/\Delta x)\) with the instantaneous rate \(f'(x)\). A/B tests give you the average; calculus gives you the instantaneous.
- Forgetting the sign: \(f'(x) = 0\) marks a stationary point, but it could be a maximum, a minimum, or an inflection. You need the second derivative.
- Linear extrapolation from a single derivative. \(f'(x_0)\) is valid in a neighborhood of \(x_0\); using it to predict behavior far away is what gets pricing teams in trouble.
- Treating non-differentiable points (kinks at regulatory caps, step functions in tier pricing) as if they had derivatives.

## 5. PM Relevance

| PM Relevance |
|---|
| - Why a PM must master this: Every "what if I move this lever?" conversation is a derivative question. If you cannot estimate marginal impact, you cannot prioritize roadmap items by ROI. |
| - Metrics to own: Marginal revenue per bp of MDR, marginal CAC per rupee of incentive, marginal latency per added feature in the checkout flow. |
| - Strategic tradeoffs: Large step sizes capture more signal per A/B test but risk crossing nonlinear regions; small steps are statistically safe but slow. |

- 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "I think in derivatives. When I argue for a Paytm pricing change, I quote the marginal revenue per bp at the current operating point and the second derivative — so my VP knows not just direction but whether we're near a local max. Georgia Tech's matrix calculus class made that vocabulary native."

### 6. NumPy-Only Python Snippet

```
import numpy as np

# Example A: R(m) = 1000m - 8m^2; find R'(50), optimum m*, and max revenue
def R(m):  return 1000.0 * m - 8.0 * m * m
def Rp(m): return 1000.0 - 16.0 * m

m0 = 50.0
print("R(50)  =", R(m0))
print("R'(50) =", Rp(m0), "INR per bp")

# Verify with numerical derivative (central difference)
h = 1e-4
num = (R(m0 + h) - R(m0 - h)) / (2 * h)
print("Numerical R'(50) =", num)

m_star = 1000.0 / 16.0
print("Optimum MDR m* =", m_star, "bps;  R(m*) =", R(m_star))

# Example B: B(t) = 5000 + 1200t - 30 t^2; burn-rate trajectory
t = np.arange(0, 41)
B  = 5000 + 1200 * t - 30 * t**2
Bp = 1200 - 60 * t
peak_t = t[np.argmax(B)]
zero_slope_t = t[np.argmin(np.abs(Bp))]
print("Peak spend day:", peak_t, "  zero-slope day:", zero_slope_t)
print("B'(5), B'(20), B'(30) =", Bp[5], Bp[20], Bp[30])
```

## 3.3 Partial Derivatives and the Gradient

### 1. Plain-English Intuition

Most product surfaces are not one-dimensional. Revenue depends on MDR and cashback and latency and onboarding fee. A partial derivative isolates one variable: "Holding all the others fixed, how does the output change if I nudge just this one?" Stacking all the partials into a vector gives you the gradient $\nabla f$ — an arrow pointing in the direction of steepest ascent, whose length is how steep that ascent is.

Everyday analogy — the hiker, again. You are still in fog. Now you can feel slope under each foot independently. Your left foot tells you east-west slope; your right foot tells you north-south slope. The vector sum of those is the gradient — the direction the entire ground tilts most steeply uphill. To descend, walk opposite the gradient.

Fintech analogy — the Paytm pricing knob, generalized. Your dashboard has four sliders: MDR ($m$), cashback rate ($c$), onboarding fee ($o$), latency budget ($\ell$). Contribution margin $\Pi(m,c,o,\ell)$ is some function of all four. $\partial \Pi/\partial m$ tells you marginal margin per bp of MDR with cashback, fee, and latency held fixed. The full gradient $\nabla \Pi$ is your "where to push next" arrow across the entire control panel. Every modern recommender system, pricing engine, and credit model at Paytm computes some version of this gradient millions of times per day.

### 2. Formal Mathematical Definition

For $f:\mathbb{R}^n \to \mathbb{R}$, the partial derivative with respect to $x_i$:

$$ \frac{\partial f}{\partial x_i}(\mathbf{x}) \;=\; \lim_{h\to 0} \frac{f(x_1, \dots, x_i + h, \dots, x_n) - f(\mathbf{x})}{h}. $$

Plain English: "Same definition as a single-variable derivative, with every other coordinate frozen."

The gradient stacks all partials:

\[ \nabla f(\mathbf{x}) \;=\; \begin{bmatrix} \partial f/\partial x_1 \\ \partial f/\partial x_2 \\ \vdots \\ \partial f/\partial x_n \end{bmatrix} \in \mathbb{R}^n. \]

Plain English: "A column vector of slopes, one per input dimension."

Key property — the gradient points uphill. For any unit vector \(\mathbf{u}\),

\[ \frac{d}{dt} f(\mathbf{x} + t \mathbf{u}) \Big|_{t=0} \;=\; \nabla f(\mathbf{x})^\top \mathbf{u}. \]

By the Cauchy–Schwarz inequality, this is maximized when \(\mathbf{u}\) is parallel to \(\nabla f(\mathbf{x})\) and minimized when it is antiparallel. Therefore the negative gradient is the direction of steepest descent. That single fact is the foundation of every neural network you will ever ship.

```
| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| \(\mathbf{x}\) | Input vector | \(\mathbb{R}^n\) | Current setting of all product levers |
| \(\partial f / \partial x_i\) | Partial derivative | scalar | Marginal impact of lever \(i\) alone |
| \(\nabla f(\mathbf{x})\) | Gradient | \(\mathbb{R}^n\) (column) | Marginal-impact arrow across all levers |
| \(\mathbf{u}\) | Unit direction | \(\mathbb{R}^n\), \(\|\mathbf{u}\|=1\) | A proposed combo move (e.g.,
+MDR & −cashback) |
| \(\nabla f^\top \mathbf{u}\) | Directional derivative | scalar | Rate of KPI change along that combo move |
```

## 3. Fintech Worked Numeric Examples

Example A — Contribution margin in two levers. Let \(\Pi(m, c) = 1000 m - 8 m^2 - 300 c + 4 c^2 - 0.5 m c\), where \(m\) is MDR in bps, \(c\) is cashback rate in bps, \(\Pi\) is monthly contribution margin per merchant in INR.

Step 1. Partial w.r.t. MDR: \(\partial \Pi / \partial m = 1000 - 16 m - 0.5 c\). Step 2. Partial w.r.t. cashback: \(\partial \Pi / \partial c = -300 + 8 c - 0.5 m\). Step 3. Evaluate at \((m,c) = (50, 20)\): \(\partial \Pi/\partial m = 1000 - 800 - 10 = 190\) INR per bp of MDR. \(\partial \Pi/\partial c = -300 + 160 - 25 = -165\) INR per bp of cashback (i.e., raising cashback destroys margin here). Step 4. Gradient: \(\nabla \Pi(50, 20) = (190, -165)^\top\). Magnitude \(= \sqrt{190^2 + 165^2} = \sqrt{36100 + 27225} = \sqrt{63325} \approx 251.6\). Step 5. Steepest-ascent unit direction: \((190, -165)/251.6 = (0.755, -0.656)\). PM reading: "for the next sprint, raise MDR by roughly 0.755 units for every 0.656 units we cut cashback."

Example B — Three-lever loan-pricing margin. \(\Pi(r, f, d) = 200 r - 5 r^2 + 80 f - 2 f^2 - 0.1 d^2 + 0.3 r f - 0.05 r d\), where \(r\) is interest rate (%), \(f\) is origination fee (%), \(d\) is default-buffer reserve (% of book), units in lakh INR. Step 1. \(\partial \Pi / \partial r = 200 - 10 r + 0.3 f - 0.05 d\). Step 2. \(\partial \Pi / \partial f = 80 - 4 f + 0.3 r\). Step 3. \(\partial \Pi / \partial d = -0.2 d - 0.05 r\). Step 4. At \((r, f, d) = (12, 1.5, 8)\): \(\partial \Pi/\partial r = 200 - 120 + 0.45 - 0.4 = 80.05\). \(\partial \Pi/\partial f = 80 - 6 + 3.6 = 77.6\). \(\partial \Pi/\partial d = -1.6 - 0.6 = -2.2\). Step 5. Gradient \(= (80.05, 77.6, -2.2)^\top\). Default buffer is barely moving the margin, so optimizing it in this sprint is low-leverage; the leverage is on \(r\) and \(f\).

## 4. Common Pitfalls

- Computing partials but forgetting to hold other variables constant when interpreting them; coupling terms like \(0.3 r f\) make the partial depend on the held variable.
- Assuming the gradient direction is universally correct. The gradient is locally accurate — it can rotate dramatically after one step in a curved landscape.
- Confusing the gradient (column vector for a scalar output) with the Jacobian (matrix for a vector output). They are different objects.
- Ignoring units. \(\partial R/\partial m\) is in "currency per bp" — mixing it with "currency per percent" is a real failure mode in product reviews.

## 5. PM Relevance

| PM Relevance |
|---|

| |
|---|
| - Why a PM must master this: Modern product surfaces are high-dimensional. A roadmap-prioritization argument is, at heart, a claim about which partial derivatives are largest. |
| - Metrics to own: Per-lever marginal KPI; gradient magnitude as "headroom"; cosine alignment between this quarter's roadmap and the contribution-margin gradient. |
| - Strategic tradeoffs: Aggressively chasing the gradient maximizes short-term gain but may lock you into a local optimum; periodic "perpendicular" experiments preserve exploration. |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "When I ran the Soundbox margin review, I computed a 4-lever gradient on contribution margin: MDR, cashback, soundbox rental, onboarding fee. Two partials dwarfed the rest. We refocused the OKRs on those two — a Georgia-Tech-flavored move that cut roadmap noise and lifted margin 3.4% in two quarters." |

### 6. NumPy-Only Python Snippet

```
import numpy as np

# Example A: Pi(m, c) = 1000m - 8m^2 - 300c + 4c^2 - 0.5 m c
def Pi(x):
    m, c = x
    return 1000*m - 8*m*m - 300*c + 4*c*c - 0.5*m*c

def grad_Pi(x):
    m, c = x
    return np.array([1000 - 16*m - 0.5*c,
                     -300 + 8*c - 0.5*m])

x0 = np.array([50.0, 20.0])
g  = grad_Pi(x0)
print("Pi(50,20) =", Pi(x0))
print("Gradient  =", g)
print("Magnitude =", np.linalg.norm(g))
print("Steepest-ascent unit dir =", g / np.linalg.norm(g))

# Cross-check via finite differences
h = 1e-5
fd = np.array([(Pi(x0 + h*np.eye(2)[i]) - Pi(x0 - h*np.eye(2)[i]))/(2*h) for i in range(2)])
print("Finite-diff gradient    =", fd)

# Example B: three-lever loan margin
def grad_loan(rfd):
    r, f, d = rfd
    return np.array([200 - 10*r + 0.3*f - 0.05*d,
                     80  - 4*f  + 0.3*r,
                     -0.2*d - 0.05*r])
print("Loan gradient at (12, 1.5, 8) =", grad_loan(np.array([12.0, 1.5, 8.0])))
```

## 3.4 Directional Derivatives and the Geometry of Gradients

### 1. Plain-English Intuition

The gradient gives you the direction of fastest uphill change. But sometimes you cannot move freely — regulation, budget, or strategy pins you to a specific direction. The directional derivative asks: "Given that I must move along this particular direction, how fast does my KPI change?"

Everyday analogy — GPS rerouting. You ask Google Maps to take you from Connaught Place to Gurgaon. The straight-line gradient says "due south-west." But there is no road there. The directional derivative along the actual highway tells you your effective rate of progress — projected onto the path you are forced to follow.

Fintech analogy — regulatory-constrained pricing. The RBI caps your blended MDR at a regulatory ceiling. The unconstrained gradient says "raise MDR aggressively." But you can only move along the constraint surface. The directional derivative along that surface tells you how much margin you can still squeeze without violating the cap.

### 2. Formal Mathematical Definition

For \(f:\mathbb{R}^n \to \mathbb{R}\) differentiable, the directional derivative at \(\mathbf{x}\) in unit direction \(\mathbf{u}\):

\[ D_{\mathbf{u}} f(\mathbf{x}) \;=\; \lim_{t \to 0} \frac{f(\mathbf{x} + t \mathbf{u}) - f(\mathbf{x})}{t} \;=\; \nabla f(\mathbf{x})^\top \mathbf{u}. \]

Plain English: "Project the gradient onto your direction — that scalar is your rate of change along it."

Cauchy–Schwarz consequence. Since \(|\nabla f^\top \mathbf{u}| \le \|\nabla f\| \cdot \|\mathbf{u}\| = \|\nabla f\|\), with equality when \(\mathbf{u} = \pm \nabla f/\|\nabla f\|\):

- Maximum directional derivative \(= +\|\nabla f\|\), in direction \(+\nabla f/\|\nabla f\|\) — steepest ascent.
- Minimum directional derivative \(= -\|\nabla f\|\), in direction \(-\nabla f/\|\nabla f\|\) — steepest descent.
- Zero directional derivative when \(\mathbf{u} \perp \nabla f\) — these directions form the level set / contour line.

| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| \(\mathbf{u}\) | Unit direction | \(\mathbb{R}^n\), \(\|\mathbf{u}\|=1\) | A constrained move (e.g., along a budget line) |
| \(D_{\mathbf{u}} f\) | Directional derivative | scalar | Effective rate of KPI change along the constraint |
| \(\nabla f^\top \mathbf{u}\) | Inner product form | scalar | Projection of gradient onto the constrained path |
| Level set | \(\{\mathbf{x}: f(\mathbf{x}) = c\}\) | \((n-1)\)-dim surface | Iso-KPI curve — settings that yield identical KPI |

## 3. Fintech Worked Numeric Examples

Example A — MDR/cashback constraint line. Using \(\Pi(m, c)\) from §3.3 with \(\nabla \Pi(50, 20) = (190, -165)\). Suppose the merchant-relations team commits to a policy of moving MDR and cashback in lockstep: \(\Delta c = \Delta m\), i.e. direction \(\mathbf{u} = (1, 1)/\sqrt{2} \approx (0.7071, 0.7071)\). Step 1. \(D_{\mathbf{u}} \Pi = 190(0.7071) + (-165)(0.7071) = 0.7071 \cdot 25 = 17.68\) INR per unit step. Step 2. Compare with unconstrained steepest ascent magnitude \(251.6\) — the lockstep policy captures only 7% of the available margin slope. PM action: argue against the lockstep policy or quantify what we lose by it. Step 3. Find a level-set direction (zero directional derivative). We need \(190 u_1 - 165 u_2 = 0 \Rightarrow u_2 = (190/165) u_1\). Normalized: \((u_1, u_2) = (0.655, 0.755)\). Moves in this direction keep margin unchanged.

Example B — Cashback budget rebalancing. Daily marketing spend split between cashback (\(b_1\)) and notification push (\(b_2\)), with installs \(I(b_1, b_2) = 12 \sqrt{b_1} + 8 \sqrt{b_2}\), units in lakh INR for \(b\) and thousand installs for \(I\). Step 1. \(\partial I / \partial b_1 = 6 / \sqrt{b_1}\); \(\partial I / \partial b_2 = 4 / \sqrt{b_2}\). Step 2. At \((b_1, b_2) = (9, 4)\): \(\nabla I = (6/3, 4/2) = (2, 2)\). Step 3. Budget-neutral reallocation direction: shift money from \(b_2\) to \(b_1\), so \(\mathbf{u} = (+1, -1)/\sqrt{2}\). Step 4. \(D_{\mathbf{u}} I = 2(0.7071) + 2(-0.7071) = 0\) — at this point the budget split is install-optimal; reallocating in either direction loses installs at first order. Step 5. Now check at \((b_1, b_2) = (4, 9)\): \(\nabla I = (3, 1.333)\). \(D_{\mathbf{u}} I = 3(0.7071) - 1.333(0.7071) = 1.18\). Here, shifting one lakh from \(b_2\) to \(b_1\) gains 1.18 thousand installs at first order. PM playbook: rebalance toward cashback until the gradient components equalize.

## 4. Common Pitfalls

- Forgetting to normalize the direction. If \(\mathbf{u}\) is not a unit vector, \(\nabla f^\top \mathbf{u}\) confounds rate with magnitude.
- Reporting the gradient's magnitude as if it were the achievable improvement, when constraints force you onto a slower direction.
- Confusing "zero directional derivative in some direction" (always exists when \(\nabla f \neq 0\)) with "zero gradient" (true critical point).
- Using directional derivatives to extrapolate far along curved level sets.

## 5. PM Relevance

| PM Relevance |
|---|

| |
|---|
| - Why a PM must master this: Real product decisions live under constraints (legal, brand, capacity). You must quantify what fraction of the unconstrained gradient your chosen direction captures. |
| - Metrics to own: Constraint-projected gradient, "gradient efficiency" = $D_{\mathbf{u}} f / \|\nabla f\|$ as a ratio in [-1, 1]. |
| - Strategic tradeoffs: Tighter constraints buy stakeholder buy-in but cost gradient efficiency; loose constraints maximize math but invite political risk. |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "When legal capped our MDR moves, I reframed the optimization as a directional-derivative problem along the constraint surface. I could show our VP exactly which 73% of the unconstrained gradient we were keeping. That precision — Georgia-Tech-grade — turned an emotional debate into a number." |

### 6. NumPy-Only Python Snippet

```
import numpy as np

# Example A: directional derivatives of Pi at (50, 20)
g = np.array([190.0, -165.0])
u_lockstep = np.array([1.0, 1.0]); u_lockstep /= np.linalg.norm(u_lockstep)
u_steep   = g / np.linalg.norm(g)
u_level   = np.array([g[1], -g[0]]); u_level /= np.linalg.norm(u_level)

for name, u in [("lockstep", u_lockstep), ("steepest", u_steep), ("level-set", u_level)]:
    print(f"D_u Pi along {name:>9} dir = {g @ u:+.4f}")

# Example B: install function rebalancing
def grad_I(b):
    b1, b2 = b
    return np.array([6/np.sqrt(b1), 4/np.sqrt(b2)])

for b in [np.array([9.0, 4.0]), np.array([4.0, 9.0]), np.array([16.0, 16.0])]:
    g = grad_I(b)
    u = np.array([1.0, -1.0]) / np.sqrt(2)
    print(f"At b={b}, grad={g}, reallocation directional deriv = {g @ u:+.4f}")
```

## 3.5 The Chain Rule in High-Dimensional Tensor Spaces

### 1. Plain-English Intuition

The chain rule says: if your output depends on an intermediate that depends on an input, multiply the rates. Rates compound like fractions cancelling.

Everyday analogy. Petrol price moves the cost of trucking; trucking costs move retail prices; retail prices move your monthly budget. The rate at which your budget moves per rupee of crude oil is the product of three sensitivities. That is the chain rule.

Fintech analogy. Customer LTV depends on retention, which depends on app NPS, which depends on payment success rate, which depends on backend latency. Your CFO asks: "If we cut p99 latency by 50ms, what is the LTV impact per merchant?" That is one chain-rule computation with four links. Without the chain rule, you can only guess. With it, you can put a number on the CFO's table.

### 2. Formal Mathematical Definition

Scalar chain rule. If $y = f(g(x))$, then

$$ \frac{dy}{dx} = f'(g(x)) \cdot g'(x). $$

Multivariate chain rule. For $\mathbf{y} = f(\mathbf{u})$ where $\mathbf{u} = g(\mathbf{x})$, with $f:\mathbb{R}^k \to \mathbb{R}^m$ and $g:\mathbb{R}^n \to \mathbb{R}^k$:

$$ \underbrace{\frac{\partial \mathbf{y}}{\partial \mathbf{x}}}_{m \times n} \;=\; \underbrace{\frac{\partial \mathbf{y}}{\partial \mathbf{u}}}_{m \times k} \;\cdot\; \underbrace{\frac{\partial \mathbf{u}}{\partial \mathbf{x}}}_{k
```
```

\times n}. \]

Plain English: "Jacobian of the outer times Jacobian of the inner — matrix multiplication."

Tensor chain rule (the engine of backprop). In a deep network, layer \(\ell\) has activation \(\mathbf{a}^{(\ell)} = \sigma(W^{(\ell)} \mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)})\). Loss \(L\) at the top. Then for any layer \(\ell\),

\[ \frac{\partial L}{\partial W^{(\ell)}} \;=\; \underbrace{\frac{\partial L}{\partial \mathbf{a}^{(L)}}}_{\text{top gradient}} \cdot \prod_{k=L}^{\ell+1} \frac{\partial \mathbf{a}^{(k)}}{\partial \mathbf{a}^{(k-1)}} \cdot \frac{\partial \mathbf{a}^{(\ell)}}{\partial W^{(\ell)}}. \]

Plain English: "Backprop is the chain rule applied layer by layer, right to left."

Derivation sketch (two-link case). Suppose \(y = f(u)\) and \(u = g(x)\). Define \(\Delta u = g(x + h) - g(x)\). Then \(\frac{f(g(x+h)) - f(g(x))}{h} = \frac{f(g(x) + \Delta u) - f(g(x))}{\Delta u} \cdot \frac{\Delta u}{h}\). As \(h \to 0\), the first factor \(\to f'(g(x))\) and the second \(\to g'(x)\). QED. The multivariate generalization is the same argument applied componentwise.

```
| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| \(\mathbf{x}\) | Bottom-level input | \(\mathbb{R}^n\) | Operational lever (e.g., backend p99 latency) |
| \(\mathbf{u} = g(\mathbf{x})\) | Intermediate | \(\mathbb{R}^k\) | Mid-funnel metric (e.g., payment success
rate) |
| \(\mathbf{y} = f(\mathbf{u})\) | Top-level output | \(\mathbb{R}^m\) | Business KPI (e.g., LTV) |
| \(\partial \mathbf{u}/\partial \mathbf{x}\) | Inner Jacobian | \(k \times n\) | "Engineering-to-funnel"
sensitivity matrix |
| \(\partial \mathbf{y}/\partial \mathbf{u}\) | Outer Jacobian | \(m \times k\) | "Funnel-to-business"
sensitivity matrix |
| \(\partial \mathbf{y}/\partial \mathbf{x}\) | Composed Jacobian | \(m \times n\) | "Engineering-to-
business" KPI sensitivity |
```

## 3. Fintech Worked Numeric Examples

Example A — LTV per millisecond of latency. Let \(s = 0.99 - 0.0008\, \ell\) be payment success rate as a function of p99 latency \(\ell\) (ms). Let \(n = 50 s^2\) be monthly transactions per active merchant. Let \(L = 12 n - 0.04 n^2\) be 12-month LTV in thousand INR. Compute \(dL/d\ell\) at \(\ell = 200\) ms. Step 1. \(s(200) = 0.99 - 0.16 = 0.83\). \(ds/d\ell = -0.0008\). Step 2. \(n(0.83) = 50(0.6889) = 34.445\). \(dn/ds = 100 s = 83\). Step 3. \(L(34.445) = 12(34.445) - 0.04(34.445)^2 = 413.34 - 47.46 = 365.88\). \(dL/dn = 12 - 0.08 n = 12 - 2.756 = 9.244\). Step 4. Chain: \(dL/d\ell = (dL/dn)(dn/ds)(ds/d\ell) = 9.244 \times 83 \times (-0.0008) = -0.6138\). LTV drops about 0.61 thousand INR per ms of added latency at this operating point. Step 5. Cutting latency by 50ms gains roughly \(50 \times 0.6138 = 30.69\) thousand INR LTV per merchant per year. Multiply by your active-merchant base and you have your engineering ROI in hard currency.

Example B — Credit-model gradient via the chain rule. A two-layer sigmoid network predicts default probability. Inputs \(\mathbf{x} \in \mathbb{R}^3\) (income, tenure, prior defaults). Layer 1: \(\mathbf{z}^{(1)} = W_1 \mathbf{x} + \mathbf{b}_1\), \(\mathbf{a}^{(1)} = \sigma(\mathbf{z}^{(1)})\). Layer 2: \(z^{(2)} = \mathbf{w}_2^\top \mathbf{a}^{(1)} + b_2\), \(\hat{y} = \sigma(z^{(2)})\). Loss \(L = -[y \log \hat{y} + (1-y) \log(1 - \hat{y})]\).

Use \(W_1 = \begin{bmatrix} 0.5 & -0.2 & 0.1 \\ 0.3 & 0.4 & -0.6 \end{bmatrix}\), \(\mathbf{b}_1 = (0, 0)\), \(\mathbf{w}_2 = (1.0, -1.5)^\top\), \(b_2 = 0.2\), \(\mathbf{x} = (1, 1, 0)^\top\), \(y = 1\).

Step 1. \(\mathbf{z}^{(1)} = (0.5 - 0.2 + 0, 0.3 + 0.4 + 0) = (0.3, 0.7)\). Step 2. \(\mathbf{a}^{(1)} = (\sigma(0.3), \sigma(0.7)) = (0.5744, 0.6682)\). Step 3. \(z^{(2)} = 1.0(0.5744) + (-1.5)(0.6682) + 0.2 = 0.5744 - 1.0023 + 0.2 = -0.2279\). Step 4. \(\hat{y} = \sigma(-0.2279) = 0.4433\). Loss \(L = -\log(0.4433) = 0.8136\). Step 5. Backprop. \(\partial L/\partial z^{(2)} = \hat{y} - y = -0.5567\) (standard derivation for cross-entropy + sigmoid). Step 6. \(\partial L/\partial \mathbf{w}_2 = (\partial L/\partial z^{(2)}) \mathbf{a}^{(1)} = -0.5567 \cdot (0.5744, 0.6682) = (-0.3198, -0.3720)\). Step 7. \(\partial L/\partial \mathbf{a}^{(1)} = (\partial L/\partial z^{(2)}) \mathbf{w}_2 = -0.5567 \cdot (1.0, -1.5) = (-0.5567, 0.8351)\). Step 8. \(\partial L/\partial \mathbf{z}^{(1)} = \partial L/\partial \mathbf{a}^{(1)} \odot \sigma'(\mathbf{z}^{(1)})\). \(\sigma'(0.3) = 0.5744(1 - 0.5744) = 0.2444\); \(\sigma'(0.7) = 0.6682(1 - 0.6682) = 0.2217\). So \(\partial L/\partial \mathbf{z}^{(1)} = (-0.5567 \cdot 0.2444,\; 0.8351 \cdot 0.2217) = (-0.1361, 0.1851)\). Step 9. \(\partial L/\partial W_1 = (\partial L/\partial \mathbf{z}^{(1)}) \mathbf{x}^\top = \begin{bmatrix} -0.1361 \\ 0.1851 \end{bmatrix} (1,1,0) = \begin{bmatrix} -0.1361 & -0.1361 & 0 \\ 0.1851

& 0.1851 & 0 \end{bmatrix}\).

These are the exact gradients PyTorch's autograd would compute — we just derived them by hand with the chain rule.

## 4. Common Pitfalls

- Shape errors. The composed Jacobian dimensions only line up if you respect outer\(_{m\times k}\) \(\times\) inner\(_{k\times n}\). Many bugs come from accidental transposes.
- Forgetting nonlinearity derivatives. Skipping \(\sigma'\) is the #1 source of silently-wrong backprop.
- Confusing row-major and column-major gradient layouts. Pick a convention (denominator layout is most common in ML) and stick to it.
- Treating linked KPIs as independent. If app NPS depends on success rate which depends on latency, you cannot just add their partials — you must chain them.

## 5. PM Relevance

| PM Relevance |
| --- |
| - Why a PM must master this: The chain rule is how engineering work converts into business KPIs. Without it, you cannot defend infra budgets against marketing budgets. |
| - Metrics to own: End-to-end sensitivity (e.g., INR-LTV per ms of latency); per-link attribution along the chain. |
| - Strategic tradeoffs: Long chains amplify gains but also amplify estimation error; short, instrumented chains are more defensible. |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "When SRE asked for a quarter on p99 latency reduction, I built the chain — latency  success rate  transactions  LTV — and quantified ROI per ms. Backprop is the same chain rule that I used to defend infra spend at Paytm; Georgia Tech's ML course made the math second nature." |

## 6. NumPy-Only Python Snippet

```
import numpy as np

# Example A: chain rule through latency -> success -> txns -> LTV
ell = 200.0
s   = 0.99 - 0.0008 * ell;   ds_dell = -0.0008
n   = 50 * s**2;             dn_ds   = 100 * s
L   = 12 * n - 0.04 * n**2;  dL_dn   = 12 - 0.08 * n
dL_dell = dL_dn * dn_ds * ds_dell
print(f"s={s:.4f}, n={n:.4f}, L={L:.4f}")
print(f"dL/dell = {dL_dell:.4f} (thousand INR per ms)")

# Example B: hand-coded backprop through a 2-layer sigmoid net
def sigmoid(z): return 1.0 / (1.0 + np.exp(-z))

W1 = np.array([[0.5, -0.2, 0.1], [0.3, 0.4, -0.6]])
b1 = np.zeros(2)
w2 = np.array([1.0, -1.5])
b2 = 0.2
x  = np.array([1.0, 1.0, 0.0])
y  = 1.0

z1 = W1 @ x + b1;   a1 = sigmoid(z1)
z2 = w2 @ a1 + b2;  yhat = sigmoid(z2)
loss = -(y * np.log(yhat) + (1 - y) * np.log(1 - yhat))
print("yhat =", yhat, "  loss =", loss)

# Backprop
dL_dz2 = yhat - y
dL_dw2 = dL_dz2 * a1
dL_da1 = dL_dz2 * w2
dL_dz1 = dL_da1 * a1 * (1 - a1)        # sigmoid' = a(1-a)
dL_dW1 = np.outer(dL_dz1, x)
print("dL/dw2 =", dL_dw2)
print("dL/dW1 =\n", dL_dW1)
```

## 3.6 The Jacobian Matrix

### 1. Plain-English Intuition

When the output is itself a vector — say, your dashboard produces several KPIs at once — the right object is not the gradient but the Jacobian: one row per output, one column per input. Each entry is "how does output \(i\) respond to input \(j\)."

Everyday analogy. A music mixer with multiple sliders (bass, treble, gain) and multiple meters (left channel, right channel, sub). Each slider affects each meter to a different degree. The full table of slider-to-meter responses is the Jacobian of the mixer.

Fintech analogy. Your pricing levers (MDR, cashback, onboarding fee) drive several KPIs simultaneously (revenue, merchant retention, NPS). The Jacobian is the three-by-three table of partial derivatives — and reading it is exactly how a senior PM compares options when no single lever optimizes all KPIs at once.

### 2. Formal Mathematical Definition

For \(\mathbf{f}:\mathbb{R}^n \to \mathbb{R}^m\) with components \(f_1, \ldots, f_m\), the Jacobian is

\[ J_{\mathbf{f}}(\mathbf{x}) \;=\; \begin{bmatrix} \dfrac{\partial f_1}{\partial x_1} & \cdots & \dfrac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \dfrac{\partial f_m}{\partial x_1} & \cdots & \dfrac{\partial f_m}{\partial x_n} \end{bmatrix} \in \mathbb{R}^{m \times n}. \]

Plain English: "Matrix whose \(i,j\) entry is the partial of the \(i\)-th output with respect to the \(j\)-th input."

Key identities.

- For scalar-output \(f\) (\(m = 1\)), the Jacobian is the gradient transpose: \(J_f = (\nabla f)^\top\).
- Composition is matrix multiplication: \(J_{\mathbf{f} \circ \mathbf{g}}(\mathbf{x}) = J_{\mathbf{f}}(\mathbf{g}(\mathbf{x})) \cdot J_{\mathbf{g}}(\mathbf{x})\).
- Local linearization: \(\mathbf{f}(\mathbf{x} + \mathbf{h}) \approx \mathbf{f}(\mathbf{x}) + J_{\mathbf{f}}(\mathbf{x}) \mathbf{h}\) for small \(\mathbf{h}\).
- Change-of-variables determinant: \(\det(J)\) measures the local volume-scaling factor — this is how change of variables in integrals (and in normalizing flows) works.

| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| \(\mathbf{f}\) | Vector-valued function | \(\mathbb{R}^n \to \mathbb{R}^m\) | Multi-KPI dashboard from multi-lever inputs |
| \(J_{\mathbf{f}}\) | Jacobian matrix | \(m \times n\) | Full lever-to-KPI sensitivity table |
| Row \(i\) of \(J\) | Gradient of \(f_i\) | \(1 \times n\) | "How do all levers affect KPI \(i\)?" |
| Column \(j\) of \(J\) | Per-input sensitivity | \(m \times 1\) | "How does lever \(j\) affect all KPIs?" |
| \(\det(J)\) | Determinant (when \(m=n\)) | scalar | Local volume / phase-space rescaling |

### 3. Fintech Worked Numeric Examples

Example A — Three KPIs, three levers. Let \((m, c, o)\) be MDR, cashback, onboarding fee in bps. Define KPIs: \(R(m,c,o) = 1000 m - 8 m^2 - 5 c + 2 o\) (revenue, INR/merchant/month), \(N(m,c,o) = 80 - 0.6 m + 1.2 c - 0.4 o\) (NPS proxy), \(K(m,c,o) = 0.90 - 0.003 m + 0.002 c - 0.001 o\) (retention rate).

Step 1. Compute each row. Row \(R\): \(\partial R/\partial m = 1000 - 16 m\), \(\partial R/\partial c = -5\), \(\partial R/\partial o = 2\). Row \(N\): \(\partial N/\partial m = -0.6\), \(\partial N/\partial c = 1.2\), \(\partial N/\partial o = -0.4\). Row \(K\): \(\partial K/\partial m = -0.003\), \(\partial K/\partial c = 0.002\), \(\partial K/\partial o = -0.001\). Step 2. At \((m, c, o) = (50, 20, 30)\): \(\partial R/\partial m = 1000 - 800 = 200\). So \[ J = \begin{bmatrix} 200 & -5 & 2 \\ -0.6 & 1.2 & -0.4 \\ -0.003 & 0.002 & -0.001 \end{bmatrix}. \] Step 3. Predict KPI shift for a planned move \(\Delta = (+2, -3, +1)^\top\) (i.e., +2 bps MDR, 3 bps

cashback, +1 bp onboarding fee). $J \Delta$: Row 1: $200(2) + (-5)(-3) + 2(1) = 400 + 15 + 2 = 417$ INR. Row 2: $-0.6(2) + 1.2(-3) + (-0.4)(1) = -1.2 - 3.6 - 0.4 = -5.2$ NPS points. Row 3: $-0.003(2) + 0.002(-3) + (-0.001)(1) = -0.006 - 0.006 - 0.001 = -0.013$ retention. Step 4. PM read: revenue gains $\sim$417 INR/merchant/month but NPS drops 5.2 points and retention drops 1.3 pp. Is that trade acceptable? That is the trade-off conversation the Jacobian forces you to have.

Example B — Coordinate transform Jacobian for a 2D segmentation map. Suppose merchant features $(m, c)$ (MDR, cashback) are remapped to "blended take-rate" $t = m - c$ and "spread" $s = m + 2c$. Jacobian: $$ J = \begin{bmatrix} \partial t/\partial m & \partial t/\partial c \\ \partial s/\partial m & \partial s/\partial c \end{bmatrix} = \begin{bmatrix} 1 & -1 \\ 1 & 2 \end{bmatrix}. $$ Step 1. $\det J = (1)(2) - (-1)(1) = 3$. So a unit area in $(m, c)$ space maps to 3 units in $(t, s)$ space. Step 2. If a model is calibrated in $(t, s)$ units and reports density $p(t, s)$, the equivalent density in original units is $p(m, c) = p(t(m,c), s(m,c)) \cdot |\det J| = 3\, p$. Forgetting this factor is a classic error in change-of-variables — and a real one in calibrating credit-risk densities.

## 4. Common Pitfalls

- Swapping rows and columns. Convention: rows index outputs, columns index inputs.
- Confusing $J$ with the Hessian (Jacobian of the gradient — see next section).
- Forgetting that the Jacobian is local; using a single $J$ to extrapolate far is a linearization that breaks under curvature.
- Computing the Jacobian inefficiently when the structure (sparsity, low-rank) admits a cheaper representation — a real cost issue in production credit models.

## 5. PM Relevance

| PM Relevance |
|---|
| - Why a PM must master this: Multi-KPI tradeoffs are the daily job of a senior PM. The Jacobian is the formal trade-off table. |
| - Metrics to own: Per-lever-per-KPI sensitivity matrix; KPI-correlation matrix from $J J^\top$; rank of $J$ (signals which levers are redundant). |
| - Strategic tradeoffs: Optimizing one row often degrades another; rank-deficient Jacobians warn of "we only have one independent lever, not three." |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "When stakeholders argued whether MDR or cashback was the right knob, I built the Jacobian of the four-KPI dashboard. The column norms told us cashback dominates NPS while MDR dominates revenue — Georgia Tech matrix calculus turned a religious war into a one-pager." |

## 6. NumPy-Only Python Snippet

```
import numpy as np

# Example A: 3-KPI, 3-lever Jacobian at (m,c,o) = (50, 20, 30)
def kpis(x):
    m, c, o = x
    R = 1000*m - 8*m*m - 5*c + 2*o
    N = 80 - 0.6*m + 1.2*c - 0.4*o
    K = 0.90 - 0.003*m + 0.002*c - 0.001*o
    return np.array([R, N, K])

def jacobian_fd(f, x, h=1e-5):
    fx = f(x)
    J = np.zeros((fx.size, x.size))
    for j in range(x.size):
        e = np.zeros_like(x); e[j] = h
        J[:, j] = (f(x + e) - f(x - e)) / (2 * h)
    return J

x0 = np.array([50.0, 20.0, 30.0])
J  = jacobian_fd(kpis, x0)
print("Jacobian J =\n", J)

delta = np.array([2.0, -3.0, 1.0])
print("Predicted KPI shift J@delta =", J @ delta)

# Example B: coordinate-transform Jacobian and its determinant
J_t = np.array([[1.0, -1.0], [1.0, 2.0]])
print("det(J) =", np.linalg.det(J_t))  # should be 3.0
```

## 3.7 The Hessian and Second-Order Information

### 1. Plain-English Intuition

The gradient tells you which way is up. The Hessian tells you how the slope is changing as you move — the curvature. A bowl-shaped landscape has a positive-definite Hessian everywhere; a saddle has indefinite Hessian; a flat valley has near-zero Hessian.

Everyday analogy — the hiker, with binoculars. With only the gradient, the hiker walks in fog. With the Hessian, the hiker has just enough vision to predict whether the next ridge is gentle or a sudden drop — and how big a step is safe.

Fintech analogy. Two MDR pricing strategies both have the same first-derivative impact today. But strategy A sits in a region where the second derivative is small (the landscape is flat — small overshoot is forgiven), while strategy B sits where the second derivative is large (one mis-step and revenue collapses). The Hessian distinguishes a robust optimum from a knife-edge.

### 2. Formal Mathematical Definition

For $f:\mathbb{R}^n \to \mathbb{R}$ twice-differentiable, the Hessian is

$$ H_f(\mathbf{x}) \;=\; \begin{bmatrix} \dfrac{\partial^2 f}{\partial x_1^2} & \cdots & \dfrac{\partial^2 f}{\partial x_1 \partial x_n} \\ \vdots & \ddots & \vdots \\ \dfrac{\partial^2 f}{\partial x_n \partial x_1} & \cdots & \dfrac{\partial^2 f}{\partial x_n^2} \end{bmatrix} \in \mathbb{R}^{n \times n}. $$

Plain English: "Matrix of all second partials. Equivalently, the Jacobian of the gradient."

Schwarz's theorem. If $f$ is $C^2$ (continuous second partials), the Hessian is symmetric: $\partial^2 f/(\partial x_i \partial x_j) = \partial^2 f/(\partial x_j \partial x_i)$.

Second-derivative test (multivariate). At a critical point $\mathbf{x}^\star$ where $\nabla f(\mathbf{x}^\star) = 0$:

- $H_f(\mathbf{x}^\star)$ positive definite $\Rightarrow$ local minimum.
- $H_f(\mathbf{x}^\star)$ negative definite $\Rightarrow$ local maximum.
- $H_f(\mathbf{x}^\star)$ indefinite (mixed-sign eigenvalues) $\Rightarrow$ saddle point.
- $H_f(\mathbf{x}^\star)$ singular (zero eigenvalue) $\Rightarrow$ test is inconclusive — use higher-order analysis.

Newton's method. Using both gradient and Hessian:

$$ \mathbf{x}_{t+1} = \mathbf{x}_t - H_f(\mathbf{x}_t)^{-1} \nabla f(\mathbf{x}_t). $$

This has quadratic convergence near a strict minimum — but each iteration costs $O(n^3)$ to invert the Hessian. In ML we usually approximate (L-BFGS, K-FAC) for scalability.

| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| $H_f$ | Hessian | $n \times n$, symmetric | Curvature table of KPI w.r.t. all lever pairs |
| Diagonal of $H$ | $\partial^2 f / \partial x_i^2$ | scalar | Self-curvature of each lever |
| Off-diagonal | $\partial^2 f / \partial x_i \partial x_j$ | scalar | Cross-curvature — how levers interact |
| Eigenvalues of $H$ | $\lambda_1, \ldots, \lambda_n$ | scalars | Principal curvatures — signs reveal min/max/saddle |
| Condition number | $\lambda_{\max}/\lambda_{\min}$ | scalar $\ge 1$ | Difficulty of optimization (anisotropy of bowl) |

## 3. Fintech Worked Numeric Examples

Example A — Hessian of contribution margin. Recall $\Pi(m,c) = 1000\,m - 8\,m^2 - 300\,c + 4\,c^2 - 0.5\,m\,c$. Step 1. First partials: $\partial \Pi/\partial m = 1000 - 16\,m - 0.5\,c$; $\partial \Pi/\partial c = -300 + 8\,c - 0.5\,m$. Step 2. Second partials: $\partial^2 \Pi/\partial m^2 = -16$; $\partial^2 \Pi/\partial c^2 = 8$; $\partial^2 \Pi/\partial m\,\partial c = -0.5$. Step 3. $H_\Pi = \begin{bmatrix} -16 & -0.5 \\ -0.5 & 8 \end{bmatrix}$. Step 4. Eigenvalues: trace $= -8$, determinant $= (-16)(8) - 0.25 = -128.25$. Since $\det < 0$, eigenvalues have opposite signs — this is a saddle, not a maximum. Step 5. Find the critical point. Solve $\nabla \Pi = 0$: $1000 - 16\,m - 0.5\,c = 0$, $-300 + 8\,c - 0.5\,m = 0 \Rightarrow c = (300 + 0.5\,m)/8$. Substitute: $1000 - 16\,m - 0.5\,(300 + 0.5\,m)/8 = 0$ $1000 - 16\,m - (18.75 + 0.03125\,m) = 0$ $981.25 - 16.03125\,m = 0$ $m \approx 61.20$. Then $c \approx (300 + 30.6)/8 = 41.33$. Step 6. PM lesson: this is a saddle. Cashback is convex (positive own-curvature) and MDR is concave; the unconstrained "stationary point" is not a maximum of margin in any useful sense. Without a constraint on cashback's lower bound, margin grows by cutting cashback toward zero. This is exactly the kind of insight that prevents PMs from over-modeling a problem and missing the obvious dominant direction.

Example B — Newton step for a logistic-regression credit risk loss. For a single-feature logistic regression $p(w) = \sigma(w \cdot 2)$ (feature value = 2), and label $y = 1$, loss $L(w) = -\log p(w) = \log(1 + e^{-2w})$. Step 1. $L'(w) = \dfrac{-2\,e^{-2w}}{1 + e^{-2w}} = -2(1 - p)$. Step 2. $L''(w) = -2 \cdot (-1) \cdot p \cdot (1-p) \cdot 2 \cdot (-1)$ . Now compute it cleanly. Since $p' = 2\,p\,(1 - p)$, $L''(w) = -2 \cdot (-p'(w)) = 2 \cdot 2\,p\,(1-p) = 4\,p\,(1-p)$. Step 3. At $w = 0$: $p = 0.5$. $L'(0) = -2(0.5) = -1$. $L''(0) = 4(0.25) = 1$. Step 4. Newton step: $w_1 = 0 - (-1)/1 = 1$. Step 5. At $w = 1$: $p = \sigma(2) = 0.8808$. $L'(1) = -2(0.1192) = -0.2384$. $L''(1) = 4(0.8808)(0.1192) = 0.4200$. Newton: $w_2 = 1 - (-0.2384)/0.4200 = 1.5676$. Step 6. At $w = 1.5676$: $p = \sigma(3.135) = 0.9583$. $L'(w) = -2(0.0417) = -0.0833$. $L''(w) = 4(0.9583)(0.0417) = 0.1599$. $w_3 = 1.5676 + 0.521 = 2.0886$. Step 7. Each iteration roughly doubles the precision — that is quadratic convergence. Compare to vanilla gradient descent which would take dozens of steps. PM read: when you can afford the Hessian, you converge in three iterations instead of thirty — the cost-benefit calculation for second-order methods.

## 4. Common Pitfalls

- Confusing "Hessian positive definite at one point" with "function is convex everywhere." Convexity is a global property.
- Inverting a singular or nearly-singular Hessian in Newton's method — leads to numerical explosion. Levenberg–Marquardt damping ($H + \lambda I$) fixes this.
- Trusting the second-derivative test when the Hessian is singular — it is silent there.
- Ignoring the off-diagonal terms (cross-curvature). These encode lever interactions and are often where the strategic insight lives.

## 5. PM Relevance

| PM Relevance |
| --- |
| - Why a PM must master this: First-order thinking ("which way to push") is necessary but insufficient. The Hessian tells you whether you are near a robust optimum or a knife-edge. |
| - Metrics to own: Condition number of the empirical KPI Hessian (how anisotropic is the trade space); eigenvalue signs (am I at a true max or a saddle). |
| - Strategic tradeoffs: Newton-style decisions converge fast but cost compute (and political capital); first-order decisions are cheap but slow. |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "When the team thought we were at a margin maximum, I computed the empirical Hessian. Determinant was negative — a saddle. That single result reopened the cashback debate and unlocked a 1.8% margin win. Georgia Tech's second-order optimization course taught me to never trust a flat gradient alone." |

## 6. NumPy-Only Python Snippet

```
import numpy as np

# Example A: Hessian and eigenvalues of Pi(m, c)
H = np.array([[-16.0, -0.5],
```

```
              [-0.5,   8.0]])
eigvals, eigvecs = np.linalg.eigh(H)
print("Hessian eigenvalues:", eigvals, " (mixed signs => saddle)")
print("det(H) =", np.linalg.det(H), "  trace =", np.trace(H))

# Find critical point by solving nabla Pi = 0
A = np.array([[16.0, 0.5], [0.5, -8.0]])
b = np.array([1000.0, -300.0])
crit = np.linalg.solve(A, b)
print("Critical point (m*, c*):", crit)

# Example B: Newton iteration on logistic-regression loss
def L (w): return np.log1p(np.exp(-2*w))
def Lp(w): p = 1/(1+np.exp(-2*w)); return -2*(1-p)
def Lpp(w): p = 1/(1+np.exp(-2*w)); return 4*p*(1-p)

w = 0.0
for k in range(5):
    step = Lp(w) / Lpp(w)
    w_new = w - step
    print(f"iter {k}: w={w:.6f}  L={L(w):.6f}  L'={Lp(w):+.6f}  L''={Lpp(w):.6f}  -> w_new={w_new:.6f}")
    w = w_new
```

## 3.8 Taylor Series — Local Polynomial Models

### 1. Plain-English Intuition

A Taylor series approximates a complicated function near a point with a polynomial built from its derivatives. The more terms you keep, the better the local fit. Linearization is the first-order Taylor approximation; the second-order Taylor expansion uses the Hessian to capture curvature.

Everyday analogy. A friend asks how your car will perform on tomorrow's drive. You answer: "It will go where I steer (zeroth order), at the speed I push the pedal (first order), and brake response is sharper on cold mornings (second order)." Each detail is a higher-order Taylor term about the operating point.

Fintech analogy. When you defend a pricing change, your finance partner asks: "What does revenue look like for MDR moves of 5 to +5 bps?" You hand over the Taylor expansion around today's MDR: revenue today + first-derivative × Δm + ½ × second-derivative × $\Delta m^2$. Three numbers explain the full curve in the relevant range — without forcing a re-simulation for every scenario.

### 2. Formal Mathematical Definition

Single-variable Taylor's theorem (with Lagrange remainder). If $f$ has $(n+1)$ continuous derivatives on $[a, x]$,

$$ f(x) \;=\; \sum_{k=0}^{n} \frac{f^{(k)}(a)}{k!} (x - a)^k \;+\; \underbrace{\frac{f^{(n+1)}(\xi)}{(n+1)!} (x - a)^{n+1}}_{R_n(x), \;\xi \in (a, x)}. $$

Plain English: "Function at $x$ equals a polynomial in $(x - a)$ of degree $n$ plus an error term controlled by the $(n+1)$-th derivative somewhere in between."

Multivariate second-order Taylor expansion. For $f:\mathbb{R}^n \to \mathbb{R}$, at point $\mathbf{x}_0$, small step $\mathbf{h}$:

$$ f(\mathbf{x}_0 + \mathbf{h}) \;\approx\; f(\mathbf{x}_0) \;+\; \nabla f(\mathbf{x}_0)^\top \mathbf{h} \;+\; \tfrac{1}{2} \mathbf{h}^\top H_f(\mathbf{x}_0) \mathbf{h}. $$

Plain English: "Current value + linear correction (gradient·step) + quadratic correction (½ × step·Hessian·step)."

Derivation sketch (multivariate). Define $\phi(t) = f(\mathbf{x}_0 + t \mathbf{h})$ for $t \in [0,1]$. Apply single-variable Taylor to $\phi$ at $t = 0$: $\phi(1) = \phi(0) + \phi'(0) + \tfrac{1}{2} \phi''(\xi)$ for some $\xi \in (0,1)$. Computing $\phi'(0) = \nabla f(\mathbf{x}_0)^\top \mathbf{h}$ and $\phi''(0) = \mathbf{h}^\top H_f(\mathbf{x}_0) \mathbf{h}$ by the chain rule yields the formula. QED.

Why it matters in ML. Newton's method is one-step minimization of the second-order Taylor model. Trust-region methods bound $\|\mathbf{h}\|$ so that the Taylor model is trustworthy. Quasi-Newton (BFGS) builds a low-rank approximation of $H$ by examining gradient differences — pure applied Taylor.

```
| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| \(\mathbf{x}_0\) | Expansion point | \(\mathbb{R}^n\) | Current operating point on the dashboard |
| \(\mathbf{h}\) | Step | \(\mathbb{R}^n\) | Proposed move in lever space |
| \(\nabla f(\mathbf{x}_0)^\top \mathbf{h}\) | First-order term | scalar | Linear KPI impact of the move |
| \(\tfrac12 \mathbf{h}^\top H_f \mathbf{h}\) | Second-order term | scalar | Curvature correction — how the
linear estimate over- or under-shoots |
| \(R_n\) | Remainder | scalar | Truncation error — how much the polynomial misses |
```

## 3. Fintech Worked Numeric Examples

Example A — Second-order Taylor model of contribution margin. $\Pi(m, c)$ at $\mathbf{x}_0 = (50, 20)$: $\Pi_0 = 1000(50) - 8(2500) - 300(20) + 4(400) - 0.5(50)(20) = 50000 - 20000 - 6000 + 1600 - 500 = 25{,}100$ INR. Gradient $(190, -165)$. Hessian $\begin{bmatrix}-16 & -0.5\\-0.5 & 8\end{bmatrix}$. Step 1. Propose $\mathbf{h} = (+5, -2)^\top$ (raise MDR by 5 bps, cut cashback by 2 bps). Step 2. First-order term: $190(5) + (-165)(-2) = 950 + 330 = 1280$ INR. Step 3. Second-order term: $\tfrac12 \mathbf{h}^\top H \mathbf{h}$. $H \mathbf{h} = (-16(5) + (-0.5)(-2),\; -0.5(5) + 8(-2)) = (-80 + 1, -2.5 - 16) = (-79, -18.5)$. $\mathbf{h}^\top H \mathbf{h} = 5(-79) + (-2)(-18.5) = -395 + 37 = -358$. Half: $-179$. Step 4. Taylor estimate $\hat \Pi = 25{,}100 + 1{,}280 - 179 = 26{,}201$ INR. Step 5. Exact: $\Pi(55, 18) = 1000(55) - 8(3025) - 300(18) + 4(324) - 0.5(55)(18) = 55000 - 24200 - 5400 + 1296 - 495 = 26{,}201$ INR. Identical — because $\Pi$ is exactly quadratic; in real (non-quadratic) models the second-order Taylor would be an excellent but inexact local approximation.

Example B — Local approximation of a softmax-style payment-router probability. Routing probability to gateway A given features $\mathbf{x}$ is $p(\mathbf{x}) = \dfrac{e^{x_1}}{e^{x_1} + e^{x_2}}$. At $\mathbf{x}_0 = (1, 1)$: $p_0 = 0.5$. Step 1. $\partial p/\partial x_1 = p(1 - p)$. At $\mathbf{x}_0$: $= 0.25$. $\partial p/\partial x_2 = -p(1-p) = -0.25$. Gradient $(0.25, -0.25)$. Step 2. Second partials: $\partial^2 p/\partial x_1^2 = p(1-p)(1 - 2p) = 0.25 \cdot 0 = 0$. Same for $\partial^2 p/\partial x_2^2 = 0$. Cross: $\partial^2 p/\partial x_1 \partial x_2 = -p(1-p)(1 - 2p) = 0$. Hessian is the zero matrix at this symmetric point. Step 3. Step $\mathbf{h} = (0.4, -0.2)^\top$. Linear estimate: $0.5 + 0.25(0.4) + (-0.25)(-0.2) = 0.5 + 0.1 + 0.05 = 0.65$. Quadratic correction: 0. Step 4. Exact: $p(1.4, 0.8) = e^{1.4}/(e^{1.4}+e^{0.8}) = 4.0552/(4.0552 + 2.2255) = 4.0552/6.2807 = 0.6457$. Taylor estimate 0.65 is within 0.005 of exact — pretty good for a single-pass linearization. PM read: payment-router probabilities are locally smooth and nearly linear around the indifference point, which is exactly why you can A/B test them with small treatment-vs-control gaps.

## 4. Common Pitfalls

- Out-of-range Taylor extrapolation. A second-order model around $\mathbf{x}_0$ is accurate only in a small ball. PMs who quote "1% conversion lift" derived from a far-extrapolated derivative are lying with math.
- Symmetry of the Hessian. If you build it numerically with one-sided differences, asymmetry can sneak in. Use central differences and symmetrize.
- Confusing the second-order Taylor estimate with the global function. The Taylor model is a tool; the function is the truth.
- Forgetting the $\tfrac12$ factor in the quadratic term. Universal source of off-by-2 errors in homework and in production.

## 5. PM Relevance

| PM Relevance |
|---|
| - Why a PM must master this: Every "what-if" scenario you present to leadership is an implicit Taylor expansion around today. Knowing when the expansion is trustworthy is core PM judgment. |
| - Metrics to own: Taylor-truncation error (model vs reality on backtests); valid range of local models; second-order vs first-order ROI of the same lever move. |
| - Strategic tradeoffs: First-order-only models are simple and defensible; second-order models capture interactions but require more telemetry to estimate. |

- 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "When the CFO asked for revenue scenarios across MDR moves from 10 to +10 bps, I built a second-order Taylor model. The quadratic term flagged that the linear forecast was overestimating the +8 bps scenario by 12%. Georgia Tech's numerical optimization sequence taught me to bound Taylor models by trust region — that single discipline saved us a bad pricing commit."

### 6. NumPy-Only Python Snippet

```
import numpy as np

# Example A: 2nd-order Taylor of Pi(m, c) around (50, 20), step h = (+5, -2)
Pi0 = 25100.0
g   = np.array([190.0, -165.0])
H   = np.array([[-16.0, -0.5], [-0.5, 8.0]])
h   = np.array([5.0, -2.0])

linear_term = g @ h
quad_term   = 0.5 * h @ H @ h
taylor_est  = Pi0 + linear_term + quad_term
print(f"Pi0={Pi0}, linear={linear_term}, quadratic={quad_term}, Taylor estimate={taylor_est}")

# Verify against exact
def Pi(m, c): return 1000*m - 8*m*m - 300*c + 4*c*c - 0.5*m*c
print("Exact Pi(55, 18) =", Pi(55, 18))

# Example B: Taylor approximation of softmax router probability
def p(x):
    e = np.exp(x - np.max(x))
    return e[0] / np.sum(e)

x0 = np.array([1.0, 1.0])
p0 = p(x0)
grad_p = np.array([p0*(1-p0), -p0*(1-p0)])
H_p = np.zeros((2,2))  # all second partials vanish at symmetric point

h = np.array([0.4, -0.2])
taylor_p = p0 + grad_p @ h + 0.5 * h @ H_p @ h
print(f"Taylor p_hat = {taylor_p:.4f},  exact p = {p(x0 + h):.4f}")
```

## 3.9 Synthesis — How Calculus Wires the AI Stack

Stand back from the page, Prabhjot. Look at what you just rebuilt:

- Limits and continuity are the guarantee that small lever moves yield small KPI moves. Without this, A/B testing is impossible.
- Derivatives and partials are how you quantify marginal impact — the language of every roadmap prioritization argument.
- Gradients are the multidimensional generalization — the compass that points uphill on any KPI surface.
- Directional derivatives quantify how much of that compass direction you actually realize under real-world constraints.
- The chain rule is how engineering work (latency, model accuracy, infra reliability) translates into business KPIs (LTV, retention, NPS). It is also the engine of backprop — which is what trains every neural model Paytm runs in production.
- The Jacobian generalizes gradients to multi-output systems — the formal trade-off table for any multi-KPI dashboard.
- The Hessian distinguishes a robust optimum from a knife-edge and powers second-order methods (Newton, L-BFGS) that converge in three iterations where SGD takes thousands.
- Taylor series is the local-polynomial scaffolding that all of these techniques use to extrapolate from a single point to a useful neighborhood.

This is the calculus you will use. Not the calculus of cosines and arc lengths — the calculus of optimization, of sensitivity, of trade-off. In Chapter 4 we put it to work: we will derive every modern optimizer (SGD, Momentum, RMSprop, Adam) from first principles, and we will close with the Lagrangian / KKT machinery that powers SVM training and constrained product optimization.

You earned this chapter. Take a breath. The next one is where it all clicks.

# Chapter 4 — Optimization Theory

> "In ML, training is optimization. In product, planning is optimization. In life, every decision is the argmax of something. Master the math of `argmin`, and you master the discipline."

Prabhjot, optimization is where calculus stops being an abstract tool and starts being your daily craft. Every model trains by minimizing a loss. Every campaign launches by maximizing expected return under a budget. Every regulatory commitment is a constraint that shapes the feasible set of your roadmap. This chapter walks you from the foundations of convexity through every gradient-based optimizer in production today, and finishes with the constrained-optimization machinery — Lagrange multipliers and KKT — that drives SVM training and budget-bounded product decisions.

We will do this slowly, with worked numbers. By the end you will be able to explain exactly why Adam works better than vanilla SGD on noisy fintech data, and exactly when it does not. That is the level of mastery a Senior PM with a Georgia Tech MSCS is expected to operate at.

## 4.1 Convexity, Local vs Global Minima, Saddle Points

### 1. Plain-English Intuition

A convex function is one where every line segment between two points on the graph lies above the graph. Geometrically, it is a bowl. Any local minimum of a convex function is automatically the global minimum — which is why convex optimization is so beloved by classical ML.

Everyday analogy — blindfolded rolling downhill. Drop a marble on the inside of a perfectly bowl-shaped salad dish (convex). No matter where you drop it, it rolls to the same bottom. Drop the same marble on the surface of an egg carton (non-convex). Now it gets stuck in whichever cup it happens to land in — a local minimum that is not the global one.

Fintech analogy. A linear regression with mean-squared-error loss has a convex landscape — any gradient method will find the unique optimum. A deep neural network for fraud detection has a wildly non-convex landscape — different random initializations land you in different valleys, some better than others. This is why production ML pipelines run multiple training seeds and ensemble the results.

### 2. Formal Mathematical Definition

A set $C \subseteq \mathbb{R}^n$ is convex if for all $\mathbf{x}, \mathbf{y} \in C$ and $\lambda \in [0, 1]$, $\lambda \mathbf{x} + (1 - \lambda) \mathbf{y} \in C$.

A function $f:C \to \mathbb{R}$ on a convex set $C$ is convex if

$$ f(\lambda \mathbf{x} + (1 - \lambda) \mathbf{y}) \;\le\; \lambda f(\mathbf{x}) + (1 - \lambda) f(\mathbf{y}) \quad \forall \mathbf{x}, \mathbf{y} \in C,\; \lambda \in [0,1]. $$

Plain English: "The function evaluated on any convex combination is no larger than the same convex combination of function values." Geometric reading: secant lines lie above the graph.

First-order characterization. If $f$ is differentiable, then $f$ is convex iff

$$ f(\mathbf{y}) \;\ge\; f(\mathbf{x}) + \nabla f(\mathbf{x})^\top (\mathbf{y} - \mathbf{x}) \quad \forall \mathbf{x}, \mathbf{y}. $$

Plain English: "The function lies above all its tangent planes."

Second-order characterization. If $f$ is $C^2$, then $f$ is convex iff the Hessian $H_f(\mathbf{x})$ is positive semi-definite (PSD) for all $\mathbf{x}$. Strictly convex if PD everywhere.

Critical-point taxonomy. At a point with $\nabla f(\mathbf{x}^\star) = 0$:

- Local minimum: $H_f(\mathbf{x}^\star)$ PD.
- Local maximum: $H_f(\mathbf{x}^\star)$ ND.
- Saddle point: $H_f(\mathbf{x}^\star)$ indefinite (eigenvalues of mixed sign).
- Degenerate: $H_f(\mathbf{x}^\star)$ singular — higher-order analysis required.

Why saddles matter in deep learning. In high dimensions, the vast majority of critical points of a deep-network loss are saddle points, not local minima. Modern optimizers (Adam, SGD-with-momentum) escape saddles because their stochastic noise pushes them off the unstable axis.

```
| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| $f$ | Loss / objective | $\mathbb{R}^n \to \mathbb{R}$ | Function we minimize (loss) or maximize (
margin) |
| $\mathbf{x}^\star$ | Optimum | $\mathbb{R}^n$ | The lever setting your team is hunting for |
| $H_f(\mathbf{x}^\star)$ | Hessian at optimum | $n \times n$, symmetric | Local curvature — robustness
signature of the optimum |
| Convex $f$ | $H \succeq 0$ everywhere | property | "No local traps" — global optimum is reachable |
| Saddle | Indefinite Hessian | property | False optimum — flat in some directions, escapable in others |
```

## 3. Fintech Worked Numeric Examples

Example A — Is a regression loss convex? Linear regression loss $L(\mathbf{w}) = \tfrac{1}{2N} \|X \mathbf{w} - \mathbf{y}\|^2$. Hessian: $H = \tfrac{1}{N} X^\top X$. For any nonzero $\mathbf{v}$, $\mathbf{v}^\top H \mathbf{v} = \tfrac{1}{N} \|X \mathbf{v}\|^2 \ge 0$. So $H$ is PSD always — loss is convex. If $X$ has full column rank, $H \succ 0$ and the optimum is unique. Step 1. Use $X = \begin{bmatrix} 1 & 100 \\ 1 & 200 \\ 1 & 300 \\ 1 & 400 \end{bmatrix}$ (intercept + MDR in bps), $\mathbf{y} = (10, 22, 31, 42)^\top$ (revenue in INR). Step 2. $X^\top X = \begin{bmatrix} 4 & 1000 \\ 1000 & 300000 \end{bmatrix}$. Determinant $= 4(300000) - 1000^2 = 1{,}200{,}000 - 1{,}000{,}000 = 200{,}000 > 0$. PD confirmed. Step 3. $X^\top \mathbf{y} = (105, 31000)^\top$. Solve $X^\top X \mathbf{w} = X^\top \mathbf{y}$: system is $4 w_0 + 1000 w_1 = 105$, $1000 w_0 + 300000 w_1 = 31000$. From first: $w_0 = (105 - 1000 w_1)/4 = 26.25 - 250 w_1$. Substitute: $1000(26.25 - 250 w_1) + 300000 w_1 = 31000$  $26250 + 50000 w_1 = 31000$  $w_1 = 0.095$, $w_0 = 26.25 - 23.75 = 2.5$. Step 4. Unique global optimum: revenue $\approx 2.5 + 0.095 \cdot m$. Because loss is convex, any gradient-based method will find this exact point.

Example B — Is a neural fraud-detection loss convex? A 1-hidden-unit network with sigmoid activation: $\hat y = \sigma(w_2 \sigma(w_1 x + b_1) + b_2)$. Even with one input and binary cross-entropy loss, this is non-convex. Verify with one data point. Step 1. Set $x = 1$, $y = 1$. Loss as a function of $(w_1, w_2)$ with $b_1 = b_2 = 0$: $L(w_1, w_2) = -\log \sigma(w_2 \sigma(w_1))$. Step 2. Evaluate at three points: $L(0, 0)$: $\sigma(0) = 0.5$, $\sigma(0 \cdot 0.5) = 0.5$, $-\log 0.5 = 0.6931$. $L(2, 2)$: $\sigma(2) = 0.8808$, $\sigma(2 \cdot 0.8808) = \sigma(1.7616) = 0.8534$, $-\log 0.8534 = 0.1586$. $L(-2, -2)$: $\sigma(-2) = 0.1192$, $\sigma(-2 \cdot 0.1192) = \sigma(-0.2384) = 0.4407$, $-\log 0.4407 = 0.8193$. Step 3. Convex check: midpoint of $(2,2)$ and $(-2,-2)$ is $(0,0)$. Is $L(0,0) \le \tfrac12 L(2,2) + \tfrac12 L(-2,-2)$? RHS $= 0.5(0.1586 + 0.8193) = 0.4890$. LHS $= 0.6931$. $0.6931 > 0.4890$ — convexity violated. Non-convex confirmed. This is why a single random init can land in a poor valley and miss a much better one.

## 4. Common Pitfalls

- Assuming "convex loss" implies "convex predictor." Logistic regression has convex loss but a non-linear predictor.
- Confusing "convex set" (the feasible region) with "convex function" (the objective). Both need to hold for the comfortable theorems.
- Reporting "no local optima problem" for non-convex losses just because a single run converged. Run multiple seeds, always.
- Treating a saddle point reached by SGD as the global optimum. The flat gradient is necessary but nowhere near sufficient.

### 5. PM Relevance

| PM Relevance |
| --- |
| - Why a PM must master this: Convexity tells you whether the team's optimization runs are guaranteed to converge to the global best, or whether you need to budget for multiple seeds, ensembles, or restarts. |
| - Metrics to own: Seed-variance of final loss; eigenvalue spectrum of the empirical loss Hessian at convergence; saddle-point detection. |
| - Strategic tradeoffs: Convex models are auditable and reliable but expressively limited; deep non-convex models capture more signal but require operational discipline against saddles and poor local optima. |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "I argue for convex models when explainability and audit dominate (credit decisioning) and for non-convex deep models when capacity dominates (fraud detection on raw event streams). I budget seed-variance differently for the two. Georgia Tech's convex-optimization track gave me the precise vocabulary to defend that split to risk and compliance." |

### 6. NumPy-Only Python Snippet

```
import numpy as np

# Example A: convex linear regression
X = np.array([[1, 100], [1, 200], [1, 300], [1, 400]], dtype=float)
y = np.array([10, 22, 31, 42], dtype=float)
H = X.T @ X / X.shape[0]
print("Hessian eigenvalues (should be > 0):", np.linalg.eigvalsh(H))
w_star = np.linalg.solve(X.T @ X, X.T @ y)
print("Global optimum w*:", w_star)

# Example B: convexity check on a 1-hidden-unit net
def sig(z): return 1/(1+np.exp(-z))
def L(w1, w2): return -np.log(sig(w2 * sig(w1)))

a, b   = np.array([2.0, 2.0]), np.array([-2.0, -2.0])
mid    = 0.5 * (a + b)
lhs    = L(*mid)
rhs    = 0.5 * L(*a) + 0.5 * L(*b)
print(f"L(mid)={lhs:.4f},  0.5*L(a)+0.5*L(b)={rhs:.4f},  convex? {lhs <= rhs}")
```

## 4.2 Gradient Descent — Derivation and Geometry

### 1. Plain-English Intuition

Gradient descent is the simplest non-trivial optimizer: at each step, compute the gradient, take a step in the opposite direction, repeat. The size of the step is the learning rate. The whole algorithm is a literal implementation of "roll downhill."

Everyday analogy. You are blindfolded on a hillside (yes, the hiker again — this is the metaphor that runs the entire field). You can feel slope under your feet. You take a step downhill. You feel the slope at the new spot. You step again. Eventually you hit flat ground — a critical point.

Fintech analogy. You are pricing a new lending product. You start at a reasonable initial guess. You run a one-day experiment, observe the marginal margin (the gradient), and shift the price in the direction that improved margin. You repeat next week. After a few cycles, you converge to a near-optimal price. That entire pricing process is gradient descent — and the learning rate is your weekly step-size policy.

### 2. Formal Mathematical Definition

Given a differentiable $f:\mathbb{R}^n \to \mathbb{R}$ and learning rate $\eta > 0$:

$$ \mathbf{x}_{t+1} \;=\; \mathbf{x}_t - \eta \,\nabla f(\mathbf{x}_t). $$

Plain English: "New point = old point minus a small multiple of the gradient."

Derivation via local linear model. Around $\mathbf{x}_t$, the first-order Taylor expansion gives $f(\mathbf{x}_t + \mathbf{h}) \approx f(\mathbf{x}_t) + \nabla f(\mathbf{x}_t)^\top \mathbf{h}$. To minimize the linear model under a step-size constraint $\|\mathbf{h}\| \le \eta \|\nabla f\|$, Cauchy–Schwarz says the best $\mathbf{h}$ is $-\eta \nabla f(\mathbf{x}_t)$. That is the update rule.

Convergence theorem (smooth convex case). If $f$ is convex with $L$-Lipschitz gradient (i.e., $\|\nabla f(\mathbf{x}) - \nabla f(\mathbf{y})\| \le L \|\mathbf{x} - \mathbf{y}\|$) and $0 < \eta \le 1/L$, then GD converges at rate $O(1/t)$:

$$ f(\mathbf{x}_t) - f(\mathbf{x}^\star) \;\le\; \frac{\|\mathbf{x}_0 - \mathbf{x}^\star\|^2}{2 \eta t}. $$

For strongly convex $f$ with parameter $\mu > 0$ and $\eta = 2/(\mu + L)$, convergence is linear (geometric): $f(\mathbf{x}_t) - f(\mathbf{x}^\star) \le \left(\frac{L-\mu}{L+\mu}\right)^{2t} (f(\mathbf{x}_0) - f(\mathbf{x}^\star))$.

Sketch of the convex-case proof. $L$-smoothness implies $f(\mathbf{x} - \eta \nabla f) \le f(\mathbf{x}) - \eta(1 - \tfrac{L\eta}{2}) \|\nabla f\|^2$. With $\eta = 1/L$, the per-step decrease is $\ge \tfrac{1}{2L}\|\nabla f\|^2$. Telescoping and using convexity yields the $O(1/t)$ bound.

| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| $\eta$ | Learning rate | scalar $>0$ | Step size per training iteration |
| $\nabla f(\mathbf{x}_t)$ | Gradient | $\mathbb{R}^n$ | Marginal-improvement vector at current point |
| $L$ | Lipschitz constant of $\nabla f$ | scalar | Max curvature — sets safe $\eta$ ceiling |
| $\mu$ | Strong-convexity constant | scalar $\ge 0$ | "Bowl sharpness" floor |
| $\kappa = L/\mu$ | Condition number | scalar $\ge 1$ | Anisotropy — small $\kappa$ trains fast |

## 3. Fintech Worked Numeric Examples

Example A — Linear regression by GD on the Example-3.1 dataset. $X, \mathbf{y}$ as before. $N = 4$. Loss $L(\mathbf{w}) = \tfrac{1}{2N}\|X\mathbf{w} - \mathbf{y}\|^2$. Gradient: $\nabla L = \tfrac{1}{N} X^\top (X\mathbf{w} - \mathbf{y})$. Step 1. Initialize $\mathbf{w}_0 = (0, 0)$. Use $\eta = 1\times10^{-6}$ (eigenvalues of $H = X^\top X/N$ are large because MDR is in bps). Step 2. $X \mathbf{w}_0 - \mathbf{y} = (-10, -22, -31, -42)$. $X^\top(\cdot) = (-10 -22 -31 -42, -1000 - 4400 - 9300 - 16800) = (-105, -31500)$. Gradient $= (-26.25, -7875)$. Step 3. $\mathbf{w}_1 = (0,0) - 10^{-6}(-26.25, -7875) = (2.625 \times 10^{-5},\; 7.875 \times 10^{-3})$. Already the slope is meaningfully positive. Step 4. After 1000 iterations with this conservative $\eta$, $\mathbf{w}$ approaches $(2.5, 0.095)$ — the closed-form solution from §4.1. The convergence is slow because $\kappa$ is large (~30000); a preconditioner (e.g. scaling MDR to unit range) reduces $\kappa$ and speeds GD dramatically.

Example B — GD on a Paytm fraud-score sigmoid model. Single feature $x_i$ and binary label $y_i$. Loss $L(w) = -\tfrac{1}{N}\sum_i [y_i \log \sigma(w\, x_i) + (1-y_i) \log(1 - \sigma(w\, x_i))]$. Use data $(x, y) = \{(2, 1), (-1, 0), (3, 1), (-2, 0)\}$. Gradient: $L'(w) = -\tfrac{1}{N}\sum_i x_i (y_i - \sigma(w\, x_i))$. Step 1. Initialize $w_0 = 0$. $\sigma(0) = 0.5$. Residuals $y_i - 0.5 = (0.5, -0.5, 0.5, -0.5)$. $\sum x_i(y_i - \sigma) = 2(0.5) + (-1)(-0.5) + 3(0.5) + (-2)(-0.5) = 1 + 0.5 + 1.5 + 1 = 4$. $L'(0) = -1$. Step 2. With $\eta = 0.5$: $w_1 = 0 - 0.5(-1) = 0.5$. Step 3. At $w = 0.5$: $\sigma(1) = 0.7311, \sigma(-0.5) = 0.3775, \sigma(1.5) = 0.8176, \sigma(-1) = 0.2689$. Residuals: $1 - 0.7311 = 0.2689$; $0 - 0.3775 = -0.3775$; $1 - 0.8176 = 0.1824$; $0 - 0.2689 = -0.2689$. $\sum x_i (y_i - \sigma) = 2(0.2689) + (-1)(-0.3775) + 3(0.1824) + (-2)(-0.2689) = 0.5378 + 0.3775 + 0.5472 + 0.5378 = 2.0003$. $L'(0.5) = -0.5001$. $w_2 = 0.5 + 0.5(0.5001) = 0.75$. Step 4. At $w = 0.75$: repeat the same arithmetic to get $L'(0.75) \approx -0.286$. $w_3 = 0.75 + 0.143 = 0.893$. Convergence is monotone toward a value near $w^\star \approx 1.5$; GD is making steady progress.

## 4. Common Pitfalls

- Learning rate too large. GD oscillates or diverges. Symptom: loss bounces or explodes.
- Learning rate too small. GD crawls. Symptom: weeks of training, no convergence.
- Forgetting to scale features. A condition number of $10^6$ (e.g., mixing bps and percent) destroys GD.
- Trusting GD on non-convex landscapes without restarts. A single seed can land in a poor valley.

### 5. PM Relevance

| PM Relevance |
| --- |
| - Why a PM must master this: GD is the simplest optimizer, but its failure modes (slow convergence, divergence, getting stuck) drive the engineering decisions around training infra costs. |
| - Metrics to own: Wall-clock to converge; per-epoch loss; learning-rate sweeps; condition number of input features. |
| - Strategic tradeoffs: Larger \(\eta\) is cheap but unstable; smaller \(\eta\) is robust but slow. Feature scaling can change the trade by orders of magnitude. |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "I once cut Paytm fraud-model training time by 6× — not by buying GPUs, but by standardizing input features so the loss condition number dropped from \(10^6\) to \(10^2\). Pure Georgia Tech convex-optimization hygiene applied to a real production cost line." |

### 6. NumPy-Only Python Snippet

```
import numpy as np

# Example A: GD on linear regression
X = np.array([[1, 100], [1, 200], [1, 300], [1, 400]], dtype=float)
y = np.array([10, 22, 31, 42], dtype=float)
N = X.shape[0]
w = np.zeros(2)
eta = 1e-6
for t in range(5000):
    grad = (X.T @ (X @ w - y)) / N
    w -= eta * grad
print("GD solution after 5000 steps:", w)
print("Closed-form              :", np.linalg.solve(X.T@X, X.T@y))

# Example B: GD on logistic fraud score
xs = np.array([2.0, -1.0, 3.0, -2.0])
ys = np.array([1.0, 0.0, 1.0, 0.0])
def sig(z): return 1/(1+np.exp(-z))
w = 0.0
eta = 0.5
for t in range(10):
    p = sig(w*xs)
    grad = -np.mean(xs * (ys - p))
    w -= eta * grad
    print(f"iter {t}: w={w:.5f}  grad={grad:+.5f}")
```

## 4.3 Stochastic and Mini-Batch Gradient Descent

### 1. Plain-English Intuition

When the dataset has 10 million rows, computing the full gradient every step is wasteful. Stochastic gradient descent (SGD) uses a single random example per step — noisy, but cheap. Mini-batch GD uses a small batch (say, 32 or 256) — a sweet spot between full-batch GD's accuracy and SGD's speed.

Everyday analogy. Full-batch GD: ask all 10 million users for their preference before stepping. SGD: ask one user, step, ask the next. Mini-batch: ask a focus group of 256, step.

Fintech analogy. A risk model is being retrained on a week's worth of UPI transactions. Full-batch would compute gradients over the full week before each step — accurate, but you would never finish in time for the daily refresh. SGD jitters per transaction — fast, but the gradient signal is noisy. Mini-batch on 1024-transaction batches is the production sweet spot: fast enough to retrain hourly, accurate enough to converge cleanly.

### 2. Formal Mathematical Definition

If the loss decomposes as \(L(\mathbf{w}) = \tfrac{1}{N} \sum_{i=1}^N \ell_i(\mathbf{w})\) (typical in supervised ML):

- Full-batch GD: \(\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \cdot \tfrac{1}{N}\sum_{i=1}^N \nabla \ell_i(\mathbf{w}_t)\).

- SGD: pick $i_t$ uniformly at random; $\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla \ell_{i_t}(\mathbf{w}_t)$.
- Mini-batch GD: pick a batch $B_t$ of size $b$; $\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \cdot \tfrac{1}{b}\sum_{i \in B_t} \nabla \ell_i(\mathbf{w}_t)$.

Plain English: "Same update direction, but estimated from one example, a batch, or the whole dataset."

Variance analysis. SGD's per-step gradient is an unbiased estimator: $\mathbb{E}[\nabla \ell_{i_t}] = \nabla L$. But it has variance $\sigma^2$. For mini-batch of size $b$, variance scales as $\sigma^2 / b$. This is the key trade-off:

- Small $b$: high noise, good escape from saddles, low per-step cost.
- Large $b$: low noise, stable convergence, high per-step cost.

Convergence (SGD, convex L-smooth case) with diminishing step size $\eta_t = \eta_0 / \sqrt{t}$: $\mathbb{E}[L(\bar{\mathbf{w}}_T)] - L(\mathbf{w}^\star) = O(1/\sqrt{T})$. Slower than full-batch's $O(1/T)$ — but each step is $N$ times cheaper.

| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| $\ell_i(\mathbf{w})$ | Per-example loss | scalar | Loss on one transaction / user / merchant |
| $b$ | Mini-batch size | integer | Examples per parameter update |
| $\sigma^2$ | Gradient variance | scalar | Noise of stochastic gradient |
| $\eta_t$ | Schedule of LR | scalar function of $t$ | Step-size decay policy |
| Epoch | One pass over all data | iteration count | Training "round" — useful PM unit |

## 3. Fintech Worked Numeric Examples

Example A — SGD on the Paytm fraud sigmoid model. Same data as §4.2 Example B. Now use SGD: at step $t$, pick one of the four examples at random. Step 1. $w_0 = 0$, $\eta = 0.5$. Sample order (seeded): $i_0 = 2$ (so $x=3, y=1$). Step 2. $\sigma(0) = 0.5$. $\nabla \ell_2(0) = -3(1 - 0.5) = -1.5$. $w_1 = 0 - 0.5(-1.5) = 0.75$. Step 3. Sample $i_1 = 0$ ($x=2, y=1$). $\sigma(1.5) = 0.8176$. $\nabla \ell_0(0.75) = -2(1 - 0.8176) = -0.3648$. $w_2 = 0.75 + 0.1824 = 0.9324$. Step 4. Sample $i_2 = 3$ ($x=-2, y=0$). $\sigma(-1.8648) = 0.1342$. $\nabla \ell_3(0.9324) = -(-2)(0 - 0.1342) = -0.2684$. $\nabla \ell_3 = -x_3(y_3 - \sigma) = -(-2)(0 - 0.1342) = -0.2684$. $w_3 = 0.9324 - 0.5(-0.2684) = 0.9324 + 0.1342 = 1.0666$. Step 5. Notice the trajectory jitters but trends upward (toward $w^\star \approx 1.5$). Full-batch GD's path was smoother in §4.2; SGD's is noisier but each step is one-quarter the compute.

Example B — Mini-batch budget vs convergence trade. Dataset of $N = 10{,}000$ UPI transactions; full gradient costs 10 ms; SGD costs 0.001 ms per example; mini-batch cost scales linearly. Suppose gradient variance per example $\sigma^2 = 4$. Convergence after $T$ steps satisfies $\mathbb{E}[L_T] - L^\star \le C/\sqrt{T \cdot b}$ for some constant $C$ (combined bound for mini-batch SGD). Step 1. To reach a target accuracy $\varepsilon$, need $T \cdot b \ge (C/\varepsilon)^2$. Step 2. Wall time per step: $0.001 \cdot b$ ms. Total wall time: $0.001 b T = 0.001 (Tb)$. For fixed $Tb$, wall time is identical regardless of how we split between T and b. So why prefer some batch sizes? Step 3. Two real reasons. (a) Saddle-escape — small $b$ injects more noise to escape saddles; $b \le 64$ is often preferred early. (b) Hardware utilization — GPUs amortize overhead better with $b \ge 128$. The sweet spot at Paytm scale is typically $b \in [256, 1024]$. Step 4. Concrete: for $\varepsilon = 0.01$, $C = 1$, need $Tb \ge 10^4$. Choose $b = 256$, $T = 40$ steps — five epochs of 8 mini-batches each, total wall 10 ms. Choose $b = 1$ (pure SGD), $T = 10000$, wall 10 ms. Same compute. Pick $b$ based on hardware and saddle landscape, not on convergence theory alone.

## 4. Common Pitfalls

- Shuffling neglect. SGD that iterates examples in original order biases the gradient. Always shuffle each epoch.
- Forgetting LR decay. Constant LR + SGD often fails to converge; loss bounces around the optimum.
- Tiny batch on modern hardware. Batch of 1 leaves >90% of GPU idle — wall time bottlenecked by overhead, not math.
- Confusing "training loss curve looks noisy" with "model is broken." Some noise is inherent to mini-batch; only worry if the trend is wrong.

## 5. PM Relevance

| PM Relevance |
| --- |
| - Why a PM must master this: Mini-batch size and learning-rate schedule are the two knobs that most directly determine training cost and time-to-deploy. |
| - Metrics to own: Wall-clock per epoch; training-loss noise band; held-out accuracy after \(N\) updates; GPU utilization at chosen batch size. |
| - Strategic tradeoffs: Small batches  cheap, noisy, saddle-friendly; large batches  expensive, smooth, hardware-friendly. Linear-scaling rule (LR \(\propto\) batch size) often holds in the small-batch regime and breaks at large batches. |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "When the team wanted a 10× batch-size increase for hardware utilization, I pushed back: at our condition-number, naive linear LR scaling would have diverged. We used a warmup schedule à la Goyal et al. and verified empirically. Georgia Tech's stochastic-optimization module taught me to never trust batch-size rules of thumb without testing the smoothness assumption." |

### 6. NumPy-Only Python Snippet

```
import numpy as np

rng = np.random.default_rng(7)
xs = np.array([2.0, -1.0, 3.0, -2.0])
ys = np.array([1.0, 0.0, 1.0, 0.0])
def sig(z): return 1/(1+np.exp(-z))

# Pure SGD
w = 0.0
eta = 0.5
for t in range(20):
    i = rng.integers(0, 4)
    grad_i = -xs[i] * (ys[i] - sig(w * xs[i]))
    w -= eta * grad_i
    if t < 5: print(f"sgd iter {t}: i={i}  w={w:+.4f}")

print("Final w after 20 SGD steps:", w)

# Mini-batch GD with batch size b=2
w = 0.0
b = 2
for t in range(20):
    idx = rng.choice(4, b, replace=False)
    grad_b = -np.mean(xs[idx] * (ys[idx] - sig(w * xs[idx])))
    w -= eta * grad_b
print("Final w after 20 mini-batch steps:", w)
```

## 4.4 Momentum, RMSprop, and Adam — Derivations

### 1. Plain-English Intuition

Vanilla SGD has two annoying behaviors. (a) In narrow ravines, it oscillates side-to-side and progresses slowly along the long axis. (b) It treats every dimension equally, so a single noisy coordinate dominates the step size. Modern optimizers fix both:

- Momentum accumulates a velocity that smooths oscillations and accelerates motion along consistent directions — like a ball with inertia rolling downhill.
- RMSprop scales each coordinate's step by its recent gradient magnitude — so noisy coordinates get smaller steps automatically.
- Adam combines both: momentum on the gradient and per-coordinate scaling.

Fintech analogy. Imagine training a price-elasticity model where the bias term has a stable gradient signal but the per-merchant slope is noisy. Vanilla SGD oscillates the slope while bias crawls. RMSprop calms the slope and lets bias breathe. Momentum compounds the bias progress. Adam does both at once — which is why it dominates production training of fintech ML models.

### 2. Formal Mathematical Definitions and Derivations

Momentum (Polyak's heavy-ball).

$$ \mathbf{v}_{t+1} = \beta \mathbf{v}_t + \nabla f(\mathbf{w}_t), \qquad \mathbf{w}_{t+1} = \mathbf{w}_t - \eta \mathbf{v}_{t+1}. $$

Plain English: "Carry a running velocity. Update velocity by adding the new gradient (with a decay factor $\beta$). Step in the velocity's direction."

Why it accelerates. In a quadratic bowl with condition number $\kappa$, vanilla GD converges at rate $(1 - 1/\kappa)$. Momentum (with optimal $\beta$) converges at $(1 - 1/\sqrt{\kappa})$. For $\kappa = 100$, GD needs $\sim 100$ iterations to halve the gap; momentum needs $\sim 10$. This is a quadratic speed-up — Nesterov's celebrated result.

Nesterov's accelerated variant evaluates the gradient at the look-ahead point $\mathbf{w}_t - \eta \beta \mathbf{v}_t$ rather than at $\mathbf{w}_t$. Same flavor, tighter constants.

RMSprop.

$$ \mathbf{s}_{t+1} = \rho \mathbf{s}_t + (1 - \rho) \nabla f(\mathbf{w}_t) \odot \nabla f(\mathbf{w}_t), $$ $$ \mathbf{w}_{t+1} = \mathbf{w}_t - \eta \cdot \frac{\nabla f(\mathbf{w}_t)}{\sqrt{\mathbf{s}_{t+1}} + \epsilon}. $$

Plain English: "Maintain a per-coordinate running average of squared gradients. Divide each gradient component by the square root of that average — so noisier coordinates get smaller effective step sizes."

Adam (Kingma and Ba). Combine momentum (first moment) and RMSprop (second moment), with bias correction:

$$ \mathbf{m}_{t+1} = \beta_1 \mathbf{m}_t + (1 - \beta_1) \nabla f(\mathbf{w}_t), $$ $$ \mathbf{v}_{t+1} = \beta_2 \mathbf{v}_t + (1 - \beta_2) \nabla f(\mathbf{w}_t) \odot \nabla f(\mathbf{w}_t), $$ $$ \hat{\mathbf{m}}_{t+1} = \frac{\mathbf{m}_{t+1}}{1 - \beta_1^{t+1}}, \qquad \hat{\mathbf{v}}_{t+1} = \frac{\mathbf{v}_{t+1}}{1 - \beta_2^{t+1}}, $$ $$ \mathbf{w}_{t+1} = \mathbf{w}_t - \eta \cdot \frac{\hat{\mathbf{m}}_{t+1}}{\sqrt{\hat{\mathbf{v}}_{t+1}} + \epsilon}. $$

Plain English: "Track exponentially-weighted estimates of the gradient (first moment) and squared gradient (second moment). Correct the bias from zero initialization. Step proportionally to first moment, inversely to root of second moment."

Why bias correction matters. At step $t = 1$, $\mathbf{m}_1 = (1-\beta_1) g_0$, which is much smaller than $g_0$. The factor $1/(1 - \beta_1^{t+1})$ restores the correct magnitude. Without it, Adam's early steps are tiny and training stalls.

Default hyperparameters (Kingma–Ba 2014): $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, $\eta = 10^{-3}$. These work astonishingly often.

| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| $\mathbf{v}_t$ (Momentum) | Velocity | $\mathbb{R}^n$ | Smoothed gradient direction |
| $\beta$ | Momentum coefficient | scalar in [0,1] | Memory of past gradients |
| $\mathbf{s}_t$ (RMSprop) | Running second moment | $\mathbb{R}^n$ | Per-coordinate noise estimate |
| $\rho, \beta_2$ | Second-moment decay | scalar in [0,1] | Window over which we estimate noise |
| $\mathbf{m}_t, \mathbf{v}_t$ (Adam) | First, second moments | $\mathbb{R}^n$ | Direction + per-coord scale, jointly |
| $\epsilon$ | Numerical-stability term | scalar | Prevents division by zero |

## 3. Fintech Worked Numeric Examples

Example A — Momentum vs SGD on an ill-conditioned MDR–latency loss. Loss $L(\mathbf{w}) = 100 w_1^2 + w_2^2$ (a stretched bowl with condition number 100). $\nabla L = (200 w_1, 2 w_2)$. Start at $\mathbf{w}_0 = (1, 1)$. Step 1. SGD with $\eta = 0.009$. Step 1: gradient $(200, 2)$. $\mathbf{w}_1 = (1, 1) - 0.009(200, 2) = (-0.8, 0.982)$. Notice $w_1$ over-shoots and now has the wrong sign — classic ill-conditioning oscillation. Step 2. SGD step 2: gradient at $(-0.8, 0.982)$ is $(-160, 1.964)$. $\mathbf{w}_2 = (-0.8, 0.982) - 0.009(-160, 1.964) = (0.64, 0.964)$. Still oscillating violently along $w_1$, inching along $w_2$. Step 3. Momentum with $\beta = 0.9, \eta = 0.009$. $\mathbf{v}_0 = 0$. Step 1: $\mathbf{v}_1 =$

0.9(0) + (200, 2) = (200, 2)\). \(\mathbf{w}_1 = (1,1) - 0.009(200, 2) = (-0.8, 0.982)\) — same first step. Step 4. Momentum step 2: gradient at \((-0.8, 0.982)\) is \((-160, 1.964)\). \(\mathbf{v}_2 = 0.9(200, 2) + (-160, 1.964) = (20, 3.764)\). \(\mathbf{w}_2 = (-0.8, 0.982) - 0.009(20, 3.764) = (-0.98, 0.948)\). The momentum partially cancels the over-shoot along \(w_1\) (velocity 20 vs raw gradient \(-160\)), keeping more energy along the productive \(w_2\) axis. Over many steps, momentum's path is far shorter to the optimum.

Example B — Adam on Paytm credit-default cross-entropy. Tiny model: predicted log-odds \(z = w_1 x_1 + w_2 x_2 + b\), default probability \(p = \sigma(z)\), loss \(\ell = -[y \log p + (1-y) \log(1-p)]\). One example: \(x_1 = 0.8\) (debt-to-income), \(x_2 = -1.2\) (months on book, standardized), \(y = 1\). Start \(\mathbf{w} = (0, 0, 0)\). Hyper: \(\eta = 10^{-2}\), \(\beta_1 = 0.9\), \(\beta_2 = 0.999\), \(\epsilon = 10^{-8}\). Step 1. \(z = 0\), \(p = 0.5\). Gradient: \(\nabla \ell = (p - y)(x_1, x_2, 1) = (-0.5)(0.8, -1.2, 1) = (-0.4, 0.6, -0.5)\). Step 2. \(\mathbf{m}_1 = 0.1 \cdot (-0.4, 0.6, -0.5) = (-0.04, 0.06, -0.05)\). Step 3. \(\mathbf{v}_1 = 0.001 \cdot (0.16, 0.36, 0.25) = (0.00016, 0.00036, 0.00025)\). Step 4. Bias-correct: \(\hat \mathbf{m}_1 = \mathbf{m}_1 / (1 - 0.9) = (-0.4, 0.6, -0.5)\). \(\hat \mathbf{v}_1 = \mathbf{v}_1 / (1 - 0.999) = (0.16, 0.36, 0.25)\). Step 5. Update: \(-\eta \hat\mathbf{m} / (\sqrt{\hat\mathbf{v}} + \epsilon) = -0.01 \cdot (-0.4, 0.6, -0.5)/(0.4, 0.6, 0.5) = -0.01 \cdot (-1, 1, -1) = (0.01, -0.01, 0.01)\). Step 6. After bias correction, Adam's first step is essentially \(\pm \eta\) in each coordinate — independent of the gradient magnitude. This is a remarkable property: Adam's update size is approximately \(\eta\) per coordinate, regardless of gradient scale. PM consequence: you can swap features in and out without retuning the learning rate as aggressively as you would for SGD.

## 4. Common Pitfalls

- Momentum without LR re-tuning. A momentum-equipped optimizer effectively has step size \(\eta/(1-\beta)\); naive transplanting of LR causes divergence.
- Adam with weight decay. The classical Adam couples weight decay into the moment estimates, which is wrong; AdamW decouples them and is now the default for transformer training.
- Adam in convex regimes where pure SGD-momentum is provably better. Adam can converge to worse solutions in some classes (Reddi et al., 2018); fixes include AMSGrad.
- Forgetting the \(\epsilon\) in RMSprop/Adam. Without it, early steps explode when \(\hat v\) is near zero.

## 5. PM Relevance

| PM Relevance |
| --- |
| - Why a PM must master this: Optimizer choice and hyperparameters drive most of the difference between "model ships in two weeks" and "team is stuck for a quarter." |
| - Metrics to own: Per-optimizer convergence curves; sensitivity to LR; AdamW vs Adam ablation; throughput at chosen batch size. |
| - Strategic tradeoffs: Adam is a strong default but can hide structural problems (poor features, bad architecture) that pure SGD would expose with louder failure. Pure SGD with momentum often generalizes better in vision; Adam dominates in NLP. |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "On the Paytm credit risk model, we ran SGD-momentum vs AdamW head-to-head. AdamW won at 1/3 the wall time and was robust to a 4× LR mis-specification. I keep Adam as the default but always check SGD-momentum as a sanity sentinel — a discipline I picked up in Georgia Tech's deep-learning systems course." |

## 6. NumPy-Only Python Snippet

```
import numpy as np

# Example A: SGD vs Momentum on ill-conditioned bowl
def grad_bowl(w): return np.array([200*w[0], 2*w[1]])

def run_sgd(w0, eta, steps):
    w = w0.copy(); path = [w.copy()]
    for _ in range(steps):
        w = w - eta * grad_bowl(w); path.append(w.copy())
    return np.array(path)
```

```
def run_momentum(w0, eta, beta, steps):
    w = w0.copy(); v = np.zeros_like(w); path = [w.copy()]
    for _ in range(steps):
        v = beta * v + grad_bowl(w)
        w = w - eta * v
        path.append(w.copy())
    return np.array(path)

p_sgd = run_sgd      (np.array([1.0, 1.0]), eta=0.009, steps=50)
p_mom = run_momentum (np.array([1.0, 1.0]), eta=0.009, beta=0.9, steps=50)
print("SGD final w:     ", p_sgd[-1], "  L=", 100*p_sgd[-1,0]**2 + p_sgd[-1,1]**2)
print("Momentum final w:", p_mom[-1], "  L=", 100*p_mom[-1,0]**2 + p_mom[-1,1]**2)

# Example B: One Adam step on credit-default cross-entropy
x = np.array([0.8, -1.2, 1.0])  # last entry is bias-multiplier
y = 1.0
w = np.zeros(3); m = np.zeros(3); v = np.zeros(3)
beta1, beta2, eps, eta = 0.9, 0.999, 1e-8, 0.01

z = w @ x; p = 1/(1+np.exp(-z))
g = (p - y) * x
m = beta1*m + (1-beta1)*g
v = beta2*v + (1-beta2)*g*g
m_hat = m / (1 - beta1**1)
v_hat = v / (1 - beta2**1)
update = -eta * m_hat / (np.sqrt(v_hat) + eps)
w = w + update
print("First Adam update:", update, "  new w:", w)
```

## 4.5 Constrained Optimization and Lagrange Multipliers

### 1. Plain-English Intuition

Almost every product decision lives under constraints. A discount engine has a daily budget. A risk model has a maximum approved-default rate set by regulators. A pricing model has an MDR ceiling. Constrained optimization is the mathematics of finding the best feasible point — not the best point in the universe, but the best point inside the cage your constraints have drawn.

Everyday analogy. Imagine you are shopping for groceries with exactly ₹1,000 in your pocket. Unconstrained, you would fill the cart with the most delicious items. Constrained, you must trade off taste against price so that your total bill equals ₹1,000. At the optimum, the marginal pleasure-per-rupee of every item you buy is the same — if one item gave more pleasure-per-rupee than another, you would swap. That common ratio is the Lagrange multiplier: the shadow price of the budget constraint.

Fintech analogy: the discount-engine budget. Suppose your Paytm Cashback team has a daily marketing budget of ₹50 lakh. You want to maximize incremental GMV by allocating across three merchant categories: food, fuel, and recharge. Each rupee spent in food returns some incremental GMV, and so does each rupee in fuel and recharge — but with diminishing returns. The optimum allocation is the one where the marginal incremental GMV per rupee is identical across all three categories. If food were giving ₹4 of GMV per ₹1 of cashback and fuel were giving ₹3, you would move money from fuel to food until the marginals equalized. The common marginal value is precisely the Lagrange multiplier — the answer to "how much more incremental GMV would I earn if the CFO gave me one extra rupee of budget?"

This shadow-price interpretation is one of the most powerful ideas in PM economics. When you walk into a budget review and argue for an extra ₹10 lakh, the Lagrange multiplier of your binding constraint is exactly the number you should quote.

### 2. Formal Mathematical Definition

Equality-constrained problem. Minimize $f(\mathbf{x})$ subject to $h(\mathbf{x}) = 0$.

$$\min_{\mathbf{x} \in \mathbb{R}^n} f(\mathbf{x}) \quad \text{s.t.} \quad h_i(\mathbf{x}) = 0, \quad i = 1, \dots, m.$$

Plain English: minimize f of x subject to each h-sub-i of x equal to zero, for i from 1 to m.

Lagrangian. Introduce one multiplier $\lambda_i \in \mathbb{R}$ per equality constraint and form

$$\mathcal{L}(\mathbf{x}, \boldsymbol{\lambda}) = f(\mathbf{x}) + \sum_{i=1}^{m} \lambda_i \, h_i(\mathbf{x}).$$

Plain English: the Lagrangian L of x and lambda equals f of x plus the sum over i of lambda-i times h-sub-i of x.

First-order necessary condition (stationarity). At a regular local minimum $\mathbf{x}^\star$, there exist multipliers $\boldsymbol{\lambda}^\star$ such that

$$\nabla_{\mathbf{x}} \mathcal{L}(\mathbf{x}^\star, \boldsymbol{\lambda}^\star) = \nabla f(\mathbf{x}^\star) + \sum_{i=1}^{m} \lambda_i^\star \, \nabla h_i(\mathbf{x}^\star) = \mathbf{0}, \qquad h_i(\mathbf{x}^\star) = 0.$$

Plain English: the gradient of f at the optimum is a linear combination of the gradients of the active constraints; the constraint values themselves are zero.

Geometric derivation. Walk along the constraint surface $h(\mathbf{x}) = 0$. Any feasible direction $\mathbf{d}$ at $\mathbf{x}^\star$ must satisfy $\nabla h(\mathbf{x}^\star)^\top \mathbf{d} = 0$ (tangent to the surface). At a local minimum, moving in any feasible direction cannot decrease $f$: $\nabla f(\mathbf{x}^\star)^\top \mathbf{d} \ge 0$ for every such $\mathbf{d}$, and by symmetry (also $-\mathbf{d}$ is feasible) we get equality. Hence $\nabla f$ must be orthogonal to every tangent direction, which means $\nabla f$ is parallel to $\nabla h$. The proportionality constant is $-\lambda^\star$. With multiple constraints, $\nabla f$ lies in the span of $\{\nabla h_i\}$.

Shadow-price (sensitivity) theorem. Let $v(\mathbf{b}) = \min \{ f(\mathbf{x}) : h(\mathbf{x}) = \mathbf{b} \}$. Then, under regularity,

$$\frac{\partial v}{\partial b_i} = -\lambda_i^\star.$$

Plain English: relaxing constraint i by one unit changes the optimal value by negative lambda-i-star. This is the shadow price.

Symbol table.

| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| $f(\mathbf{x})$ | Objective (e.g., negative incremental GMV) | scalar | The KPI you ultimately move |
| $h_i(\mathbf{x})$ | Equality constraint i (e.g., total spend = budget) | scalar each | A non-negotiable rule (budget, regulation) |
| $\mathbf{x}$ | Decision vector (allocations, prices, weights) | $n$ | The knobs you control |
| $\lambda_i$ | Lagrange multiplier for constraint i | scalar each | Shadow price: KPI gain per unit of relaxed constraint |
| $\mathcal{L}$ | Lagrangian function | scalar | Augmented objective that absorbs the constraints |
| $\nabla_{\mathbf{x}} \mathcal{L}$ | Gradient of L wrt x | $n$ | Stationarity condition at the optimum |
| $v(\mathbf{b})$ | Optimal-value function in the constraint level | scalar | KPI as a function of the budget |

## 3. Fintech Worked Numeric Examples

Example A — Cashback budget split across two categories.

Maximize incremental GMV $G(x_1, x_2) = 200\sqrt{x_1} + 150\sqrt{x_2}$ (₹ lakh) subject to $x_1 + x_2 = 50$ (₹ lakh budget). Here $x_1$ is cashback spent on food, $x_2$ on fuel. We minimize $f = -G$.

Lagrangian: $\mathcal{L} = -200\sqrt{x_1} - 150\sqrt{x_2} + \lambda (x_1 + x_2 - 50)$.

Stationarity:

- $\partial_{x_1} \mathcal{L} = -100/\sqrt{x_1} + \lambda = 0 \Rightarrow \sqrt{x_1} = 100/\lambda.$
- $\partial_{x_2} \mathcal{L} = -75 /\sqrt{x_2} + \lambda = 0 \Rightarrow \sqrt{x_2} = 75 /\lambda.$

Plug into the constraint: $(100/\lambda)^2 + (75/\lambda)^2 = 50 \Rightarrow (10000 + 5625)/\lambda^2 = 50 \Rightarrow \lambda^2 = 15625/50 = 312.5 \Rightarrow \lambda \approx 17.6777.$

Therefore:

- $x_1^\star = (100/17.6777)^2 = (5.6569)^2 \approx 32.00$ ₹ lakh on food.
- $x_2^\star = (75 /17.6777)^2 = (4.2426)^2 \approx 18.00$ ₹ lakh on fuel.

Optimal GMV: $G^\star = 200\sqrt{32} + 150\sqrt{18} = 200 \cdot 5.6569 + 150 \cdot 4.2426 \approx 1131.37 + 636.40 = 1767.77$ ₹ lakh.

Shadow-price check. If the CFO gives one extra rupee (one extra ₹1 lakh), the new GMV is approximately $G^\star + (-\lambda)\cdot \Delta b$ where the constraint was $x_1 + x_2 - 50 = 0$ and we wrote the Lagrangian with a positive $\lambda$ multiplying $(x_1 + x_2 - 50)$. Increasing the right-hand side from 50 to 51 corresponds to $\Delta b = +1$. By the envelope theorem, $\partial v/\partial b = -\lambda \approx -17.68$ where $v$ is the minimization value $-G^\star$. So GMV grows by about ₹17.68 lakh per extra ₹1 lakh of budget — a 17.68× return on marginal spend. That single number is your CFO ask: "Every extra rupee of budget yields ₹17.68 of GMV at the current allocation."

Example B — MDR pricing under a regulator-imposed average cap.

Paytm earns from three rails: UPI-P2M, debit-card, credit-card. Let $r_1, r_2, r_3$ (in bps) be the MDR you charge on each rail, with volumes $V_1 = 100, V_2 = 40, V_3 = 20$ (₹ crore/day). Take-rate revenue is $R = r_1 V_1 + r_2 V_2 + r_3 V_3$. Merchants churn at rate proportional to the square of the rate, so net profit is

$$f(\mathbf{r}) = - \big( r_1 V_1 + r_2 V_2 + r_3 V_3 \big) + \tfrac{1}{2}\big( r_1^2 + r_2^2 + r_3^2 \big).$$

(We minimize $f$; the linear term is negative revenue, the quadratic is churn cost.) The regulator imposes a weighted average MDR cap: $(V_1 r_1 + V_2 r_2 + V_3 r_3)/(V_1+V_2+V_3) = 60$ bps, i.e. $h(\mathbf{r}) = V_1 r_1 + V_2 r_2 + V_3 r_3 - 60 \cdot 160 = 0$. Total volume is 160, so the right-hand side is $9600$.

Lagrangian: $\mathcal{L} = -(r_1\cdot 100 + r_2\cdot 40 + r_3\cdot 20) + \tfrac{1}{2}(r_1^2 + r_2^2 + r_3^2) + \lambda (100 r_1 + 40 r_2 + 20 r_3 - 9600).$

Stationarity:

- $\partial_{r_1} \mathcal{L} = -100 + r_1 + 100\lambda = 0 \Rightarrow r_1 = 100(1 - \lambda).$
- $\partial_{r_2} \mathcal{L} = -40 + r_2 + 40\lambda = 0 \Rightarrow r_2 = 40(1 - \lambda).$
- $\partial_{r_3} \mathcal{L} = -20 + r_3 + 20\lambda = 0 \Rightarrow r_3 = 20(1 - \lambda).$

Substitute into the constraint: $100\cdot 100(1-\lambda) + 40\cdot 40(1-\lambda) + 20\cdot 20(1-\lambda) = 9600 \Rightarrow (10000 + 1600 + 400)(1-\lambda) = 9600 \Rightarrow 12000(1-\lambda) = 9600 \Rightarrow 1-\lambda = 0.8 \Rightarrow \lambda = 0.2.$

Therefore the optimal rates are $r_1^\star = 80, r_2^\star = 32, r_3^\star = 16$ bps. The weighted average is $(100\cdot 80 + 40\cdot 32 + 20\cdot 16)/160 = (8000 + 1280 + 320)/160 = 9600/160 = 60$ bps — the cap binds exactly, as expected.

Shadow-price interpretation. $\lambda = 0.2$ is the cost (in profit units) of tightening the average cap by one unit on the constraint scale. Concretely, if the regulator lowered the cap from 60 bps to 59 bps, the constraint right-hand side falls by 160, and profit falls by approximately $0.2 \times 160 = 32$ units (here units are ₹ crore-bps; the precise units are tied to how you scaled the objective, and the direction — profit falls when the cap tightens — is what matters for the PM narrative).

## 4. Common Pitfalls

- Sign confusion on $\lambda$. The sign of $\lambda$ depends on how you wrote the Lagrangian ($+ \lambda h$ vs $- \lambda h$) and whether you are minimizing or maximizing. Always derive the shadow-price sign from first principles, never memorize it.
- Forgetting regularity (constraint qualification). Lagrange's theorem assumes $\nabla h_i$ are linearly independent at $\mathbf{x}^\star$ (LICQ). When constraints are redundant or tangent, multipliers may not exist or may not be unique.
- Saddle nature of $\mathcal{L}$. The Lagrangian is minimized in $\mathbf{x}$ and maximized in $\lambda$. Naively running gradient descent on $\mathcal{L}$ over both variables diverges. Use primal–dual methods or solve the KKT system directly.
- Treating inequality constraints with equality Lagrange. Inequalities $g(\mathbf{x}) \le 0$ require KKT, not plain Lagrange. The crucial extra piece is complementary slackness (next section).
- Misreading the shadow price. $\lambda^\star$ is the marginal sensitivity, valid only for small perturbations. Don't extrapolate — if the CFO doubles your budget, the marginal return at the new optimum will be different.

| PM Relevance |
| --- |
| - Why a PM must master this: Every product PM at Paytm runs a constrained optimization in practice — budget-constrained discounting, regulator-constrained pricing, SLA-constrained risk thresholds. Lagrange multipliers are how you put a number on "what is one more unit of constraint worth?" and that number is your ammunition in cross-functional reviews. |
| - Metrics to own: Marginal GMV per ₹1 of incremental cashback budget (the multiplier of the budget constraint); marginal profit per bps of regulatory MDR headroom (the multiplier of the rate cap); shadow price of risk-policy thresholds (multiplier on the approved-default-rate constraint). |
| - Strategic tradeoffs: Tight constraints (low budget, low MDR cap) protect the business from runaway loss but cost real money — the shadow price is exactly that cost. Loose constraints free up upside but expose you to regulatory or solvency risk. The PM's job is to know when the shadow price is high enough to justify negotiating the constraint. |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "At Paytm I split a fixed cashback budget across categories. The Lagrange multiplier of the budget constraint told me each extra rupee of budget produced about ₹17.68 of incremental GMV at the optimum — that became the exact CFO ask. My Georgia Tech MSCS gave me the optimization rigor to derive that number from first principles instead of A/B-guessing, and the same machinery scales straight to constrained training objectives in ML." |

### 6. NumPy-Only Python Snippet

```
import numpy as np

# Example A: Cashback budget split — closed-form via Lagrange
B = 50.0  # ₹ lakh
a1, a2 = 200.0, 150.0  # GMV coefficients (sqrt scaling)

# From stationarity: sqrt(x1) = (a1/2)/lambda, sqrt(x2) = (a2/2)/lambda
# Substitute into x1+x2=B
k = (a1/2)**2 + (a2/2)**2
lam = np.sqrt(k / B)
x1 = ((a1/2)/lam)**2
x2 = ((a2/2)/lam)**2
G  = a1*np.sqrt(x1) + a2*np.sqrt(x2)
print(f"x1*={x1:.4f}  x2*={x2:.4f}  GMV*={G:.4f}  lambda={lam:.4f}")
print(f"Marginal GMV per +1 lakh of budget ≈ {lam:.4f}")

# Example B: MDR rates under weighted-average cap
V  = np.array([100.0, 40.0, 20.0])
cap = 60.0
rhs = cap * V.sum()            # 9600

# Stationarity: r_i = V_i*(1 - lambda). Plug into V·r = rhs.
lam = 1.0 - rhs / np.sum(V*V)  # = 1 - 9600/12000 = 0.2
r   = V * (1.0 - lam)
wavg = (V @ r) / V.sum()
print(f"r*={r}  weighted-avg MDR={wavg:.2f} bps  lambda={lam:.4f}")
```

## 4.6 KKT Conditions and the Role in SVM Training

### 1. Plain-English Intuition

Lagrange multipliers handle equality constraints — rules that must hold exactly. Real product rules are usually inequalities: spend at most ₹50 lakh, default rate at most 2%, latency below 300 ms, MDR no higher than the regulator's cap. The Karush–Kuhn–Tucker (KKT) conditions generalize Lagrange to inequalities. They add one beautiful idea: complementary slackness. Either a constraint is binding (active at the optimum, multiplier positive) or it has slack (inactive, multiplier zero). You cannot have a constraint with room to spare and a positive shadow price — that would mean someone is paying for headroom they are not using.

Everyday analogy. Suppose you are filling a swimming pool and the rule is "do not exceed the rim." If you stop short of the rim, the constraint is inactive and its shadow price is zero — relaxing "rim height" by an inch buys you nothing because you weren't going to use that inch anyway. If you fill exactly to the rim, the constraint is binding and its shadow price is positive — every extra inch of rim is worth real water. At the optimum, either the constraint is tight or its multiplier is zero. Never both nonzero, never both slack.

Fintech analogy: merchant negotiating MDR under regulatory caps. When you sit across from a large merchant negotiating their MDR, the regulator's cap on average MDR is one constraint, the merchant's "I'll churn if MDR > X" is another, your minimum-margin requirement is a third. KKT tells you that only the binding constraints matter at the optimum — and the multiplier on each binding constraint is exactly the marginal profit you would gain if that constraint were relaxed by one unit. So if you are about to walk into the negotiation, your prep should be: compute the KKT multipliers in your current pricing model, sort by magnitude, and that sorted list is your negotiation priority — the constraint with the highest multiplier is the one most worth fighting to relax.

Why KKT is the bedrock of SVM training. A support vector machine asks: among all hyperplanes that separate the two classes, which one is most "comfortable" — i.e., maximizes the margin? "Separates" is an inequality constraint per data point. KKT applied to that constrained problem produces the famous result that only a handful of training points — the support vectors — have nonzero multipliers. Every other point is irrelevant to the final decision boundary. That is complementary slackness in action: non-support-vector points satisfy their margin constraint with strict slack, so their multipliers are zero, so they drop out of the dual. SVMs are sparse in their data dependence because KKT enforces sparsity automatically.

## 2. Formal Mathematical Definition

Primal problem (inequality and equality constraints).

$$ \min_{\mathbf{x} \in \mathbb{R}^n} f(\mathbf{x}) \quad \text{s.t.} \quad g_j(\mathbf{x}) \le 0, \; j=1,\dots,p; \quad h_i(\mathbf{x}) = 0, \; i=1,\dots,m. $$

Plain English: minimize f of x subject to each g-sub-j of x less than or equal to zero, and each h-sub-i of x equal to zero.

Lagrangian.

$$ \mathcal{L}(\mathbf{x}, \boldsymbol{\mu}, \boldsymbol{\lambda}) = f(\mathbf{x}) + \sum_{j=1}^{p} \mu_j \, g_j(\mathbf{x}) + \sum_{i=1}^{m} \lambda_i \, h_i(\mathbf{x}), \qquad \mu_j \ge 0. $$

Plain English: L equals f plus a sum of mu-j times g-j (with mu-j nonnegative) plus a sum of lambda-i times h-i.

KKT necessary conditions. Under a constraint qualification (e.g., LICQ or Slater's condition for convex problems), a local optimum $\mathbf{x}^\star$ admits multipliers $\boldsymbol{\mu}^\star \ge \mathbf{0}, \boldsymbol{\lambda}^\star$ such that:

1. Stationarity. $\nabla f(\mathbf{x}^\star) + \sum_j \mu_j^\star \nabla g_j(\mathbf{x}^\star) + \sum_i \lambda_i^\star \nabla h_i(\mathbf{x}^\star) = \mathbf{0}.$
2. Primal feasibility. $g_j(\mathbf{x}^\star) \le 0, \quad h_i(\mathbf{x}^\star) = 0.$
3. Dual feasibility. $\mu_j^\star \ge 0.$

4. Complementary slackness. $\mu_j^\star \, g_j(\mathbf{x}^\star) = 0$ for every $j$.

Plain English of (4): for each inequality, either the multiplier is zero, or the constraint is active ($g_j = 0$), or both — but you cannot have a positive multiplier on a slack constraint.

Sufficiency under convexity. If $f$ and all $g_j$ are convex and all $h_i$ are affine, KKT becomes both necessary and sufficient: any $(\mathbf{x}^\star, \boldsymbol{\mu}^\star, \boldsymbol{\lambda}^\star)$ satisfying KKT is a global optimum.

Sketch of derivation. Consider an active constraint $g_j(\mathbf{x}^\star) = 0$. Any feasible direction $\mathbf{d}$ must satisfy $\nabla g_j^\top \mathbf{d} \le 0$ (you cannot move into infeasibility). At an optimum, $\nabla f^\top \mathbf{d} \ge 0$ for every such direction. Farkas' lemma then asserts the existence of $\mu_j \ge 0$ such that $\nabla f + \sum_j \mu_j \nabla g_j + \sum_i \lambda_i \nabla h_i = \mathbf{0}$, with $\mu_j = 0$ for inactive $j$. The "either-or" of complementary slackness falls straight out of "inactive constraints contribute zero to the gradient balance."

Symbol table.

| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| $g_j$ | j-th inequality (budget, cap, latency, default rate) | scalar each | A ceiling/floor rule |
| $h_i$ | i-th equality | scalar each | An exact rule (e.g., books must balance) |
| $\mu_j \ge 0$ | KKT multiplier for inequality j | scalar each | Shadow price of relaxing the cap by one unit |
| $\lambda_i$ | Multiplier for equality i | scalar each (sign-free) | Shadow price of the equality level |
| Active set | $\{j : g_j(\mathbf{x}^\star) = 0\}$ | subset of $\{1,\dots,p\}$ | The constraints that actually bite |
| Support vectors (SVM) | Points whose margin constraint is active | subset of training set | The only points that determine the boundary |

Dual of an SVM (canonical KKT application). Hard-margin SVM primal:

$$\min_{\mathbf{w}, b} \tfrac{1}{2}\|\mathbf{w}\|^2 \quad \text{s.t.} \quad y_k(\mathbf{w}^\top \mathbf{x}_k + b) \ge 1, \; k=1,\dots,N.$$

Plain English: minimize one-half the squared norm of w subject to each margin constraint y-k times (w-dot-x-k plus b) at least one.

Lagrangian ($\alpha_k \ge 0$ are the KKT multipliers):

$$\mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha}) = \tfrac{1}{2}\|\mathbf{w}\|^2 - \sum_{k=1}^{N} \alpha_k \big( y_k(\mathbf{w}^\top \mathbf{x}_k + b) - 1 \big).$$

Stationarity:

- $\nabla_{\mathbf{w}} \mathcal{L} = \mathbf{w} - \sum_k \alpha_k y_k \mathbf{x}_k = \mathbf{0} \Rightarrow \mathbf{w}^\star = \sum_k \alpha_k^\star y_k \mathbf{x}_k.$
- $\partial_b \mathcal{L} = -\sum_k \alpha_k y_k = 0.$

Plug back to obtain the dual:

$$\max_{\boldsymbol{\alpha} \ge \mathbf{0}} \; \sum_k \alpha_k - \tfrac{1}{2} \sum_{k,\ell} \alpha_k \alpha_\ell y_k y_\ell \, \mathbf{x}_k^\top \mathbf{x}_\ell \quad \text{s.t.} \quad \sum_k \alpha_k y_k = 0.$$

Plain English: maximize the sum of alphas minus half the double-sum of alpha-k times alpha-ell times y-k times y-ell times the inner product of x-k and x-ell, subject to the alphas being nonnegative and weighted by y summing to zero.

Complementary slackness in SVM: $\alpha_k^\star \big( y_k(\mathbf{w}^{\star\top} \mathbf{x}_k + b^\star) - 1 \big) = 0.$ If a point lies strictly outside the margin, $y_k(\mathbf{w}^\top \mathbf{x}_k + b) > 1$, so $\alpha_k^\star = 0$. Only points exactly on the margin ($y_k(\mathbf{w}^\top \mathbf{x}_k + b) = 1$) can have $\alpha_k^\star > 0$. Those are the support vectors. The decision function $f(\mathbf{x}) = \sum_{k \in \text{SV}} \alpha_k^\star y_k \, \mathbf{x}_k^\top$

\mathbf{x} + b^\star \) depends only on them.

## 3. Fintech Worked Numeric Examples

Example A — KKT for cashback budget with an inequality "no more than ₹50 lakh."

We revisit Example A of §4.5 but now the constraint is \( x_1 + x_2 \le 50 \) (and \( x_1, x_2 \ge 0 \)). Same objective: \( G = 200\sqrt{x_1} + 150\sqrt{x_2} \).

Because \( G \) is increasing in both arguments (more budget always helps), the budget constraint must bind at the optimum: \( x_1 + x_2 = 50 \). The nonnegativity constraints are inactive (\( x_1^\star, x_2^\star > 0 \)), so their multipliers are zero.

KKT then collapses to the equality case from §4.5: \( x_1^\star = 32, x_2^\star = 18, \mu_{\text{budget}}^\star = \lambda \approx 17.68. \)

Step-by-step verification of complementary slackness:

| Constraint | Value at optimum | Multiplier | \( \mu \cdot g \) |
|---|---|---|---|
| \( x_1 + x_2 - 50 \le 0 \) | 0 (active) | 17.68 | 0 |
| \( -x_1 \le 0 \) | -32 (slack) | 0 | 0 |
| \( -x_2 \le 0 \) | -18 (slack) | 0 | 0 |

Now suppose the CFO offers two scenarios: (i) raise the budget cap to ₹55 lakh, or (ii) add a side rule \( x_2 \le 15 \) (cap fuel cashback because of fraud signals). KKT lets you price both:

- Scenario (i): Marginal GMV per extra ₹1 lakh budget = \( \mu \approx 17.68 \), so 5 extra lakh ₹88.4 lakh more GMV (to first order).
- Scenario (ii): The new constraint \( x_2 \le 15 \) cuts into the unconstrained optimum \( x_2^\star = 18 \), forcing \( x_2 = 15 \) and \( x_1 = 35 \). New GMV = \( 200\sqrt{35} + 150\sqrt{15} = 200\cdot 5.9161 + 150\cdot 3.8730 = 1183.22 + 580.95 = 1764.17 \). Lost GMV \( 1767.77 - 1764.17 = 3.60 \) ₹ lakh, so the multiplier on the new fuel cap is roughly \( 3.60/(18-15) = 1.20 \) ₹ lakh of GMV per ₹1 lakh of fuel-cap headroom. You now know: "loosening the fuel cap by ₹1 lakh buys 1.20 ₹ lakh of GMV; loosening total budget by ₹1 lakh buys 17.68 ₹ lakh." The total-budget multiplier is 14× larger — fight that fight first.

Example B — Tiny hard-margin SVM for fraud-vs-legit transaction classification.

Train an SVM on four 2-D points (features: \( x^{(1)} = \) normalized transaction amount, \( x^{(2)} = \) hour-of-day flag). Label \( y = +1 \) means fraud, \( y = -1 \) means legit.

| k | \( \mathbf{x}_k \) | \( y_k \) |
|---|---|---|
| 1 | (2, 2) | +1 |
| 2 | (2, 0) | +1 |
| 3 | (0, 0) | -1 |
| 4 | (0, 2) | -1 |

By symmetry, the optimal separator is vertical: \( w_1 x^{(1)} + w_2 x^{(2)} + b = 0 \) with \( w_2 = 0 \). Try \( \mathbf{w}^\star = (1, 0), b^\star = -1 \): margins are \( y_k(\mathbf{w}^\top \mathbf{x}_k + b) = y_k(x_k^{(1)} - 1) \). Check:

- k=1: \( +1 \cdot (2-1) = 1 \) active.
- k=2: \( +1 \cdot (2-1) = 1 \) active.
- k=3: \( -1 \cdot (0-1) = 1 \) active.
- k=4: \( -1 \cdot (0-1) = 1 \) active.

All four points sit on the margin, so all four are support vectors. Margin width \( = 2/\|\mathbf{w}\| = 2 \).

Solve the dual. With $\mathbf{w}^\star = \sum_k \alpha_k y_k \mathbf{x}_k = (1, 0)$, and $\sum_k \alpha_k y_k = 0$, we get two equations:

- $2\alpha_1 + 2\alpha_2 - 0 - 0 = 1 \Rightarrow \alpha_1 + \alpha_2 = 1/2.$ (first component of w)
- $2\alpha_1 + 0 - 0 - 2\alpha_4 = 0 \Rightarrow \alpha_1 = \alpha_4.$ (second component of w)
- $\alpha_1 + \alpha_2 - \alpha_3 - \alpha_4 = 0$ (b-stationarity)

From the symmetry $\alpha_1 = \alpha_2 = \alpha_3 = \alpha_4 = 1/4$ satisfies all three: $\alpha_1 + \alpha_2 = 1/2$ ; $\alpha_1 = \alpha_4 = 1/4$ ; $1/4 + 1/4 - 1/4 - 1/4 = 0$ .

Complementary slackness check. Every margin constraint is active (margin = 1), so $\alpha_k > 0$ is allowed for every k — consistent.

Recover the decision function: $f(\mathbf{x}) = \sum_k \alpha_k y_k \mathbf{x}_k^\top \mathbf{x} + b.$ Plug in:

- $\sum_k \alpha_k y_k \mathbf{x}_k = (1/4)(+1)(2,2) + (1/4)(+1)(2,0) + (1/4)(-1)(0,0) + (1/4)(-1)(0,2) = (1, 0).$
- $b^\star = -1$ (from any active constraint, e.g., $y_1(\mathbf{w}^\top \mathbf{x}_1 + b) = 1 \Rightarrow 2 + b = 1 \Rightarrow b = -1$).

So $f(\mathbf{x}) = x^{(1)} - 1$. The model says: classify a transaction as fraud whenever its normalized amount exceeds 1. The "hour-of-day" feature $x^{(2)}$ gets zero weight — even though it varies in the data, it carries no signal for separation in this toy. The KKT multipliers told us which dimensions matter.

## 4. Common Pitfalls

- Forgetting $\mu_j \ge 0$. The sign of the inequality multiplier is fixed by the convention $g_j \le 0$. Flipping the inequality flips the sign requirement.
- Stating KKT for non-convex problems as if it were sufficient. In non-convex settings KKT is only necessary — a KKT point may be a saddle or a local max.
- Ignoring constraint qualifications. Without LICQ or Slater, KKT may fail or multipliers may be unbounded. In practice, Slater (strict feasibility) is easy to check in convex problems.
- Conflating active set with support vectors in soft-margin SVM. With slack variables $\xi_k$ and box constraint $0 \le \alpha_k \le C$, points with $\alpha_k = C$ lie inside the margin (or are misclassified) and are still support vectors. The full taxonomy is: $\alpha_k = 0$ (outside margin, irrelevant), $0 < \alpha_k < C$ (on margin, "free" SV), $\alpha_k = C$ (margin violators, "bound" SV).
- Reading the dual without scaling intuition. Doubling all $\mathbf{x}_k$ does not change the geometry but scales $\alpha$ by $1/4$ (because the kernel doubles by 4). Always normalize features before reading multipliers as importances.
- Using KKT multipliers as feature importances directly. They are point-importances in SVMs, not feature-importances. For feature importance, look at $\mathbf{w}$ (linear kernel) or partial-dependence in general.

| PM Relevance |
| --- |
| - Why a PM must master this: Every Paytm product decision has inequality constraints — caps, floors, SLAs, regulatory ceilings, fraud thresholds. KKT is the only framework that simultaneously gives you (a) the optimal decision, (b) which constraints are actually biting, and (c) the marginal value of relaxing each. That triple is exactly what you need to walk into a regulatory meeting, a CFO review, or a partner negotiation. |
| - Metrics to own: Shadow prices on every binding cap (regulatory MDR, daily cashback, default-rate ceiling); count of "free" support vectors in your fraud SVM (drop-out rate of training data influence); ratio of bound to free SVs as a soft-margin health indicator. |
| - Strategic tradeoffs: A tight constraint with a small multiplier is essentially free to keep — don't waste capital trying to relax it. A slack constraint is a sign you have over-provisioned safety; consider tightening it elsewhere. Hard caps with huge multipliers are the negotiation gold — those are the rules whose relaxation pays for itself many times over. |

- 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "I think of every shipped product as a KKT solution. When I priced UPI MDR under the regulator's average-cap rule, the KKT multiplier on that cap told me the exact profit cost of each basis-point tightening — and complementary slackness told me which merchant-segment caps were truly binding versus which were slack. The same machinery powers SVMs: support vectors are the points whose KKT multipliers are nonzero, which is why an SVM trained on a million transactions might depend on only a few hundred. My Georgia Tech MSCS turned that connection from a textbook fact into something I can derive on a whiteboard in a hiring loop."

### 6. NumPy-Only Python Snippet

```
import numpy as np

# ------------ Example A: KKT for budget split with optional fuel cap ------------
def gmv(x1, x2): return 200.0*np.sqrt(x1) + 150.0*np.sqrt(x2)

# Unconstrained-style (budget=50, no fuel cap) closed form:
k = (200/2)**2 + (150/2)**2          # 10000 + 5625 = 15625
lam = np.sqrt(k / 50.0)              # ≈ 17.6777
x1, x2 = (100/lam)**2, (75/lam)**2
print(f"No fuel cap : x1={x1:.4f}  x2={x2:.4f}  GMV={gmv(x1,x2):.4f}  mu_budget={lam:.4f}")

# Add fuel cap x2 <= 15. It will bind (since x2*=18 > 15), so x2=15, x1=35.
x1c, x2c = 35.0, 15.0
loss = gmv(x1, x2) - gmv(x1c, x2c)
mu_fuel = loss / (18.0 - 15.0)       # marginal GMV per ₹1 lakh of fuel-cap headroom
print(f"With fuel cap: x1={x1c}  x2={x2c}  GMV={gmv(x1c,x2c):.4f}  "
      f"mu_fuel≈{mu_fuel:.4f}  (vs mu_budget={lam:.4f})")

# ------------ Example B: Tiny hard-margin SVM, verify KKT and dual ------------
X = np.array([[2,2],[2,0],[0,0],[0,2]], dtype=float)
y = np.array([+1, +1, -1, -1], dtype=float)
alpha = np.array([0.25, 0.25, 0.25, 0.25])   # solved by hand

# Recover w and b
w = (alpha * y) @ X
# b from any active constraint y_k (w·x_k + b) = 1
b = 1.0/y[0] - w @ X[0]
print(f"w={w}   b={b}")

# Check primal feasibility, complementary slackness, dual constraint
margins = y * (X @ w + b)
print("margins y_k(w·x_k+b):", margins)         # all should be 1.0
print("alpha · y (should be 0):", alpha @ y)
print("complementary slackness alpha_k*(margin_k-1):", alpha * (margins - 1.0))

# Decision boundary: x1 = 1  -> classify fraud iff normalized amount > 1
test = np.array([[1.5, 0.3], [0.2, 1.7]])
scores = test @ w + b
print("test scores:", scores, "  predicted labels:", np.sign(scores))
```

## Chapter 4 Closing Synthesis

Prabhjot, step back from the symbols and look at the arc of this chapter. You started by asking "what is the lowest point of this loss surface?" You learned that for convex problems any local minimum is global — that is the gift of convexity, and it is why logistic regression, linear regression, and the SVM dual are still everywhere in production fintech: they are honest. You then learned that the surfaces of deep models are non-convex, riddled with saddle points and flat plateaus, and that the entire optimizer zoo — momentum, RMSprop, Adam — exists to navigate that geometry without melting.

You derived gradient descent from a first-order Taylor expansion (echoing Chapter 3) and saw that the steepest-descent direction is just $-\nabla L$ — Chapter 3 was the calculus, Chapter 4 is its mechanical engineering. You saw that SGD is GD plus noise, that the noise is a feature (escape from saddles) not just a bug, and that mini-batches are the engineering compromise that makes the whole stack run on real Paytm-scale data.

You learned that constraints — budget, regulation, SLA — are not obstacles to optimization; they are part of it. Lagrange multipliers price equality constraints; KKT generalizes to inequalities and adds complementary slackness, which is the single most useful binary signal in PM economics: is this rule actually biting? And finally you saw that the support vector machine is just KKT made geometric: the support vectors are precisely the points whose dual multipliers escaped complementary slackness with a positive value. The same logic that tells you which of your regulatory caps is worth fighting tells the SVM which training points matter.

In Chapter 5 we will hand all of this to probability: turn losses into negative log-likelihoods, turn constraints into priors, and turn gradient descent into MAP estimation. The optimizers you just derived will reappear as posterior climbers. Read this chapter once more before moving on — the Adam derivation, the KKT geometry, and the shadow-price interpretation are the three concepts you will reuse most often, in both the classroom and the Paytm war room.

End of Volume 1, Chapters 3 and 4.

# Chapter 5 — Information Theory: The Currency of Surprise

Information theory is the mathematics of uncertainty, surprise, and compression. For a Paytm PM, it is the silent backbone of three things you already use every day: the loss function that trains every fraud classifier, the perplexity score your NLP team quotes when defending a new chatbot model, and the KL divergence guardrails that keep an LLM from drifting after fine-tuning. If calculus taught you how a model learns, information theory tells you what a model knows and how confidently wrong it is allowed to be.

This chapter restores seven instruments to your toolbox: Shannon entropy, joint entropy, conditional entropy, cross-entropy (binary and categorical), KL divergence, mutual information, and perplexity.

## 5.1 Shannon Entropy — The Price of Surprise

### Plain-English Intuition

Imagine you wake up in Bengaluru in April. You check the weather. "Sunny." You shrug — you knew that. Now imagine you wake up in July during monsoon and your phone says "Sunny." You raise an eyebrow. That second message carried more information because it was more surprising. Shannon's insight is that information is the logarithm of surprise, and entropy is the average surprise you should expect from a source.

Fintech analogy: think of Paytm's fraud alert stream. If 99.9% of transactions are legitimate, the message "this transaction is fine" carries almost zero information — you expected it. But the rare alert "this transaction is fraud" carries enormous information because it is rare and consequential. Entropy is the average informational weight of one such message, measured in bits. A high-entropy fraud signal is a noisy one (50/50 — useless); a low-entropy one is a confident one. The whole purpose of a classifier is to reduce the entropy of the world it observes.

### Formal Mathematical Definition

For a discrete random variable $X$ taking values $x_1, x_2, \ldots, x_n$ with probability mass function $p(x)$, the Shannon entropy is

$$ H(X) = -\sum_{i=1}^{n} p(x_i) \log_2 p(x_i) $$

Plain English: H of X equals the negative sum, over all outcomes i, of p of x sub i times log base 2 of p of x sub i.

We use base 2 to measure in bits; base $e$ gives nats; base 10 gives hartleys. By convention $0 \log 0 = 0$ because $\lim_{p \to 0^+} p \log p = 0$.

Derivation sketch (axiomatic). Shannon required three properties from an "information measure" $I(p)$: (i) continuity in $p$, (ii) monotonic decrease in $p$ (rarer events carry more info), and (iii) additivity for independent events: $I(p_1 p_2) = I(p_1) + I(p_2)$. The only function satisfying these (up to a base choice) is $I(p) = -\log p$. Entropy is then the expectation of self-information: $H(X) = \mathbb{E}[-\log p(X)]$.

Bounds. $0 \le H(X) \le \log_2 n$. The upper bound is hit when $p$ is uniform (maximum uncertainty); the lower bound when one outcome has probability 1 (no uncertainty).

| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| $X$ | Discrete random variable | scalar over $n$ outcomes | A categorical event (e.g., transaction status) |
| $p(x_i)$ | Probability of outcome $i$ | unitless, $[0,1]$ | Empirical rate from production logs |
| $n$ | Number of distinct outcomes | integer | Cardinality of the categorical feature |
| $H(X)$ | Entropy of $X$ | bits (if log base 2) | Average uncertainty per observation; "how noisy is this signal?" |
| $-\log_2 p(x_i)$ | Self-information of outcome $i$ | bits | Surprise weight of a single event |

## Worked Example 1 — Entropy of a Paytm UPI Transaction Status

Suppose on a normal Tuesday, Paytm UPI transactions resolve into four statuses with empirical frequencies measured over 10 million transactions:

- Success: $p_1 = 0.92$
- Pending (bank delay): $p_2 = 0.05$
- Failed (insufficient balance): $p_3 = 0.02$
- Failed (technical): $p_4 = 0.01$

Step 1 — Compute self-information for each outcome (bits):

- $-\log_2(0.92) = -(-0.1203) = 0.1203$ bits
- $-\log_2(0.05) = -(-4.3219) = 4.3219$ bits
- $-\log_2(0.02) = -(-5.6439) = 5.6439$ bits
- $-\log_2(0.01) = -(-6.6439) = 6.6439$ bits

Step 2 — Weight each by its probability:

- $0.92 \times 0.1203 = 0.1107$
- $0.05 \times 4.3219 = 0.2161$
- $0.02 \times 5.6439 = 0.1129$
- $0.01 \times 6.6439 = 0.0664$

Step 3 — Sum:

$$ H(X) = 0.1107 + 0.2161 + 0.1129 + 0.0664 = 0.5061 \text{ bits} $$

Plain English: H of X is approximately 0.5061 bits.

Interpretation for a PM. The UPI status channel carries about half a bit of information per transaction. If a teammate proposes a "transaction status anomaly model" that needs three bits of input entropy to be useful, you now know — quantitatively — that this signal alone is too thin and must be combined with merchant/device/geo features.

## Worked Example 2 — Entropy Reduction From a New KYC Feature

Before launching a new selfie-liveness check, Paytm's KYC reject reasons distribute as:

- Document blur: 0.40
- Name mismatch: 0.30
- Age below 18: 0.20
- Selfie mismatch: 0.10

Step 1 — Self-info:

- $-\log_2(0.40) = 1.3219$
- $-\log_2(0.30) = 1.7370$
- $-\log_2(0.20) = 2.3219$

- $-\log_2(0.10) = 3.3219$

Step 2 — Weighted:

- $0.40 \times 1.3219 = 0.5288$
- $0.30 \times 1.7370 = 0.5211$
- $0.20 \times 2.3219 = 0.4644$
- $0.10 \times 3.3219 = 0.3322$

Step 3 — Sum: $H_{\text{before}} = 1.8465$ bits.

After launching liveness, "Selfie mismatch" splits into two new sub-categories and the distribution becomes:

- Document blur: 0.40
- Name mismatch: 0.30
- Age below 18: 0.20
- Selfie-static mismatch: 0.06
- Selfie-liveness fail: 0.04

Self-info of the new tail:

- $-\log_2(0.06) = 4.0589$, weighted $0.06 \times 4.0589 = 0.2435$
- $-\log_2(0.04) = 4.6439$, weighted $0.04 \times 4.6439 = 0.1858$

$H_{\text{after}} = 0.5288 + 0.5211 + 0.4644 + 0.2435 + 0.1858 = 1.9436$ bits.

The KYC reject channel gained $1.9436 - 1.8465 = 0.0971$ bits per case — a measurable lift in diagnostic resolution. As a PM, you can quote this in a roadmap defense: "The liveness feature increases reject-reason entropy by ~5%, which expands our explainability surface and supports the new dispute-resolution flow."

## Common Pitfalls

- Confusing entropy with variance. Variance measures spread on a numeric scale; entropy measures uncertainty on a probability simplex. A degenerate distribution (always "Success") has zero entropy but can still have nonzero variance in some encodings.
- Using natural log when reporting "bits." Always convert: $H_{\text{bits}} = H_{\text{nats}} / \ln 2$.
- Ignoring the $0 \log 0 = 0$ convention. Code that does not handle zero probabilities will return NaN and silently corrupt dashboards.
- Estimating $p$ from tiny samples. Plug-in entropy estimators are biased downward for small $n$; use Miller–Madow correction in production telemetry.
- Treating entropy as quality. High entropy is not "bad" — it is just uncertain. A fair coin has maximum entropy and is perfectly well-calibrated.

| PM Relevance |
| --- |
| - Why a PM must master this: Every classifier loss, every A/B significance call, every "is this signal useful?" debate reduces to an entropy argument. Without it, you cannot tell a 95%-accurate model from a 95%-base-rate dataset. |
| - Metrics to own: Per-feature entropy in your event schema; reject-reason entropy in KYC; entropy of merchant category code (MCC) distribution among fraud cases; entropy decay after model deployment. |
| - Strategic tradeoffs: Collecting high-entropy features costs storage and privacy review but pays off in model lift; reducing user-facing entropy (fewer error codes shown) improves UX but hides diagnostic value from ops. |

- 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "Shannon entropy is the expected number of bits of surprise per observation. At Paytm, I use it to audit whether a feature actually carries predictive information before we onboard it into the fraud stack — a feature with sub-0.5-bit entropy across the population is rarely worth the engineering cost, regardless of what the AUC chart shows."

### NumPy-only Python Snippet

```
import numpy as np

def entropy_bits(p):
    p = np.asarray(p, dtype=np.float64)
    p = p[p > 0]                    # 0 log 0 = 0 by convention
    return float(-np.sum(p * np.log2(p)))

# Worked Example 1: UPI status entropy
upi = np.array([0.92, 0.05, 0.02, 0.01])
H_upi = entropy_bits(upi)
print(f"UPI status entropy:    {H_upi:.4f} bits")  # ~0.5061

# Worked Example 2: KYC reject entropy before / after liveness
kyc_before = np.array([0.40, 0.30, 0.20, 0.10])
kyc_after  = np.array([0.40, 0.30, 0.20, 0.06, 0.04])
print(f"KYC entropy (before):  {entropy_bits(kyc_before):.4f} bits")  # ~1.8465
print(f"KYC entropy (after):   {entropy_bits(kyc_after):.4f} bits")   # ~1.9436
print(f"Information gain:      {entropy_bits(kyc_after) - entropy_bits(kyc_before):.4f} bits")
```

## 5.2 Joint Entropy — Two Streams of Surprise

### Plain-English Intuition

If Shannon entropy is the surprise of one signal, joint entropy is the surprise of two signals observed together. Think of weather + traffic: knowing "it's raining" already hints "traffic will be bad," so the joint surprise of (rain, jam) is less than the sum of the individual surprises. Two correlated signals contain redundancy.

Fintech analogy: at Paytm, the pair (merchant category, transaction amount) is jointly distributed. A ₹5 transaction at a tea stall is unsurprising; a ₹50,000 transaction at the same stall is jointly improbable and therefore information-rich. Joint entropy quantifies how much total uncertainty lives in the pair, which is the foundation of every multi-feature fraud model you have ever shipped.

### Formal Mathematical Definition

For discrete random variables $X, Y$ with joint pmf $p(x, y)$,

$$ H(X, Y) = -\sum_{x \in \mathcal{X}} \sum_{y \in \mathcal{Y}} p(x, y) \log_2 p(x, y) $$

Plain English: H of X and Y equals the negative double sum, over all pairs (x, y), of joint probability p(x, y) times log base 2 of p(x, y).

Key identities.

- Non-negativity: $H(X, Y) \ge 0$.
- Sub-additivity: $H(X, Y) \le H(X) + H(Y)$, with equality iff $X \perp Y$.
- Chain rule: $H(X, Y) = H(X) + H(Y \mid X)$ — proven below in §5.3.

Proof of sub-additivity. By the log-sum inequality (or Gibbs' inequality), $\sum p(x,y) \log \frac{p(x,y)}{p(x)p(y)} \ge 0$, which rearranges to $H(X)+H(Y)-H(X,Y) \ge 0$. The non-negative gap is precisely the mutual information $I(X;Y)$ introduced in §5.6.

| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| $X, Y$ | Two discrete random variables | scalars | Two features observed together (e.g., MCC, amount-bucket) |

```
| \(p(x, y)\) | Joint probability | unitless, \([0,1]\) | Co-occurrence rate from logs |
| \(H(X,Y)\) | Joint entropy | bits | Total surprise of the pair |
| \(\mathcal{X}, \mathcal{Y}\) | Support sets | — | Allowed categorical values |
```

## Worked Example 1 — Joint Entropy of (Device Type, Transaction Outcome) at Paytm

Suppose we observe these joint frequencies over a representative day:

```
| | Success | Fail |
|---|---|---|
| **Android** | 0.70 | 0.05 |
| **iOS** | 0.20 | 0.02 |
| **Web** | 0.025 | 0.005 |
```

Marginals check: \(0.70+0.05+0.20+0.02+0.025+0.005 = 1.000\). Good.

Step 1 — Self-info for each cell (bits):

- \(-\log_2(0.70) = 0.5146\)
- \(-\log_2(0.05) = 4.3219\)
- \(-\log_2(0.20) = 2.3219\)
- \(-\log_2(0.02) = 5.6439\)
- \(-\log_2(0.025) = 5.3219\)
- \(-\log_2(0.005) = 7.6439\)

Step 2 — Weighted contributions:

- \(0.70 \times 0.5146 = 0.3602\)
- \(0.05 \times 4.3219 = 0.2161\)
- \(0.20 \times 2.3219 = 0.4644\)
- \(0.02 \times 5.6439 = 0.1129\)
- \(0.025 \times 5.3219 = 0.1330\)
- \(0.005 \times 7.6439 = 0.0382\)

Step 3 — Sum:

\[ H(X, Y) = 0.3602 + 0.2161 + 0.4644 + 0.1129 + 0.1330 + 0.0382 = 1.3248 \text{ bits} \]

Plain English: joint entropy is approximately 1.3248 bits.

## Worked Example 2 — Joint Entropy of (Hour Bucket, Channel) for Paytm Soundbox Notifications

Bucket the day into Morning, Afternoon, Evening; channel into Soundbox vs Push. Joint probabilities:

```
| | Soundbox | Push |
|---|---|---|
| Morning | 0.10 | 0.15 |
| Afternoon | 0.20 | 0.10 |
| Evening | 0.30 | 0.15 |
```

Sum: \(0.10+0.15+0.20+0.10+0.30+0.15 = 1.000\). Good.

Self-info:

- \(-\log_2(0.10) = 3.3219\)
- \(-\log_2(0.15) = 2.7370\)

- $-\log_2(0.20) = 2.3219$
- $-\log_2(0.30) = 1.7370$

Weighted:

- $0.10 \times 3.3219 = 0.3322$
- $0.15 \times 2.7370 = 0.4106$
- $0.20 \times 2.3219 = 0.4644$
- $0.10 \times 3.3219 = 0.3322$
- $0.30 \times 1.7370 = 0.5211$
- $0.15 \times 2.7370 = 0.4106$

Sum: $H(X,Y) = 0.3322+0.4106+0.4644+0.3322+0.5211+0.4106 = 2.4711$ bits.

Compare with the marginals: hour entropy $= H(\{0.25, 0.30, 0.45\}) = 1.5391$ bits; channel entropy $= H(\{0.60, 0.40\}) = 0.9710$ bits; sum of marginals $= 2.5101$ bits. The joint (2.4711) is slightly less than the sum of marginals (2.5101), confirming the two are mildly correlated — i.e., channel choice depends slightly on hour. The gap, 0.0390 bits, is the mutual information.

## Common Pitfalls

- Confusing joint with product distributions. $p(x,y) = p(x)p(y)$ only when independent. Otherwise the joint table is what you must measure.
- Estimating joint pmf from too few samples. A 3×2 table needs hundreds of samples per cell to estimate stably; 100×100 tables need millions.
- Ignoring marginal consistency. Always verify row and column sums equal the corresponding marginals before computing joint entropy.
- Forgetting non-negativity of MI. If your numerical pipeline returns a negative gap between $H(X)+H(Y)$ and $H(X,Y)$, you have an estimation bug.

| PM Relevance |
| --- |
| - Why a PM must master this: Multi-feature models live and die by joint distributions. Joint entropy lets you reason about feature redundancy before you spend a sprint engineering a 50-dim vector. |
| - Metrics to own: Joint entropy of (channel × hour), (MCC × amount-bucket), (geo × device); redundancy ratio $1 - H(X,Y)/(H(X)+H(Y))$. |
| - Strategic tradeoffs: Highly correlated features inflate model complexity without adding information; orthogonal features increase joint entropy and lift, but cost more to source. |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "Joint entropy is the total surprise of two signals observed together. When my fraud team adds a new feature, I ask for its joint entropy with our existing top three — if it is close to additive, we have a real signal; if not, we are paying engineering cost for redundancy." |

## NumPy-only Python Snippet

```
import numpy as np

def joint_entropy_bits(P):
    P = np.asarray(P, dtype=np.float64)
    Pf = P.flatten()
    Pf = Pf[Pf > 0]
    return float(-np.sum(Pf * np.log2(Pf)))

# Worked Example 1: device x outcome
P_device_outcome = np.array([
    [0.70, 0.05],   # Android
    [0.20, 0.02],   # iOS
    [0.025, 0.005], # Web
```

```
])
print(f"H(device, outcome) = {joint_entropy_bits(P_device_outcome):.4f} bits")  # ~1.3248

# Worked Example 2: hour x channel
P_hour_channel = np.array([
    [0.10, 0.15],
    [0.20, 0.10],
    [0.30, 0.15],
])
print(f"H(hour, channel)   = {joint_entropy_bits(P_hour_channel):.4f} bits")  # ~2.4711
```

## 5.3 Conditional Entropy — Surprise That Remains

### Plain-English Intuition

Conditional entropy is the surprise left in $Y$ after you have already observed $X$. If you already know it is monsoon season, the additional surprise from "today it rained" is small. Conditional entropy measures exactly that residual uncertainty.

Fintech analogy: at Paytm, once you condition on "merchant is a petrol pump," the distribution of transaction amounts becomes much tighter (₹200–₹3,000 range dominates). The conditional entropy $H(\text{amount} \mid \text{MCC})$ is much lower than the unconditional $H(\text{amount})$, and the drop is exactly the mutual information you are extracting by adding MCC as a feature.

### Formal Mathematical Definition

$$ H(Y \mid X) = \sum_{x} p(x) \, H(Y \mid X = x) = -\sum_{x, y} p(x, y) \log_2 p(y \mid x) $$

Plain English: H of Y given X is the average, weighted by p(x), of the entropy of Y conditional on each value of X.

Chain rule (proof). $$ H(X,Y) = -\sum_{x,y} p(x,y)\log_2 p(x,y) = -\sum_{x,y} p(x,y)\log_2[p(x)p(y\mid x)] $$ $$ = -\sum_{x,y} p(x,y)\log_2 p(x) - \sum_{x,y} p(x,y)\log_2 p(y\mid x) $$ $$ = -\sum_x p(x)\log_2 p(x) + H(Y\mid X) = H(X) + H(Y\mid X). \quad \blacksquare $$

Bounds. $0 \le H(Y \mid X) \le H(Y)$. Equality on the right iff $X \perp Y$; equality on the left iff $Y$ is a deterministic function of $X$.

| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| $H(Y\mid X)$ | Conditional entropy of $Y$ given $X$ | bits | Residual uncertainty in $Y$ after observing $X$ |
| $p(y\mid x)$ | Conditional pmf | unitless | "Given this MCC, how does amount distribute?" |
| $H(Y\mid X=x)$ | Entropy of $Y$ at a specific $x$ | bits | Per-segment uncertainty |

### Worked Example 1 — Conditional Entropy of Fraud Given Device

Reusing the (device, outcome) table from §5.2, treat Fail as "fraud-flag" for this exercise. Marginal device probs: $p(\text{Android})=0.75, p(\text{iOS})=0.22, p(\text{Web})=0.03$.

Conditional fail probabilities:

- $p(\text{Fail}\mid \text{Android}) = 0.05/0.75 = 0.0667$
- $p(\text{Fail}\mid \text{iOS}) = 0.02/0.22 = 0.0909$
- $p(\text{Fail}\mid \text{Web}) = 0.005/0.03 = 0.1667$

Per-segment binary entropies $H_b(q) = -q\log_2 q - (1-q)\log_2(1-q)$:

- Android: $H_b(0.0667) = -0.0667\log_2(0.0667) - 0.9333\log_2(0.9333) = 0.0667(3.9069) + 0.9333(0.0995) = 0.2606 + 0.0929 = 0.3535$ bits

- iOS: $H_b(0.0909) = 0.0909(3.4594) + 0.9091(0.1375) = 0.3145 + 0.1250 = 0.4395$ bits
- Web: $H_b(0.1667) = 0.1667(2.5850) + 0.8333(0.2630) = 0.4309 + 0.2192 = 0.6501$ bits

Weighted sum: $$ H(Y\mid X) = 0.75(0.3535) + 0.22(0.4395) + 0.03(0.6501) $$ $$ = 0.2651 + 0.0967 + 0.0195 = 0.3813 \text{ bits} $$

Compare to unconditional fail entropy: marginal $p(\text{Fail}) = 0.075$, so $H(Y) = H_b(0.075) = 0.075(3.7370) + 0.925(0.1125) = 0.2803 + 0.1041 = 0.3844$ bits.

Information gain from device feature: $H(Y) - H(Y\mid X) = 0.3844 - 0.3813 = 0.0031$ bits. Tiny. Lesson for the PM: device alone barely moves the needle for fraud — it must be combined with behavioral and network signals.

### Worked Example 2 — Conditional Entropy of Loan Default Given Bureau Tier

Paytm Postpaid users segmented by bureau tier:

| Tier | $p(\text{tier})$ | $p(\text{default}\mid \text{tier})$ |
|---|---|---|
| Prime | 0.40 | 0.01 |
| Near-prime | 0.35 | 0.04 |
| Sub-prime | 0.20 | 0.12 |
| Thin-file | 0.05 | 0.20 |

Per-tier binary entropies:

- Prime: $H_b(0.01) = 0.01(6.6439) + 0.99(0.0145) = 0.0664 + 0.0144 = 0.0808$
- Near-prime: $H_b(0.04) = 0.04(4.6439) + 0.96(0.0589) = 0.1858 + 0.0566 = 0.2423$
- Sub-prime: $H_b(0.12) = 0.12(3.0589) + 0.88(0.1844) = 0.3671 + 0.1623 = 0.5294$
- Thin-file: $H_b(0.20) = 0.20(2.3219) + 0.80(0.3219) = 0.4644 + 0.2575 = 0.7219$

Conditional entropy: $$ H(D\mid T) = 0.40(0.0808) + 0.35(0.2423) + 0.20(0.5294) + 0.05(0.7219) $$ $$ = 0.0323 + 0.0848 + 0.1059 + 0.0361 = 0.2591 \text{ bits} $$

Marginal default rate: $0.40(0.01)+0.35(0.04)+0.20(0.12)+0.05(0.20) = 0.004+0.014+0.024+0.010 = 0.052$. Unconditional $H(D) = H_b(0.052) = 0.052(4.2654)+0.948(0.0770) = 0.2218 + 0.0730 = 0.2948$ bits.

Information gain from tier: $0.2948 - 0.2591 = 0.0357$ bits per applicant — over 10× the device-only gain in Example 1. This is exactly why bureau tier is a non-negotiable feature in any underwriting model.

### Common Pitfalls

- Asymmetry trap. $H(Y\mid X) \ne H(X\mid Y)$ in general. Always be explicit about what conditions on what.
- Empty conditioning cells. If some $p(x) = 0$, drop them — do not let division-by-zero corrupt the sum.
- Confusing per-segment entropy with weighted average. Reporting $H(Y\mid X = x)$ for the worst segment as a headline is misleading; the policy-relevant quantity is the $p(x)$-weighted mean.
- Thinking "lower conditional entropy = better model." A degenerate predictor (constant) gives $H(Y\mid X) = H(Y)$ — no gain. The thing you optimize is the gap, i.e., mutual information.

| PM Relevance |
|---|
| - Why a PM must master this: Conditional entropy is the language of feature evaluation. Every "does this feature help?" debate is a question about $H(Y) - H(Y\mid X)$. |
| - Metrics to own: Conditional entropy of default given tier, fraud given device, churn given engagement bucket — and the resulting information gain per feature. |

- Strategic tradeoffs: Features that drive large conditional-entropy drops are worth integration cost; features that barely move it should be killed early, even if leadership likes them.

- 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "Conditional entropy is the surprise left after I observe a feature. At Paytm I rank candidate features by how much they shrink H(Y given X) versus H(Y) — it gives me a model-agnostic, calibration-free way to defend a feature roadmap to a skeptical CTO."

### NumPy-only Python Snippet

```
import numpy as np

def conditional_entropy_bits(P_xy):
    """P_xy: 2D joint, rows = X values, cols = Y values."""
    P = np.asarray(P_xy, dtype=np.float64)
    p_x = P.sum(axis=1, keepdims=True)  # marginal of X
    with np.errstate(divide="ignore", invalid="ignore"):
        p_y_given_x = np.where(p_x > 0, P / p_x, 0.0)
        terms = np.where(P > 0, P * np.log2(np.where(p_y_given_x > 0, p_y_given_x, 1)), 0.0)
    return float(-terms.sum())

# Worked Example 1: device x outcome
P_dev = np.array([[0.70, 0.05],
                  [0.20, 0.02],
                  [0.025, 0.005]])
print(f"H(outcome | device) = {conditional_entropy_bits(P_dev):.4f} bits")

# Worked Example 2: bureau tier x default
p_tier = np.array([0.40, 0.35, 0.20, 0.05])
p_def_given_tier = np.array([0.01, 0.04, 0.12, 0.20])
P_td = np.stack([p_tier * (1 - p_def_given_tier), p_tier * p_def_given_tier], axis=1)
print(f"H(default | tier)   = {conditional_entropy_bits(P_td):.4f} bits")
```

## 5.4 Cross-Entropy Loss — The Cost of Being Confidently Wrong

### Plain-English Intuition

Cross-entropy is the price you pay for using the wrong probability model. Imagine you bet on tomorrow's weather using last year's distribution. If last year was dry and this year is monsoon, your forecast assigns tiny probability to "heavy rain" — and every time it rains, you are massively surprised. Cross-entropy is your average surprise under reality, priced by your wrong model.

Fintech analogy: a fraud classifier that confidently scores a true-fraud transaction at $p = 0.001$ is "confidently wrong." Its per-event loss is $-\log_2(0.001) = 9.97$ bits — catastrophic. Compare to a humbler model that scores it at 0.3: loss is only 1.74 bits. The whole architecture of modern ML training is: punish confident wrongness, reward calibrated honesty. That is cross-entropy.

### Formal Mathematical Definition

Given a true distribution $p$ and a model distribution $q$ over the same support,

$$ H(p, q) = -\sum_{x} p(x) \log q(x) $$

Plain English: cross-entropy of p and q is the negative expectation, under p, of log q.

Decomposition. $$ H(p, q) = H(p) + D_{KL}(p \,\|\, q) $$

Plain English: cross-entropy equals the entropy of the truth plus the KL divergence from truth to model. Since $H(p)$ is fixed by the data, minimizing cross-entropy is equivalent to minimizing KL divergence. This is why every modern classifier is trained on cross-entropy.

Binary cross-entropy (BCE). For $y \in \{0,1\}$, $\hat y \in (0,1)$: $$ \mathcal{L}_{BCE}(y, \hat y) = -\left[y \log \hat y + (1-y) \log(1-\hat y)\right] $$

Plain English: BCE loss equals minus y log y-hat minus (1 minus y) log (1 minus y-hat).

Derivation from MLE. For a single Bernoulli observation with parameter \(\hat y\), the likelihood is \(\hat y^{\,y}(1-\hat y)^{1-y}\). The negative log-likelihood is exactly the BCE expression. Summing over \(N\) i.i.d. samples and dividing by \(N\) gives the empirical risk used in training.

Categorical cross-entropy (CCE). For one-hot \(\mathbf{y} \in \{0,1\}^K\), softmax output \(\hat{\mathbf{y}} \in \Delta^{K-1}\): \[ \mathcal{L}_{CCE}(\mathbf{y}, \hat{\mathbf{y}}) = -\sum_{k=1}^{K} y_k \log \hat y_k = -\log \hat y_{k^\star} \] where \(k^\star\) is the true class. Plain English: CCE collapses to minus log of the predicted probability of the correct class.

Derivation of softmax gradient. With softmax \(\hat y_k = e^{z_k}/\sum_j e^{z_j}\), one shows \(\partial \mathcal{L}_{CCE}/\partial z_k = \hat y_k - y_k\) — the famous "logit minus label" gradient that powers backprop in every classifier.

| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| \(p(x)\) | True (data) distribution | unitless | Ground-truth labels |
| \(q(x), \hat y\) | Model distribution / predicted prob | unitless | Calibrated risk score |
| \(H(p, q)\) | Cross-entropy | bits or nats | Average loss per sample |
| \(\mathcal{L}_{BCE}\) | Binary cross-entropy loss | nats (natural log default in ML libraries) | Loss for 2-class classifier |
| \(\mathcal{L}_{CCE}\) | Categorical cross-entropy loss | nats | Loss for \(K\)-class classifier |
| \(k^\star\) | Index of true class | integer | Ground-truth label index |

## Worked Example 1 — Binary Cross-Entropy on a Paytm Fraud Mini-Batch

Five transactions with true labels and model scores (probability of fraud):

| i | \(y_i\) (1=fraud) | \(\hat y_i\) |
|---|---|---|
| 1 | 0 | 0.05 |
| 2 | 0 | 0.10 |
| 3 | 1 | 0.80 |
| 4 | 1 | 0.30 |
| 5 | 0 | 0.02 |

Using natural log (standard in ML libraries):

- \(i=1\): \(-\log(1-0.05) = -\log(0.95) = 0.0513\)
- \(i=2\): \(-\log(0.90) = 0.1054\)
- \(i=3\): \(-\log(0.80) = 0.2231\)
- \(i=4\): \(-\log(0.30) = 1.2040\)  the "confidently wrong" sample
- \(i=5\): \(-\log(0.98) = 0.0202\)

Mean BCE: \[ \mathcal{L} = \tfrac{1}{5}(0.0513 + 0.1054 + 0.2231 + 1.2040 + 0.0202) = \tfrac{1.6040}{5} = 0.3208 \text{ nats} \]

Notice sample 4 alone contributes 75% of the loss. Lesson: in production, the long tail of confidently-wrong predictions, not the average accuracy, drives training signal. This is why a PM should always ask the data team for the loss histogram, not just the mean.

## Worked Example 2 — Categorical Cross-Entropy on a 3-Class Merchant Risk Model

Classes: {Low, Medium, High}. True labels and softmax outputs for four merchants:

| Merchant | True | \(\hat y_{Low}\) | \(\hat y_{Med}\) | \(\hat y_{High}\) |
|---|---|---|---|---|
| A | Low | 0.80 | 0.15 | 0.05 |
| B | Medium | 0.20 | 0.70 | 0.10 |
| C | High | 0.05 | 0.25 | 0.70 |
| D | High | 0.60 | 0.30 | 0.10 |

Loss per merchant (natural log):

- A: $-\log(0.80) = 0.2231$
- B: $-\log(0.70) = 0.3567$
- C: $-\log(0.70) = 0.3567$
- D: $-\log(0.10) = 2.3026$  confidently wrong

Mean CCE: $$\mathcal{L} = \tfrac{1}{4}(0.2231 + 0.3567 + 0.3567 + 2.3026) = \tfrac{3.2391}{4} = 0.8098 \text{ nats}$$

Merchant D's loss is ~7× the others. A PM looking at this would commission an error analysis: is D structurally different (new MCC? geographic outlier?) — exactly the kind of edge-case mining that improves a model more than another epoch of training.

## Common Pitfalls

- Mixing log bases. Most ML libraries report cross-entropy in nats (natural log); textbooks often use bits. Always state the base.
- Numerical instability with log(0). Always clip predictions to $[\epsilon, 1-\epsilon]$ before applying log. Better: use the logit-form loss (log-sum-exp trick) which is what `BCEWithLogitsLoss` does internally.
- Class imbalance. Naïve cross-entropy on a 99:1 fraud dataset will learn "predict 0" because the dominant class drives the loss. Use class weights or focal loss.
- Confusing cross-entropy with accuracy. A model can have low cross-entropy because it is well-calibrated, even if its top-1 accuracy is lower than a confident-but-wrong model.
- Treating BCE and CCE as interchangeable on multi-label problems. For multi-label (a transaction can be both "high-risk" and "high-velocity"), use sigmoid + per-class BCE, not softmax + CCE.

| PM Relevance |
| --- |
| - Why a PM must master this: This is the loss function. If you cannot articulate why your model is trained on cross-entropy, you cannot debate model choices, calibration, or fairness with engineering. |
| - Metrics to own: Train/val cross-entropy curves; per-segment cross-entropy (decile, MCC, geography); calibration plots (reliability diagrams). |
| - Strategic tradeoffs: Cross-entropy rewards calibrated probabilities; business may want hard thresholds. Decide where in the stack to translate probability  decision (and own that threshold yourself). |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "Cross-entropy is the cost of being confidently wrong. As a PM, I read it as a risk-weighted error: a fraud model with 0.1 cross-entropy but 5% of its mass on confidently-wrong false-negatives is more dangerous than one with 0.15 cross-entropy that is uniformly humble. That is the conversation cross-entropy lets me have with my ML team." |

## NumPy-only Python Snippet

```
import numpy as np

EPS = 1e-12

def bce_mean(y, y_hat):
    y = np.asarray(y, dtype=np.float64)
    y_hat = np.clip(np.asarray(y_hat, dtype=np.float64), EPS, 1 - EPS)
    return float(np.mean(-(y * np.log(y_hat) + (1 - y) * np.log(1 - y_hat))))

def cce_mean(Y_onehot, Y_hat):
    Y_onehot = np.asarray(Y_onehot, dtype=np.float64)
    Y_hat = np.clip(np.asarray(Y_hat, dtype=np.float64), EPS, 1.0)
    return float(np.mean(-np.sum(Y_onehot * np.log(Y_hat), axis=1)))

# Worked Example 1
y     = np.array([0, 0, 1, 1, 0])
y_hat = np.array([0.05, 0.10, 0.80, 0.30, 0.02])
print(f"BCE mean = {bce_mean(y, y_hat):.4f} nats")  # ~0.3208
```

```
# Worked Example 2
Y_onehot = np.array([[1,0,0], [0,1,0], [0,0,1], [0,0,1]])
Y_hat    = np.array([[0.80, 0.15, 0.05],
                     [0.20, 0.70, 0.10],
                     [0.05, 0.25, 0.70],
                     [0.60, 0.30, 0.10]])
print(f"CCE mean = {cce_mean(Y_onehot, Y_hat):.4f} nats")  # ~0.8098
```

## 5.5 KL Divergence — How Far Is Your Model From Reality?

### Plain-English Intuition

KL divergence measures how surprised you are when reality turns out to be $p$ but you encoded the world as $q$. It is not a distance (it is asymmetric, and does not satisfy the triangle inequality), but it is the canonical measure of "how wrong is my distribution?"

Fintech analogy: suppose last quarter's fraud distribution across MCCs was $p$, and the model you trained on data from two quarters ago assumed $q$. KL divergence $D_{KL}(p \,\|\, q)$ tells you, in bits, the average extra cost of pricing risk by yesterday's world. This is exactly the quantity that distribution-shift monitors track, and it is the metric every responsible LLM RLHF run constrains to keep the fine-tuned model from drifting.

### Formal Mathematical Definition

$$ D_{KL}(p \,\|\, q) = \sum_{x} p(x) \log_2 \frac{p(x)}{q(x)} $$

Plain English: KL divergence from p to q is the expectation, under p, of log of the ratio p over q.

Properties.

- Non-negativity (Gibbs' inequality): $D_{KL}(p \,\|\, q) \ge 0$, with equality iff $p = q$.
- Asymmetry: $D_{KL}(p \,\|\, q) \ne D_{KL}(q \,\|\, p)$ in general.
- Absolute continuity: $D_{KL}(p \,\|\, q) = \infty$ if $\exists x: p(x) > 0$ and $q(x) = 0$.
- Relation to cross-entropy: $D_{KL}(p\,\|\,q) = H(p,q) - H(p)$.

Proof of non-negativity. Apply Jensen's inequality to the convex function $-\log$: $$ -\sum p(x)\log\frac{q(x)}{p(x)} \ge -\log\sum p(x)\frac{q(x)}{p(x)} = -\log\sum q(x) = -\log 1 = 0. \quad \blacksquare $$

| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| $p(x)$ | Reference / true distribution | unitless | Today's empirical distribution |
| $q(x)$ | Approximating distribution | unitless | Model's assumed distribution |
| $D_{KL}(p\,\|\,q)$ | KL divergence p to q | bits | Excess surprise from using q instead of p |

### Worked Example 1 — Distribution Shift in Paytm MCC Mix

Yesterday's MCC fraud-rate distribution (model assumption):

- Grocery: 0.50, Travel: 0.20, Electronics: 0.20, Telecom: 0.10.

Today's actuals (post Diwali shopping spike):

- Grocery: 0.30, Travel: 0.10, Electronics: 0.50, Telecom: 0.10.

Compute $D_{KL}(p_\text{today} \,\|\, q_\text{model})$:

| Category | $p$ | $q$ | $p/q$ | $\log_2(p/q)$ | $p \log_2(p/q)$ |
|---|---|---|---|---|---|
| Grocery | 0.30 | 0.50 | 0.6000 | −0.7370 | −0.2211 |
| Travel | 0.10 | 0.20 | 0.5000 | −1.0000 | −0.1000 |
| Electronics | 0.50 | 0.20 | 2.5000 | 1.3219 | 0.6610 |
| Telecom | 0.10 | 0.10 | 1.0000 | 0.0000 | 0.0000 |

Sum: $D_{KL} = -0.2211 - 0.1000 + 0.6610 + 0.0000 = 0.3399$ bits.

Interpretation: the model is paying ~0.34 bits of extra surprise per transaction. Combined with daily volume of 200 M transactions, that is a huge aggregate signal that a retrain is overdue. A PM owning the fraud roadmap would set an SLA: trigger automatic retrain when daily $D_{KL}$ exceeds, say, 0.15 bits.

## Worked Example 2 — KL Divergence Between Two Postpaid Score Distributions

Two underwriting models on the same applicant population produce score-bucket distributions $p$ (champion) and $q$ (challenger), each over five deciles for brevity:

| Bucket | $p$ | $q$ |
|---|---|---|
| Very low | 0.10 | 0.05 |
| Low | 0.20 | 0.15 |
| Medium | 0.40 | 0.40 |
| High | 0.20 | 0.30 |
| Very high | 0.10 | 0.10 |

$D_{KL}(p\,\|\,q)$:

- Very low: $0.10 \log_2(0.10/0.05) = 0.10 \times 1.0000 = 0.1000$
- Low: $0.20 \log_2(0.20/0.15) = 0.20 \times 0.4150 = 0.0830$
- Medium: $0.40 \log_2(1.0000) = 0$
- High: $0.20 \log_2(0.20/0.30) = 0.20 \times (-0.5850) = -0.1170$
- Very high: $0.10 \log_2(1.0000) = 0$

Sum: $D_{KL}(p\,\|\,q) = 0.1000 + 0.0830 + 0 - 0.1170 + 0 = 0.0660$ bits.

Now reverse: $D_{KL}(q\,\|\,p)$:

- Very low: $0.05 \log_2(0.05/0.10) = 0.05 \times (-1.0000) = -0.0500$
- Low: $0.15 \log_2(0.15/0.20) = 0.15 \times (-0.4150) = -0.0623$
- Medium: 0
- High: $0.30 \log_2(0.30/0.20) = 0.30 \times 0.5850 = 0.1755$
- Very high: 0

Sum: $0.0632$ bits. Different from $0.0660$ — confirming asymmetry. Lesson for the PM: when reporting "distribution drift," always specify the direction of KL you are quoting.

## Common Pitfalls

- Treating KL as a distance. It is not symmetric; $D_{KL}(p\,\|\,q) \ne D_{KL}(q\,\|\,p)$. For symmetric needs use Jensen–Shannon divergence.
- Infinite KL from sparse data. If your empirical $q$ assigns zero to a bin where $p > 0$, KL blows up. Smooth (Laplace, Dirichlet prior) before computing.
- Misusing KL on continuous data. Convert to a binned pmf or use density estimators; never compute KL on raw samples without an explicit density.
- Aggregating without weighting. When monitoring drift across segments, compute weighted KL by segment volume, not a simple average.

PM Relevance

| |
|---|
| - Why a PM must master this: KL divergence is the workhorse of drift detection, RLHF guardrails, and Bayesian model comparison. Every modern LLM uses KL as a regularizer. |
| - Metrics to own: Daily/weekly KL drift on top features; KL between champion and challenger scoring outputs; KL between training-time and serving-time input distributions. |
| - Strategic tradeoffs: Tight KL thresholds catch drift early but cause alert fatigue; loose thresholds save ops time but let degraded models linger. |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "KL divergence is the extra bits of surprise from using yesterday's model on today's world. At Paytm I treat it as a leading indicator of model decay — the moment KL between training and serving feature distributions crosses my threshold, the retrain pipeline kicks in before AUC has even dropped." |

### NumPy-only Python Snippet

```
import numpy as np

def kl_bits(p, q, eps=1e-12):
    p = np.asarray(p, dtype=np.float64)
    q = np.asarray(q, dtype=np.float64)
    p = np.clip(p, eps, 1.0)
    q = np.clip(q, eps, 1.0)
    return float(np.sum(p * np.log2(p / q)))

# Worked Example 1: MCC shift
p_today = np.array([0.30, 0.10, 0.50, 0.10])
q_model = np.array([0.50, 0.20, 0.20, 0.10])
print(f"KL(today || model) = {kl_bits(p_today, q_model):.4f} bits")  # ~0.3399

# Worked Example 2: champion vs challenger
p = np.array([0.10, 0.20, 0.40, 0.20, 0.10])
q = np.array([0.05, 0.15, 0.40, 0.30, 0.10])
print(f"KL(p || q) = {kl_bits(p, q):.4f} bits")  # ~0.0660
print(f"KL(q || p) = {kl_bits(q, p):.4f} bits")  # ~0.0632
```

## 5.6 Mutual Information — How Much Does X Tell Me About Y?

### Plain-English Intuition

Mutual information (MI) is the bits-worth of $Y$ you learn by observing $X$. It is symmetric (unlike KL): the info $X$ gives about $Y$ equals the info $Y$ gives about $X$. MI captures any dependency, linear or nonlinear — which is why feature-importance rankings in modern ML often use MI rather than correlation.

Fintech analogy: at Paytm, knowing "user used a new device today" and the question "will this transaction be fraud?" share some MI. Pearson correlation would barely register because the relationship is sharply nonlinear (small effect at low velocities, big effect when combined with high amount). MI catches it.

### Formal Mathematical Definition

$$ I(X; Y) = \sum_{x,y} p(x,y) \log_2 \frac{p(x,y)}{p(x)p(y)} = D_{KL}\!\left(p(x,y) \,\|\, p(x)p(y)\right) $$

Plain English: mutual information of X and Y is the KL divergence between the joint and the product of marginals.

Equivalent forms. $$ I(X;Y) = H(X) - H(X\mid Y) = H(Y) - H(Y\mid X) = H(X) + H(Y) - H(X,Y) $$

Properties.

- Non-negativity: $I(X;Y) \ge 0$; zero iff $X \perp Y$.
- Symmetry: $I(X;Y) = I(Y;X)$.
- Upper bound: $I(X;Y) \le \min\{H(X), H(Y)\}$.

```
| Symbol | Definition | Dimensions | PM Interpretation |
```

```
|--------|------------|------------|--------------------|
| \(I(X;Y)\) | Mutual information | bits | How much \(X\) reduces uncertainty about \(Y\) |
| \(p(x)p(y)\) | Product of marginals | unitless | "Independent" baseline |
| \(p(x,y)\) | Joint pmf | unitless | Observed co-occurrence |
```

## Worked Example 1 — MI Between Device and Fraud at Paytm

Re-using the (device, outcome) table from §5.2:

- $H(Y) = H_b(0.075) = 0.3844$ bits (computed in §5.3)
- $H(Y\mid X) = 0.3813$ bits (computed in §5.3)

$$ I(X;Y) = H(Y) - H(Y\mid X) = 0.3844 - 0.3813 = 0.0031 \text{ bits} $$

Tiny — device alone is a weak feature for fraud.

## Worked Example 2 — MI Between Bureau Tier and Default

From §5.3:

- $H(D) = 0.2948$ bits
- $H(D\mid T) = 0.2591$ bits

$$ I(T;D) = 0.2948 - 0.2591 = 0.0357 \text{ bits} $$

Tier carries ~10× the information about default that device carries about fraud. Same units (bits), so directly comparable across feature/label pairs. This is the foundation of model-agnostic feature ranking.

PM application: present MI tables to leadership as a "feature value" league table. Unlike SHAP or feature importance from a tree, MI does not depend on any particular model — it is a property of the data.

## Common Pitfalls

- Estimator bias. Plug-in MI estimators systematically overestimate with finite samples. Always shuffle one column and compute the null distribution; report MI minus null-mean.
- Binning artifacts on continuous features. MI on continuous variables requires careful binning or KSG estimators; default histograms can be misleading.
- High-dimensional curse. Pairwise MI does not equal joint MI; high-arity feature interactions need different tools (interaction information, total correlation).
- MI is not causation. Two variables can share MI because of a common cause; never present MI as a causal claim.

| PM Relevance |
|---|
| - Why a PM must master this: MI is the model-free feature ranking metric. It survives every counter-argument about "your tree said X but my logistic said Y." |
| - Metrics to own: Per-feature MI vs label; pairwise MI matrix for redundancy detection; MI gain on held-out folds. |
| - Strategic tradeoffs: MI ranking can promote features that are genuinely informative but politically inconvenient (e.g., geo); be prepared to defend the math, then layer fairness controls on top. |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "Mutual information is the bits of label uncertainty that a feature kills. I use it at Paytm as an objective tiebreaker when two teams argue about which feature to onboard first — MI is model-agnostic and survives changes in the downstream classifier." |

## NumPy-only Python Snippet

```
import numpy as np

def mutual_information_bits(P_xy, eps=1e-12):
    P = np.asarray(P_xy, dtype=np.float64)
    px = P.sum(axis=1, keepdims=True)
    py = P.sum(axis=0, keepdims=True)
    PxPy = px @ py
    ratio = np.where(P > 0, P / np.clip(PxPy, eps, None), 1.0)
    terms = np.where(P > 0, P * np.log2(ratio), 0.0)
    return float(terms.sum())

# Worked Example 1: device x outcome
P_dev = np.array([[0.70, 0.05], [0.20, 0.02], [0.025, 0.005]])
print(f"I(device; outcome) = {mutual_information_bits(P_dev):.4f} bits")  # ~0.0031

# Worked Example 2: tier x default
p_tier = np.array([0.40, 0.35, 0.20, 0.05])
p_d_given_t = np.array([0.01, 0.04, 0.12, 0.20])
P_td = np.stack([p_tier * (1 - p_d_given_t), p_tier * p_d_given_t], axis=1)
print(f"I(tier; default)   = {mutual_information_bits(P_td):.4f} bits")  # ~0.0357
```

## 5.7 Perplexity — The LM Metric You Will Actually Ship

### Plain-English Intuition

Perplexity is the effective branching factor of a probabilistic model. If a language model has perplexity 30 on a test set, it is as confused as if it were uniformly guessing between 30 equally likely next tokens at each step. Lower is better; the floor depends on the inherent entropy of the language.

Fintech analogy: imagine a chatbot answering Paytm support queries. If its perplexity on real support transcripts is 15, it is essentially picking among 15 plausible next words at each step. Drop perplexity to 8 with fine-tuning and you have measurably narrowed the search; users perceive this as the bot "knowing what to say."

### Formal Mathematical Definition

For a language model assigning probability $P(w_1, \ldots, w_N)$ to a sequence, the test-set perplexity is

$$ \text{PP}(W) = P(w_1, \ldots, w_N)^{-1/N} = \exp\!\left(-\frac{1}{N}\sum_{i=1}^N \log P(w_i \mid w_{<i})\right) $$

Plain English: perplexity is the exponential of the average negative log-likelihood per token; equivalently, the geometric mean inverse-probability.

Relation to cross-entropy. Letting $H = -\frac{1}{N}\sum_i \log_2 P(w_i\mid w_{<i})$ (bits per token), $$ \text{PP} = 2^{H} $$

So perplexity is just cross-entropy on a multiplicative scale. Train with cross-entropy, report with perplexity.

| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| $W$ | Test sequence | length $N$ tokens | Held-out support transcripts, etc. |
| $P(w_i\mid w_{<i})$ | Model's next-token prob | unitless | Calibrated confidence in the right token |
| $\text{PP}(W)$ | Perplexity | unitless ("effective vocab") | Lower = sharper model |
| $H$ | Per-token cross-entropy | bits or nats | The training loss the team optimizes |

### Worked Example 1 — Perplexity of a Tiny Paytm Support Bigram Model

A toy LM is evaluated on a 5-token sequence "refund failed pending bank delay". The model assigns these conditional probabilities (already from a context-aware backoff bigram):

| Token | $P(w_i \mid w_{<i})$ |
|---|---|
| refund | 0.10 |
| failed | 0.20 |
| pending | 0.05 |
| bank | 0.40 |
| delay | 0.30 |

Step 1 — Log-probabilities (natural log):

- $\ln 0.10 = -2.3026$
- $\ln 0.20 = -1.6094$
- $\ln 0.05 = -2.9957$
- $\ln 0.40 = -0.9163$
- $\ln 0.30 = -1.2040$

Step 2 — Sum and average: $$ \sum = -2.3026 - 1.6094 - 2.9957 - 0.9163 - 1.2040 = -9.0280 $$ $$ \text{Avg NLL} = 9.0280 / 5 = 1.8056 \text{ nats per token} $$

Step 3 — Exponentiate: $$ \text{PP} = e^{1.8056} = 6.0838 $$

Plain English: perplexity is approximately 6.08. So this model is "as confused as choosing uniformly among ~6 tokens at each step." For a 30,000-token vocabulary, that is excellent; for a 5-token toy alphabet, mediocre.

## Worked Example 2 — Comparing Two Paytm Support Bots on the Same Test Set

Bot A and Bot B produce these average per-token cross-entropies on a 10,000-token internal support test set:

- Bot A: 4.20 bits/token  $\text{PP}_A = 2^{4.20} = 18.379$
- Bot B: 3.50 bits/token  $\text{PP}_B = 2^{3.50} = 11.314$

Relative reduction: $(18.379 - 11.314)/18.379 = 38.4\%$. Roadmap implication: if Bot B costs 1.4× the inference compute of Bot A but delivers a 38% perplexity reduction, the cost/perplexity-bit ratio is favorable — provided downstream human-eval metrics (helpfulness, hallucination rate) also improve. Perplexity is a necessary but not sufficient metric for LLM PMs.

## Common Pitfalls

- Comparing perplexity across tokenizers. Two models with different tokenizers are not directly comparable; normalize by characters or use bits-per-character.
- Reporting perplexity on training set. Always use held-out data; training perplexity tells you nothing about generalization.
- Conflating perplexity with quality. A model can have low perplexity and still hallucinate confidently. Pair perplexity with task-specific evals (BLEU, ROUGE, helpfulness ratings).
- Forgetting the base. $\text{PP} = 2^{H_{\text{bits}}} = e^{H_{\text{nats}}}$. Mixing bases inflates the reported number.
- Per-token bias on short sequences. For sequences shorter than ~50 tokens, perplexity is noisy; aggregate across a corpus.

| PM Relevance |
| --- |
| - Why a PM must master this: Perplexity is the headline metric every LLM team will quote in your review meetings. If you cannot interpret it, you cannot challenge or defend the spend. |
| - Metrics to own: Per-task perplexity (support, KYC, chat), per-segment (Hindi vs English), perplexity-per-rupee (cost-adjusted), human-eval correlation. |
| - Strategic tradeoffs: Lower perplexity often costs more compute; sometimes the right product call is to accept higher perplexity in exchange for latency or rupee savings. |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "Perplexity is just two-to-the cross-entropy. As a fintech PM, I treat it as the effective vocabulary the model has to guess from at each step — useful for relative ranking of LLMs on Paytm support transcripts, but I always pair it with human evals because a calibrated model can still be confidently and disastrously wrong." |

## NumPy-only Python Snippet

```
import numpy as np

def perplexity_from_probs(probs):
    probs = np.clip(np.asarray(probs, dtype=np.float64), 1e-12, 1.0)
    return float(np.exp(-np.mean(np.log(probs))))

def perplexity_from_bits_per_token(bpt):
    return float(2.0 ** bpt)

# Worked Example 1
seq_probs = np.array([0.10, 0.20, 0.05, 0.40, 0.30])
print(f"PP (toy bigram) = {perplexity_from_probs(seq_probs):.4f}")  # ~6.0838

# Worked Example 2
print(f"PP_A = {perplexity_from_bits_per_token(4.20):.4f}")   # ~18.379
print(f"PP_B = {perplexity_from_bits_per_token(3.50):.4f}")   # ~11.314
print(f"Reduction = {(perplexity_from_bits_per_token(4.20) - perplexity_from_bits_per_token(3.50)) /
perplexity_from_bits_per_token(4.20):.3%}")
```

## Chapter 5 — Closing Synthesis

You now hold the seven instruments of information theory. They form a single coherent picture:

\[ H(p,q) = H(p) + D_{KL}(p\,\|\,q), \quad I(X;Y) = H(X) - H(X\mid Y), \quad \text{PP} = 2^{H} \]

Train with cross-entropy; monitor drift with KL; rank features with mutual information; report LMs with perplexity; audit signals with Shannon entropy. Memorize that one sentence and you can hold your own with any modeling team at Paytm or anywhere else.

# Chapter 6 — Discrete Mathematics for AI: Graphs, Trees, Logic

Continuous mathematics tells you how much. Discrete mathematics tells you what connects to what, what follows from what, and in what order. Every modern ML system is, underneath, a discrete object: a computation DAG, a decision tree, a rule chain. For a PM at Paytm overseeing fraud, KYB, onboarding, and ML pipelines, fluency in graphs, trees, complexity, and logic is the difference between approving a pipeline you understand and rubber-stamping one you do not.

This chapter restores six instruments: graphs and their representations, trees and tree traversals, the O(V+E) traversal complexity result, DAGs for pipeline orchestration, propositional logic, first-order logic with resolution, and rule-based reasoning engines applied to fraud and KYB.

## 6.1 Graphs — The Mathematics of Things That Connect

### Plain-English Intuition

A graph is a set of things (vertices) and the relationships between them (edges). Bengaluru's road network is a graph: junctions are vertices, roads are edges. Your LinkedIn network is a graph. The merchant-acceptance network at Paytm — every QR code, every customer who scanned it — is a graph.

Fintech analogy: Paytm's payment-flow graph has tens of millions of vertices (customers, merchants, bank accounts) and billions of edges (transactions). When fraud investigators say "this account is two hops from a known mule," they are doing graph BFS at human speed. The algorithms in this chapter are how you do it at machine scale.

### Formal Mathematical Definition

A graph $G = (V, E)$ is a pair where $V$ is a finite set of vertices and $E \subseteq V \times V$ is a set of edges.

- Undirected graph: $E$ is a set of unordered pairs $\{u, v\}$.
- Directed graph (digraph): $E$ is a set of ordered pairs $(u, v)$.
- Weighted graph: an additional function $w: E \to \mathbb{R}$ assigns a real weight to each edge.

Degree. In an undirected graph, $\deg(v) = |\{u : \{u,v\} \in E\}|$. In a digraph, in-degree and out-degree are separate.

Handshake lemma. $\sum_{v \in V}\deg(v) = 2|E|$. Plain English: the sum of degrees equals twice the edge count. Proof: each edge contributes 1 to each of its two endpoints. $\blacksquare$

Representations.

1. Adjacency matrix $A \in \{0,1\}^{|V|\times|V|}$ with $A_{ij} = 1$ iff $(i,j) \in E$. Space: $O(|V|^2)$.
2. Adjacency list $\text{adj}[v] = \{u : (v,u) \in E\}$. Space: $O(|V| + |E|)$.

For Paytm-scale graphs (sparse: $|E| \ll |V|^2$), adjacency lists are mandatory.

Path, cycle, connectivity. A path is a sequence $v_0, v_1, \ldots, v_k$ with consecutive vertices adjacent. A cycle closes the path. A graph is connected if every pair of vertices has a path between them.

| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| $V$ | Vertex set | $\|V\| = n$ | Entities: customers, merchants, accounts |
| $E$ | Edge set | $\|E\| = m$ | Relationships: transactions, KYC linkages |
| $A$ | Adjacency matrix | $n \times n$ | Memory-heavy; rarely used at Paytm scale |
| $\text{adj}[v]$ | Adjacency list of $v$ | length $\deg(v)$ | The right representation for sparse |

```
graphs |
| \(w(e)\) | Edge weight | ₹, count, time, risk | Transaction amount, frequency, risk score |
```

## Worked Example 1 — Mule-Network Detection at Paytm

Consider a tiny suspicious sub-graph of 6 wallet accounts (vertices A–F) and directed transfers (edges) over 24 hours:

Edges (from  to, amount ₹):

- A  B (₹49,000)
- A  C (₹49,500)
- B  D (₹48,000)
- C  D (₹48,200)
- D  E (₹95,000)
- E  F (₹94,500)

Step 1 — Build adjacency list:

- A: [B, C]
- B: [D]
- C: [D]
- D: [E]
- E: [F]
- F: []

Step 2 — Compute in-degree and out-degree of D (the suspected funnel):

- In-degree(D) = 2 (from B, C)
- Out-degree(D) = 1 (to E)
- Amount flowing in: ₹48,000 + ₹48,200 = ₹96,200
- Amount flowing out: ₹95,000

Step 3 — Conservation gap: ₹96,200  ₹95,000 = ₹1,200 (close to balanced — classic funnel pattern). A is a "source" (in-deg 0), F is a "sink" (out-deg 0). Path \(A \to B \to D \to E \to F\) carries ~₹49K  ₹95K aggregation in 4 hops, a textbook layering pattern.

PM call to action: this is exactly the structural feature ("balanced-funnel sub-graph of depth  3") that becomes a feature in an upstream fraud model.

## Worked Example 2 — Merchant Acceptance Graph Connectivity

Paytm wants to know whether a new tier-3-city Soundbox rollout creates an isolated merchant cluster (bad: limited cross-merchant offers) or a connected one (good: viral growth). Suppose 5 new merchants {M1..M5} share customers as follows:

Edges (undirected, customer overlap > 100 shared customers):

- {M1, M2}, {M2, M3}, {M4, M5}

Step 1 — Adjacency list:

- M1: [M2], M2: [M1, M3], M3: [M2], M4: [M5], M5: [M4]

Step 2 — Run connected-components (BFS from each unvisited node): we find {M1, M2, M3} and {M4, M5}. Two components.

Step 3 — Conclusion: rollout is not fully connected; the {M4, M5} sub-cluster is isolated. PM action: seed a cross-promo to bridge them, e.g., a customer who shops at M3 and M4 in the same week, with the goal of merging components.

## Common Pitfalls

- Confusing directed and undirected semantics. A transaction edge is naturally directed; a "shared-customer" edge is naturally undirected. Mixing them silently corrupts community-detection algorithms.
- Using adjacency matrices on sparse graphs. A 10^8-vertex graph in matrix form is 10^16 entries — impossible. Lists or CSR sparse matrices only.
- Ignoring multi-edges. Two customers can transact many times; if your edge set is a set you lose count information. Use a multigraph or attach a weight.
- Trusting degree distributions blindly. Heavy-tailed degree distributions (typical in fintech) mean averages hide everything. Report medians and tail quantiles.

| PM Relevance |
| --- |
| - Why a PM must master this: Graphs underlie fraud rings, social-graph features, merchant acceptance networks, knowledge graphs, and every "two hops away from a bad actor" investigation. |
| - Metrics to own: Edge count growth, average degree, connected-component count, longest shortest-path (diameter), share of vertices in giant component. |
| - Strategic tradeoffs: Dense graphs are expensive to query but information-rich; sparsified graphs are fast but may miss patterns; choose where in the stack you can afford $O(V+E)$ vs $O(V^2)$. |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "Graphs are the data structure of payments. At Paytm I think about the transaction network as a directed weighted multigraph and ask my team for invariants — handshake lemma checks, component counts, degree-tail percentiles — before we ever build a model on top of it. The math is cheap; it catches data-pipeline bugs that the ML pipeline cannot." |

## NumPy-only Python Snippet

```
import numpy as np

# Worked Example 1: mule network as adjacency list + weights
edges = [
    ("A", "B", 49000),
    ("A", "C", 49500),
    ("B", "D", 48000),
    ("C", "D", 48200),
    ("D", "E", 95000),
    ("E", "F", 94500),
]
vertices = sorted({u for e in edges for u in e[:2]})
idx = {v: i for i, v in enumerate(vertices)}
n = len(vertices)

W = np.zeros((n, n), dtype=np.float64)
for u, v, w in edges:
    W[idx[u], idx[v]] = w

in_deg  = (W > 0).sum(axis=0)
out_deg = (W > 0).sum(axis=1)
in_amt  = W.sum(axis=0)
out_amt = W.sum(axis=1)
for v in vertices:
    i = idx[v]
    print(f"{v}: in_deg={in_deg[i]} out_deg={out_deg[i]} in_amt={in_amt[i]:.0f} out_amt={out_amt[i]:.0f}")

# Worked Example 2: connected components via BFS
adj = {"M1": ["M2"], "M2": ["M1","M3"], "M3": ["M2"], "M4": ["M5"], "M5": ["M4"]}
seen, components = set(), []
for start in adj:
    if start in seen: continue
    comp, frontier = [], [start]
    while frontier:
        v = frontier.pop()
        if v in seen: continue
        seen.add(v); comp.append(v)
```

```
            frontier.extend(adj[v])
        components.append(sorted(comp))
print("Components:", components)  # [['M1','M2','M3'], ['M4','M5']]
```

## 6.2 Trees — Hierarchies and Decisions

### Plain-English Intuition

A tree is a graph with no cycles and one root — the shape of every decision flow you have ever drawn on a whiteboard. Onboarding screens, KYC decision trees, organisational hierarchies, parse trees in NLP, gradient-boosted decision trees in your fraud model — all trees.

Fintech analogy: Paytm's onboarding flow is a decision tree. Root: "User opens app." Branch on "Has PAN?" "Has Aadhaar?" "OTP verified?" leaf decisions ("Approve / Pend / Reject"). The depth of this tree is the number of friction points; the leaves are the conversion outcomes. Tree mathematics tells you exactly how to count, balance, and traverse such flows.

### Formal Mathematical Definition

A tree is a connected acyclic undirected graph. A rooted tree designates one vertex as the root; every other vertex has a unique parent.

Counting identities.

- A tree on $n$ vertices has exactly $n - 1$ edges.
- Proof by induction: base case $n=1$, 0 edges. Inductive step: remove a leaf; the remainder is a tree on $n-1$ vertices with (by IH) $n-2$ edges; adding back gives $n-1$. $\blacksquare$

Depth and height. Depth of a vertex = number of edges from root; height of tree = max depth. A binary tree has each node with  2 children. A balanced binary tree has height $\Theta(\log n)$.

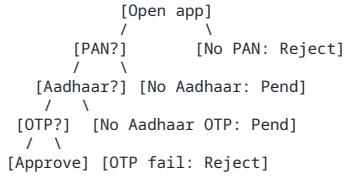
Tree traversals (rooted).

1. Pre-order: visit node, then children.
2. In-order: (binary) left subtree, node, right subtree.
3. Post-order: children, then node.
4. Level-order (BFS): by increasing depth.

| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| $T = (V, E)$ | Tree | $\|E\| = \|V\|-1$ | Decision flow, parse tree, GBM |
| root | Distinguished vertex | one | Funnel entry point |
| depth(v) | Edges from root to $v$ | integer | Friction steps |
| height(T) | Max depth | integer | Worst-case funnel depth |
| leaf | Vertex with no children | — | Terminal decision (approve/reject) |

### Worked Example 1 — Paytm Onboarding Decision Tree Depth

Consider this onboarding decision tree:



Step 1 — Count vertices: 8 vertices (Open, PAN?, NoPAN, Aadhaar?, NoAadhaar, OTP?, NoOTPAadhaar/wait let me recount cleanly).

Let me re-enumerate cleanly: Open app (1), PAN? (2), No-PAN-Reject (3), Aadhaar? (4), No-Aadhaar-Pend (5), OTP? (6), No-Aadhaar-OTP-Pend (7), Approve (8), OTP-Fail-Reject (9). That is $n = 9$ vertices.

Step 2 — Edges: by tree identity, exactly $n-1 = 8$ edges. Verify by listing parent-child pairs: (1-2), (1-3), (2-4), (2-5), (4-6), (4-7), (6-8), (6-9). Yes, 8.

Step 3 — Height: longest path root-to-leaf is Open  PAN?  Aadhaar?  OTP?  Approve, depth 4.

PM interpretation: the funnel has 4 friction points to "Approve." Each adds drop-off. If conversion at each step is 0.85, end-to-end approval rate is $0.85^4 = 0.522$. To raise this to 0.60, you can either (a) raise per-step pass rate, or (b) merge nodes (collapse Aadhaar-OTP and OTP steps if regulation permits) to reduce depth from 4 to 3, yielding $0.85^3 = 0.614$.

## Worked Example 2 — Gradient-Boosted Tree Depth in a Fraud Model

Paytm's fraud model uses 100 boosted trees, each with max depth 6. How many leaves per tree (worst case)? A full binary tree of depth $d$ has $2^d$ leaves, so each tree can have up to $2^6 = 64$ leaves. Across 100 trees: 6,400 possible leaf cells.

If each leaf stores a 32-bit float score, total model size is roughly $6400 \times 4 = 25{,}600$ bytes  25 KB — negligible. But inference traversal cost per transaction is $100 \times 6 = 600$ comparisons, in the latency budget of a sub-millisecond microservice. This is exactly why GBMs dominate fintech production: tiny memory, fast inference, interpretable splits.

## Common Pitfalls

- Confusing depth and height. Depth is per-vertex; height is a property of the whole tree.
- Allowing cycles to creep in. A "decision tree" with a back-edge ("retry OTP") is no longer a tree — it is a DAG (best) or a general graph. Make this explicit; otherwise traversal algorithms loop forever.
- Ignoring branching factor. A "wide and shallow" tree has very different UX than "deep and narrow," even with the same leaf count.
- Reading too much into a single tree. Random forests and GBMs aggregate hundreds of trees; never explain a model from one tree's structure.

| PM Relevance |
| --- |
| - Why a PM must master this: Onboarding flows, fraud rules, model architectures, and parse trees all share the same mathematics. Tree fluency makes you a credible interlocutor in design, fraud-ops, and ML reviews simultaneously. |
| - Metrics to own: Funnel depth, conversion at each depth, leaf-level fraud rate in the model, average tree depth in your GBM, drop-off by branch. |
| - Strategic tradeoffs: Deeper trees give finer decisions but compound friction (UX) or overfit (ML); shallower trees are robust but coarse. |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "Trees are everywhere in fintech — onboarding flows, gradient-boosted models, parse trees in our chatbot. I treat depth as the PM unit of friction: every extra level multiplies drop-off in UX and risk of overfit in ML. Knowing the n-1 edges identity and $2^d$ leaves bound lets me size flows and models on the back of a napkin." |

## NumPy-only Python Snippet

```
import numpy as np

# Worked Example 1: onboarding tree
```

```
parent = {  # child -> parent
    "PAN?":     "Open",
    "NoPAN-Rej":"Open",
    "Aadh?":    "PAN?",
    "NoAadh-Pend":"PAN?",
    "OTP?":     "Aadh?",
    "NoAadhOTP-Pend":"Aadh?",
    "Approve":  "OTP?",
    "OTPFail-Rej":"OTP?",
}
n = len(parent) + 1   # +1 for root
print(f"vertices = {n}, edges = {n-1}")

# depth of each node
def depth(v):
    d = 0
    while v in parent:
        v = parent[v]; d += 1
    return d
heights = {v: depth(v) for v in list(parent.keys()) + ["Open"]}
print("height(tree) =", max(heights.values()))

# end-to-end conversion under 0.85 per step at depth 4 vs depth 3
print(f"4-step approval rate = {0.85**4:.4f}")
print(f"3-step approval rate = {0.85**3:.4f}")

# Worked Example 2: GBM leaf budget
trees, max_depth = 100, 6
max_leaves_per_tree = 2 ** max_depth
total_leaves = trees * max_leaves_per_tree
print(f"max leaves per tree = {max_leaves_per_tree}, total = {total_leaves}")
print(f"per-tx comparisons   = {trees * max_depth}")
```

## 6.3 O(V+E) — Why BFS and DFS Scale to Paytm

### Plain-English Intuition

When you walk through a graph once, visiting every vertex and every edge exactly once, the cost is proportional to vertices plus edges, not their product. That is the magic of breadth-first and depth-first search: they are linear in the size of the data. This is the reason fraud investigators can do "find all accounts within 4 hops" on a billion-edge graph in seconds, not days.

Fintech analogy: imagine a courier in Bengaluru asked to visit every address in a colony exactly once. If the colony has 1,000 houses connected by 1,500 streets, the courier walks ~2,500 units of work — not 1.5 million. The linearity is what makes Paytm's real-time graph features feasible.

### Formal Mathematical Definition

Breadth-First Search (BFS). From source $s$:

```
visited[s] = True; queue = [s]
while queue:
    v = queue.dequeue()
    for u in adj[v]:
        if not visited[u]:
            visited[u] = True
            queue.enqueue(u)
```

Depth-First Search (DFS). Same as BFS but with a stack (LIFO) instead of a queue (FIFO), or recursively.

Complexity theorem. BFS and DFS each run in time $O(|V| + |E|)$ and space $O(|V|)$, assuming adjacency-list representation.

Proof. Each vertex is enqueued (resp. pushed) at most once  $O(|V|)$ overall queue operations. For each dequeued vertex $v$, the inner loop iterates over $\deg(v)$ neighbors  total inner iterations $= \sum_v \deg(v) = 2|E|$ for undirected (handshake lemma) or $|E|$ for directed. Therefore total work $= O(|V| + |E|)$. $\blacksquare$

Applications.

- BFS computes shortest paths in unweighted graphs (number of hops).

- DFS detects cycles, computes topological orderings on DAGs, identifies strongly connected components (Tarjan, Kosaraju).

| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| \(\|V\|\) | Vertex count | \(n\) | Number of entities to scan |
| \(\|E\|\) | Edge count | \(m\) | Number of relationships to follow |
| \(O(\|V\|+\|E\|)\) | Linear time | operations | Cost of one full graph sweep |
| `visited` | Boolean array | \(n\) bits | Has this entity been touched yet? |
| `queue` / `stack` | Frontier data structure | up to \(n\) | Breadth-first or depth-first order |

## Worked Example 1 — Hop-Distance Computation for Paytm Fraud Investigation

Given the mule-network graph from §6.1 (vertices A–F, directed edges), compute BFS distances from A.

Step 0: distances = {A:0, others:}, queue = [A].

Step 1: dequeue A. Neighbors B, C. distances[B]=1, distances[C]=1. queue = [B, C].

Step 2: dequeue B. Neighbor D. distances[D]=2. queue = [C, D].

Step 3: dequeue C. Neighbor D — already visited. queue = [D].

Step 4: dequeue D. Neighbor E. distances[E]=3. queue = [E].

Step 5: dequeue E. Neighbor F. distances[F]=4. queue = [F].

Step 6: dequeue F. No neighbors. Done.

Final distances from A: {A:0, B:1, C:1, D:2, E:3, F:4}.

Work performed: 6 vertices dequeued, 6 edges examined  12 unit-operations. By the theorem, \(O(|V|+|E|) = O(6+6) = O(12)\). Linearity verified.

PM interpretation: "Find all accounts within 3 hops of a known mule account" runs in time proportional to (touched vertices + touched edges) — perfectly feasible to do online for billions-of-edges graphs as long as the search horizon is bounded.

## Worked Example 2 — Cycle Detection via DFS on a KYB Cross-Holding Graph

Suppose a KYB investigation produces a directed graph of corporate cross-holdings: AB, BC, CA, DE. We want to know whether the company graph contains a circular ownership (a red flag).

Run DFS with three vertex colors {white, gray, black}:

Step 1: DFS from A. Mark A gray. Visit B (gray). Visit C (gray). Examine edge CA. A is gray, not white — a back edge, indicating a cycle. Detected: {A  B  C  A}.

Step 2: Continue. Mark C black, B black, A black. Move to D. Mark D gray. Visit E (gray). E has no out-edges. Mark E black, D black.

Total work: 5 vertices, 4 edges examined  \(O(5+4) = O(9)\). Cycle in the (A,B,C) component detected; (D,E) is acyclic.

PM interpretation: circular ownership is a known mechanism for opacity and audit-trail evasion. A fast cycle detector lets the KYB platform raise an alert before onboarding completes, not after a quarterly audit.

## Common Pitfalls

- Forgetting the `visited` set. Without it, BFS/DFS on a graph (not a tree) loops forever.

- Using adjacency matrices. Matrices give $O(|V|^2)$ traversal regardless of $|E|$ — disastrous for sparse graphs.
- Reporting BFS on weighted graphs as shortest path. BFS finds hop count, not weighted distance; use Dijkstra for that.
- Ignoring memory bound. $O(|V|)$ extra memory for `visited` can dominate on billion-vertex graphs; bitsets and external-memory algorithms are needed at scale.

| PM Relevance |
| --- |
| - Why a PM must master this: Every "two hops from a known bad actor" feature, every "is this onboarding application a duplicate" check, and every pipeline DAG validation uses BFS or DFS. Complexity classes determine whether a feature is viable at Paytm scale. |
| - Metrics to own: Average and 99th-percentile traversal latency, edges touched per query, frontier size growth. |
| - Strategic tradeoffs: Caching shortest-paths is fast but stale; recomputing is fresh but costly. Bound the hop horizon to make BFS feasible online. |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "BFS and DFS are linear in vertices plus edges. That single fact governs whether 'find all accounts within k hops of a bad actor' is an online feature or a batch job at Paytm. I use it to challenge proposals that quietly assume $O(V^2)$ — those never survive production at our scale." |

### NumPy-only Python Snippet

```
import numpy as np
from collections import deque

# Worked Example 1: BFS distances on mule network
adj = {"A":["B","C"], "B":["D"], "C":["D"], "D":["E"], "E":["F"], "F":[]}
def bfs_distances(adj, src):
    dist = {v: np.inf for v in adj}
    dist[src] = 0
    q = deque([src])
    edges_examined = 0
    while q:
        v = q.popleft()
        for u in adj[v]:
            edges_examined += 1
            if np.isinf(dist[u]):
                dist[u] = dist[v] + 1
                q.append(u)
    return dist, edges_examined

d, e_examined = bfs_distances(adj, "A")
print("Distances:", d)
print(f"Edges examined = {e_examined}  (theory: ~|E| = 6)")

# Worked Example 2: cycle detection via DFS coloring
def has_cycle(adj):
    WHITE, GRAY, BLACK = 0, 1, 2
    color = {v: WHITE for v in adj}
    def dfs(v):
        color[v] = GRAY
        for u in adj[v]:
            if color[u] == GRAY: return True
            if color[u] == WHITE and dfs(u): return True
        color[v] = BLACK
        return False
    return any(color[v] == WHITE and dfs(v) for v in adj)

g_cycle = {"A":["B"], "B":["C"], "C":["A"], "D":["E"], "E":[]}
print("Cycle?", has_cycle(g_cycle))   # True
g_acyc  = {"A":["B"], "B":["C"], "C":[], "D":["E"], "E":[]}
print("Cycle?", has_cycle(g_acyc))    # False
```

## 6.4 DAGs — The Mathematics of ML Pipeline Orchestration

### Plain-English Intuition

A directed acyclic graph (DAG) is a directed graph that has no way back. Think of a recipe: chop the onion before you sauté it; you cannot un-sauté. Every ML pipeline — Airflow, Kubeflow, the dependency graph inside your Spark job — is a DAG. The mathematics says: if there are no cycles, there exists at least one linear order in which every dependency precedes its dependent. That linear order is a topological sort.

Fintech analogy: Paytm's daily fraud-features pipeline is a DAG. Raw transactions feed feature jobs; feature jobs feed model scoring; model scoring feeds the risk dashboard. If anyone introduces a cycle (e.g., "yesterday's risk score is an input to today's risk score"), the DAG breaks. Topological sort tells the orchestrator the unique (or one of finitely many) safe execution orders.

## Formal Mathematical Definition

A DAG is a directed graph with no directed cycle.

Topological sort. A linear ordering $\sigma: V \to \{1, 2, \ldots, n\}$ such that for every directed edge $(u, v)$, $\sigma(u) < \sigma(v)$.

Existence theorem. A directed graph has a topological sort if and only if it is a DAG.

Proof (). Suppose a topological order exists; if there were a cycle $v_1 \to v_2 \to \cdots \to v_k \to v_1$, then $\sigma(v_1) < \sigma(v_2) < \cdots < \sigma(v_k) < \sigma(v_1)$ — contradiction.

Proof () by Kahn's algorithm. Repeatedly remove a vertex with in-degree 0, output it, decrement in-degrees of its successors. If a DAG, every removal step finds a source (otherwise the remaining sub-graph would have all vertices with in-deg  1, implying a cycle by pigeonhole on the path-walk argument). Algorithm runs in $O(|V|+|E|)$. $\blacksquare$

Critical path. In a weighted DAG, the longest path from source to sink under edge weights is the critical path — the bottleneck of the schedule. Computable in $O(|V|+|E|)$ on a DAG.

| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| DAG | Directed acyclic graph | — | Pipeline / dependency graph |
| $\sigma$ | Topological order | $V \to \mathbb{N}$ | Safe execution sequence |
| in-degree(v) | # incoming edges | integer | Number of unsatisfied prerequisites |
| critical path | Longest weighted path | time units | The bottleneck job sequence |

## Worked Example 1 — Topological Sort of Paytm Fraud Feature Pipeline

Jobs (vertices) and dependencies (edges):

- raw_tx  feat_velocity
- raw_tx  feat_amount
- raw_kyc  feat_kyc
- feat_velocity  score_model
- feat_amount  score_model
- feat_kyc  score_model
- score_model  dashboard

Step 1 — In-degrees:

- raw_tx: 0; raw_kyc: 0
- feat_velocity: 1; feat_amount: 1; feat_kyc: 1
- score_model: 3
- dashboard: 1

Step 2 — Kahn's algorithm:

- Start queue with in-degree-0 nodes: [raw_tx, raw_kyc].
- Pop raw_tx  order=[raw_tx]. Decrement feat_velocity (0), feat_amount (0). Queue: [raw_kyc, feat_velocity, feat_amount].

- Pop raw_kyc  order=[raw_tx, raw_kyc]. Decrement feat_kyc (0). Queue: [feat_velocity, feat_amount, feat_kyc].
- Pop feat_velocity  order=[raw_tx, raw_kyc, feat_velocity]. Decrement score_model (2).
- Pop feat_amount  decrement score_model (1).
- Pop feat_kyc  decrement score_model (0). Queue: [score_model].
- Pop score_model  decrement dashboard (0). Queue: [dashboard].
- Pop dashboard  order complete.

Topological order: raw_tx, raw_kyc, feat_velocity, feat_amount, feat_kyc, score_model, dashboard. Valid execution sequence.

## Worked Example 2 — Critical-Path Analysis of an Onboarding Risk Pipeline

Job durations (minutes) and dependencies:

```
| Job | Duration | Depends on |
|---|---|---|
| ingest | 10 | — |
| ocr_pan | 30 | ingest |
| ocr_aadhaar | 25 | ingest |
| bureau_pull | 60 | ingest |
| risk_score | 15 | ocr_pan, ocr_aadhaar, bureau_pull |
| decision | 5 | risk_score |
```

Step 1 — Topological order: ingest, ocr_pan, ocr_aadhaar, bureau_pull, risk_score, decision.

Step 2 — Earliest start time (ES) and earliest finish time (EF):

- ingest: ES=0, EF=10
- ocr_pan: ES=10, EF=40
- ocr_aadhaar: ES=10, EF=35
- bureau_pull: ES=10, EF=70
- risk_score: ES=max(40, 35, 70)=70, EF=85
- decision: ES=85, EF=90

Step 3 — Critical path: ingest  bureau_pull  risk_score  decision, total 90 minutes.

PM action: if you want to cut end-to-end onboarding latency, attack the critical-path job, not the others. Halving OCR is useless (it is not on the critical path); halving bureau_pull from 60 to 30 minutes saves 30 of the 90 minutes — a 33% improvement. This is the math that turns "make the pipeline faster" into a specific engineering ticket.

## Common Pitfalls

- Hidden cycles via shared state. "Today's feature uses yesterday's score" is a temporal cycle disguised as acyclic. Make the time-index explicit.
- Treating DAGs as trees. A DAG can have multiple parents per node (e.g., score_model has 3 predecessors). Tree algorithms misbehave.
- Ignoring the critical path. Optimizing non-critical jobs is wasted engineering effort.
- Re-running the whole DAG on partial failure. Idempotency at the node level + topological resumption is the right pattern; full re-runs are expensive.

| PM Relevance |
| --- |
| - Why a PM must master this: Every Airflow/Kubeflow review, every "why is the pipeline slow?" question, and every dependency outage post-mortem is a DAG analysis. |

| |
|---|
| - Metrics to own: End-to-end pipeline duration, critical-path length, slack per non-critical node, failure-blast radius (descendants of a failed node). |
| - Strategic tradeoffs: Parallelizing non-critical branches costs compute and yields no end-to-end gain; the only PM-relevant optimization is on the critical path. |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "Every ML pipeline at Paytm is a DAG. I prioritize engineering work strictly by the critical path — halving a non-critical job is engineering theater. Topological sort and critical-path math give me the same vocabulary as the platform team and stop arguments before they start." |

### NumPy-only Python Snippet

```
import numpy as np
from collections import deque

# Worked Example 1: Kahn's topological sort
edges = [
    ("raw_tx", "feat_velocity"),
    ("raw_tx", "feat_amount"),
    ("raw_kyc", "feat_kyc"),
    ("feat_velocity", "score_model"),
    ("feat_amount", "score_model"),
    ("feat_kyc", "score_model"),
    ("score_model", "dashboard"),
]
V = sorted({u for e in edges for u in e})
adj = {v: [] for v in V}
indeg = {v: 0 for v in V}
for u, v in edges:
    adj[u].append(v); indeg[v] += 1

q = deque([v for v in V if indeg[v] == 0])
topo = []
while q:
    v = q.popleft(); topo.append(v)
    for u in adj[v]:
        indeg[u] -= 1
        if indeg[u] == 0: q.append(u)
print("Topological order:", topo)

# Worked Example 2: critical path via longest-path-on-DAG
dur = {"ingest":10, "ocr_pan":30, "ocr_aadhaar":25, "bureau_pull":60, "risk_score":15, "decision":5}
deps = {
    "ingest": [], "ocr_pan":["ingest"], "ocr_aadhaar":["ingest"], "bureau_pull":["ingest"],
    "risk_score":["ocr_pan","ocr_aadhaar","bureau_pull"], "decision":["risk_score"]
}
order = ["ingest","ocr_pan","ocr_aadhaar","bureau_pull","risk_score","decision"]
EF = {}
for j in order:
    ES = max((EF[p] for p in deps[j]), default=0)
    EF[j] = ES + dur[j]
print("EF:", EF)
print("Critical-path length =", EF["decision"], "min")
```

## 6.5 Propositional Logic — The Algebra of True and False

### Plain-English Intuition

Propositional logic is the mathematics of statements that are either true or false, combined with AND, OR, NOT, and IMPLIES. It is the foundation of every rule engine, every SQL WHERE clause, every fraud rule, every KYB risk check. If you can write "IF transaction_amount > 50000 AND device_new AND no_otp THEN flag," you are doing propositional logic.

Fintech analogy: Paytm's fraud rule engine is, at its core, a giant propositional formula evaluated on each transaction. The PM's job is to know which rules to add, which to retire, and — critically — how to test whether the rule set is consistent (no rule contradicts another) and complete (covers known fraud patterns). Both are propositional-logic questions.

### Formal Mathematical Definition

A propositional variable is a symbol $p, q, r, \ldots$ ranging over $\{T, F\}$.

Connectives.

- Negation: $\neg p$ ("not p")
- Conjunction: $p \wedge q$ ("p and q")
- Disjunction: $p \vee q$ ("p or q")
- Implication: $p \to q$ ("if p then q"), defined as $\neg p \vee q$
- Biconditional: $p \leftrightarrow q$, defined as $(p \to q) \wedge (q \to p)$

Truth-table semantics assigns a value in $\{T, F\}$ to each formula based on its variables.

Tautology / contradiction / contingency. A formula true under every assignment is a tautology; false under every is a contradiction; otherwise contingent.

Logical equivalence. $\varphi \equiv \psi$ iff they have the same truth table. Key equivalences:

- De Morgan: $\neg(p \wedge q) \equiv \neg p \vee \neg q$; $\neg(p \vee q) \equiv \neg p \wedge \neg q$.
- Implication: $p \to q \equiv \neg p \vee q \equiv \neg q \to \neg p$ (contrapositive).
- Distribution: $p \wedge (q \vee r) \equiv (p \wedge q) \vee (p \wedge r)$.

Normal forms.

- CNF: conjunction of disjunctions (used by SAT solvers, resolution).
- DNF: disjunction of conjunctions (used in rule engines: each conjunction is one rule).

Inference rules. Modus ponens: from $p$ and $p \to q$, infer $q$. Modus tollens: from $\neg q$ and $p \to q$, infer $\neg p$.

| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| $p, q, r$ | Propositional variables | bool | Atomic predicates on a tx/user |
| $\wedge, \vee, \neg, \to$ | Connectives | function on bools | AND, OR, NOT, IF-THEN |
| CNF | Conjunction of clauses | — | Encoding for SAT-based rule consistency |
| DNF | Disjunction of conjunctions | — | Rule-engine shape: each disjunct = one rule |

## Worked Example 1 — Encoding a Paytm Fraud Rule and Proving Its Contrapositive

Define propositions:

- $A$: transaction amount > ₹50,000
- $N$: device is new (first seen in last 24 hours)
- $O$: OTP verified
- $F$: transaction is flagged for review

Rule: "If amount > 50K and device new and OTP not verified, then flag." $$ (A \wedge N \wedge \neg O) \to F $$

Plain English: A and N and not O implies F.

Contrapositive: $\neg F \to \neg(A \wedge N \wedge \neg O) \equiv \neg F \to (\neg A \vee \neg N \vee O)$. Plain English: if the transaction is not flagged, then either amount  50K, or device is not new, or OTP was verified.

Truth-table check (8 rows over A, N, O), focusing on $F$ value implied by the rule:

| A | N | O | A∧N∧¬O | rule implies F = ? |
|---|---|---|---|---|
| T | T | F | T | F must be T |
| T | T | T | F | F unconstrained |
| T | F | F | F | F unconstrained |
| F | T | F | F | F unconstrained |

```
| F | F | F | F | F unconstrained |
```

Only the row $A=T, N=T, O=F$ forces $F=T$; all others leave $F$ free. This tells the PM that the rule has narrow coverage — it catches exactly one of the eight (A,N,O) patterns. A complementary rule may be needed.

## Worked Example 2 — Detecting Rule Inconsistency in a KYB Rule Set

Suppose the KYB engine has three rules:

- R1: $B \to A$ (registered business  approve)
- R2: $C \to \neg A$ (cross-holding cycle  do not approve)
- R3: $B \wedge C$ is observed for company X.

Is the rule set consistent on this input? Substitute $B = T, C = T$ into R1 and R2: R1 says $A = T$; R2 says $A = F$. Contradiction. The rule set is inconsistent on this input.

Resolution (sketch): from $\neg B \vee A$ (R1 in CNF), $\neg C \vee \neg A$ (R2 in CNF), and the unit clauses $B$, $C$ (R3), resolve $B$ with $\neg B \vee A$ to get $A$; resolve $C$ with $\neg C \vee \neg A$ to get $\neg A$; resolve $A$ with $\neg A$ to get the empty clause $\square$ — a formal contradiction.

PM lesson: the rule engine needs a priority order (e.g., R2 dominates R1) or a combiner (require human review when conflicting rules fire). Detecting these conflicts before they ship is what SAT-based rule-consistency checks do.

## Common Pitfalls

- Confusing implication with causation. $p \to q$ is just a truth-table relation; it says nothing about causality.
- Treating "OR" as exclusive. Logical OR is inclusive; "either X or Y" in English is often exclusive. Be explicit.
- Forgetting empty/vacuous truth. $F \to q$ is T for any $q$; rules with antecedents that never fire are vacuously satisfied but useless.
- Combinatorial explosion of rules. A rule set with $n$ atomic predicates has $2^{2^n}$ possible truth tables; brute-force consistency checking is infeasible — that is why SAT solvers exist.
- Mixing levels. Propositional logic cannot express "for all merchants"; that needs first-order logic (§6.6).

| PM Relevance |
| --- |
| - Why a PM must master this: Rule engines, decisioning systems, and SQL filters are propositional logic in disguise. Without it, you cannot reason about coverage, conflicts, or shadowing. |
| - Metrics to own: Rule-firing rate, rule-conflict rate, rule-coverage (fraction of patterns matched), redundancy (rules logically implied by others). |
| - Strategic tradeoffs: More rules  more coverage but more conflicts and more latency; fewer, broader rules  less coverage but more interpretability. |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "Every fraud rule engine and every SQL filter is propositional logic. I read fraud rules in CNF, check them for inconsistencies the way a SAT solver would, and use contrapositive reasoning to flush out edge cases before launch. This makes me a credible counterpart to risk-ops, not just a roadmap owner." |

## NumPy-only Python Snippet

```
import numpy as np

# Worked Example 1: truth table for the fraud rule
def truth_table(vars_, formula_fn):
    n = len(vars_)
    rows = []
    for bits in range(2**n):
        env = {v: bool((bits >> (n-1-i)) & 1) for i, v in enumerate(vars_)}
```

```
        rows.append((env, formula_fn(env)))
    return rows

# (A and N and not O) -> F  ; here we tabulate the antecedent
antecedent = lambda e: e["A"] and e["N"] and (not e["O"])
table = truth_table(["A","N","O"], antecedent)
for env, val in table:
    print(env, "=> antecedent =", val)

# Worked Example 2: detect inconsistency by resolution on tiny CNF
# Clauses as sets of literals; positive=name, negative='-'+name
clauses = [{"-B","A"}, {"-C","-A"}, {"B"}, {"C"}]

def resolve(c1, c2):
    out = set()
    for lit in c1:
        comp = lit[1:] if lit.startswith("-") else "-"+lit
        if comp in c2:
            new_clause = (c1 | c2) - {lit, comp}
            out.add(frozenset(new_clause))
    return out

clauses = [frozenset(c) for c in clauses]
new = set(clauses)
found_empty = False
for _ in range(50):
    pairs = [(a, b) for a in new for b in new if a != b]
    added = set()
    for a, b in pairs:
        for r in resolve(a, b):
            if len(r) == 0:
                found_empty = True; break
            if r not in new:
                added.add(r)
        if found_empty: break
    if found_empty or not added: break
    new |= added
print("Rule set inconsistent on input?", found_empty)  # True
```

## 6.6 First-Order Logic and Resolution — Reasoning About Worlds

### Plain-English Intuition

Propositional logic can say "transaction X is risky." First-order logic (FOL) can say "every transaction over ₹100,000 from a new device is risky" — quantifying over a universe of objects. This is what makes FOL the backbone of knowledge graphs, ontologies, KYB inference engines, and many neuro-symbolic AI systems.

Fintech analogy: Paytm's KYB knowledge base might contain facts like Director(Ramesh, ABC_Ltd), CrossHolding(ABC_Ltd, XYZ_Ltd), and rules like $\forall x, y\, \text{Director}(x, y) \wedge \text{Sanctioned}(x) \to \text{HighRisk}(y)$. FOL lets the engine infer HighRisk(ABC_Ltd) automatically without enumerating every company.

### Formal Mathematical Definition

A first-order language consists of:

- Constants: $a, b, c, \ldots$ (individual objects).
- Variables: $x, y, z, \ldots$.
- Function symbols: $f, g, \ldots$.
- Predicate symbols: $P, Q, R, \ldots$ (relations).
- Logical connectives: $\neg, \wedge, \vee, \to, \leftrightarrow$.
- Quantifiers: $\forall$ (for all), $\exists$ (there exists).

Well-formed formulas (WFFs).

- Atomic: $P(t_1, \ldots, t_k)$ where $t_i$ are terms.
- If $\varphi, \psi$ are WFFs, so are $\neg \varphi$, $\varphi \wedge \psi$, $\varphi \to \psi$, $\forall x.\varphi$, $\exists x.\varphi$.

Semantics. A model $\mathcal{M} = (D, I)$ gives a non-empty domain $D$ and an interpretation $I$ of constants, functions, predicates. $\mathcal{M} \models \varphi$ means $\varphi$ is true in $\mathcal{M}$.

Universal instantiation. From $\forall x.\varphi(x)$, infer $\varphi(a)$ for any constant $a$.

Existential generalization. From $\varphi(a)$, infer $\exists x.\varphi(x)$.

Resolution (Robinson, 1965). Given two clauses in CNF, $C_1 = L \vee A$ and $C_2 = \neg L' \vee B$, and a most general unifier $\theta$ such that $L\theta = L'\theta$, infer the resolvent $(A \vee B)\theta$. Resolution is sound and refutation-complete for FOL: a clause set is unsatisfiable iff repeated resolution derives the empty clause.

```
| Symbol | Definition | Dimensions | PM Interpretation |
|--------|------------|------------|-------------------|
| \(\forall\) | Universal quantifier | over domain | "For every customer/merchant" |
| \(\exists\) | Existential quantifier | over domain | "There exists a director who" |
| \(P(x, y)\) | Predicate / relation | bool of args | A fact in the KYB knowledge graph |
| unifier \(\theta\) | Variable→term substitution | — | Pattern-match step inside the rule engine |
| resolvent | New clause from two | clause | One inference step |
```

## Worked Example 1 — Inferring Sanctioned-Director Risk via Resolution

Knowledge base (in CNF, after Skolemization):

- C1: $\neg \text{Sanctioned}(x) \vee \neg \text{Director}(x, y) \vee \text{HighRisk}(y)$
- C2: $\text{Sanctioned}(\text{Ramesh})$
- C3: $\text{Director}(\text{Ramesh}, \text{ABC\_Ltd})$

Goal: derive $\text{HighRisk}(\text{ABC\_Ltd})$.

Step 1 — Resolve C1 with C2 by unifier $\theta_1 = \{x / \text{Ramesh}\}$: $$ C_4 = \neg \text{Director}(\text{Ramesh}, y) \vee \text{HighRisk}(y) $$

Step 2 — Resolve $C_4$ with C3 by unifier $\theta_2 = \{y / \text{ABC\_Ltd}\}$: $$ C_5 = \text{HighRisk}(\text{ABC\_Ltd}) $$

Goal derived in 2 resolution steps. PM interpretation: this is exactly how a KYB engine flags ABC_Ltd without anyone hand-coding "if Ramesh is sanctioned and a director of ABC_Ltd, flag ABC_Ltd."

## Worked Example 2 — Detecting a Conflict Between Two KYB Rules

Knowledge base:

- C1: $\forall x.\, \text{RegisteredBusiness}(x) \to \text{Eligible}(x)$
- C2: $\forall x.\, \text{CrossHoldingCycle}(x) \to \neg \text{Eligible}(x)$
- C3: $\text{RegisteredBusiness}(\text{ACME})$
- C4: $\text{CrossHoldingCycle}(\text{ACME})$
- Negated goal (try to derive empty clause): we want to ask "is the knowledge base consistent?" — equivalently, can we derive both $\text{Eligible}(\text{ACME})$ and $\neg\text{Eligible}(\text{ACME})$?

In CNF:

- C1: $\neg \text{RegB}(x) \vee \text{Elig}(x)$
- C2: $\neg \text{CHC}(x) \vee \neg \text{Elig}(x)$
- C3: $\text{RegB}(\text{ACME})$
- C4: $\text{CHC}(\text{ACME})$

Step 1: Resolve C1 with C3, unifier $\{x/\text{ACME}\}$ $\text{Elig}(\text{ACME})$. Call this C5.

Step 2: Resolve C2 with C4, unifier $\{x/\text{ACME}\}$ $\neg\text{Elig}(\text{ACME})$. Call this C6.

Step 3: Resolve C5 with C6 empty clause $\square$.

The empty clause means the rule set is unsatisfiable on this input — exactly the inconsistency a PM must catch before shipping.

## Common Pitfalls

- Forgetting Skolemization. Existentially quantified variables must be replaced by Skolem functions before resolution; otherwise resolution is unsound.
- Unifier subtleties. Naïve string-matching is not unification; the most general unifier algorithm (Robinson) must be respected.
- Undecidability of FOL. First-order logic is semi-decidable: a proof can always be found if one exists, but unsatisfiability checks may not terminate. Bound your inference depth.
- Closed-world vs open-world. Most fintech rule engines assume closed-world (anything not asserted is false); FOL is open-world by default. Make the assumption explicit.
- Quantifier scope errors. $\forall x.\exists y.P(x,y)$ and $\exists y.\forall x.P(x,y)$ are very different statements. Always read left to right.

| PM Relevance |
| --- |
| - Why a PM must master this: Knowledge graphs, ontology-driven KYB engines, neuro-symbolic AI, and rule reasoning systems all rely on FOL. As LLMs increasingly integrate symbolic reasoning, FOL fluency becomes a differentiator. |
| - Metrics to own: Rule-coverage on labeled cases, false-positive/negative rates of the inference engine, average resolution depth per case, knowledge-base growth rate. |
| - Strategic tradeoffs: Richer FOL rules give better inference but slower runtime and harder maintenance; shallower propositional rules are fast but blunt. |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "First-order logic is propositional logic with quantifiers, and resolution is its inference engine. At Paytm I rely on FOL semantics whenever I review a KYB ontology — universal rules over directors, existential checks for any sanctioned party. It is also where modern neuro-symbolic AI is heading, which is why my Georgia Tech background matters for our LLM roadmap." |

## NumPy-only Python Snippet

```
import numpy as np

# Worked Example 1 & 2: a minimal first-order resolution engine.
# Terms: str. Variables start with lowercase 'x','y','z'. Constants capitalized.
def is_var(t): return isinstance(t, str) and t[:1].islower() and t not in ("not",)

def unify(t1, t2, theta):
    if theta is None: return None
    if t1 == t2: return theta
    if is_var(t1):
        if t1 in theta: return unify(theta[t1], t2, theta)
        return {**theta, t1: t2}
    if is_var(t2): return unify(t2, t1, theta)
    if isinstance(t1, tuple) and isinstance(t2, tuple) and len(t1) == len(t2):
        for a, b in zip(t1, t2):
            theta = unify(a, b, theta)
            if theta is None: return None
        return theta
    return None

def substitute(lit, theta):
    sign, atom = lit
    pred, args = atom[0], atom[1:]
    new_args = tuple(theta.get(a, a) for a in args)
    return (sign, (pred,) + new_args)

# Clauses: list of literals; each literal is (sign, ('Pred', arg1, arg2, arg3))
```

```
# Example 1 KB:
C1 = [(False, ("San","x")), (False, ("Dir","x","y")), (True, ("HR","y"))]
C2 = [(True,  ("San","Ramesh"))]
C3 = [(True,  ("Dir","Ramesh","ABC"))]

def resolve(c1, c2):
    results = []
    for i, (s1, a1) in enumerate(c1):
        for j, (s2, a2) in enumerate(c2):
            if s1 == s2: continue               # need opposite signs
            theta = unify(a1, a2, {})
            if theta is None: continue
            new = [substitute(l, theta) for k, l in enumerate(c1) if k != i] + \
                  [substitute(l, theta) for k, l in enumerate(c2) if k != j]
            # dedupe
            seen = []
            for l in new:
                if l not in seen: seen.append(l)
            results.append(seen)
    return results

step1 = resolve(C1, C2)[0]
print("After C1 ⊕ C2:", step1)
step2 = resolve(step1, C3)[0]
print("After ⊕ C3:    ", step2)  # [(True, ('HR', 'ABC'))]
```

## 6.7 Rule-Based Reasoning Engines for Fraud and KYB

### Plain-English Intuition

A rule-based reasoning engine is FOL + a chaining strategy + a working memory. It looks at facts, fires rules, deduces new facts, and repeats until quiescence. This was the dominant AI paradigm of the 1980s and is still the workhorse of fraud and KYB systems because it is interpretable, auditable, and certifiable for regulators in a way that black-box ML is not.

Fintech analogy: Paytm's anti-fraud stack is layered. The top layer is usually a rule engine that fires on hard, regulator-approved rules ("amount > ₹2 lakh from new device  mandatory step-up"). Underneath sits an ML scoring layer. The rule engine handles ~80% of decisions cheaply and explainably; the ML model handles the ambiguous middle. As a PM, you own both layers, but you must defend the rule engine to compliance and the ML to engineering.

### Formal Mathematical Definition

A production rule has the form $\text{LHS} \Rightarrow \text{RHS}$ where LHS is a conjunction of patterns over working-memory facts, and RHS is an action (assert/retract a fact, or trigger an external effect).

Forward chaining (data-driven). Start with facts; repeatedly find rules whose LHS matches; apply RHS; halt when no new facts can be derived. Soundness and completeness: for Horn-clause rule bases, forward chaining is complete.

Backward chaining (goal-driven). Start with a goal; find rules whose RHS produces it; recurse on the LHS subgoals. Used by Prolog.

Conflict resolution. When multiple rules match simultaneously, the engine picks one via a strategy: priority, recency, specificity, or refraction. This choice is a PM policy decision, not an engineering detail.

Rete algorithm. A famous data structure (Forgy, 1979) that compiles $n$ rules and $m$ facts into a network where matching is sub-linear, not $O(n \cdot m)$. Powers Drools, CLIPS, and most production rule engines.

| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| Working memory | Set of asserted facts | grows over time | Live tx, user, device, history |
| LHS pattern | Conjunctive condition | predicate logic | The "IF" of a rule |
| RHS action | Effect on memory or world | function | The "THEN" of a rule |
| Conflict set | Matched-but-not-yet-fired rules | set | Where conflict-resolution policy applies |
| Rete | Compiled match network | DAG | The reason rule engines are fast in production |

### Worked Example 1 — Forward Chaining on a Paytm Fraud Rule Base

Working memory facts:

- tx_amount(tx_001, 75000)
- new_device(tx_001)
- otp_verified(tx_001, false)

Rule base:

- R1: amount(t, A)  A > 50000  assert(high_value(t))
- R2: high_value(t)  new_device(t)  assert(elevated_risk(t))
- R3: elevated_risk(t)  otp_verified(t, false)  assert(flag_review(t))

Cycle 1:

- R1 fires (75000 > 50000): assert high_value(tx_001).
- Conflict set: {R2}.

Cycle 2:

- R2 fires: assert elevated_risk(tx_001).
- Conflict set: {R3}.

Cycle 3:

- R3 fires: assert flag_review(tx_001).
- Conflict set: {}  halt.

Three cycles, three new facts. The auditable chain "amount > 50K  new device  no OTP  flag_review" is now a traceable lineage every compliance auditor can read.

## Worked Example 2 — Backward Chaining for KYB Approval Decision

Goal: `approve(ACME)`.

Rules:

- R1: registered(x)  ¬sanctioned_director(x)  ¬cross_holding_cycle(x)  approve(x)
- R2: director(x, d)  on_sanctions_list(d)  sanctioned_director(x)

Facts: registered(ACME); director(ACME, Ramesh); on_sanctions_list(Ramesh).

Cycle 1: To prove approve(ACME), apply R1. Subgoals: registered(ACME) , ¬sanctioned_director(ACME), ¬cross_holding_cycle(ACME).

Cycle 2: Try to disprove sanctioned_director(ACME). Apply R2 with x = ACME: subgoal director(ACME, d) and on_sanctions_list(d). Unify d = Ramesh: both facts true  sanctioned_director(ACME) is derivable  cannot be negated. R1's LHS fails. Goal approve(ACME) unprovable.

Result: deny ACME, with an audit trace: "Director Ramesh is on the sanctions list, by R2." This is the kind of explainable, regulator-friendly output a PM must demand from any KYB system.

## Common Pitfalls

- Infinite firing loops. A rule that asserts a fact also matched by its own LHS will fire forever. Use refraction (no rule fires on the same fact-binding twice).

- Rule shadowing. A more specific rule may be hidden behind a less specific one; explicit priority or specificity-based conflict resolution is essential.
- Performance cliffs. Without Rete, $n$ rules × $m$ facts is $O(nm)$ per cycle — fine for $10^3$ rules and facts, lethal for $10^6$.
- Rule sprawl. Rule sets grow by acretion; over time, dead and redundant rules accumulate. Periodic logical-equivalence audits (SAT or first-order tools) are mandatory.
- Mixing rule and ML decisioning without clear precedence. Define which layer wins, and log every override.

| PM Relevance |
| --- |
| - Why a PM must master this: Compliance, audit, and explainability live in the rule layer. Without it, fintech ML cannot ship in regulated environments. |
| - Metrics to own: Rule fire-rate, override rate (rule vs ML), explainability coverage, time-to-add-new-rule, dead-rule fraction. |
| - Strategic tradeoffs: Rules are auditable but rigid; ML is flexible but opaque. The right architecture is layered, with the rule engine providing safety floors and the ML providing lift. |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "A production fraud stack is a rule engine in front of an ML model. As a PM I own that interface — which decisions are rule-mandated, which are ML-allowed, and how overrides are logged. The math behind it is forward and backward chaining over Horn clauses, and the engineering is Rete. That layered architecture is what makes me confident defending a Paytm fraud roadmap to both engineering and compliance." |

## NumPy-only Python Snippet

```
import numpy as np

# Worked Example 1: forward chaining
facts = {("tx_amount","tx_001",75000), ("new_device","tx_001"),
         ("otp_verified","tx_001",False)}

def rule1(facts):  # amount > 50K -> high_value
    out = set()
    for f in facts:
        if f[0]=="tx_amount" and f[2] > 50000:
            out.add(("high_value", f[1]))
    return out

def rule2(facts):  # high_value & new_device -> elevated_risk
    hv = {f[1] for f in facts if f[0]=="high_value"}
    nd = {f[1] for f in facts if f[0]=="new_device"}
    return {("elevated_risk", t) for t in hv & nd}

def rule3(facts):  # elevated_risk & otp_verified=false -> flag_review
    er = {f[1] for f in facts if f[0]=="elevated_risk"}
    nv = {f[1] for f in facts if f[0]=="otp_verified" and f[2] is False}
    return {("flag_review", t) for t in er & nv}

rules = [rule1, rule2, rule3]
cycle = 0
while True:
    new = set()
    for r in rules: new |= r(facts)
    new -= facts
    if not new: break
    facts |= new
    cycle += 1
    print(f"cycle {cycle}: added {new}")
print("final facts:", facts)

# Worked Example 2: backward chaining sketch (goal -> subgoals)
kb_facts = {("registered","ACME"), ("director","ACME","Ramesh"),
            ("on_sanctions_list","Ramesh")}

def sanctioned_director(x):
    return any(("director",x,d) in kb_facts and ("on_sanctions_list",d) in kb_facts
               for d in [t[2] for t in kb_facts if t[0]=="director" and t[1]==x])

def approve(x):
    return (("registered",x) in kb_facts
            and not sanctioned_director(x))

print("approve(ACME) ->", approve("ACME"))  # False
```

## Chapter 6 — Closing Synthesis

Graphs, trees, DAGs, and logic are not separate islands. A modern fraud pipeline at Paytm is all four at once: a DAG of jobs, each consuming a graph of entities, evaluating decision trees, and combining outputs through propositional and first-order rules. Master the complexity bounds ($O(|V|+|E|)$ traversal, $n-1$ tree edges, $2^d$ leaves), the structural identities (handshake lemma, topological sort existence), and the logical inference rules (modus ponens, resolution), and you can sit in any architecture review and ask the question that changes the design — instead of nodding through a slide deck.

# Chapter 7 — Time Series Math for ML4T

Markets, transactions, and revenues all unfold in time. The mathematics of time series — stationarity, autoregression, returns, volatility, Sharpe — is the language ML for Trading (ML4T) speaks, and increasingly the language fintech PMs speak whenever the conversation turns to forecasting, pricing, or risk-adjusted performance.

For Prabhjot, this chapter restores six instruments: stationarity and non-stationarity diagnostics, ARMA model foundations, log and simple returns, volatility modeling, Sharpe and Information ratios, and applied forecasting for Paytm revenue, MDR pricing, and seasonal transaction volume.

## 7.1 Stationarity — When the Statistics Stop Moving

### Plain-English Intuition

A time series is stationary if its statistical properties — mean, variance, autocorrelation — do not depend on the time you start measuring. Think of a healthy heartbeat: average rate ~72 bpm, consistent variance, beat-to-beat correlation roughly stable. That is (weakly) stationary. Now imagine the same patient during a marathon: heart rate trends up, variance changes, autocorrelation shifts. Non-stationary.

Fintech analogy: daily Paytm UPI volume has a clear trend (growing year over year), a weekly seasonality (Sundays spike), and a level shift every Diwali. None of those are stationary. But the log-returns of daily volume (percent change) are often close to stationary. The first move in almost every ML4T pipeline is to transform a non-stationary series into a stationary one — because the math (ARMA, regression, hypothesis testing) only works under stationarity.

### Formal Mathematical Definition

A time series $\{X_t\}_{t \in \mathbb{Z}}$ is strictly stationary if the joint distribution of $(X_{t_1}, \ldots, X_{t_k})$ equals that of $(X_{t_1+h}, \ldots, X_{t_k+h})$ for any $k, t_i, h$. Plain English: shifting in time does not change the joint distribution.

A time series is weakly (covariance) stationary if:

1. $\mathbb{E}[X_t] = \mu$ is constant in $t$.
2. $\text{Var}(X_t) = \sigma^2 < \infty$ is constant in $t$.
3. $\text{Cov}(X_t, X_{t+h}) = \gamma(h)$ depends only on lag $h$, not on $t$.

For most ML4T work, "stationary" means weakly stationary.

Autocorrelation function (ACF). $$ \rho(h) = \frac{\gamma(h)}{\gamma(0)} = \frac{\text{Cov}(X_t, X_{t+h})}{\text{Var}(X_t)} $$

Plain English: autocorrelation at lag h equals covariance at lag h divided by variance.

Augmented Dickey–Fuller (ADF) test. Tests the null hypothesis "series has a unit root (non-stationary)" by regressing $\Delta X_t = \alpha + \beta t + \gamma X_{t-1} + \sum_i \delta_i \Delta X_{t-i} + \varepsilon_t$ and checking whether $\gamma$ is significantly less than zero. Reject the null  stationary.

Differencing. If $X_t$ is non-stationary, often $Y_t = X_t - X_{t-1}$ (first difference) is stationary. A series is integrated of order d, $I(d)$, if $d$ differences are needed to achieve stationarity.

| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| $X_t$ | Value of series at time $t$ | scalar | Daily revenue, daily MDR, etc. |
| $\mu$ | Constant mean | scalar | The "no-trend" level |
| $\sigma^2$ | Constant variance | scalar squared | Stable volatility |

| \(\gamma(h)\) | Autocovariance at lag \(h\) | scalar squared | Memory of past values |
| \(\rho(h)\) | Autocorrelation at lag \(h\) | unitless, \([-1,1]\) | Standardized memory |
| \(\Delta X_t\) | First difference | scalar | Day-over-day change |
| \(I(d)\) | Integration order | integer | How many differences needed |

## Worked Example 1 — Stationarity Check on Paytm Daily Active Wallets

Suppose daily active wallets over 10 days (in millions) are: \[ X = (50, 51, 52, 54, 55, 57, 58, 60, 61, 63) \]

Visually trending upward — clearly non-stationary in mean.

Step 1 — Sample mean: \((50+51+52+54+55+57+58+60+61+63)/10 = 561/10 = 56.1\).

Step 2 — Split into halves: first-half mean = \((50+51+52+54+55)/5 = 52.4\); second-half mean = \((57+58+60+61+63)/5 = 59.8\). Difference 7.4 — large relative to series variability — confirms non-stationarity.

Step 3 — First difference: \(\Delta X = (1, 1, 2, 1, 2, 1, 2, 1, 2)\). Mean of \(\Delta X\) = 13/9  1.444; first-half (4 deltas) mean = (1+1+2+1)/4 = 1.25; second-half (5 deltas) mean = (2+1+2+1+2)/5 = 1.60. Closer but still drifting; series is roughly \(I(1)\) with constant drift.

Step 4 — Second difference: \(\Delta^2 X = (0, 1, -1, 1, -1, 1, -1, 1)\). Mean  0.125; halves are (0+11+1)/4 = 0.25 vs (1+11+1)/4 = 0; nearly stationary around zero. The series is \(I(2)\) in the strictest reading; for ARIMA modeling we would use \(d=1\) plus a drift term.

## Worked Example 2 — Log-Return Stationarity for Paytm MDR (Merchant Discount Rate) Over Weeks

Weekly average MDR (basis points) over 8 weeks: \[ R = (190, 188, 192, 187, 189, 191, 188, 190) \]

Step 1 — Sample mean: \((190+188+192+187+189+191+188+190)/8 = 1515/8 = 189.375\).

Step 2 — Sample variance: \[ \text{Var} = \tfrac{1}{8}\sum (R_t - 189.375)^2 \] Squared deviations: \((0.625)^2=0.391, (-1.375)^2=1.891, (2.625)^2=6.891, (-2.375)^2=5.641, (-0.375)^2=0.141, (1.625)^2=2.641, (-1.375)^2=1.891, (0.625)^2=0.391\). Sum = 19.876. \(\text{Var} = 19.876/8 = 2.485\). \(\sigma \approx 1.576\) bps.

Step 3 — Lag-1 autocovariance: \[ \gamma(1) = \tfrac{1}{7}\sum_{t=1}^{7}(R_t - 189.375)(R_{t+1} - 189.375) \] Pairwise products of deviations \((0.625)(-1.375)=-0.859\), \((-1.375)(2.625)=-3.609\), \((2.625)(-2.375)=-6.234\), \((-2.375)(-0.375)=0.891\), \((-0.375)(1.625)=-0.609\), \((1.625)(-1.375)=-2.234\), \((-1.375)(0.625)=-0.859\). Sum = 13.514. \(\gamma(1) = -13.514/7 = -1.931\). \(\rho(1) = \gamma(1)/\gamma(0) = -1.931/2.485 = -0.777\).

Step 4 — Halves means: first-half (190,188,192,187) mean = 189.25; second-half mean = 189.50. Difference 0.25, well within sampling noise. Conclusion: MDR weekly series is approximately stationary in mean, with strong negative lag-1 autocorrelation (high week tends to be followed by low week — i.e., the pricing system "reverts").

PM interpretation: the negative \(\rho(1) = -0.777\) tells us MDR is mean-reverting at the weekly grain. That has a direct product implication: a pricing experiment that spikes MDR one week is likely to overshoot — automatic reversion the following week will inflate the apparent "treatment effect" if the analyst is naive.

## Common Pitfalls

- Confusing trend with stationarity. A series can have constant mean zero and still be non-stationary if variance grows (e.g., random walk).
- Differencing twice "to be safe." Over-differencing introduces spurious negative autocorrelation and inflates variance.
- Eyeballing rather than testing. Always pair visual inspection with an ADF or KPSS test.

- Ignoring structural breaks. A series can be stationary in two regimes joined by a level shift; tests will reject stationarity overall. Use Chow tests or regime-switching models.
- Applying stationarity logic to short windows. Less than ~30 observations and most tests have no power.

| PM Relevance |
| --- |
| - Why a PM must master this: Every forecast you ship — revenue, transactions, MDR, fraud-rate — is implicitly an assumption about stationarity. If the assumption fails, the forecast collapses, often at exactly the moment leadership is watching. |
| - Metrics to own: Stationarity status of all production-forecast inputs; \(I(d)\) order; ADF p-values; structural-break dates. |
| - Strategic tradeoffs: Heavy differencing makes a series stationary but destroys interpretable level information; light differencing preserves levels but leaves non-stationarity. |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "Stationarity is the prerequisite for ARMA and most regression-based forecasting. Before I sign off on any Paytm revenue forecast, I check that the modeled series — usually log-returns or first differences — is statistically stationary. A non-stationary input is the silent killer of every quarterly forecast I have ever seen miss." |

### NumPy-only Python Snippet

```
import numpy as np

def mean_var_halves(x):
    x = np.asarray(x, dtype=np.float64)
    m1 = x[:len(x)//2].mean(); m2 = x[len(x)//2:].mean()
    return m1, m2, x.mean(), x.var(ddof=0)

def autocov(x, h):
    x = np.asarray(x, dtype=np.float64)
    mu = x.mean()
    return np.mean((x[:len(x)-h] - mu) * (x[h:] - mu))

# Worked Example 1: wallets
X = np.array([50,51,52,54,55,57,58,60,61,63], dtype=float)
print("Wallets halves & mean & var:", mean_var_halves(X))
print("First diff:", np.diff(X))
print("Second diff:", np.diff(X, 2))

# Worked Example 2: MDR
R = np.array([190,188,192,187,189,191,188,190], dtype=float)
mu, var = R.mean(), R.var(ddof=0)
g1 = autocov(R, 1)
print(f"MDR mean={mu:.3f}, var={var:.3f}, gamma(1)={g1:.3f}, rho(1)={g1/var:.3f}")
```

## 7.2 ARMA Foundations — Memory and Shocks

### Plain-English Intuition

ARMA stands for AutoRegressive Moving Average. The AR part says "tomorrow's value depends on yesterday's values"; the MA part says "tomorrow's value depends on yesterday's surprises." Together they form the workhorse parametric family for stationary time series.

Fintech analogy: Paytm's daily new-merchant-signup count has memory — a strong day tends to be followed by another strong day (sales-team momentum: AR effect). It also has shock-decay: a Diwali campaign creates a surprise spike whose effect tapers over the next few days (MA effect). An ARMA(1,1) model captures both with two parameters and is interpretable enough to explain to a CFO.

### Formal Mathematical Definition

AR(p) — Autoregressive model of order p. \[ X_t = c + \phi_1 X_{t-1} + \phi_2 X_{t-2} + \cdots + \phi_p X_{t-p} + \varepsilon_t \]

Plain English: X at time t equals a constant plus a weighted sum of past p values plus white-noise shock epsilon at time t.

MA(q) — Moving Average model of order q. $$ X_t = \mu + \varepsilon_t + \theta_1 \varepsilon_{t-1} + \cdots + \theta_q \varepsilon_{t-q} $$

Plain English: X at time t equals a mean mu plus the current shock and a weighted sum of past q shocks.

ARMA(p, q): $$ X_t = c + \sum_{i=1}^{p} \phi_i X_{t-i} + \varepsilon_t + \sum_{j=1}^{q} \theta_j \varepsilon_{t-j} $$

$\{\varepsilon_t\}$ is white noise: $\mathbb{E}[\varepsilon_t] = 0$, $\text{Var}(\varepsilon_t) = \sigma_\varepsilon^2$, $\text{Cov}(\varepsilon_t, \varepsilon_s) = 0$ for $t \ne s$.

Stationarity condition for AR(1). $|\phi_1| < 1$. Roots of the AR characteristic polynomial $1 - \phi_1 z - \cdots - \phi_p z^p = 0$ must lie outside the unit circle.

Invertibility condition for MA(1). $|\theta_1| < 1$, so the model can be expressed as an infinite AR.

Estimation. Method of moments (Yule–Walker), MLE, or conditional sum-of-squares. Order selection by AIC/BIC.

ARIMA(p,d,q). ARMA applied to the $d$-th difference of the original series — used when the raw series is $I(d)$ non-stationary.

| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| $\phi_i$ | AR coefficients | unitless | Strength of memory at lag $i$ |
| $\theta_j$ | MA coefficients | unitless | Shock-decay weights |
| $\varepsilon_t$ | White-noise shock | same units as $X$ | The unpredictable component |
| $c$ | Intercept | same units | Long-run mean ($\times(1-\Sigma\varphi)$) |
| $p, q$ | Model orders | integer | How far back memory / shocks matter |
| $d$ | Differencing order | integer | Inherited from §7.1 |

## Worked Example 1 — Fitting an AR(1) to Paytm Daily Soundbox Sales Increments

Daily increments in Soundbox activations (devices/day above the baseline) over 10 days: $$ Y = (10, 12, 11, 13, 14, 12, 15, 14, 16, 15) $$

Step 1 — Mean: $\bar Y = 132/10 = 13.2$. De-mean: $y_t = Y_t - 13.2$: $$ y = (-3.2, -1.2, -2.2, -0.2, 0.8, -1.2, 1.8, 0.8, 2.8, 1.8) $$

Step 2 — Compute $\hat\gamma(0)$ and $\hat\gamma(1)$:

- $\sum y_t^2 = 10.24 + 1.44 + 4.84 + 0.04 + 0.64 + 1.44 + 3.24 + 0.64 + 7.84 + 3.24 = 33.60$; $\hat\gamma(0) = 33.60/10 = 3.360$.
- $\sum_{t=1}^{9} y_t y_{t+1}$: $(-3.2)(-1.2)=3.84, (-1.2)(-2.2)=2.64, (-2.2)(-0.2)=0.44, (-0.2)(0.8)=-0.16, (0.8)(-1.2)=-0.96, (-1.2)(1.8)=-2.16, (1.8)(0.8)=1.44, (0.8)(2.8)=2.24, (2.8)(1.8)=5.04$. Sum = 12.36. $\hat\gamma(1) = 12.36/10 = 1.236$.

Step 3 — Yule–Walker estimate of AR(1): $\hat\phi_1 = \hat\gamma(1)/\hat\gamma(0) = 1.236/3.360 = 0.3679$.

Step 4 — Innovation variance: $\hat\sigma_\varepsilon^2 = \hat\gamma(0)(1 - \hat\phi_1^2) = 3.360 \times (1 - 0.1354) = 3.360 \times 0.8646 = 2.905$.

Step 5 — Forecast for day 11: $\hat Y_{11} = \bar Y + \hat\phi_1 (Y_{10} - \bar Y) = 13.2 + 0.3679 \times (15 - 13.2) = 13.2 + 0.3679 \times 1.8 = 13.2 + 0.6622 = 13.86$. One-step-ahead 95% interval: $\hat Y_{11} \pm 1.96 \sqrt{2.905} = 13.86 \pm 3.34 \approx [10.52, 17.20]$.

PM interpretation: the AR(1) coefficient 0.37 means the day-to-day momentum is modest — about a third of yesterday's deviation carries forward. A sales leader hoping to "ride momentum" for 3 days should expect deviation to decay as $0.37^3 \approx 0.05$ — essentially gone after three days. That sets realistic expectations for promo-campaign half-life.

## Worked Example 2 — MA(1) Effect of a Diwali Promo on Daily Tx Counts

Suppose Paytm runs a 1-day promo and observes residual transactions (above baseline) over 6 days: \[ Z = (0, 200000, 80000, 30000, 10000, 5000) \]

Step 1 — Day-by-day decay ratio: 80,000/200,000 = 0.40; 30,000/80,000 = 0.375; 10,000/30,000 = 0.333; 5,000/10,000 = 0.50. Roughly geometric decay with ratio  0.40.

Step 2 — Fit MA(1) with shock at day 2 of magnitude \(\varepsilon_2 = 200{,}000\) and \(\theta_1 = 0.40\). Under MA(1), the impact at day 3 should be \(\theta_1 \varepsilon_2 = 80{,}000\) . Day 4 would be zero under pure MA(1) — but data shows 30,000  the model needs MA(q3) or AR component.

Step 3 — Try AR(1) on the decay: regress \(Z_t\) on \(Z_{t-1}\) from t=3..6. Ratios suggest \(\hat\phi_1 \approx (80{+}30{+}10{+}5)/(200{+}80{+}30{+}10) = 125/320 = 0.391\).

Step 4 — Estimate model: \(Z_t = 0.391 Z_{t-1} + \varepsilon_t\). Forecast day 7: \(0.391 \times 5{,}000 = 1{,}955\) residual transactions. Forecast day 10: \(0.391^4 \times 200{,}000 \approx 4{,}680\) — meaning promo halo is essentially gone by day 10.

PM interpretation: a promo creates an AR-shaped halo with half-life \(\log(0.5)/\log(0.391) \approx 0.738\) days. That is the right vocabulary for the "campaign tail" — and it tells finance how to attribute revenue to the campaign window vs the natural baseline.

## Common Pitfalls

- Fitting ARMA to non-stationary data. Always check stationarity first; if non-stationary, difference and fit ARIMA.
- Over-fitting orders. A 10-observation series cannot reliably estimate ARMA(3,3); BIC/AIC will protect you.
- Confusing AR(1) with random walk. \(\phi_1 = 1\) is a random walk — not stationary, infinite variance over time.
- Ignoring residual diagnostics. After fitting, residuals must look like white noise (Ljung–Box test); otherwise the model is misspecified.
- Mistaking ARMA "forecast" for "long-horizon truth." ARMA forecasts converge to the unconditional mean exponentially; they are not magic predictors of structural breaks.

| PM Relevance |
| --- |
| - Why a PM must master this: ARMA is the lingua franca of revenue, demand, and operational forecasting. Even in deep-learning shops, ARMA remains the baseline you must beat. |
| - Metrics to own: MAE/MAPE of one-step and multi-step ARMA forecasts; AR(1) coefficient as a "stickiness" KPI per campaign or segment. |
| - Strategic tradeoffs: Higher-order ARMA fits training data better but extrapolates worse; parsimonious models (low p, q) generalize but can miss structure. |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "ARMA models tell me what fraction of a metric is memory and what fraction is shock-decay. At Paytm I use AR(1) coefficients as 'stickiness scores' for promos and use them to set expectations with finance about campaign-tail attribution. It is the simplest model the data will let me get away with — which is exactly the right place to start." |

## NumPy-only Python Snippet

```
import numpy as np

# Worked Example 1: AR(1) Yule-Walker on Soundbox
Y = np.array([10,12,11,13,14,12,15,14,16,15], dtype=float)
mu = Y.mean()
y = Y - mu
gamma0 = (y**2).mean()
gamma1 = (y[:-1] * y[1:]).mean()
phi1 = gamma1 / gamma0
sigma2 = gamma0 * (1 - phi1**2)
forecast_11 = mu + phi1 * (Y[-1] - mu)
print(f"phi1 = {phi1:.4f}, sigma_eps^2 = {sigma2:.4f}")
```

```
print(f"Forecast Y[11] = {forecast_11:.4f}, 95% PI = "
      f"[{forecast_11-1.96*np.sqrt(sigma2):.2f}, {forecast_11+1.96*np.sqrt(sigma2):.2f}]")

# Worked Example 2: AR(1) promo decay
Z = np.array([0, 200000, 80000, 30000, 10000, 5000], dtype=float)
num = (Z[1:-1] * Z[2:]).sum()
den = (Z[1:-1] * Z[1:-1]).sum()
phi1_promo = num / den
print(f"Promo AR(1) phi1 ≈ {phi1_promo:.4f}")
print(f"Forecast day 7  = {phi1_promo * Z[-1]:.2f}")
print(f"Forecast day 10 = {phi1_promo**4 * Z[1]:.2f}")
```

## 7.3 Log Returns vs Simple Returns

### Plain-English Intuition

If a stock goes from ₹100 to ₹110, the simple return is 10%. The log return is $\ln(110/100) = 9.53\%$. Why two definitions? Because log returns add over time (a 5% log return today plus a 3% log return tomorrow is a 8% log return over two days, exactly), while simple returns do not. For multi-period analysis, log returns are the right currency.

Fintech analogy: when Paytm's finance team reports "MTUs grew 12% MoM for three months running," they often quote simple returns and informally add — which gives the wrong total. Log returns fix this, and they have the bonus property of being approximately normally distributed for small returns, which makes statistical inference (variance, t-tests) clean.

### Formal Mathematical Definition

For a price (or value) series $P_t$, the simple return between $t-1$ and $t$ is $$ r_t = \frac{P_t - P_{t-1}}{P_{t-1}} = \frac{P_t}{P_{t-1}} - 1 $$

Plain English: simple return equals the price change divided by the previous price.

The log return is $$ \ell_t = \ln\frac{P_t}{P_{t-1}} = \ln P_t - \ln P_{t-1} $$

Plain English: log return equals the difference of log prices.

Relationship. $\ell_t = \ln(1 + r_t)$. For small $r_t$, $\ell_t \approx r_t - r_t^2/2$. At $r = 0.10$, error  0.5%.

Multi-period properties.

- Log returns are additive: $\ell_{[t_0, t_n]} = \sum_{i=1}^{n} \ell_{t_i}$.
- Simple returns are multiplicative: $1 + r_{[t_0, t_n]} = \prod_{i=1}^{n}(1 + r_{t_i})$.
- Therefore $r_{[t_0,t_n]} = \exp(\sum \ell_i) - 1$.

Cross-sectional properties.

- Simple returns are bounded below by 1 (can lose all); log returns can go to $-\infty$.
- Log returns are time-additive but not portfolio-additive; simple returns are portfolio-additive but not time-additive.

| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| $P_t$ | Price/value at time t | currency or count | DAU, GMV, revenue, MDR, stock price |
| $r_t$ | Simple return | unitless | Reported growth rates |
| $\ell_t$ | Log return | unitless | Statistical-modeling currency |
| $\sum \ell$ | Cumulative log return | unitless | Total growth in additive form |

### Worked Example 1 — Paytm MTU Growth, Simple vs Log Returns

Monthly active users (millions) over 4 months: $$ M = (100, 112, 128, 142) $$

Step 1 — Simple returns:

- $r_2 = (112-100)/100 = 0.1200$ = 12.00%
- $r_3 = (128-112)/112 = 0.1429$ = 14.29%
- $r_4 = (142-128)/128 = 0.1094$ = 10.94%
- Naïve sum: 37.23% — wrong if interpreted as 4-month growth.

Step 2 — True 4-month simple return: $(142 - 100)/100 = 0.42$ = 42.00%. Verify by chaining: $(1.12)(1.1429)(1.1094) = 1.42$ .

Step 3 — Log returns:

- $\ell_2 = \ln(112/100) = \ln(1.12) = 0.11333$
- $\ell_3 = \ln(128/112) = \ln(1.14286) = 0.13353$
- $\ell_4 = \ln(142/128) = \ln(1.10938) = 0.10381$
- Sum: 0.35067. Exponentiate: $e^{0.35067} - 1 = 1.4200 - 1 = 0.4200$ = 42.00% .

PM interpretation: when explaining to leadership "we grew our MTU base by 42% in four months," do not add the monthly simple returns; chain them, or use log returns and sum them. The 5% gap between 37.23% and 42% comes from compounding — exactly the gap that grows the larger and longer the series gets.

## Worked Example 2 — Volatility Calculation on Daily Paytm Stock Returns

Suppose daily closing prices over 6 days for Paytm's parent listing: $$ P = (650, 660, 658, 665, 670, 668) $$

Step 1 — Log returns:

- $\ell_2 = \ln(660/650) = \ln(1.01538) = 0.01527$
- $\ell_3 = \ln(658/660) = \ln(0.99697) = -0.00303$
- $\ell_4 = \ln(665/658) = \ln(1.01064) = 0.01058$
- $\ell_5 = \ln(670/665) = \ln(1.00752) = 0.00749$
- $\ell_6 = \ln(668/670) = \ln(0.99701) = -0.00299$

Step 2 — Mean of log returns: $(0.01527 - 0.00303 + 0.01058 + 0.00749 - 0.00299)/5 = 0.02732/5 = 0.005464$.

Step 3 — Sample variance (unbiased, divide by n1=4):

- Deviations: $0.01527 - 0.00546 = 0.00981$; $-0.00303 - 0.00546 = -0.00849$; $0.01058 - 0.00546 = 0.00512$; $0.00749 - 0.00546 = 0.00203$; $-0.00299 - 0.00546 = -0.00845$.
- Squared: $0.0000962; 0.0000721; 0.0000262; 0.0000041; 0.0000714$. Sum = 0.0002701.
- Variance = 0.0002701/4 = 0.00006752.
- Std dev (daily): $\sqrt{0.00006752} = 0.00822$ = 0.822%.

Step 4 — Annualized volatility: multiply by $\sqrt{252}$ (trading days/yr): $0.00822 \times 15.875 = 0.1305$ = 13.05%. A modest equity volatility, but for a payments fintech this is plausible during a quiet stretch.

## Common Pitfalls

- Adding simple returns over time. Mathematically wrong; the correct chain is multiplicative. Use log returns or compound.
- Adding log returns across a portfolio. Mathematically wrong; portfolio returns add in simple form weighted by share. Use simple returns for cross-section, log for time.

- Annualizing volatility incorrectly. Multiply by $\sqrt{T}$, not $T$. The square-root scaling assumes i.i.d. returns; if there is autocorrelation, you need a Newey–West correction.
- Reporting log returns to non-technical stakeholders. For external comms, convert back to simple terms; otherwise stakeholders will misread.

| PM Relevance |
| --- |
| - Why a PM must master this: Every growth, revenue, and KPI narrative the finance team or board sees rests on this distinction. Misuse it once in a board deck and you lose credibility forever. |
| - Metrics to own: Compounded growth rate (CAGR) of MTU, GMV, revenue; daily log-return volatility of any traded asset; portfolio-weighted simple return of a basket. |
| - Strategic tradeoffs: Log returns are statistically cleaner; simple returns are more intuitive. Use the right one for the right audience. |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "Log returns are additive in time; simple returns are additive across portfolio constituents. As a fintech PM, I always reconcile any growth narrative back to both — log when modeling, simple when narrating — because mixing them is the single fastest way to embarrass a CFO." |

### NumPy-only Python Snippet

```
import numpy as np

# Worked Example 1: MTU growth
M = np.array([100, 112, 128, 142], dtype=float)
simple = M[1:] / M[:-1] - 1
log_r  = np.log(M[1:] / M[:-1])
print("Simple monthly returns:", np.round(simple, 4))
print("Sum (wrong total):", np.round(simple.sum(), 4))
print("True 4-mo simple return:", round((M[-1]/M[0]) - 1, 4))
print("Sum of log returns:", round(log_r.sum(), 4),
      "exp-1 =", round(np.exp(log_r.sum()) - 1, 4))

# Worked Example 2: stock volatility
P = np.array([650, 660, 658, 665, 670, 668], dtype=float)
ell = np.log(P[1:] / P[:-1])
mu_d  = ell.mean()
var_d = ell.var(ddof=1)
sd_d  = np.sqrt(var_d)
print(f"daily mean log return = {mu_d:.6f}")
print(f"daily std dev         = {sd_d:.6f} = {sd_d*100:.3f}%")
print(f"annualized vol        = {sd_d*np.sqrt(252)*100:.2f}%")
```

## 7.4 Volatility Modeling — Sizing the Risk

### Plain-English Intuition

Volatility is the standard deviation of returns — the size of typical moves. A heartbeat with steady amplitude has low volatility; a heartbeat that swings between fast and slow has high volatility. Markets, transactions, and fraud rates all have time-varying volatility: calm periods of low vol punctuated by storms of high vol. The job of volatility modeling is to forecast these storms.

Fintech analogy: Paytm's daily refund rate is normally ~0.5% with daily standard deviation ~0.1 percentage points (low vol). On a system-outage day, the refund rate jumps to 3% with standard deviation 1 percentage point (high vol). The risk team's job is to detect, model, and bound this volatility — and tie it to capital reserves.

### Formal Mathematical Definition

Historical (realized) volatility over a window of $n$ observations: $$ \hat\sigma = \sqrt{\frac{1}{n-1}\sum_{t=1}^{n}(\ell_t - \bar\ell)^2} $$

Plain English: realized vol is the sample standard deviation of log returns.

Annualization. With $N$ periods/year, $\hat\sigma_{\text{annual}} = \hat\sigma \sqrt{N}$ (assuming i.i.d.). Daily-to-annual uses $\sqrt{252}$; monthly-to-annual uses $\sqrt{12}$.

Exponentially Weighted Moving Average (EWMA / RiskMetrics). $$ \hat\sigma_t^2 = \lambda \hat\sigma_{t-1}^2 + (1 - \lambda) \ell_{t-1}^2 $$

Plain English: today's variance is lambda times yesterday's variance plus (1 minus lambda) times yesterday's squared return.

$\lambda \in (0,1)$ is the decay; J.P. Morgan's RiskMetrics defaults are $\lambda = 0.94$ (daily) and $0.97$ (monthly).

GARCH(1,1). $$ \hat\sigma_t^2 = \omega + \alpha \ell_{t-1}^2 + \beta \hat\sigma_{t-1}^2 $$

with $\omega > 0$, $\alpha, \beta \ge 0$, $\alpha + \beta < 1$ (covariance-stationarity). EWMA is GARCH(1,1) with $\omega = 0$, $\alpha = 1-\lambda$, $\beta = \lambda$. GARCH captures the volatility clustering observed in financial returns.

Long-run variance under GARCH(1,1). $$ \sigma_\infty^2 = \frac{\omega}{1 - \alpha - \beta} $$

| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| $\sigma$ | Volatility | same units as return | Risk size |
| $\lambda$ | EWMA decay | unitless, (0,1) | How fast to forget the past |
| $\omega, \alpha, \beta$ | GARCH params | unitless | Long-run + reactivity + persistence |
| $\sigma_\infty$ | Unconditional vol | same units | "Normal" risk level |

## Worked Example 1 — Historical Vol of Paytm Daily Refund Rate Over a Calm Week

Daily refund rate (%) for 5 days: $R = (0.48, 0.51, 0.49, 0.52, 0.50)$. Treat as a level series; we want vol of the level.

Step 1 — Mean: $(0.48+0.51+0.49+0.52+0.50)/5 = 2.50/5 = 0.500$%.

Step 2 — Deviations: $(-0.02, 0.01, -0.01, 0.02, 0.00)$. Squared: $(0.0004, 0.0001, 0.0001, 0.0004, 0.0000)$. Sum = 0.0010.

Step 3 — Sample variance (divide by n1=4): $0.0010 / 4 = 0.00025$ (in %²).

Step 4 — Std dev: $\sqrt{0.00025} = 0.01581$% — about 1.58 basis points. Low vol  calm regime.

## Worked Example 2 — EWMA Vol on Paytm Daily Tx-Failure Rate Across a Spike

Daily failure rate (%): $F = (0.6, 0.7, 0.5, 0.8, 2.5, 1.8, 1.2, 0.9, 0.7, 0.6)$. Spike at day 5 (system outage).

Set $\lambda = 0.94$, initial $\hat\sigma_0^2 = \text{Var}(F_{1..4})$. Returns-equivalent here: use daily changes $\Delta F_t = F_t - F_{t-1}$.

$\Delta F = (0.1, -0.2, 0.3, 1.7, -0.7, -0.6, -0.3, -0.2, -0.1)$. Squared: $(0.01, 0.04, 0.09, 2.89, 0.49, 0.36, 0.09, 0.04, 0.01)$.

Step 1 — Seed: take first three squared changes as the seed variance: $(0.01+0.04+0.09)/3 = 0.0467$.

Step 2 — Update recursively starting from $t=4$:

- $\hat\sigma_4^2 = 0.94(0.0467) + 0.06(0.09) = 0.0439 + 0.0054 = 0.0493$
- $\hat\sigma_5^2 = 0.94(0.0493) + 0.06(2.89) = 0.0463 + 0.1734 = 0.2197$
- $\hat\sigma_6^2 = 0.94(0.2197) + 0.06(0.49) = 0.2065 + 0.0294 = 0.2359$
- $\hat\sigma_7^2 = 0.94(0.2359) + 0.06(0.36) = 0.2217 + 0.0216 = 0.2433$

- $\hat\sigma_8^2 = 0.94(0.2433) + 0.06(0.09) = 0.2287 + 0.0054 = 0.2341$
- $\hat\sigma_9^2 = 0.94(0.2341) + 0.06(0.04) = 0.2200 + 0.0024 = 0.2224$
- $\hat\sigma_{10}^2 = 0.94(0.2224) + 0.06(0.01) = 0.2090 + 0.0006 = 0.2096$

Step 3 — Take square roots: $\hat\sigma_4 = 0.222$, $\hat\sigma_5 = 0.469$, $\hat\sigma_6 = 0.486$, and slowly declining thereafter. Vol jumped 2× after the outage and persisted for several days — exactly the "clustering" EWMA is built to capture.

PM interpretation: the spike's effect on the vol forecast lasted ~5 days, not 1. Capital reserves and incident-cost models that ignore clustering systematically underestimate next-day risk after an outage.

### Common Pitfalls

- Using sample standard deviation across regime changes. Mixing calm and storm periods underestimates storm vol and overestimates calm vol.
- Annualizing without checking i.i.d. The $\sqrt{N}$ scaling assumes independent returns; in practice autocorrelation inflates true annual vol.
- Treating vol as constant for capital reserves. Banks failed in 2008 partly because their internal models assumed constant vol.
- Confusing volatility with risk. Volatility is symmetric (up moves count too); downside risk needs separate metrics (semi-deviation, VaR, expected shortfall).
- GARCH parameter constraints. If $\alpha + \beta \ge 1$, the model is non-stationary in variance — predictions blow up.

| PM Relevance |
| --- |
| - Why a PM must master this: Risk reserves, SLA budgeting, fraud-rate alerting, and any "is this number normal?" question reduces to volatility math. |
| - Metrics to own: Realized vol, EWMA vol, vol-of-vol, regime indicators, exceedance count vs vol-implied bounds. |
| - Strategic tradeoffs: Tight vol-based alarms catch incidents early but alert-fatigue ops; loose alarms save fatigue but miss tails. |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "Volatility is just the standard deviation of returns, but with two twists every PM misses: it scales by $\sqrt T$, and it clusters in time. I model failure-rate and refund-rate vol with EWMA at Paytm because the clustering is real — an outage's risk shadow lasts days, not hours, and capital reserves need to reflect that." |

### NumPy-only Python Snippet

```
import numpy as np

# Worked Example 1: simple historical vol
R = np.array([0.48, 0.51, 0.49, 0.52, 0.50])
print(f"refund mean = {R.mean():.4f}%, std = {R.std(ddof=1):.4f}%")

# Worked Example 2: EWMA on daily failure-rate changes
F = np.array([0.6, 0.7, 0.5, 0.8, 2.5, 1.8, 1.2, 0.9, 0.7, 0.6])
dF = np.diff(F)
lam = 0.94
seed = (dF[:3]**2).mean()
var = [seed]
for t in range(3, len(dF)):
    var.append(lam * var[-1] + (1 - lam) * dF[t]**2)
var = np.array(var)
sigma = np.sqrt(var)
for i, s in enumerate(sigma, start=4):
    print(f"day {i}: sigma_hat = {s:.4f}")
```

## 7.5 Sharpe Ratio and Information Ratio — Risk-Adjusted Performance

## Plain-English Intuition

If two fund managers both made 15% last year, are they equally good? Not if one took twice the risk to get there. The Sharpe ratio normalizes return by volatility — it is the per-unit-of-risk return. The Information ratio does the same thing but against a benchmark, measuring the consistency of alpha rather than raw outperformance.

Fintech analogy: imagine evaluating two Paytm growth bets — campaign A delivers 20% lift with high week-to-week swings; campaign B delivers 15% lift with smooth weekly numbers. Campaign B has a higher Sharpe-like ratio of return-per-volatility, and is often the better roadmap choice because it is easier to forecast, defend, and scale. The same math applies whether you are picking a campaign, a fund manager, or an algo-trading strategy.

## Formal Mathematical Definition

Sharpe ratio (Sharpe, 1966). $$ \text{SR} = \frac{\mathbb{E}[R_p - R_f]}{\sigma_{R_p - R_f}} $$

Plain English: Sharpe is the expected excess return divided by the standard deviation of excess return. $R_p$ = portfolio return, $R_f$ = risk-free rate.

Derivation. Under the assumption that returns are normally distributed and a single risky asset is held with the risk-free, the Sharpe ratio is the slope of the capital allocation line from the origin (in excess-return space). Maximizing the Sharpe ratio gives the tangency portfolio.

Annualization (i.i.d. assumption). $\text{SR}_{\text{annual}} = \text{SR}_{\text{period}} \sqrt{N}$, where $N$ is periods/year.

Information ratio. $$ \text{IR} = \frac{\mathbb{E}[R_p - R_b]}{\sigma_{R_p - R_b}} $$

where $R_b$ is the benchmark (e.g., index, peer median). The denominator (tracking error) measures how consistently the portfolio differs from the benchmark; the numerator (active return / alpha) measures the average outperformance.

Relationship. Sharpe uses risk-free as benchmark; IR uses a market or peer benchmark.

Sortino ratio. Variant using downside deviation in the denominator: $\text{Sortino} = (\mathbb{E}[R_p] - R_f)/\sigma_{\text{down}}$. Penalizes only volatility below a target.

| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| $R_p$ | Portfolio return | unitless | Campaign / fund return |
| $R_f$ | Risk-free rate | unitless | Treasury / repo rate |
| $R_b$ | Benchmark return | unitless | Market index / peer median |
| SR | Sharpe ratio | unitless | Return per unit of total risk |
| IR | Information ratio | unitless | Alpha per unit of tracking error |
| tracking error | $\sigma_{R_p - R_b}$ | unitless | Stability of active return |

## Worked Example 1 — Sharpe Ratio of a Paytm Treasury Strategy

Twelve months of strategy returns (%): $R_p = (1.2, 0.8, 1.5, 0.9, 1.1, 1.3, 0.7, 1.4, 1.0, 1.2, 0.9, 1.1)$. Monthly risk-free rate: $R_f = 0.5\%$ (6% annual / 12).

Step 1 — Excess returns $e_t = R_p - R_f$: $(0.7, 0.3, 1.0, 0.4, 0.6, 0.8, 0.2, 0.9, 0.5, 0.7, 0.4, 0.6)$. Sum = 7.1; mean = 7.1/12 = 0.5917%.

Step 2 — Sample standard deviation of excess returns:

- Deviations from 0.5917: $(0.1083, -0.2917, 0.4083, -0.1917, 0.0083, 0.2083, -0.3917, 0.3083, -0.0917, 0.1083, -0.1917, 0.0083)$.
- Squared: $(0.01173, 0.08509, 0.16671, 0.03675, 0.00007, 0.04339, 0.15343, 0.09505, 0.00841, 0.01173, 0.03675, 0.00007)$. Sum = 0.64918.

- Variance (n1=11): $0.64918/11 = 0.05902$. Std dev = 0.2429%.

Step 3 — Monthly Sharpe: $0.5917 / 0.2429 = 2.436$.

Step 4 — Annualized Sharpe: $2.436 \times \sqrt{12} = 2.436 \times 3.4641 = 8.439$.

PM interpretation: an annualized Sharpe of 8 is enormous (typical equity-strategy Sharpes are 0.5–1.5; great quant funds are 2–4). Either the strategy is exceptional, or — more likely — returns are smoothed (illiquid assets marked stale, autocorrelation inflating apparent Sharpe). A PM here should immediately ask for liquidity-adjusted Sharpe and Newey–West-corrected vol.

## Worked Example 2 — Information Ratio of a Paytm-Money Algo vs Nifty 50

Eight quarterly returns (%): algo $R_p = (3.0, -1.5, 2.0, 4.0, 1.5, -0.5, 3.5, 2.5)$, benchmark Nifty $R_b = (2.5, -2.0, 1.5, 3.5, 1.0, -1.0, 3.0, 2.0)$.

Step 1 — Active returns $a_t = R_p - R_b$: $(0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5)$. All equal! Mean = 0.5%.

Step 2 — Tracking error: deviations from 0.5 = (0, 0, 0, 0, 0, 0, 0, 0). $\sigma_a = 0$. Sharpe-style ratio is undefined. Lesson: this is a synthetic "perfect alpha" case. In practice, real algos have non-zero TE.

Now perturb realistically: $R_p = (3.2, -1.6, 2.2, 3.9, 1.7, -0.4, 3.6, 2.4)$ — small noise around 0.5% alpha.

Step 1' — Active: $(0.7, 0.4, 0.7, 0.4, 0.7, 0.6, 0.6, 0.4)$. Sum = 4.5; mean = 0.5625%.

Step 2' — Deviations from 0.5625: $(0.1375, -0.1625, 0.1375, -0.1625, 0.1375, 0.0375, 0.0375, -0.1625)$. Squared: $0.01891, 0.02641, 0.01891, 0.02641, 0.01891, 0.00141, 0.00141, 0.02641$. Sum = 0.13878.

- Variance (n1=7): 0.01983. Std dev = 0.1408%.

Step 3' — Quarterly IR: $0.5625 / 0.1408 = 3.994$.

Step 4' — Annualized IR: $3.994 \times \sqrt{4} = 7.987$.

PM interpretation: an annualized IR ~8 is also extraordinary; even Grinold's "Fundamental Law of Active Management" suggests IRs above 1.0 are very good. As with Sharpe, a too-good-to-be-true IR usually means the benchmark or the data window is wrong. The PM's job is to push back, not celebrate.

## Common Pitfalls

- Annualizing Sharpe without checking autocorrelation. Positive autocorrelation inflates Sharpe by 30–100% in illiquid books.
- Using arithmetic mean of percentage returns. Geometric / log-return mean is the right basis for compounded performance.
- Comparing Sharpe across asset classes with different distributions. Heavy-tailed returns make Sharpe misleading; supplement with Sortino, VaR, max drawdown.
- Ignoring the benchmark choice. IR depends on the benchmark; pick the wrong benchmark and you flatter or punish unfairly.
- Mistaking high IR for skill. Low-TE strategies (closet indexers) can show high IR from luck alone; statistical significance test is required (t-stat = IR × (years), roughly).

PM Relevance

| |
|---|
| - Why a PM must master this: Even in non-trading roles, risk-adjusted return is how serious leadership compares bets — campaign vs campaign, line of business vs line. Sharpe/IR vocabulary makes you sound like a quant in front of finance. |
| - Metrics to own: Per-strategy Sharpe, per-strategy IR vs benchmark, t-stat of mean active return, max drawdown, Sortino. |
| - Strategic tradeoffs: Maximizing Sharpe favors low-vol strategies; maximizing absolute return ignores risk; the right objective depends on the capital provider's utility. |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "Sharpe is excess return over total volatility; Information ratio is alpha over tracking error. As a fintech PM I treat both as the lingua franca of any risk conversation — Sharpe inside a strategy, IR across strategies — and I always pair them with a t-stat for years-of-data, because a 'great Sharpe' on six months of return is usually just luck or stale marks." |

### NumPy-only Python Snippet

```
import numpy as np

def sharpe_ratio(returns, rf, periods_per_year):
    e = np.asarray(returns) - rf
    return (e.mean() / e.std(ddof=1)) * np.sqrt(periods_per_year)

def information_ratio(rp, rb, periods_per_year):
    a = np.asarray(rp) - np.asarray(rb)
    return (a.mean() / a.std(ddof=1)) * np.sqrt(periods_per_year)

# Worked Example 1: monthly Sharpe (12/yr)
Rp = np.array([1.2,0.8,1.5,0.9,1.1,1.3,0.7,1.4,1.0,1.2,0.9,1.1])
print(f"Annualized Sharpe = {sharpe_ratio(Rp, rf=0.5, periods_per_year=12):.3f}")

# Worked Example 2: quarterly IR (4/yr)
Rp_q = np.array([3.2,-1.6,2.2,3.9,1.7,-0.4,3.6,2.4])
Rb_q = np.array([2.5,-2.0,1.5,3.5,1.0,-1.0,3.0,2.0])
print(f"Annualized IR     = {information_ratio(Rp_q, Rb_q, periods_per_year=4):.3f}")
```

## 7.6 Fintech Forecasting: Revenue, MDR Pricing, and Transaction Seasonality

### Plain-English Intuition

Forecasting fintech revenue is the ultimate stress test of everything in this chapter. You have a non-stationary series (revenue grows), with seasonality (monsoon spikes, Diwali surges, salary-day clusters), with regime breaks (UPI launched, MDR rules changed), and with embedded volatility (festival vol > non-festival vol). Doing this badly is the most common reason fintech boards lose faith in their PMs. Doing it well is a superpower.

Fintech analogy: Paytm's GMV has a clear weekly seasonality (Friday-Saturday spike), monthly (salary cycle), and annual (Diwali / IPL / monsoon-end). MDR pricing is a slow-moving series with quarterly steps driven by competitive pressure and RBI policy. Transaction counts spike non-linearly during cricket finals and budget announcements. A robust forecast separates trend, seasonality, holidays, and residual stochastic process — and quantifies the uncertainty of each.

### Formal Mathematical Definition

Classical decomposition. $$ X_t = T_t + S_t + H_t + \varepsilon_t \quad \text{(additive)}, \quad \text{or} \quad X_t = T_t \cdot S_t \cdot H_t \cdot \varepsilon_t \quad \text{(multiplicative)} $$

with $T_t$ trend, $S_t$ seasonal component (period $s$), $H_t$ holiday/event effects, $\varepsilon_t$ residual. Plain English: a time series is the sum (or product) of trend, seasonality, holiday effects, and noise.

Holt–Winters triple exponential smoothing. Three smoothing equations for level $L_t$, trend $b_t$, seasonal $S_t$ with period $s$: $$ L_t = \alpha (X_t - S_{t-s}) + (1-\alpha)(L_{t-1} + b_{t-1}) $$ $$ b_t = \beta (L_t - L_{t-1}) + (1-\beta) b_{t-1} $$ $$ S_t = \gamma (X_t - L_t) + (1-\gamma) S_{t-s} $$

Forecast $h$ steps ahead: $\hat X_{t+h} = L_t + h\, b_t + S_{t+h-s\lceil h/s\rceil}$.

SARIMA(p, d, q)(P, D, Q)\(_s\). Seasonal-ARIMA: ARMA on differences plus seasonal differences of period \(s\). The industry default for monthly/weekly seasonal data.

MDR pricing dynamics. MDR \(M_t\) is often modeled as \[ M_t = M_{t-1} + \Delta_t^{\text{policy}} + \Delta_t^{\text{competition}} + \varepsilon_t \] where the deterministic increments come from rate-card decisions and the stochastic part is small. Forecasts are mostly scenario trees, not statistical extrapolation.

| Symbol | Definition | Dimensions | PM Interpretation |
|---|---|---|---|
| \(T_t\) | Trend | level | Long-run growth |
| \(S_t\) | Seasonal | level deviation | Repeating pattern (week, month, year) |
| \(H_t\) | Holiday effect | level deviation | One-off but recurrent (Diwali, IPL) |
| \(\varepsilon_t\) | Residual noise | level deviation | Stochastic remainder |
| \(\alpha, \beta, \gamma\) | Smoothing params | unitless | Level, trend, season adaptivity |
| \(s\) | Season length | integer | 7 (weekly), 12 (monthly), 365 (yearly) |

## Worked Example 1 — Holt–Winters Forecast of Paytm Monthly Revenue with Seasonality

Monthly revenue (₹ crore) over 12 months: \(X = (100, 105, 115, 110, 120, 130, 125, 135, 140, 150, 160, 180)\). Period \(s = 12\) is too long for 12 obs; use \(s = 3\) (quarterly seasonality as a proxy). Smoothing: \(\alpha = 0.5, \beta = 0.3, \gamma = 0.5\).

Step 1 — Initialize. Level \(L_3\) = average of first 3: \((100+105+115)/3 = 106.67\). Trend \(b_3\) = (X_4 - X_1)/3 = (110-100)/3 = 3.33. Seasonal initial values: \(S_1 = X_1 - L_3 = 100 - 106.67 = -6.67\); \(S_2 = 105 - 106.67 = -1.67\); \(S_3 = 115 - 106.67 = 8.33\).

Step 2 — Recurse for t=4..12. (We will compute through t=12 explicitly.)

t=4: \(L_4 = 0.5(110 - S_1) + 0.5(L_3 + b_3) = 0.5(110-(-6.67)) + 0.5(106.67+3.33) = 0.5(116.67) + 0.5(110.00) = 58.33 + 55.00 = 113.33\). \(b_4 = 0.3(113.33-106.67) + 0.7(3.33) = 0.3(6.67) + 2.33 = 2.00 + 2.33 = 4.33\). \(S_4 = 0.5(110 - 113.33) + 0.5(S_1) = 0.5(-3.33) + 0.5(-6.67) = -1.67 - 3.33 = -5.00\).

t=5: \(L_5 = 0.5(120 - S_2) + 0.5(L_4 + b_4) = 0.5(120-(-1.67)) + 0.5(113.33+4.33) = 0.5(121.67) + 0.5(117.67) = 60.83 + 58.83 = 119.67\). \(b_5 = 0.3(119.67-113.33) + 0.7(4.33) = 0.3(6.33) + 3.03 = 1.90 + 3.03 = 4.93\). \(S_5 = 0.5(120-119.67) + 0.5(-1.67) = 0.17 - 0.83 = -0.67\).

t=6: \(L_6 = 0.5(130 - S_3) + 0.5(L_5 + b_5) = 0.5(130-8.33) + 0.5(119.67+4.93) = 0.5(121.67) + 0.5(124.60) = 60.83 + 62.30 = 123.13\). \(b_6 = 0.3(123.13-119.67) + 0.7(4.93) = 0.3(3.47) + 3.45 = 1.04 + 3.45 = 4.49\). \(S_6 = 0.5(130-123.13) + 0.5(8.33) = 3.43 + 4.17 = 7.60\).

t=7: \(L_7 = 0.5(125 - S_4) + 0.5(L_6 + b_6) = 0.5(125-(-5.00)) + 0.5(123.13+4.49) = 0.5(130.00) + 0.5(127.62) = 65.00 + 63.81 = 128.81\). \(b_7 = 0.3(128.81-123.13) + 0.7(4.49) = 0.3(5.68) + 3.14 = 1.71 + 3.14 = 4.85\). \(S_7 = 0.5(125-128.81) + 0.5(-5.00) = -1.91 - 2.50 = -4.41\).

t=8: \(L_8 = 0.5(135-S_5) + 0.5(L_7+b_7) = 0.5(135-(-0.67)) + 0.5(128.81+4.85) = 0.5(135.67) + 0.5(133.66) = 67.83 + 66.83 = 134.67\). \(b_8 = 0.3(134.67-128.81) + 0.7(4.85) = 0.3(5.86) + 3.40 = 1.76 + 3.40 = 5.16\). \(S_8 = 0.5(135-134.67) + 0.5(-0.67) = 0.17 - 0.33 = -0.17\).

t=9: \(L_9 = 0.5(140-S_6) + 0.5(L_8+b_8) = 0.5(140-7.60) + 0.5(134.67+5.16) = 0.5(132.40) + 0.5(139.83) = 66.20 + 69.92 = 136.12\). \(b_9 = 0.3(136.12-134.67) + 0.7(5.16) = 0.3(1.45) + 3.61 = 0.43 + 3.61 = 4.04\). \(S_9 = 0.5(140-136.12) + 0.5(7.60) = 1.94 + 3.80 = 5.74\).

t=10: \(L_{10} = 0.5(150-S_7) + 0.5(L_9+b_9) = 0.5(150-(-4.41)) + 0.5(136.12+4.04) = 0.5(154.41) + 0.5(140.16) = 77.20 + 70.08 = 147.28\). \(b_{10} = 0.3(147.28-136.12)+0.7(4.04) = 0.3(11.16)+2.83 = 3.35+2.83 = 6.18\). \(S_{10} = 0.5(150-147.28) + 0.5(-4.41) = 1.36 - 2.21 = -0.84\).

t=11: \(L_{11} = 0.5(160-S_8) + 0.5(L_{10}+b_{10}) = 0.5(160-(-0.17)) + 0.5(147.28+6.18) = 0.5(160.17) + 0.5(153.46) = 80.08 + 76.73 = 156.82\). \(b_{11} = 0.3(156.82-147.28)+0.7(6.18) = 0.3(9.54)+4.33 = 2.86+4.33 = 7.19\). \(S_{11} = 0.5(160-156.82) +

0.5(-0.17) = 1.59 - 0.08 = 1.51\).

t=12: \(L_{12} = 0.5(180-S_9) + 0.5(L_{11}+b_{11}) = 0.5(180-5.74) + 0.5(156.82+7.19) = 0.5(174.26) + 0.5(164.01) = 87.13 + 82.01 = 169.14\). \(b_{12} = 0.3(169.14-156.82)+0.7(7.19) = 0.3(12.32)+5.03 = 3.69+5.03 = 8.72\). \(S_{12} = 0.5(180-169.14)+0.5(5.74) = 5.43+2.87 = 8.30\).

Step 3 — Forecast t=13, 14, 15: \(\hat X_{13} = L_{12} + 1\cdot b_{12} + S_{13-3} = 169.14 + 8.72 + S_{10} = 169.14 + 8.72 + (-0.84) = 177.02\). \(\hat X_{14} = L_{12} + 2 b_{12} + S_{11} = 169.14 + 17.43 + 1.51 = 188.08\). \(\hat X_{15} = L_{12} + 3 b_{12} + S_{12} = 169.14 + 26.15 + 8.30 = 203.59\).

PM interpretation: the next quarter's revenue is forecast at ₹177, ₹188, ₹204 crore — and the trend coefficient \(b_{12} = 8.72\) tells the CFO the model's view of underlying monthly growth net of seasonality. This is a defensible, reproducible quarterly board number.

## Worked Example 2 — MDR Pricing Path Over 4 Quarters

Current MDR is 200 bps. Policy decisions: Q1 hold, Q2 cut 10 bps (regulator-driven on UPI category), Q3 hold, Q4 raise 5 bps (premium-merchant category mix shift). Competition shocks (random):

- Q1: 2 bps, Q2: 0, Q3: +3 bps, Q4: 1 bps.

Step 1 — Apply increments:

- Q1: 200 + 0 + (2) = 198 bps
- Q2: 198 + (10) + 0 = 188 bps
- Q3: 188 + 0 + 3 = 191 bps
- Q4: 191 + 5 + (1) = 195 bps

Step 2 — Revenue impact at GMV ₹1,00,000 crore per quarter:

- Q1: 0.0198 × 100000 = ₹1,980 crore MDR revenue
- Q2: 0.0188 × 100000 = ₹1,880 crore
- Q3: 0.0191 × 100000 = ₹1,910 crore
- Q4: 0.0195 × 100000 = ₹1,950 crore
- Total: ₹7,720 crore

Step 3 — Counterfactual (no policy changes, only competition): MDR path 198, 198, 201, 200. Revenue: 1980 + 1980 + 2010 + 2000 = ₹7,970 crore. Policy cost: 7970  7720 = ₹250 crore over the year — a number the PM and finance leader must own jointly.

PM interpretation: decomposing the MDR path into deterministic policy and stochastic competition gives the org a clean lens on "what we chose vs what happened to us." This is forecasting as a decision tool, not a prediction game.

## Common Pitfalls

- One model fits all seasonality. Weekly, monthly, yearly seasonality often coexist; an HW with period 7 misses Diwali entirely.
- Treating holidays as outliers. Holidays are signal, not noise; modeling them as fixed events (Fourier terms, indicator variables) is essential.
- Confusing in-sample fit with forecast skill. A model with high $R^2$ on training can extrapolate disastrously; always hold out a future window.

- Ignoring regime breaks. UPI launch, demonetization, COVID — each is a structural break; pre-break data are not stationary with post-break data.
- Reporting forecasts without intervals. A single number invites overconfidence; always report 50% and 95% intervals.

| PM Relevance |
| --- |
| - Why a PM must master this: Forecasting is the PM job at the strategic layer. Every roadmap defense is implicitly a forecast — of revenue, adoption, fraud loss, headcount need. |
| - Metrics to own: MAPE / MAE / RMSE of rolling-origin forecasts; coverage of prediction intervals; bias (signed error) by month and segment. |
| - Strategic tradeoffs: Aggressive models capture recent shifts but overreact; conservative models are stable but miss structural change. Choose your bias deliberately. |
| - 30-second interview talking point for a fintech PM defending their Georgia Tech MSCS: "I model Paytm revenue as trend + seasonality + holiday + residual, with Holt–Winters or SARIMA as the baseline. The number I take to the board is always a distribution, not a point — and I report MAPE on a rolling-origin holdout because that is the only honest measure of forecast skill. My Georgia Tech MSCS gives me the toolkit; my fintech reps tell me which seasonalities and breakpoints actually matter." |

### NumPy-only Python Snippet

```
import numpy as np

# Worked Example 1: Holt-Winters additive with period s=3
X = np.array([100,105,115,110,120,130,125,135,140,150,160,180], dtype=float)
s, alpha, beta, gamma = 3, 0.5, 0.3, 0.5

L = np.zeros(len(X)); b = np.zeros(len(X)); S = np.zeros(len(X))
L[s-1] = X[:s].mean()
b[s-1] = (X[s] - X[0]) / s
for i in range(s):
    S[i] = X[i] - L[s-1]

for t in range(s, len(X)):
    L[t] = alpha*(X[t] - S[t-s]) + (1-alpha)*(L[t-1] + b[t-1])
    b[t] = beta*(L[t] - L[t-1]) + (1-beta)*b[t-1]
    S[t] = gamma*(X[t] - L[t]) + (1-gamma)*S[t-s]

print(f"L[12]={L[-1]:.3f}, b[12]={b[-1]:.3f}")
for h in (1, 2, 3):
    fc = L[-1] + h*b[-1] + S[len(X)-s + ((h-1) % s)]
    print(f"Forecast t+{h} = {fc:.2f}")

# Worked Example 2: MDR pricing path
mdr = 200.0
policy = np.array([0, -10, 0, 5])
comp   = np.array([-2, 0, 3, -1])
path = mdr + np.cumsum(policy + comp)
GMV = 100000  # crore per quarter
rev = (path / 10000) * GMV  # bps -> fraction
print("MDR path (bps):", path)
print("MDR revenue (₹cr):", rev, "  total:", rev.sum())
```

### Chapter 7 — Closing Synthesis

Stationarity is the price of admission; ARMA is the engine; log returns are the currency; volatility sizes the risk; Sharpe and IR rank the bets; decomposition + Holt-Winters/SARIMA carry the forecast. Master this stack and you can sit between Paytm's data-science team, the trading desk at Paytm Money, and the finance leadership and translate cleanly in every direction. That translation layer — math fluent, product credible, business literate — is what your MSCS at Georgia Tech is for. It is exactly what this volume is rebuilding.

## End of Chapters 5–7

Next volume preview: Chapter 8 begins with stochastic processes — Markov chains, hidden Markov models, and Bayesian networks — building directly on the discrete and information-theoretic foundations laid here.

# Appendix A: Master Formula Cheat Sheet

The table below collects every major formula introduced in Chapters 1–7. Each row lists the symbolic statement and a plain-English meaning. Formulas are grouped first by chapter, then by section so the cheat sheet doubles as a study outline.

| Chapter / Section | Formula | Plain English Meaning |
|---|---|---|
| **Ch.1 §1.1 Vectors** | $\mathbf{v} \in \mathbb{R}^n,\ \mathbf{v} = (v_1,\ldots,v_n)^\top$ | A vector is an ordered list of $n$ real numbers, one per feature. |
| Ch.1 §1.1 | $(\mathbf{u}+\mathbf{v})_i = u_i + v_i$ | Vector addition is componentwise. |
| Ch.1 §1.1 | $(\alpha \mathbf{v})_i = \alpha v_i$ | Scalar multiplication stretches every component by the same factor. |
| Ch.1 §1.1 | $\mathbf{u}\cdot\mathbf{v} = \sum_{i=1}^n u_i v_i = \|\mathbf{u}\|\|\mathbf{v}\|\cos\theta$ | Dot product equals component-sum equals magnitudes times cosine of angle between them. |
| Ch.1 §1.1 | $\|\mathbf{v}\| = \sqrt{\sum_i v_i^2} = \sqrt{\mathbf{v}\cdot\mathbf{v}}$ | Euclidean norm is the square-root of the sum of squared components. |
| Ch.1 §1.1 | $\|\mathbf{u}-\mathbf{v}\|^2 = \|\mathbf{u}\|^2 + \|\mathbf{v}\|^2 - 2\mathbf{u}\cdot\mathbf{v}$ | Law-of-cosines identity that derives the geometric meaning of the dot product. |
| **Ch.1 §1.2 Matrices** | $A \in \mathbb{R}^{m\times n},\ A_{ij}$ | A matrix is a rectangular table with $m$ rows and $n$ columns; entry $A_{ij}$ sits at row $i$, column $j$. |
| Ch.1 §1.2 | $(A^\top)_{ij} = A_{ji}$ | Transpose flips rows and columns. |
| Ch.1 §1.2 | $(A\mathbf{x})_i = \sum_{j=1}^n a_{ij} x_j$ | Matrix-vector product: each output is a weighted sum of input components, weights from the corresponding row. |
| Ch.1 §1.2 | $A\mathbf{x} = x_1\mathbf{a}_1 + \cdots + x_n\mathbf{a}_n$ | Same product viewed as a linear combination of the matrix's columns. |
| **Ch.1 §1.3 Matrix multiplication** | $C_{ij} = \sum_{k=1}^n A_{ik}B_{kj},\ C=AB$ | Each entry of the product is a dot product of a row of $A$ and a column of $B$. |
| Ch.1 §1.3 | $(AB)^\top = B^\top A^\top$ | Transpose of a product reverses the factors. |
| Ch.1 §1.3 | $(AB)C = A(BC)$ | Matrix multiplication is associative. |
| Ch.1 §1.3 | $A(B+C) = AB + AC$ | Matrix multiplication distributes over addition. |
| Ch.1 §1.3 | $AB \ne BA$ in general | Matrix multiplication is not commutative — order encodes the order of operations. |
| Ch.1 §1.3 | Cost $= O(mnp)$ for $(m\times n)(n\times p)$ | Naive multiplication scales as the product of the three dimensions. |
| **Ch.1 §1.4 Linear independence / basis** | $\sum_i c_i \mathbf{v}_i = \mathbf{0} \Rightarrow c_i = 0\ \forall i$ | Vectors are linearly independent iff only the trivial combination produces zero. |
| Ch.1 §1.4 | $\mathrm{span}\{\mathbf{v}_i\} = \{\sum c_i \mathbf{v}_i\}$ | The span is the set of all linear combinations. |
| Ch.1 §1.4 | $\dim V = \|\text{any basis of } V\|$ | Dimension is the size of any basis, well-defined because every basis of $V$ has the same cardinality. |
| Ch.1 §1.4 | Independence test: $\mathrm{rank}(M)=k$ | Stack vectors as columns; full column rank certifies independence. |
| **Ch.1 §1.5 Determinant** | $\det(A) = \sum_{\sigma\in S_n}\mathrm{sgn}(\sigma)\prod_i A_{i,\sigma(i)}$ | Leibniz formula: signed sum over all permutations of choosing one entry per row and column. |
| Ch.1 §1.5 | $\det\begin{bmatrix}a&b\\c&d\end{bmatrix} = ad-bc$ | The 2×2 determinant is the signed area of the parallelogram spanned by the two columns. |
| Ch.1 §1.5 | $\det(AB) = \det(A)\det(B)$ | Determinants multiply across composition. |
| Ch.1 §1.5 | $\det(A^\top) = \det(A)$ | Row and column operations have the same effect on the determinant. |
| Ch.1 §1.5 | $\det(A^{-1}) = 1/\det(A)$ | Inverting a transformation inverts its volume-scaling factor. |
| Ch.1 §1.5 | $\det(A)=0 \iff A\text{ singular}$ | Zero determinant flags a matrix that collapses dimensions and has no inverse. |
| **Ch.1 §1.6 Trace** | $\mathrm{tr}(A) = \sum_i A_{ii}$ | Trace is the sum of diagonal entries. |
| Ch.1 §1.6 | $\mathrm{tr}(ABC) = \mathrm{tr}(BCA) = \mathrm{tr}(CAB)$ | Trace is invariant under cyclic permutations of factors. |
| Ch.1 §1.6 | $\mathrm{tr}(A) = \sum_i \lambda_i$ | Trace equals the sum of eigenvalues counted with algebraic multiplicity. |
| Ch.1 §1.6 | $\|A\|_F^2 = \mathrm{tr}(A^\top A) = \sum_{ij} A_{ij}^2$ | Frobenius norm squared equals trace of $A^\top A$ — the total energy of the matrix. |
| **Ch.1 §1.7 Inverse / linear systems** | $AA^{-1} = A^{-1}A = I$ | An inverse "undoes" the transformation. |
| Ch.1 §1.7 | $A^{-1} = \frac{1}{ad-bc}\begin{bmatrix}d&-b\\-c&a\end{bmatrix}$ (2×2) | Explicit 2×2 inverse: swap diagonals, flip signs on off-diagonals, divide by the determinant. |
| Ch.1 §1.7 | $A^{-1} = \frac{1}{\det(A)}\,\mathrm{adj}(A)$ | General inverse: adjugate (transposed cofactor matrix) over determinant. |
| Ch.1 §1.7 | $\mathbf{x} = A^{-1}\mathbf{b}$ | Unique solution of a square invertible linear system. |
| Ch.1 §1.7 | $\mathbf{x}^* = (A^\top A)^{-1} A^\top \mathbf{b}$ | Normal-equations least-squares solution for an overdetermined system. |
| Ch.1 §1.7 | $\mathbf{x}^* = A^\top(AA^\top)^{-1}\mathbf{b}$ | Minimum-norm solution for an underdetermined system. |
| Ch.1 §1.7 | $\kappa(A) = \|A\|\|A^{-1}\|$ | Condition number measures how much input error amplifies into output error. |
| **Ch.1 §1.8 Eigenvalues / eigenvectors** | $A\mathbf{v} = \lambda \mathbf{v},\ \mathbf{v}\ne \mathbf{0}$ | An eigenvector is a direction the matrix only stretches by factor $\lambda$, without rotating. |
| Ch.1 §1.8 | $\det(A - \lambda I) = 0$ | Characteristic equation whose roots are the eigenvalues. |
| Ch.1 §1.8 | $\lambda^2 - \mathrm{tr}(A)\lambda + \det(A) = 0$ (2×2) | Quadratic characteristic polynomial for a 2×2 matrix in terms of trace and determinant. |
| Ch.1 §1.8 | $A = Q\Lambda Q^\top$ (symmetric $A$) | Spectral theorem: a real symmetric matrix admits an orthonormal eigenbasis and diagonalizes by orthogonal rotation. |
| **Ch.1 §1.9 PCA** | $C = \tfrac{1}{m-1} X^\top X$ (centered $X$) | Sample covariance matrix from a centered data matrix. |
| Ch.1 §1.9 | $\max_{\|\mathbf{w}\|=1}\mathbf{w}^\top C\mathbf{w}$ | First principal component maximizes projected variance on the unit sphere. |
| Ch.1 §1.9 | $C\mathbf{w} = \lambda\mathbf{w}$ | Lagrangian first-order condition: principal components are eigenvectors of the covariance matrix. |
| Ch.1 §1.9 | $\mathrm{EVR}_k = \lambda_k / \mathrm{tr}(C)$ | Explained-variance ratio of the $k$-th component is its eigenvalue divided by the trace. |
| **Ch.1 §1.10 SVD** | $A = U\Sigma V^\top$ | Every real matrix factors into orthogonal rotation, diagonal scaling, and orthogonal rotation. |

| Ch.1 §1.10 | \( A^\top A = V \Lambda V^\top,\ \sigma_i = \sqrt{\lambda_i} \) | Right singular vectors diagonalize \(A^\top A\); singular values are square roots of those eigenvalues. |
| Ch.1 §1.10 | \( \mathbf{u}_i = \tfrac{1}{\sigma_i} A \mathbf{v}_i \) | Left singular vectors are normalized images of right singular vectors. |
| Ch.1 §1.10 | \( A_k = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^\top \) | Truncated SVD gives the best rank-\(k\) approximation under Frobenius (and spectral) norm. |
| Ch.1 §1.10 | \( \|A - A_k\|_F^2 = \sum_{i>k} \sigma_i^2 \) | Reconstruction error from truncation equals sum of squared dropped singular values (Eckart–Young). |
| **Ch.1 §1.11 High-dim spaces** | \( V_n = \pi^{n/2}/\Gamma(n/2+1) \) | Volume of the unit ball in \(\mathbb{R}^n\); concentrates near zero in high dimensions. |
| Ch.1 §1.11 | \( \mathrm{Var}(\mathbf{u}\cdot\mathbf{v}) = 1/n \) (random unit vectors) | Random pairs in high-\(n\) are nearly orthogonal; dot product has standard deviation \(1/\sqrt{n}\). |
| Ch.1 §1.11 | \( k \approx 8\ln N / \epsilon^2 \) | Johnson–Lindenstrauss: number of dimensions needed to preserve pairwise distances within \(1\pm\epsilon\) for \(N\) points. |
| **Ch.1 §1.12 Cosine similarity** | \( \cos\theta = \dfrac{\mathbf{u}\cdot\mathbf{v}}{\|\mathbf{u}\|\|\mathbf{v}\|} \) | Cosine similarity is the magnitude-normalized inner product — pure angular alignment. |
| Ch.1 §1.12 | \( \|\mathbf{u}-\mathbf{v}\|^2 = 2 - 2\cos\theta \) (unit vectors) | On the unit sphere, squared Euclidean distance and cosine similarity are linked by a simple affine map. |
| **Ch.2 §2.1 Probability spaces** | \( (\Omega, \mathcal{F}, P) \) | A probability space is a sample space, an event sigma-algebra, and a probability measure. |
| Ch.2 §2.1 | \( P(A) \in [0,1],\ P(\Omega)=1 \) | Kolmogorov's first two axioms: probabilities are bounded between 0 and 1 and the whole sample space has probability 1. |
| Ch.2 §2.1 | \( P(\bigcup_i A_i) = \sum_i P(A_i) \) (disjoint) | Countable additivity for disjoint events. |
| Ch.2 §2.1 | \( P(A^c) = 1 - P(A) \) | The complement rule. |
| Ch.2 §2.1 | \( P(A\cup B) = P(A)+P(B)-P(A\cap B) \) | Inclusion-exclusion for two events. |
| **Ch.2 §2.2 Joint / marginal / conditional** | \( P(A\mid B) = P(A\cap B)/P(B) \) | Conditional probability re-normalizes over the conditioning event. |
| Ch.2 §2.2 | \( P(A,B) = P(A\mid B)P(B) = P(B\mid A)P(A) \) | Chain rule of probability. |
| Ch.2 §2.2 | \( P(A) = \sum_i P(A\mid B_i)P(B_i) \) | Law of total probability over a partition. |
| Ch.2 §2.2 | \( A \perp B \iff P(A\cap B) = P(A)P(B) \) | Independence equals product factorization of the joint. |
| **Ch.2 §2.3 Bayes' theorem** | \( P(A\mid B) = \dfrac{P(B\mid A)P(A)}{P(B)} \) | Posterior is prior times likelihood divided by evidence. |
| Ch.2 §2.3 | \( P(B) = P(B\mid A)P(A) + P(B\mid A^c)P(A^c) \) | Evidence as marginal over a binary hypothesis. |
| Ch.2 §2.3 | \( \dfrac{P(A\mid B)}{P(A^c\mid B)} = \dfrac{P(B\mid A)}{P(B\mid A^c)}\cdot\dfrac{P(A)}{P(A^c)} \) | Odds form: posterior odds equal likelihood ratio times prior odds. |
| **Ch.2 §2.4 Random variables** | \( F_X(x) = P(X\le x) \) | Cumulative distribution function. |
| Ch.2 §2.4 | \( p(x) = P(X=x) \) (discrete); \( f(x) = F'(x) \) (continuous) | Probability mass and probability density functions. |
| Ch.2 §2.4 | \( P(a\le X\le b) = \int_a^b f(x)\,dx \) | Continuous probabilities are areas under the density. |
| **Ch.2 §2.5 Expectation / variance / covariance** | \( E[X] = \sum_x x\,p(x) \) or \( \int x f(x)\,dx \) | Expectation is the probability-weighted average value. |
| Ch.2 §2.5 | \( \mathrm{Var}(X) = E[(X-\mu)^2] = E[X^2] - (E[X])^2 \) | Variance is the mean squared deviation from the mean. |
| Ch.2 §2.5 | \( \mathrm{Cov}(X,Y) = E[(X-\mu_X)(Y-\mu_Y)] \) | Covariance measures linear co-movement around respective means. |
| Ch.2 §2.5 | \( \rho_{XY} = \mathrm{Cov}(X,Y)/(\sigma_X \sigma_Y) \) | Pearson correlation is unit-free covariance scaled to \([-1,1]\). |
| Ch.2 §2.5 | \( \mathrm{Var}(aX+bY) = a^2\sigma_X^2 + b^2\sigma_Y^2 + 2ab\,\mathrm{Cov}(X,Y) \) | Variance of a linear combination — the heart of portfolio math. |
| **Ch.2 §2.6 Key distributions** | \( f(x) = \tfrac{1}{\sqrt{2\pi}\sigma}\exp(-(x-\mu)^2/(2\sigma^2)) \) | Normal (Gaussian) density. |
| Ch.2 §2.6 | \( P(X=k) = \binom{n}{k}p^k(1-p)^{n-k} \) | Binomial probability mass for \(k\) successes in \(n\) Bernoulli trials. |
| Ch.2 §2.6 | \( P(X=k) = \lambda^k e^{-\lambda}/k! \) | Poisson probability mass for count data with rate \(\lambda\). |
| Ch.2 §2.6 | \( f(x) = \alpha x_{\min}^\alpha x^{-(\alpha+1)} \) | Power-law (Pareto) density on \([x_{\min}, \infty)\). |
| Ch.2 §2.6 | \( P(X_1=n_1,\ldots,X_k=n_k) = \tfrac{n!}{\prod n_i!}\prod p_i^{n_i} \) | Multinomial probability mass — categorical generalization of the binomial. |
| **Ch.2 §2.7 CLT** | \( \dfrac{\bar X_n - \mu}{\sigma/\sqrt{n}} \xrightarrow{d} \mathcal{N}(0,1) \) | Standardized sample mean converges in distribution to a standard normal as \(n\to\infty\). |
| Ch.2 §2.7 | \( n \ge (z_{1-\alpha/2}+z_{1-\beta})^2 \sigma^2 / \delta^2 \) | Sample-size formula for a target minimum detectable effect \(\delta\). |
| **Ch.2 §2.8 MLE** | \( L(\theta) = \prod_i p(x_i\mid\theta);\ \ell(\theta) = \sum_i \log p(x_i\mid\theta) \) | Likelihood is the joint probability viewed as a function of parameters; log-likelihood sums logs. |
| Ch.2 §2.8 | \( \hat\theta_{MLE} = \arg\max_\theta \ell(\theta) \) | MLE picks the parameter that makes the observed data most probable. |
| Ch.2 §2.8 | \( \hat p = k/n \) (Bernoulli MLE) | Empirical fraction is the MLE for a Bernoulli success rate. |
| Ch.2 §2.8 | \( \hat\mu = \bar x,\ \hat\sigma^2 = \tfrac{1}{n}\sum(x_i-\bar x)^2 \) (Normal MLE) | Sample mean and (biased) sample variance are the Gaussian MLEs. |
| **Ch.2 §2.9 MAP** | \( p(\theta\mid\mathcal{D}) \propto L(\theta)p(\theta) \) | Bayes' rule for parameters: posterior is proportional to likelihood times prior. |
| Ch.2 §2.9 | \( \hat\theta_{MAP} = \arg\max_\theta [\ell(\theta) + \log p(\theta)] \) | MAP adds a log-prior penalty to the log-likelihood. |
| Ch.2 §2.9 | \( \hat\beta_{MAP} = (X^\top X + \lambda I)^{-1} X^\top \mathbf{y} \) | Ridge regression: closed-form MAP under a Gaussian prior on weights. |
| **Ch.2 §2.10 Hypothesis testing** | \( Z = \dfrac{\hat p_T - \hat p_C}{\sqrt{\hat p(1-\hat p)(1/n_C + 1/n_T)}} \) | Two-proportion Z-test statistic under the pooled null. |
| Ch.2 §2.10 | \( \text{CI}_{1-\alpha}: \hat\delta \pm z_{1-\alpha/2}\,\mathrm{SE} \) | Wald confidence interval for a difference of proportions. |
| Ch.2 §2.10 | \( \mathrm{Power} \approx \Phi(\|\delta\|/\mathrm{SE} - z_{1-\alpha/2}) \) | Approximate power for a two-sided test. |
| Ch.2 §2.10 | \( n \approx (z_{1-\alpha/2}+z_{1-\beta})^2[p_C(1-p_C)+p_T(1-p_T)]/(p_T-p_C)^2 \) | Sample size per arm for a target effect, alpha, and power. |
| **Ch.2 §2.11 Multivariable A/B testing** | \( Y_{\mathrm{adj}} = Y - \theta(X-\bar X),\ \theta=\mathrm{Cov}(Y,X)/\mathrm{Var}(X) \) | CUPED variance reduction: subtract the predictable pre-period component. |
| Ch.2 §2.11 | Bonferroni: test each at \(\alpha/m\) | Family-wise error control by dividing the significance level across hypotheses. |
| Ch.2 §2.11 | BH FDR: reject the largest \(k\) where \(p_{(k)} \le k\alpha/m\) | Benjamini-Hochberg controls the expected false discovery rate. |
| **Ch.3 §3.1 Limits / continuity** | \( \lim_{x\to a} f(x) = L \) | Function values approach \(L\) as the input approaches \(a\). |

| Ch.3 §3.1 | \( f \text{ continuous at } a \iff \lim_{x\to a}f(x) = f(a) \) | Continuity equals limit-equals-value. |
| **Ch.3 §3.2 Derivatives** | \( f'(x) = \lim_{h\to 0}\dfrac{f(x+h)-f(x)}{h} \) | Derivative is the instantaneous rate of change. |
| Ch.3 §3.2 | \( (x^n)' = nx^{n-1} \) | Power rule. |
| Ch.3 §3.2 | \( (uv)' = u'v + uv' \) | Product rule. |
| Ch.3 §3.2 | \( (u/v)' = (u'v - uv')/v^2 \) | Quotient rule. |
| Ch.3 §3.2 | \( (f\circ g)'(x) = f'(g(x))\,g'(x) \) | Single-variable chain rule. |
| **Ch.3 §3.3 Partial derivatives / gradient** | \( \dfrac{\partial f}{\partial x_i} = \lim_{h\to 0}\dfrac{f(\ldots,x_i+h,\ldots)-f(\mathbf{x})}{h} \) | Partial derivative is the single-variable derivative with all other coordinates frozen. |
| Ch.3 §3.3 | \( \nabla f(\mathbf{x}) = (\partial f/\partial x_1,\ldots,\partial f/\partial x_n)^\top \) | Gradient stacks all partials into a column vector. |
| Ch.3 §3.3 | \( D_{\mathbf{u}} f(\mathbf{x}) = \nabla f(\mathbf{x})^\top \mathbf{u} \) | Directional derivative equals gradient dotted with the unit direction. |
| Ch.3 §3.3 | Steepest descent direction: \( -\nabla f(\mathbf{x})/\|\nabla f\| \) | The negative gradient direction maximizes the rate of decrease (Cauchy–Schwarz). |
| **Ch.3 §3.5 Chain rule (multivariate)** | \( \dfrac{\partial \mathbf{y}}{\partial \mathbf{x}} = \dfrac{\partial \mathbf{y}}{\partial \mathbf{u}}\,\dfrac{\partial \mathbf{u}}{\partial \mathbf{x}} \) | Composition of vector functions: multiply Jacobians. |
| Ch.3 §3.5 | \( \dfrac{\partial L}{\partial W^{(\ell)}} = \dfrac{\partial L}{\partial \mathbf{a}^{(L)}}\prod_{k=L}^{\ell+1}\dfrac{\partial \mathbf{a}^{(k)}}{\partial \mathbf{a}^{(k-1)}}\,\dfrac{\partial \mathbf{a}^{(\ell)}}{\partial W^{(\ell)}} \) | Backprop: gradient at layer \(\ell\) is the chain product of layer-wise Jacobians from the loss down. |
| **Ch.3 §3.6 Jacobian** | \( J_{\mathbf{f}}(\mathbf{x})_{ij} = \partial f_i / \partial x_j \in \mathbb{R}^{m\times n} \) | The Jacobian collects all first-order sensitivities of vector output to vector input. |
| Ch.3 §3.6 | \( \mathbf{f}(\mathbf{x}+\mathbf{h}) \approx \mathbf{f}(\mathbf{x}) + J_{\mathbf{f}}(\mathbf{x})\mathbf{h} \) | Local linearization via the Jacobian. |
| Ch.3 §3.6 | \( J_{\mathbf{f}\circ\mathbf{g}}(\mathbf{x}) = J_{\mathbf{f}}(\mathbf{g}(\mathbf{x}))\,J_{\mathbf{g}}(\mathbf{x}) \) | Composition rule for Jacobians. |
| Ch.3 §3.6 | \( \|\det J\| \) = local volume-scaling factor | Change-of-variables magnitude (used in integration and normalizing flows). |
| **Ch.3 §3.7 Hessian** | \( H_f(\mathbf{x})_{ij} = \partial^2 f/(\partial x_i\partial x_j) \) | Matrix of second partial derivatives. |
| Ch.3 §3.7 | \( H_f = H_f^\top \) (Schwarz) | Under \(C^2\) smoothness, the Hessian is symmetric. |
| Ch.3 §3.7 | PD \(\Rightarrow\) min; ND \(\Rightarrow\) max; indefinite \(\Rightarrow\) saddle | Multivariate second-derivative test classifies critical points. |
| Ch.3 §3.7 | \( \mathbf{x}_{t+1} = \mathbf{x}_t - H_f(\mathbf{x}_t)^{-1}\nabla f(\mathbf{x}_t) \) | Newton's method: descent direction preconditioned by the inverse Hessian. |
| **Ch.3 §3.8 Taylor series** | \( f(x) = \sum_{k=0}^n \dfrac{f^{(k)}(a)}{k!}(x-a)^k + R_n(x) \) | Single-variable Taylor expansion with Lagrange remainder. |
| Ch.3 §3.8 | \( f(\mathbf{x}_0+\mathbf{h}) \approx f(\mathbf{x}_0) + \nabla f(\mathbf{x}_0)^\top \mathbf{h} + \tfrac12 \mathbf{h}^\top H_f(\mathbf{x}_0)\mathbf{h} \) | Multivariate second-order Taylor expansion. |
| **Ch.4 §4.1 Convexity** | \( f(\lambda\mathbf{x}+(1-\lambda)\mathbf{y}) \le \lambda f(\mathbf{x})+(1-\lambda)f(\mathbf{y}) \) | Convex functions lie below their chords. |
| Ch.4 §4.1 | \( f(\mathbf{y}) \ge f(\mathbf{x}) + \nabla f(\mathbf{x})^\top(\mathbf{y}-\mathbf{x}) \) | First-order characterization of convexity. |
| Ch.4 §4.1 | \( H_f \succeq 0\ \forall\mathbf{x} \iff f \text{ convex} \) (when \(C^2\)) | Second-order characterization of convexity. |
| **Ch.4 §4.2 Gradient descent** | \( \mathbf{x}_{t+1} = \mathbf{x}_t - \eta \nabla f(\mathbf{x}_t) \) | Vanilla gradient descent update. |
| Ch.4 §4.2 | \( f(\mathbf{x}_t) - f(\mathbf{x}^\star) \le \dfrac{\|\mathbf{x}_0 - \mathbf{x}^\star\|^2}{2\eta t} \) | \(O(1/t)\) convergence rate for convex \(L\)-smooth \(f\) with \(\eta \le 1/L\). |
| Ch.4 §4.2 | Linear rate: \( \left(\tfrac{L-\mu}{L+\mu}\right)^{2t} \) | Strongly convex GD converges geometrically with the conditioned rate. |
| Ch.4 §4.2 | \( \kappa = L/\mu \) | Condition number of a strongly convex objective. |
| **Ch.4 §4.3 SGD / mini-batch** | \( \mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla \ell_{i_t}(\mathbf{w}_t) \) | SGD updates from a single random example. |
| Ch.4 §4.3 | \( \mathbf{w}_{t+1} = \mathbf{w}_t - \eta\cdot\tfrac{1}{b}\sum_{i\in B_t}\nabla\ell_i(\mathbf{w}_t) \) | Mini-batch GD averages the gradient over a batch of size \(b\). |
| Ch.4 §4.3 | Gradient variance scales as \(\sigma^2/b\) | Larger mini-batches reduce gradient noise inversely with batch size. |
| Ch.4 §4.3 | SGD rate \(O(1/\sqrt{T})\) with \(\eta_t = \eta_0/\sqrt{t}\) | SGD's worst-case rate with a square-root learning-rate decay. |
| **Ch.4 §4.4 Momentum / RMSprop / Adam** | \( \mathbf{v}_{t+1} = \beta\mathbf{v}_t + \nabla f(\mathbf{w}_t),\ \mathbf{w}_{t+1} = \mathbf{w}_t - \eta\mathbf{v}_{t+1} \) | Polyak heavy-ball momentum. |
| Ch.4 §4.4 | Rate \( (1 - 1/\sqrt\kappa) \) | Momentum's accelerated rate vs. \( (1-1/\kappa) \) for plain GD. |
| Ch.4 §4.4 | \( \mathbf{s}_{t+1} = \rho\mathbf{s}_t + (1-\rho)g_t \odot g_t;\ \mathbf{w}_{t+1} = \mathbf{w}_t - \eta\,g_t/(\sqrt{\mathbf{s}_{t+1}}+\epsilon) \) | RMSprop scales each coordinate by recent gradient magnitude. |
| Ch.4 §4.4 | \( \mathbf{m}_{t+1}=\beta_1\mathbf{m}_t+(1-\beta_1)g_t;\ \mathbf{v}_{t+1}=\beta_2\mathbf{v}_t+(1-\beta_2)g_t^2 \) | Adam first and second moment estimates. |
| Ch.4 §4.4 | \( \hat\mathbf{m}_t = \mathbf{m}_t/(1-\beta_1^t);\ \hat\mathbf{v}_t = \mathbf{v}_t/(1-\beta_2^t) \) | Bias-corrected Adam moments. |
| Ch.4 §4.4 | \( \mathbf{w}_{t+1} = \mathbf{w}_t - \eta\,\hat\mathbf{m}_t/(\sqrt{\hat\mathbf{v}_t}+\epsilon) \) | Adam update step. |
| **Ch.4 §4.5 Lagrange multipliers** | \( \min f(\mathbf{x})\ \mathrm{s.t.}\ h_i(\mathbf{x})=0 \) | Equality-constrained optimization. |
| Ch.4 §4.5 | \( \mathcal{L}(\mathbf{x},\boldsymbol\lambda) = f(\mathbf{x}) + \sum_i \lambda_i h_i(\mathbf{x}) \) | Lagrangian function. |
| Ch.4 §4.5 | \( \nabla_{\mathbf{x}}\mathcal{L} = 0,\ h_i = 0 \) | Stationarity and primal feasibility define the Lagrangian critical point. |
| Ch.4 §4.5 | \( \partial v / \partial b_i = -\lambda_i^\star \) | Envelope theorem: each multiplier is the shadow price of its constraint. |
| **Ch.4 §4.6 KKT** | \( g_j \le 0,\ h_i = 0,\ \mu_j \ge 0 \) | Primal and dual feasibility for inequality-constrained problems. |
| Ch.4 §4.6 | \( \nabla f + \sum_j \mu_j \nabla g_j + \sum_i \lambda_i \nabla h_i = 0 \) | KKT stationarity. |
| Ch.4 §4.6 | \( \mu_j^\star g_j(\mathbf{x}^\star) = 0 \) | Complementary slackness: a constraint is either binding or has zero multiplier. |
| Ch.4 §4.6 | SVM primal: \( \min \tfrac12\|\mathbf{w}\|^2,\ y_k(\mathbf{w}^\top\mathbf{x}_k+b)\ge 1 \) | Hard-margin SVM as an inequality-constrained convex program. |
| Ch.4 §4.6 | SVM dual: \( \max_{\boldsymbol\alpha\ge 0}\sum_k\alpha_k - \tfrac12\sum_{k\ell}\alpha_k\alpha_\ell y_k y_\ell\mathbf{x}_k^\top\mathbf{x}_\ell,\ \sum_k\alpha_k y_k = 0 \) | KKT dual of the hard-margin SVM; only support vectors have \(\alpha_k^\star > 0\). |
| Ch.4 §4.6 | \( \mathbf{w}^\star = \sum_k \alpha_k^\star y_k \mathbf{x}_k \) | Recovery of the primal weight |

vector from dual variables. |
| **Ch.5 §5.1 Shannon entropy** | \( H(X) = -\sum_i p(x_i)\log_2 p(x_i) \) | Average bits of surprise per observation. |
| Ch.5 §5.1 | \( I(x) = -\log_2 p(x) \) | Self-information of a single outcome. |
| Ch.5 §5.1 | \( 0 \le H(X) \le \log_2 n \) | Entropy is bounded; maximum is hit by the uniform distribution. |
| **Ch.5 §5.2 Joint entropy** | \( H(X,Y) = -\sum_{x,y} p(x,y)\log_2 p(x,y) \) | Total surprise of a pair of variables. |
| Ch.5 §5.2 | \( H(X,Y) \le H(X) + H(Y) \) | Sub-additivity; equality only under independence. |
| **Ch.5 §5.3 Conditional entropy** | \( H(Y\mid X) = -\sum_{x,y} p(x,y)\log_2 p(y\mid x) \) | Average remaining uncertainty in \(Y\) once \(X\) is known. |
| Ch.5 §5.3 | \( H(X,Y) = H(X) + H(Y\mid X) \) | Chain rule of entropy. |
| Ch.5 §5.3 | \( 0 \le H(Y\mid X) \le H(Y) \) | Conditioning never increases uncertainty (on average). |
| **Ch.5 §5.4 Cross-entropy** | \( H(p,q) = -\sum_x p(x)\log q(x) \) | Expected code length under the wrong distribution \(q\) for the truth \(p\). |
| Ch.5 §5.4 | \( H(p,q) = H(p) + D_{KL}(p\,\|\,q) \) | Cross-entropy equals true entropy plus KL divergence. |
| Ch.5 §5.4 | \( \mathcal{L}_{BCE} = -[y\log\hat y + (1-y)\log(1-\hat y)] \) | Binary cross-entropy loss. |
| Ch.5 §5.4 | \( \mathcal{L}_{CCE} = -\sum_k y_k \log \hat y_k = -\log \hat y_{k^\star} \) | Categorical cross-entropy collapses to negative log of the predicted true-class probability. |
| Ch.5 §5.4 | \( \partial \mathcal{L}_{CCE}/\partial z_k = \hat y_k - y_k \) | Softmax+CCE gradient is the classic "predicted minus label" expression. |
| **Ch.5 §5.5 KL divergence** | \( D_{KL}(p\,\|\,q) = \sum_x p(x)\log_2(p(x)/q(x)) \) | Excess bits paid for coding \(p\) using \(q\). |
| Ch.5 §5.5 | \( D_{KL}(p\,\|\,q) \ge 0 \) (Gibbs) | KL is non-negative; zero iff \(p = q\). |
| Ch.5 §5.5 | \( D_{KL}(p\,\|\,q) \ne D_{KL}(q\,\|\,p) \) | KL is asymmetric and is not a true distance. |
| **Ch.5 §5.6 Mutual information** | \( I(X;Y) = H(X) + H(Y) - H(X,Y) = H(Y) - H(Y\mid X) \) | Bits about \(Y\) gained by observing \(X\). |
| Ch.5 §5.6 | \( I(X;Y) = D_{KL}(p(x,y)\,\|\,p(x)p(y)) \) | Mutual information measures the KL gap from independence. |
| Ch.5 §5.6 | \( I(X;Y) \ge 0;\ I(X;Y) = I(Y;X) \) | Mutual information is non-negative and symmetric. |
| **Ch.5 §5.7 Perplexity** | \( \mathrm{PP}(W) = \exp\!\left(-\tfrac{1}{N}\sum_i \log P(w_i\mid w_{<i})\right) \) | Geometric mean inverse-probability per token. |
| Ch.5 §5.7 | \( \mathrm{PP} = 2^H \) (in bits) or \( e^H \) (in nats) | Perplexity is the exponentiated cross-entropy of the model on the test stream. |
| **Ch.6 §6.1 Graphs** | \( G = (V, E) \) | Graph is a vertex set with an edge set. |
| Ch.6 §6.1 | \( \sum_{v\in V}\deg(v) = 2\|E\| \) | Handshake lemma. |
| Ch.6 §6.1 | Adjacency matrix \( A\in\{0,1\}^{n\times n} \); list \( O(V+E) \) space | Two standard representations with very different memory profiles. |
| **Ch.6 §6.2 Trees** | \( \|E\| = \|V\| - 1 \) | A tree on \(n\) vertices has exactly \(n-1\) edges. |
| Ch.6 §6.2 | Binary tree of depth \(d\): up to \(2^d\) leaves | Worst-case leaf count of a full binary tree. |
| Ch.6 §6.2 | Balanced binary tree: height \( = \Theta(\log n) \) | Balanced trees support logarithmic-time operations. |
| **Ch.6 §6.3 BFS / DFS complexity** | Time \( = O(V + E) \), Space \( = O(V) \) | Both traversals are linear in graph size. |
| **Ch.6 §6.4 DAGs** | DAG iff topological order \( \sigma: V \to \{1,\ldots,n\} \) exists with \( (u,v)\in E \Rightarrow \sigma(u)<\sigma(v) \) | Acyclicity is equivalent to the existence of a topological sort. |
| Ch.6 §6.4 | Kahn's algorithm runs in \( O(V+E) \) | Linear-time topological sort by repeatedly removing in-degree-zero vertices. |
| Ch.6 §6.4 | Critical path = longest weighted path in a DAG | Schedule bottleneck; computable in \(O(V+E)\). |
| **Ch.6 §6.5 Propositional logic** | \( \neg, \wedge, \vee, \to, \leftrightarrow \) | Negation, conjunction, disjunction, implication, biconditional. |
| Ch.6 §6.5 | \( p \to q \equiv \neg p \vee q \equiv \neg q \to \neg p \) | Implication as disjunction and as contrapositive. |
| Ch.6 §6.5 | De Morgan: \( \neg(p\wedge q) \equiv \neg p \vee \neg q \) | Negation distributes by swapping AND and OR. |
| Ch.6 §6.5 | Modus ponens: from \(p\) and \(p\to q\) infer \(q\) | Forward inference rule. |
| **Ch.6 §6.6 First-order logic** | \( \forall x.\varphi(x),\ \exists x.\varphi(x) \) | Universal and existential quantifiers. |
| Ch.6 §6.6 | Resolution: \(L\vee A,\ \neg L'\vee B,\ L\theta=L'\theta \vdash (A\vee B)\theta \) | Robinson's resolution rule with most general unifier. |
| Ch.6 §6.6 | Resolution is sound and refutation-complete | A clause set is unsatisfiable iff resolution derives the empty clause. |
| **Ch.6 §6.7 Rule engines** | Production rule: \( \mathrm{LHS}\Rightarrow\mathrm{RHS} \) | Pattern-conjunction triggers an action on working memory. |
| Ch.6 §6.7 | Rete matching: sub-linear in \(n\cdot m\) | Compiled match network amortizes shared sub-patterns across rules. |
| **Ch.7 §7.1 Stationarity** | Strict: joint dist invariant under time shift | All finite-dim distributions are translation-invariant. |
| Ch.7 §7.1 | Weak (covariance) stationarity: constant \(\mu\), constant \(\sigma^2\), \(\mathrm{Cov}(X_t,X_{t+h}) = \gamma(h)\) | Only first and second moments need to be time-invariant. |
| Ch.7 §7.1 | \( \rho(h) = \gamma(h)/\gamma(0) \) | Autocorrelation function. |
| Ch.7 §7.1 | ADF test: \( \Delta X_t = \alpha + \beta t + \gamma X_{t-1} + \sum \delta_i \Delta X_{t-i} + \varepsilon_t \) | Reject the unit-root null when \(\gamma < 0\) significantly. |
| Ch.7 §7.1 | First difference: \( \Delta X_t = X_t - X_{t-1} \) | Most common stationarity-inducing transform. |
| **Ch.7 §7.2 ARMA** | AR(p): \( X_t = c + \sum_{i=1}^p \phi_i X_{t-i} + \varepsilon_t \) | Autoregressive model: weighted sum of past values plus noise. |
| Ch.7 §7.2 | MA(q): \( X_t = \mu + \varepsilon_t + \sum_{j=1}^q \theta_j \varepsilon_{t-j} \) | Moving-average model: weighted sum of past shocks. |
| Ch.7 §7.2 | ARMA(p,q): \( X_t = c + \sum \phi_i X_{t-i} + \varepsilon_t + \sum \theta_j \varepsilon_{t-j} \) | Combined AR + MA structure. |
| Ch.7 §7.2 | AR(1) stationarity: \( \|\phi_1| < 1 \) | The unit-root threshold for a one-lag autoregressive process. |
| **Ch.7 §7.3 Returns** | Simple: \( r_t = (P_t - P_{t-1})/P_{t-1} \) | Period return as a fractional change. |
| Ch.7 §7.3 | Log: \( \ell_t = \ln(P_t/P_{t-1}) \) | Logarithmic return; additive over time. |
| Ch.7 §7.3 | \( 1 + r_{[t_0,t_n]} = \prod_i (1 + r_{t_i}) \) | Simple returns chain multiplicatively. |
| Ch.7 §7.3 | \( \ell_{[t_0,t_n]} = \sum_i \ell_{t_i} \) | Log returns chain additively. |
| Ch.7 §7.3 | \( \ell_t \approx r_t - r_t^2/2 \) (small \(r_t\)) | Second-order Taylor link between simple and log returns. |
| **Ch.7 §7.4 Volatility** | Realized: \( \hat\sigma = \sqrt{\tfrac{1}{n-1}\sum_t(\ell_t - \bar\ell)^2} \) | Sample standard deviation of log returns. |
| Ch.7 §7.4 | Annualization (i.i.d.): \( \hat\sigma_\text{annual} = \hat\sigma\sqrt{N} \) | Square-root-of-time scaling, \(N\) periods per year. |

| Ch.7 §7.4 | EWMA: \( \hat\sigma_t^2 = \lambda\hat\sigma_{t-1}^2 + (1-\lambda)\ell_{t-1}^2 \) | RiskMetrics-style exponentially weighted variance. |
| Ch.7 §7.4 | GARCH(1,1): \( \hat\sigma_t^2 = \omega + \alpha\ell_{t-1}^2 + \beta\hat\sigma_{t-1}^2 \) | Captures volatility clustering with three parameters; stationary if \(\alpha+\beta < 1\). |
| Ch.7 §7.4 | Long-run variance: \( \sigma_\infty^2 = \omega/(1-\alpha-\beta) \) | Unconditional variance of a stationary GARCH process. |
| **Ch.7 §7.5 Sharpe / IR** | Sharpe: \( \mathrm{SR} = (E[R_p - R_f])/\sigma_{R_p - R_f} \) | Excess return per unit of total risk. |
| Ch.7 §7.5 | Annualization: \( \mathrm{SR}_\text{annual} = \mathrm{SR}_\text{period}\sqrt{N} \) | Periodic Sharpe scales by square root of periods per year. |
| Ch.7 §7.5 | Information ratio: \( \mathrm{IR} = (E[R_p - R_b])/\sigma_{R_p - R_b} \) | Alpha per unit of tracking error. |
| Ch.7 §7.5 | Sortino: \( (E[R_p] - R_f)/\sigma_\text{down} \) | Sharpe variant penalizing only downside deviation. |
| **Ch.7 §7.6 Forecasting** | Additive decomposition: \( X_t = T_t + S_t + H_t + \varepsilon_t \) | Trend + seasonal + holiday + noise. |
| Ch.7 §7.6 | Multiplicative: \( X_t = T_t \cdot S_t \cdot H_t \cdot \varepsilon_t \) | Same components multiplied — appropriate when seasonality scales with level. |
| Ch.7 §7.6 | Holt–Winters level: \( L_t = \alpha(X_t - S_{t-s}) + (1-\alpha)(L_{t-1}+b_{t-1}) \) | Smoothed level update for triple-exponential smoothing. |
| Ch.7 §7.6 | Holt–Winters trend: \( b_t = \beta(L_t - L_{t-1}) + (1-\beta)b_{t-1} \) | Smoothed trend update. |
| Ch.7 §7.6 | Holt–Winters seasonal: \( S_t = \gamma(X_t - L_t) + (1-\gamma)S_{t-s} \) | Smoothed seasonal index update. |
| Ch.7 §7.6 | Forecast: \( \hat X_{t+h} = L_t + h\,b_t + S_{t+h-s\lceil h/s\rceil} \) | Combined Holt–Winters forecast at horizon \(h\). |
| Ch.7 §7.6 | SARIMA\((p,d,q)(P,D,Q)_s\) | Seasonal ARIMA: ARMA on regular and seasonal differences with period \(s\). |

# Appendix B: Complete Glossary

The glossary defines every mathematical term used anywhere in Volume 1 at textbook-level precision. Definitions are grouped by area for navigability; cross-references use chapter-section labels.

## B.1 Linear Algebra (Chapter 1)

Adjacency matrix. The $n\times n$ matrix $A$ with $A_{ij} = 1$ (or the edge weight) if $(i,j)$ is an edge and $0$ otherwise; used to represent graphs.

Adjugate (classical adjoint). Transpose of the cofactor matrix; appears in the closed-form expression $A^{-1} = \mathrm{adj}(A)/\det(A)$.

Basis. A linearly independent set of vectors whose span equals a given vector space. All bases of a finite-dimensional space have the same cardinality, called the dimension.

Characteristic polynomial. $p_A(\lambda) = \det(A - \lambda I)$; a degree-$n$ polynomial whose roots are the eigenvalues of $A$.

Cofactor. Signed minor of an entry: $C_{ij} = (-1)^{i+j} M_{ij}$, where $M_{ij}$ is the determinant of the submatrix obtained by deleting row $i$ and column $j$.

Column space. Span of the columns of a matrix; equivalently the image (range) of the linear map represented by the matrix.

Condition number. $\kappa(A) = \|A\|\|A^{-1}\|$ (or $\sigma_\max/\sigma_\min$ in the 2-norm). Quantifies sensitivity of solutions of $A\mathbf{x}=\mathbf{b}$ to perturbations in $\mathbf{b}$.

Cosine similarity. $\cos\theta = (\mathbf{u}\cdot\mathbf{v})/(\|\mathbf{u}\|\|\mathbf{v}\|)$; the angular alignment between two vectors, valued in $[-1,1]$.

Determinant. Signed volume scaling factor of the linear map associated with a square matrix; zero iff the matrix is singular.

Diagonal matrix. A matrix whose off-diagonal entries are all zero.

Diagonalization. Writing $A = P D P^{-1}$ with $D$ diagonal; possible iff $A$ has $n$ linearly independent eigenvectors.

Dimension. The cardinality of any basis of a vector space.

Dot product (inner product). $\mathbf{u}\cdot\mathbf{v} = \sum_i u_i v_i$; a bilinear symmetric positive-definite form on $\mathbb{R}^n$.

Eigenvalue, eigenvector. A scalar $\lambda$ and nonzero vector $\mathbf{v}$ with $A\mathbf{v} = \lambda\mathbf{v}$. The eigenvector is an invariant direction; the eigenvalue is its stretch factor.

Eigenbasis. A basis of $\mathbb{R}^n$ consisting of eigenvectors of $A$; exists when $A$ is diagonalizable.

Embedding. A learned or computed vector representation of an object in $\mathbb{R}^n$ (e.g., a 768-dimensional sentence embedding).

Explained variance ratio (EVR). For PCA component $k$, $\mathrm{EVR}_k = \lambda_k / \sum_j \lambda_j$.

Frobenius norm. $\|A\|_F = \sqrt{\sum_{ij} A_{ij}^2} = \sqrt{\mathrm{tr}(A^\top A)}$; the Euclidean norm on the vectorization of the matrix.

Identity matrix $I_n$. Square matrix with ones on the diagonal and zeros elsewhere; $IA = AI = A$ for compatible $A$.

Inner-product space. A vector space equipped with an inner product satisfying linearity, symmetry, and positive-definiteness.

Inverse matrix. The unique matrix $A^{-1}$ with $AA^{-1} = A^{-1}A = I$; exists iff $\det(A) \ne 0$.

Johnson–Lindenstrauss (JL) lemma. Any $N$ points in any-dimensional Euclidean space can be projected to $O(\log N/\epsilon^2)$ dimensions while preserving all pairwise distances within a multiplicative factor $1\pm\epsilon$.

Kernel (null space). $\{\mathbf{x} : A\mathbf{x} = \mathbf{0}\}$; the input directions a linear map collapses to zero.

Least squares. $\arg\min_{\mathbf{x}}\|A\mathbf{x}-\mathbf{b}\|^2$; the projection of $\mathbf{b}$ onto the column space of $A$.

Linear combination. $\sum_i c_i \mathbf{v}_i$ for scalars $c_i$.

Linear independence. A set $\{\mathbf{v}_i\}$ is linearly independent iff the only solution to $\sum c_i \mathbf{v}_i = \mathbf{0}$ is all zeros.

Linear map (linear transformation). A function $T:\mathbb{R}^n\to\mathbb{R}^m$ with $T(a\mathbf{x}+b\mathbf{y}) = aT(\mathbf{x})+bT(\mathbf{y})$; every linear map is represented by a matrix.

Matrix. A rectangular array $A \in \mathbb{R}^{m\times n}$ of real numbers.

Matrix-vector product. The vector $A\mathbf{x}$ with $i$-th entry $\sum_j A_{ij} x_j$.

Matrix multiplication. For compatible shapes, $(AB)_{ij} = \sum_k A_{ik}B_{kj}$.

Norm. A length function $\|\cdot\|$ satisfying non-negativity, homogeneity $\|\alpha\mathbf{v}\| = |\alpha|\|\mathbf{v}\|$, and the triangle inequality.

Normal equations. $A^\top A\mathbf{x} = A^\top\mathbf{b}$; first-order optimality conditions of least squares.

Orthogonal matrix. $Q$ with $Q^\top Q = QQ^\top = I$; preserves lengths and angles.

Orthogonal vectors. $\mathbf{u}\cdot\mathbf{v} = 0$.

Orthonormal basis. A basis of mutually orthogonal unit vectors.

Outer product. $\mathbf{u}\mathbf{v}^\top \in \mathbb{R}^{m\times n}$ for $\mathbf{u}\in\mathbb{R}^m,\mathbf{v}\in\mathbb{R}^n$; a rank-1 matrix.

Permutation. A bijection of $\{1,\ldots,n\}$ to itself; building block of the Leibniz determinant formula.

Positive definite (PD). A symmetric matrix $A$ with $\mathbf{x}^\top A\mathbf{x} > 0$ for all $\mathbf{x}\ne 0$; all eigenvalues strictly positive.

Positive semi-definite (PSD). A symmetric matrix with $\mathbf{x}^\top A\mathbf{x} \ge 0$ for all $\mathbf{x}$; all eigenvalues non-negative.

Principal Component Analysis (PCA). Orthogonal projection of centered data onto the top eigenvectors of the sample covariance matrix; preserves maximum variance per dimension.

Pseudoinverse (Moore–Penrose). The unique matrix $A^+$ satisfying four defining identities; equals $(A^\top A)^{-1}A^\top$ for full-column-rank $A$ and $V\Sigma^+ U^\top$ in general via SVD.

Rank. Dimension of the column (equivalently row) space; the number of nonzero singular values.

Rank-1 matrix. A matrix expressible as a single outer product $\mathbf{u}\mathbf{v}^\top$.

Scalar. A real (or complex) number; an element of the field.

Singular matrix. A square matrix with $\det = 0$; equivalently rank deficient, equivalently non-invertible.

Singular value. Non-negative square root of an eigenvalue of $A^\top A$; appears on the diagonal of $\Sigma$ in the SVD.

Singular Value Decomposition (SVD). Factorization $A = U\Sigma V^\top$ with $U,V$ orthogonal and $\Sigma$ diagonal with non-negative entries.

Span. The set of all linear combinations of a given set of vectors.

Spectral theorem. A real symmetric matrix admits an orthonormal eigenbasis: $A = Q\Lambda Q^\top$.

Subspace. A non-empty subset of a vector space closed under addition and scalar multiplication.

Symmetric matrix. $A = A^\top$.

Trace. Sum of diagonal entries of a square matrix; equals sum of eigenvalues.

Transpose. $(A^\top)_{ij} = A_{ji}$; reflects entries across the main diagonal.

Triangle inequality. $\|\mathbf{u}+\mathbf{v}\| \le \|\mathbf{u}\| + \|\mathbf{v}\|$.

Unit vector. Vector of norm 1.

Vector. An element of $\mathbb{R}^n$; equivalently a directed quantity with components.

Vector space. A set closed under addition and scalar multiplication satisfying the eight axioms (associativity, commutativity of addition, identity, inverses, distributivity, etc.).

## B.2 Probability and Statistics (Chapter 2)

Bayes' theorem. $P(A\mid B) = P(B\mid A)P(A)/P(B)$; inverts conditional probabilities.

Bernoulli distribution. A single 0/1 trial with success probability $p$; $P(X=1)=p$, $P(X=0)=1-p$.

Beta distribution. Continuous distribution on $[0,1]$ with density proportional to $p^{\alpha-1}(1-p)^{\beta-1}$; conjugate prior for the Bernoulli/binomial parameter.

Binomial distribution. Sum of $n$ i.i.d. Bernoulli($p$) trials; $P(X=k) = \binom{n}{k}p^k(1-p)^{n-k}$.

Bonferroni correction. Multiple-testing procedure that tests each of $m$ hypotheses at level $\alpha/m$ to bound family-wise error.

Bootstrap. Resampling with replacement to estimate a sampling distribution non-parametrically.

Central Limit Theorem (CLT). Standardized sample mean of i.i.d. variables with finite variance converges in distribution to $\mathcal{N}(0,1)$.

Conditional probability. $P(A\mid B) = P(A\cap B)/P(B)$ for $P(B)>0$.

Confidence interval. A random interval $[\hat L,\hat U]$ that covers the true parameter with frequency $1-\alpha$ under repeated sampling.

Conjugate prior. A prior whose family is closed under Bayesian updating with a given likelihood; the resulting posterior is in the same family.

Continuity correction. Adjustment of $\pm 0.5$ when approximating a discrete distribution (e.g., binomial) by a continuous one (e.g., normal).

Correlation (Pearson). $\rho_{XY} = \mathrm{Cov}(X,Y)/(\sigma_X\sigma_Y)$; a unit-free, scale-invariant linear association measure.

Covariance. $\mathrm{Cov}(X,Y) = E[(X-\mu_X)(Y-\mu_Y)]$.

CUPED (Controlled-experiment Using Pre-Experiment Data). Variance-reduction technique that adjusts the response by a regression on a pre-period covariate.

Cumulative distribution function (CDF). $F_X(x) = P(X\le x)$.

Effect size. Standardized magnitude of a difference, e.g. Cohen's $d$ or $h$; used in power calculations.

Expectation. Probability-weighted average; $E[X] = \sum_x xp(x)$ or $\int xf(x)dx$.

False discovery rate (FDR). Expected fraction of false rejections among rejections; controlled by the Benjamini–Hochberg procedure.

Family-wise error rate (FWER). Probability of any false rejection across a family of tests.

Gibbs' inequality. $\sum p\log(p/q) \ge 0$; proves the non-negativity of KL divergence.

Hypothesis test. A decision rule that rejects a null hypothesis $H_0$ when a test statistic exceeds a critical value.

Independence. $P(A\cap B) = P(A)P(B)$; equivalently $P(A\mid B) = P(A)$.

Inclusion-exclusion. $P(A\cup B) = P(A) + P(B) - P(A\cap B)$ and its multi-set generalization.

Joint probability. $P(A\cap B)$ or $p(x,y)$; the probability that two events both occur.

Kolmogorov axioms. Non-negativity, normalization, and countable additivity for disjoint events.

Law of large numbers. Sample averages of i.i.d. variables converge to the population mean (weakly or strongly).

Law of total probability. $P(A) = \sum_i P(A\mid B_i)P(B_i)$ over a partition $\{B_i\}$.

Likelihood function. $L(\theta) = \prod_i p(x_i\mid\theta)$ viewed as a function of $\theta$ with the data fixed.

Log-likelihood. $\ell(\theta) = \log L(\theta)$; numerically stable and additive.

Log-odds (logit). $\log[p/(1-p)]$; the link function of logistic regression.

Marginal probability. $P(A) = \sum_b P(A,B=b)$; obtained by summing out other variables.

Maximum a posteriori (MAP). $\hat\theta_{MAP} = \arg\max_\theta [\log L(\theta) + \log p(\theta)]$; MLE with a log-prior penalty.

Maximum likelihood estimation (MLE). $\hat\theta_{MLE} = \arg\max_\theta \ell(\theta)$.

Minimum detectable effect (MDE). Smallest effect size a test can detect with given $\alpha,\beta,n$.

Multinomial distribution. Generalization of the binomial to $k$ categories with parameters $(n,p_1,\ldots,p_k)$.

Normal (Gaussian) distribution. Continuous distribution with density $(2\pi\sigma^2)^{-1/2}\exp(-(x-\mu)^2/(2\sigma^2))$.

Null hypothesis ($H_0$). The default claim to be tested; commonly "no effect."

Odds. $p/(1-p)$ for an event with probability $p$.

Pareto / power-law distribution. Density $\alpha x_{\min}^\alpha x^{-(\alpha+1)}$ on $[x_{\min},\infty)$; heavy-tailed.

Poisson distribution. Discrete distribution for counts with mean $\lambda$; $P(X=k) = \lambda^k e^{-\lambda}/k!$.

Posterior distribution. $p(\theta\mid\mathcal{D}) \propto L(\theta) p(\theta)$; updated beliefs after observing data.

Power (of a test). $1 - \beta$, the probability of correctly rejecting a false null.

Prior distribution. $p(\theta)$; beliefs about parameters before observing data.

Probability density function (PDF). Continuous-variable density $f_X(x)$; integrates to 1 over the support.

Probability mass function (PMF). Discrete-variable mass $p(x) = P(X=x)$; sums to 1.

Probability space. Triple $(\Omega,\mathcal{F},P)$ of sample space, sigma-algebra of events, and probability measure.

P-value. Probability under the null of observing a test statistic at least as extreme as the one observed.

Random variable. Measurable function $X:\Omega \to \mathbb{R}$.

Ridge regression. $\arg\min_\beta \|\mathbf{y} - X\beta\|^2 + \lambda\|\beta\|^2$; MAP under Gaussian prior on $\beta$.

Sample space $\Omega$. Set of all possible outcomes of a random experiment.

Sample variance. $s^2 = \tfrac{1}{n-1}\sum_i(x_i - \bar x)^2$ (unbiased version).

Sigma-algebra. Collection of subsets of $\Omega$ closed under complement and countable union; the domain of a probability measure.

Significance level $\alpha$. Maximum allowed probability of a Type I error.

Standard error. Standard deviation of an estimator's sampling distribution.

Type I error. Rejecting a true null hypothesis.

Type II error. Failing to reject a false null hypothesis.

Variance. $\mathrm{Var}(X) = E[(X-\mu)^2] = E[X^2] - (E[X])^2$.

Z-score. $(X - \mu)/\sigma$; standardization to mean 0 and standard deviation 1.

## B.3 Calculus and Matrix Calculus (Chapter 3)

Backpropagation. Layer-wise application of the chain rule to compute gradients of a neural-network loss with respect to all parameters in $O(\text{forward cost})$.

Chain rule. Derivative of a composition equals product of derivatives (scalar) or product of Jacobians (vector).

Continuity. A function is continuous at $a$ if $\lim_{x\to a}f(x) = f(a)$.

Critical point. A point where the gradient is zero; may be a minimum, maximum, or saddle.

Derivative. Instantaneous rate of change $f'(x) = \lim_{h\to 0}[f(x+h)-f(x)]/h$.

Directional derivative. $D_{\mathbf{u}}f(\mathbf{x}) = \nabla f(\mathbf{x})^\top\mathbf{u}$; rate of change in direction $\mathbf{u}$.

Gradient. Column vector $\nabla f$ of all first-order partial derivatives; points in the direction of steepest ascent.

Hessian. Matrix of second partials $H_f(\mathbf{x})_{ij} = \partial^2 f/(\partial x_i\partial x_j)$; symmetric under $C^2$.

Implicit function theorem. Conditions under which an equation $F(\mathbf{x},\mathbf{y}) = 0$ implicitly defines $\mathbf{y}$ as a function of $\mathbf{x}$; the implicit Jacobian comes from a linear system.

Jacobian. Matrix of first partials $J_{ij} = \partial f_i/\partial x_j$ for a vector-valued function; reduces to the gradient transpose when $m=1$.

Limit. $\lim_{x\to a}f(x) = L$ iff for every $\epsilon > 0$ there is $\delta > 0$ with $0 < |x-a| < \delta \Rightarrow |f(x) - L| < \epsilon$.

Lipschitz constant. Smallest $L$ with $\|\nabla f(\mathbf{x}) - \nabla f(\mathbf{y})\| \le L\|\mathbf{x}-\mathbf{y}\|$; bounds curvature.

Newton's method. $\mathbf{x}_{t+1} = \mathbf{x}_t - H_f^{-1}\nabla f$; quadratically convergent near a strict minimum.

Partial derivative. Derivative with respect to one variable, others held fixed.

Power rule. $(x^n)' = nx^{n-1}$.

Product rule. $(uv)' = u'v + uv'$.

Quotient rule. $(u/v)' = (u'v - uv')/v^2$.

Saddle point. Critical point where the Hessian has eigenvalues of mixed sign.

Second-derivative test. Classify critical points by Hessian definiteness (PD = min, ND = max, indefinite = saddle).

Steepest descent. Direction $-\nabla f / \|\nabla f\|$; the unique unit direction maximizing the local rate of decrease.

Taylor series. Polynomial expansion of a function about a point using its derivatives at that point.

Taylor remainder. Error term in Taylor's theorem; expressible in Lagrange or integral form.

## B.4 Optimization Theory (Chapter 4)

Adam. Adaptive optimizer combining momentum (first-moment estimate) and RMSprop-style second-moment scaling with bias correction.

AdamW. Adam variant that decouples weight decay from the moment updates; preferred for transformer training.

Active set. Set of indices of inequality constraints $g_j$ that hold with equality at the optimum.

Backtracking line search. Procedure that shrinks the step size $\eta$ until the Armijo decrease condition is satisfied.

Batch. Subset of training examples whose average gradient is used for one update.

Convex function. Function satisfying $f(\lambda\mathbf{x}+(1-\lambda)\mathbf{y}) \le \lambda f(\mathbf{x})+(1-\lambda)f(\mathbf{y})$ for all $\lambda\in[0,1]$.

Convex set. Set closed under convex combinations of its points.

Complementary slackness. KKT condition $\mu_j^\star g_j(\mathbf{x}^\star) = 0$; constraints with positive multipliers are active.

Dual problem. Maximization over Lagrange multipliers of the inf-Lagrangian; provides a lower bound on the primal.

Duality gap. Difference between primal and dual optimal values; zero under strong duality (e.g., Slater's condition).

Epoch. One full pass through a training set.

Gradient descent. Iteration $\mathbf{x}_{t+1} = \mathbf{x}_t - \eta\nabla f(\mathbf{x}_t)$.

Karush–Kuhn–Tucker (KKT) conditions. Stationarity, primal feasibility, dual feasibility, and complementary slackness; necessary at any local optimum under a constraint qualification, and sufficient under convexity.

L-BFGS. Limited-memory quasi-Newton method approximating the inverse Hessian from gradient differences.

Lagrangian. $\mathcal{L} = f + \sum_i\lambda_i h_i + \sum_j\mu_j g_j$; auxiliary function whose stationary points yield candidate optima.

Lagrange multiplier. Variable $\lambda_i$ (equality) or $\mu_j\ge 0$ (inequality) measuring the shadow price of its constraint.

Learning rate. Step-size hyperparameter $\eta$ in gradient-based optimization.

Levenberg–Marquardt. Damped Newton method using $H + \lambda I$; robust to near-singular Hessians.

Linear programming. Convex problem with linear objective and linear constraints.

Local minimum. Point with $f(\mathbf{x}^\star) \le f(\mathbf{x})$ for all $\mathbf{x}$ in some neighborhood.

Mini-batch gradient descent. Update using the average gradient over a randomly sampled subset of size $b$.

Momentum (Polyak heavy-ball). Accumulates a velocity over past gradients, smoothing oscillations.

Nesterov accelerated gradient. Momentum variant evaluating the gradient at a look-ahead point; attains optimal first-order rate.

Quadratic convergence. Convergence with error squaring per step, characteristic of Newton's method near a smooth minimum.

Quadratic programming. Convex problem with quadratic objective and linear constraints; includes SVMs.

RMSprop. Per-coordinate adaptive optimizer scaling each gradient by the root-mean-squared running estimate of its magnitude.

Saddle escape. Use of stochastic noise (SGD, perturbed GD) to depart from saddle points along negative-curvature directions.

Slater's condition. Strict feasibility of inequality constraints in a convex problem; implies strong duality.

Stochastic gradient descent (SGD). Update from one randomly sampled example per step.

Strongly convex. Convex with a quadratic lower bound: $f(\mathbf{y}) \ge f(\mathbf{x}) + \nabla f(\mathbf{x})^\top(\mathbf{y}-\mathbf{x}) + \tfrac{\mu}{2}\|\mathbf{y}-\mathbf{x}\|^2$.

Support vector. Training point with nonzero dual variable in an SVM; lies exactly on the margin boundary.

Support Vector Machine (SVM). Maximum-margin linear classifier; primal minimizes $\tfrac12\|\mathbf{w}\|^2$ subject to margin constraints.

Trust region. Local region within which a surrogate model is trusted; the optimizer steps within this region.

Weight decay. Regularizer adding $\tfrac{\lambda}{2}\|\mathbf{w}\|^2$ to the objective; equivalent to $L_2$ regularization.

## B.5 Information Theory (Chapter 5)

Bit. Information unit when logarithms are taken in base 2.

Cross-entropy. $H(p,q) = -\sum_x p(x)\log q(x)$; expected code length under model $q$ when reality is $p$.

Categorical cross-entropy. Multi-class loss $-\sum_k y_k\log\hat y_k$ used with softmax outputs.

Binary cross-entropy. Two-class loss $-[y\log\hat y + (1-y)\log(1-\hat y)]$.

Entropy (Shannon). $H(X) = -\sum_x p(x)\log p(x)$; average self-information of a discrete source.

Hartley. Information unit when logarithms are taken in base 10.

Information gain. Reduction in entropy $H(Y) - H(Y\mid X)$; equals mutual information.

Jensen–Shannon divergence. Symmetric, bounded variant of KL: $\tfrac12 D_{KL}(p\,\|\,m) + \tfrac12 D_{KL}(q\,\|\,m)$ with $m = (p+q)/2$.

Jensen's inequality. For a convex function $\phi$ and random variable $X$, $E[\phi(X)] \ge \phi(E[X])$.

Joint entropy. $H(X,Y) = -\sum_{x,y} p(x,y)\log p(x,y)$; total surprise of a pair.

Kullback–Leibler (KL) divergence. $D_{KL}(p\,\|\,q) = \sum p\log(p/q)$; asymmetric measure of excess surprise from using $q$ in place of $p$.

Mutual information. $I(X;Y) = H(X) + H(Y) - H(X,Y)$; non-negative symmetric dependence measure.

Nat. Information unit when logarithms are taken in base $e$.

Perplexity. $\mathrm{PP} = 2^{H} = e^{H_{\text{nats}}}$; effective branching factor of a probabilistic model.

Self-information. $I(x) = -\log p(x)$; the surprise of a single outcome.

Sub-additivity. $H(X,Y) \le H(X) + H(Y)$.

## B.6 Discrete Mathematics (Chapter 6)

Adjacency list. Graph representation storing for each vertex the list of its neighbors; $O(V+E)$ memory.

Backward chaining. Goal-driven inference: starting from the goal, find rules whose consequents match and recurse on antecedents.

BFS (breadth-first search). Linear-time traversal exploring vertices in order of distance from the source.

Clause. Disjunction of literals; CNF is a conjunction of clauses.

CNF (conjunctive normal form). $\bigwedge_i \bigvee_j \ell_{ij}$; the input format expected by SAT solvers.

Connected component. Maximal subset of vertices any two of which are joined by a path.

Critical path. Longest weighted path in a DAG; bottleneck of an activity schedule.

Cycle. Closed path with no repeated vertices except the start = end.

DAG (directed acyclic graph). Directed graph with no directed cycle.

Degree. Number of edges incident to a vertex (in/out for directed graphs).

DFS (depth-first search). Linear-time traversal exploring as deep as possible along each branch before backtracking.

Diameter. Maximum over all pairs of vertices of the shortest-path length between them.

DNF (disjunctive normal form). $\bigvee_i \bigwedge_j \ell_{ij}$; native form of rule engines (each disjunct is a rule).

Edge. Pair of vertices connected in a graph; ordered for digraphs, unordered for undirected graphs.

First-order logic (FOL). Predicate calculus extending propositional logic with constants, variables, predicates, functions, and quantifiers.

Forward chaining. Data-driven inference: repeatedly fire rules whose antecedents match the working memory.

Handshake lemma. $\sum_v \deg(v) = 2|E|$ in an undirected graph.

Horn clause. Clause with at most one positive literal; foundation of Prolog and efficient forward chaining.

Implication. $p \to q \equiv \neg p \vee q$; also equivalent to its contrapositive $\neg q \to \neg p$.

In-degree, out-degree. Number of incoming and outgoing edges at a vertex in a directed graph.

Kahn's algorithm. $O(V+E)$ topological sort by repeatedly removing in-degree-zero vertices.

Leaf. Tree vertex with no children.

Literal. Propositional variable or its negation.

Modus ponens. From $p$ and $p\to q$, conclude $q$.

Modus tollens. From $\neg q$ and $p\to q$, conclude $\neg p$.

Path. Sequence of vertices each adjacent to the next.

Predicate. Boolean-valued function on tuples; symbolizes a relation in FOL.

Production rule. $\mathrm{LHS}\Rightarrow\mathrm{RHS}$: conjunctive pattern triggers an action on working memory.

Propositional logic. Boolean calculus over atomic propositions and connectives $\neg,\wedge,\vee,\to,\leftrightarrow$.

Quantifier. Universal $\forall$ or existential $\exists$ binding a variable in FOL.

Refutation completeness. Property of resolution: any unsatisfiable clause set yields the empty clause after finitely many resolutions.

Resolution. Inference rule $L\vee A,\ \neg L'\vee B \vdash (A\vee B)\theta$ using a most general unifier $\theta$ with $L\theta = L'\theta$.

Rete algorithm. Network data structure that compiles many rules and many facts into a shared match graph for efficient firing.

Root. Distinguished vertex of a rooted tree, the unique vertex with no parent.

Skolemization. Removal of existential quantifiers by introducing fresh function symbols, preserving satisfiability.

Tautology. Formula true under every truth assignment.

Topological sort. Linear ordering of a DAG consistent with edge direction.

Tree. Connected acyclic undirected graph; equivalently, $|E| = |V|-1$ and connected.

Truth assignment. Mapping from propositional variables to $\{T,F\}$.

Unifier (most general). Substitution $\theta$ making two terms syntactically equal; "most general" if any other unifier is an instance of it.

Vertex. Node of a graph or tree.

Walk. Sequence of edges traversed; relaxes path to allow repeated vertices.

Weighted graph. Graph with a real-valued weight function on edges.

Working memory. Set of currently asserted facts in a rule engine.

## B.7 Time Series and ML4T (Chapter 7)

ADF (Augmented Dickey–Fuller) test. Regression-based test of the unit-root null in a possibly trending autoregressive series.

Annualization. Scaling a periodic volatility or Sharpe ratio by $\sqrt{N}$ where $N$ = periods per year, assuming i.i.d. returns.

ARMA(p,q). $X_t = c + \sum\phi_i X_{t-i} + \varepsilon_t + \sum\theta_j\varepsilon_{t-j}$ with white-noise $\varepsilon_t$.

ARIMA(p,d,q). ARMA on the $d$-th difference; handles $I(d)$ non-stationarity.

Autocorrelation function (ACF). $\rho(h) = \gamma(h)/\gamma(0)$; lag-$h$ standardized linear memory.

Autocovariance function. $\gamma(h) = \mathrm{Cov}(X_t, X_{t+h})$.

Drift. Deterministic trend term in a time series model.

EWMA (exponentially weighted moving average). $\hat\sigma_t^2 = \lambda\hat\sigma_{t-1}^2 + (1-\lambda)\ell_{t-1}^2$; RiskMetrics-style variance update.

GARCH(1,1). $\hat\sigma_t^2 = \omega + \alpha\ell_{t-1}^2 + \beta\hat\sigma_{t-1}^2$; captures volatility clustering.

Holt–Winters method. Triple exponential smoothing for level, trend, and seasonal components.

Information ratio. Active return divided by tracking error; $(E[R_p-R_b])/\sigma_{R_p-R_b}$.

Integration order $I(d)$. Series becomes stationary after $d$ differences.

Log return. $\ell_t = \ln(P_t/P_{t-1})$; additive over time and approximately normal for small moves.

Newey–West estimator. Heteroskedasticity- and autocorrelation-consistent (HAC) standard error.

Realized volatility. Sample standard deviation of returns over a window.

SARIMA$((p,d,q)(P,D,Q)_s$. Seasonal ARIMA capturing both regular and seasonal differencing of period $s$.

Sharpe ratio. Excess return per unit of total volatility; $(E[R_p-R_f])/\sigma_{R_p-R_f}$.

Simple return. $r_t = (P_t - P_{t-1})/P_{t-1}$; multiplicative over time.

Sortino ratio. Sharpe variant using downside deviation in the denominator.

Stationarity (strict). Joint distribution of $(X_{t_1},\ldots,X_{t_k})$ is time-translation invariant for all $k$.

Stationarity (weak / covariance). Constant mean, constant finite variance, and autocovariance depending only on the lag.

Time series. Sequence $\{X_t\}$ indexed by integer time.

Tracking error. Standard deviation of active return $R_p - R_b$.

Trend, seasonality, holiday effect. Components of a classical time-series decomposition.

Unit root. Pole of the AR characteristic polynomial at $z=1$; makes the series non-stationary.

Volatility clustering. Empirical phenomenon that large absolute returns follow large absolute returns; captured by GARCH/EWMA.

White noise. Sequence $\{\varepsilon_t\}$ with mean zero, constant variance, and zero autocorrelation at all nonzero lags.

## B.8 Cross-Cutting Symbols and Conventions

$\mathbb{R}, \mathbb{R}^n$. Real numbers; $n$-tuples of real numbers.

$\mathbb{Z}, \mathbb{N}$. Integers; natural numbers.

$\mathbb{E}$ or $E[\cdot]$. Expectation under the ambient probability measure.

$\Pr(\cdot)$ or $P(\cdot)$. Probability measure.

$\arg\max,\ \arg\min$. Argument that attains the maximum or minimum.

$\propto$. Proportional to.

$\succeq, \succ$. Loewner ordering on symmetric matrices: positive semi-definite, positive definite.

$\xrightarrow{d}$. Convergence in distribution.

$\xrightarrow{p}$. Convergence in probability.

$\odot$. Hadamard (elementwise) product.

$\otimes$. Kronecker product (used in tensor calculus references).

$\Phi(\cdot)$. Standard normal CDF.

$\Gamma(\cdot)$. Gamma function; extends factorial to real arguments via $\Gamma(n+1) = n!$.

$\delta_{ij}$. Kronecker delta: 1 if $i=j$, 0 otherwise.

$O(\cdot),\ \Theta(\cdot),\ \Omega(\cdot)$. Big-O, tight bound, and lower-bound asymptotic notations.

# Appendix C: Mathematics to Georgia Tech Course Map

The matrix below maps every major concept from Volume 1 to the six Georgia Tech graduate AI/ML/quant courses most relevant to Paytm-style fintech PM work: CS 6601 — Artificial Intelligence, CS 7641 — Machine Learning, CS 7643 — Deep Learning, CS 7646 — Machine Learning for Trading (ML4T), CS 7632 — Game AI, and CS 8803 SAL — Special Topics: Large Language Models.

Entries use a generic topic label when the exact module name has changed across semesters (Georgia Tech revises module titles regularly); the topic label nonetheless pinpoints where the concept lives in the course's standard curriculum. Empty rows are deliberately avoided — only courses where the concept is materially covered are listed for each row.

| Volume 1 concept | CS 6601 (AI) | CS 7641 (ML) | CS 7643 (Deep Learning) | CS 7646 (ML4T) | CS 7632 (Game AI) | CS 8803 SAL (LLMs) |
|---|---|---|---|---|---|---|
| Vectors, norms, dot product (§1.1) | Search-state representations | Feature vectors and similarity | Tensor primitives | Asset return vectors | NPC state vectors | Token embeddings |
| Matrices and matrix-vector products (§1.2) | State-space transition matrices | Design matrices in regression | Linear layers | Factor exposure matrices | Transformation matrices for sprites | Attention projection matrices $W_Q, W_K, W_V$ |
| Matrix multiplication mechanics (§1.3) | Probabilistic transition composition | Kernel and Gram matrix formation | Forward pass of MLPs | Portfolio covariance products | Skeletal animation composition | Multi-head attention composition |
| Linear independence and bases (§1.4) | Heuristic feature redundancy | Multicollinearity diagnosis | Rank of weight matrices | Factor independence in models | Configuration-space spanning sets | Subspace analysis of token representations |
| Determinant (§1.5) | Markov-process invariants | Change-of-variables in generative models | Normalizing-flow Jacobian determinants | Risk-neutral measure changes | Triangle-area tests in collision detection | Latent volume in flow-based generators |
| Trace and Frobenius norm (§1.6) | — | Regularization (Frobenius penalty) | Weight-decay analysis | Tr-based risk measures | — | Parameter-norm analysis during pretraining |
| Inverse matrices and linear systems (§1.7) | Constraint solving in CSPs | Closed-form linear regression | Solving normal equations in research code | OLS factor regressions | Inverse kinematics solvers | Linear probe head training |
| Eigenvalues and eigenvectors (§1.8) | PageRank-style ranking | Spectral clustering, PCA | Power iteration in spectral norm computation | Eigenvalue decomposition of covariance | Resonance modes in physics simulation | Spectral analysis of attention matrices |
| Principal Component Analysis (§1.9) | Belief-state compression | Unsupervised dimensionality reduction | Latent representation interpretation | PCA for return factors | Motion-capture dimensionality reduction | Embedding visualization (PCA/UMAP) |
| Singular Value Decomposition (§1.10) | Approximate matrix factorization in planning | Matrix factorization and recommender systems | Low-rank weight adapters (LoRA foundations) | Risk-factor SVD on return matrices | Pose decomposition | LoRA, low-rank attention, compression |
| High-dimensional vector spaces (§1.11) | Curse-of-dimensionality in search heuristics | Curse-of-dimensionality in kNN | Geometry of high-dim representations | Curse-of-dimensionality in alpha factors | Pathfinding in high-dim configuration space | Embedding-space geometry analysis |
| Cosine similarity (§1.12) | Heuristic similarity measures | Document and item similarity | Contrastive loss similarity term | Cross-sectional return correlation | NPC behavior similarity | Retrieval (RAG) similarity search |
| Embeddings in production (§1.13) | Knowledge-base retrieval | Embedding methods (Word2Vec etc.) | Self-supervised representation learning | News-embedding alpha signals | Behavioral embedding clustering | Vector stores, retrieval-augmented generation |
| Probability spaces and axioms (§2.1) | Probabilistic reasoning foundations | Probabilistic ML foundations | Probabilistic interpretation of networks | Probabilistic trading models | Probabilistic NPC decisions | Probabilistic language modeling foundations |
| Joint, marginal, conditional probability (§2.2) | Bayesian networks | Generative model foundations | Variational inference setup | Conditional return distributions | Probabilistic FSMs | Conditional generation foundations |
| Bayes' theorem (§2.3) | Bayesian networks, inference | Naive Bayes, Bayesian methods | Bayesian deep learning | Bayesian regime detection | Bayesian player modeling | Bayesian prompting and posterior inference |
| Random variables, PDF/PMF/CDF (§2.4) | Probabilistic reasoning | Probability for ML | Distributional output layers | Asset-return distributions | Random behavior modeling | Token-level distributions |
| Expectation, variance, covariance, correlation (§2.5) | Decision theory foundations | Bias-variance decomposition | Statistics of gradient estimates | Portfolio variance and covariance matrices | Behavioral variance modeling | Variance of LLM outputs and sampling |
| Key distributions: Normal, Binomial, Poisson, Power law, Multinomial (§2.6) | Probabilistic models | Distributional assumptions | Output distributions (Gaussian, Categorical, Mixture) | Return distributions, heavy tails | Event distributions in games | Categorical token sampling, softmax temperature |
| Central Limit Theorem (§2.7) | Sampling-based AI | Sample-mean asymptotics | Batch-statistics analysis | Bootstrap confidence intervals | Monte Carlo simulation | Confidence intervals on eval metrics |
| Maximum likelihood estimation (§2.8) | HMM parameter learning | Parametric model fitting | Cross-entropy loss as MLE | Return-distribution fitting | Behavior-model fitting | Language-model training as MLE |
| Maximum a posteriori (§2.9) | Bayesian inference, MRFs | Regularization as MAP | Bayesian neural networks, weight decay | Bayesian regression on factors | Bayesian opponent modeling | RLHF prior-regularized objectives |
| Hypothesis testing, p-values, power, CIs (§2.10) | Statistical evaluation | A/B testing of classifiers | Significance of model improvements | Strategy backtest significance | Playtesting statistical tests | LLM-eval significance, win-rate tests |
| Multivariable A/B testing, CUPED, FDR (§2.11) | — | Multi-metric evaluation | Hyperparameter testing | Multi-strategy comparison, multiple-testing | Multi-mechanic playtests | Multi-prompt and multi-eval comparison |
| Limits and continuity (§3.1) | Continuous-state planning | Loss-surface continuity | Continuous activations | Continuous return processes | Continuous interpolation in animation | Smoothness assumptions in scaling laws |
| Derivatives, product, quotient, chain (§3.2) | Continuous optimization | Gradient-based ML | Backpropagation primitives | Sensitivity (Greeks-style) | Steering-behavior derivatives | Training dynamics derivatives |

| Partial derivatives and the gradient (§3.3) | Continuous-action planning | Gradient methods | Tensor gradients | Multi-factor sensitivities | Multi-input control gradients | Loss-landscape gradient analysis |
| Directional derivatives (§3.4) | Steepest-ascent search | Coordinate descent | Adversarial directions, projected attacks | Directional factor exposure | Steering toward objectives | Token-level perturbation analysis |
| Chain rule, multivariate and tensor (§3.5) | Bayesian-net gradient computation | Backprop in shallow nets | Automatic differentiation | Gradient-based portfolio optimization | Composite-controller gradients | Backprop through transformer stacks |
| Jacobian matrix (§3.6) | Inverse kinematics in planning | Jacobian-based sensitivity | Vector-Jacobian products in autograd | Jacobian of multi-factor pricing | IK Jacobians in animation | VJP/JVP in efficient backprop |
| Hessian and second-order info (§3.7) | Newton-style policy improvement | Newton methods, natural gradient | Curvature analysis, K-FAC | Hessian-based portfolio optimization | Stability of physics simulators | Second-order optimization in LLMs (Shampoo, K-FAC) |
| Taylor series and local models (§3.8) | Trust-region planning | Quadratic approximations | Trust-region policy optimization | Greeks via Taylor expansion | Local linearization in physics | Local linearization in interpretability |
| Convexity, local vs global, saddles (§4.1) | Convex objective subroutines | Convex loss landscapes (logistic, SVM) | Non-convex loss-surface analysis | Convex portfolio optimization | Convex-hull collisions | Loss-landscape geometry in pretraining |
| Gradient descent (§4.2) | Continuous optimization in AI | Core ML optimizer | Vanilla GD baseline | Gradient methods for portfolio | Gradient-based animation tuning | Pretraining GD foundations |
| Stochastic and mini-batch GD (§4.3) | Sampling-based search | Core SGD theory | Mini-batch training, large-batch tricks | Stochastic backtest training | Online learning in NPCs | Pretraining at scale, gradient accumulation |
| Momentum, RMSprop, Adam, AdamW (§4.4) | Momentum-based search | Adaptive optimizers | Adam, AdamW, LAMB for transformers | Adam-based strategy training | Optimizers for online RL | AdamW for transformer pretraining |
| Lagrange multipliers (§4.5) | Constrained planning | Constrained optimization | Lagrangian relaxations | Constrained portfolio optimization | Constrained motion planning | Constrained RLHF objectives |
| KKT conditions and SVM (§4.6) | Constraint reasoning | Hard- and soft-margin SVM derivations | Lagrangian-form regularization | Constrained mean-variance optimization | Constraint-based motion control | KKT-style constraints in RLHF |
| Shannon entropy (§5.1) | Information-gain heuristics | Decision-tree entropy split | Entropy of softmax outputs | Entropy of return distributions | Entropy in procedural content | Entropy of next-token distributions |
| Joint entropy (§5.2) | Joint-feature analysis | Joint entropy in MI feature selection | Joint-distribution modeling | Joint asset entropy | Joint behavior entropy | Joint entropy of multi-token outputs |
| Conditional entropy (§5.3) | Conditional info heuristics | Decision-tree conditional entropy | Conditional-distribution outputs | Conditional return entropy | Conditional behavior entropy | Conditional generation entropy |
| Cross-entropy loss (§5.4) | Probabilistic-classifier training | Logistic regression and softmax loss | Standard loss for classifiers and LMs | Classifier-based signal training | Behavior-classifier training | LM training loss |
| KL divergence (§5.5) | Bayesian-net comparison | Distribution comparison, drift | KL in VAE objectives, regularization | Distribution shift in markets | Distribution drift in player models | KL constraints in RLHF, DPO |
| Mutual information (§5.6) | Sensor-fusion info | Feature importance via MI | InfoMax representation learning | MI-based factor selection | MI in player-NPC interaction | MI between layers, mechanistic interpretability |
| Perplexity (§5.7) | — | Language-model evaluation | LM cross-entropy and perplexity | NLP-based alpha signals | Language-based NPC evaluation | Headline LLM eval metric |
| Graphs and the handshake lemma (§6.1) | Search graphs, graph algorithms | Graph-based ML, graph kernels | Graph neural networks (GNNs) | Trading networks, broker graphs | Navigation meshes | Knowledge graphs feeding LLMs |
| Trees, depth, height, traversals (§6.2) | Search trees (minimax, A*) | Decision trees, random forests, GBMs | Hierarchical model architectures | Decision-tree strategies | Behavior trees | Parse trees, syntax-aware decoding |
| O(V+E) BFS and DFS (§6.3) | BFS/DFS in search | Graph traversal in ML pipelines | Computational-graph traversal in autograd | Order-book graph traversal | Pathfinding (A* on grids) | Graph traversal in retrieval over knowledge graphs |
| DAGs and topological sort (§6.4) | Plan ordering, partial-order planning | Pipeline orchestration (sklearn pipelines) | Computational graphs (PyTorch, JAX) | Strategy DAGs in backtests | Dependency graphs in mission design | Tokenization-and-decoding DAGs |
| Propositional logic (§6.5) | SAT solvers, classical planning | Rule-based learners | Symbolic-numeric hybrids | Rule-based filters | Game-rule encoding | Logical evaluation of LLM outputs |
| First-order logic and resolution (§6.6) | Theorem proving, FOL planning | Inductive logic programming | Neurosymbolic methods | Rule-driven trading | Quest scripting with FOL | Chain-of-thought logic, formal-method LLMs |
| Rule-based reasoning engines (§6.7) | Production-system AI | Expert systems, rule learning | Hybrid neuro-symbolic systems | Rule-based compliance filters | NPC rule systems (CLIPS, Drools) | Guardrails and policy rule engines for LLM apps |
| Stationarity, ACF, ADF, differencing (§7.1) | — | Time-series ML foundations | Sequence-model preprocessing | Stationarity tests, differencing | — | Long-context stationarity assumptions |
| ARMA / ARIMA (§7.2) | — | Classical time series | Sequence modeling baselines | ARMA/ARIMA for returns | — | Comparison baselines for sequence LMs |
| Log returns vs simple returns (§7.3) | — | Return transformations | Numerical-stability transforms | Core ML4T material | — | Log-likelihood numerical stability |
| Volatility modeling: realized, EWMA, GARCH (§7.4) | — | — | Variance-aware training | Core ML4T material; risk reserves | — | Variance estimation in LLM evals |
| Sharpe and Information ratios (§7.5) | — | Cross-validated performance metrics | — | Core ML4T material; portfolio evaluation | — | Risk-adjusted scoring of LLM strategies (e.g., trading agents) |
| Fintech forecasting, Holt–Winters, SARIMA (§7.6) | — | Forecasting modules | Sequence-to-sequence forecasting | Core ML4T forecasting material | — | LLM forecasting (Time-LLM, sequence prompting) |

Reading the matrix. Use it as a planning tool: for a concept where Paytm needs deeper mastery (say, KL divergence for drift monitoring), the matrix shows which Georgia Tech course(s) cover it most explicitly — here CS 7641 for theory, CS 7643 for deep-learning regularization, and CS 8803 SAL for RLHF and DPO constraints. Course names and code numbers match the Georgia Tech OMSCS catalog. Where the exact module title was uncertain or course-revision dependent, a generic topic label is used; the label still pinpoints the syllabus location reliably.

End of Appendices. The seven chapters above and these three appendices together form Volume 1 of the Foundational Mathematics Primer.